

CODE DOCUMENTATION

qDischarge Clinic Queue — Backend

Complete annotated source: every Java class, the database schema, and the build/runtime configuration

Spring Boot 3.3.5 · Java 21 · PostgreSQL · 83 source files · ~8,600 lines

Table of Contents

1. Architecture Overview

2. Application Bootstrap

ClinicQueueApplication.java

3. Package: bot

BotMessages.java

ConfirmationChoice.java

Gender.java

Intent.java

Lang.java

WaSessionService.java

4. Package: catalog

MedicalCategory.java

5. Package: config

AppProperties.java

AsyncConfig.java

EvolutionWebhookInitializer.java

HospitalSeedRunner.java

JdbcConfig.java

MumbaiHospitalDataSeeder.java

RestTemplateConfig.java

SchemaInitRunner.java

WebConfig.java

6. Package: controller

CounterController.java

DoctorController.java

HospitalController.java

LocationController.java

OtpController.java

PatientController.java

QueueController.java

WebhookController.java

7. Package: dto

AccessGrantDto.java

AccessRequestDto.java

AssignDoctorLocationRequest.java

CreateHospitalRequest.java

CreateTimeSlotRequest.java

CreateTokenRequest.java

CreateTokenResult.java

DoctorDto.java

DoctorLoginRequest.java

DoctorPatientGroupDto.java

HospitalDepartmentDto.java

HospitalDto.java

JoinHospitalRequest.java

LoginRequest.java
OtpSendRequest.java
OtpVerifyRequest.java
PatientDocumentDto.java
QueueData.java
RegisterDoctorRequest.java
SetLocationRequest.java
Stats.java
TimeSlotDto.java
TokenDto.java
TokenHistoryDto.java
UpdateDepartmentCountersRequest.java
UpdateStatusRequest.java
UpdateStatusResult.java
VerifyRequest.java
WaButton.java
WaListRow.java
WaListSection.java

8. Package: exception

GlobalExceptionHandler.java

9. Package: geo

DigipinService.java
GeoDistanceService.java
TomTomRoutingService.java

10. Package: ratelimit

RateLimitFilter.java
RateLimiterService.java

11. Package: security

AdminAuthService.java
CurrentUser.java
JwtAuthFilter.java
JwtService.java
RestAccessDeniedHandler.java
RestAuthenticationEntryPoint.java
SecurityConfig.java

12. Package: service

AccessService.java
CounterAssignmentService.java
HospitalDepartmentService.java
HospitalService.java
OtpService.java
PatientDocumentService.java
QrCodeService.java
QueueManagerService.java
TimeSlotService.java
TreatmentTimingScheduler.java
TwilioStudioCallService.java
WhatsAppService.java

13. Database Schema

14. Configuration

15. Build

1. Architecture Overview

This is a Spring Boot port of an original Node/Express WhatsApp-based clinic queue management system ("qDischarge"). A patient books a token either through a WhatsApp chatbot or a web frontend, waits in a live queue scoped to one hospital's one department, and is notified/called automatically as their turn approaches — using real routing ETAs (TomTom) rather than a fixed average wait time.

No JPA/Hibernate — plain JDBC everywhere

There isn't a single `@Entity` class in this codebase. Every table is accessed through `NamedParameterJdbcTemplate` with hand-written SQL (see `JdbcConfig`), and DTOs (in `dto/`) are plain data carriers, not persistence-managed objects. The schema itself lives in one hand-maintained `schema.sql`, executed idempotently on every boot by `SchemaInitRunner` — there's no Flyway/Liquibase migration history to fall out of sync with the code.

Three JWT roles, one filter

`JwtService` issues short-lived, stateless JWTs for three distinct kinds of caller — `ROLE_ADMIN` (reception/staff), `ROLE_PATIENT` (a phone number, issued the moment its OTP is verified), and `ROLE_DOCTOR` (issued on doctor registration/login, carries a `doctorId` claim). `JwtAuthFilter` decodes whichever token is presented and populates Spring Security's context; `SecurityConfig` declares per-route role requirements explicitly, since a bare `authenticated()` would incorrectly accept any of the three roles for an admin-only route.

Every queue is (hospital, category)-scoped

A hospital doesn't run one queue — it runs one queue *per department it offers* (`catalog.MedicalCategory`). Nearly every method in `QueueManagerService` takes a (`hospitalId`, `category`) pair and scopes its SQL to it. A department's counter count (`hospital_departments`, see `HospitalDepartmentService`) determines whether it runs the simple single-counter flow or the look-ahead multi-counter package (`CounterAssignmentService`).

The WhatsApp bot is a server-side state machine

`WebhookController` receives every inbound WhatsApp message (from either Meta's Cloud API or the self-hosted Evolution API — both payload shapes are parsed) and drives a patient through registration and booking purely off a `session_step` column stored on their in-progress token row (`tokens.session_step`: `awaiting_name` → `awaiting_gender` → `awaiting_age` → `awaiting_category` → `awaiting_location` → `awaiting_hospital_selection` → `awaiting_confirmation` → `booked`). Every message, in three languages (English/Hindi/Marathi, see `bot.BotMessages`), is matched against both a tap-button id and free-typed text.

Real ETA-driven notifications, not fixed timers

Rather than notifying a patient a fixed N minutes before their turn, `TreatmentTimingScheduler` ticks on an interval and `QueueManagerService#runTreatmentTimingTick` compares each waiting patient's real routing ETA (`geo.TomTomRoutingService`, falling back to straight-line distance when no API key is configured) against how much queue time is genuinely still ahead of them. The moment travel time is no longer comfortably shorter, it fires a WhatsApp "head to the hospital now" message and a Twilio voice call together, and opens an "anomaly-control" grace window during which the patient's slot is held without blocking the counter for everyone behind them.

Doctor <-> patient consent model

A doctor is also just a phone+OTP account (`DoctorService`), optionally linked to one hospital via a join code. To view a patient's uploaded documents, a doctor generates a short-lived access code/QR (`AccessService`); the patient's device claims it, creating a revocable grant and firing a WhatsApp notice the patient can reply revoke <id> to instantly undo.

2. Application Bootstrap

The single entry point that boots the embedded Tomcat server and wires up scheduling.

ClinicQueueApplication.java

src/main/java/com/qdischarge/clinicqueue/ClinicQueueApplication.java

The `main()` entry point. `@ConfigurationPropertiesScan` picks up `AppProperties` without needing an explicit `@EnableConfigurationProperties`; `@EnableScheduling` is what makes `TreatmentTimingScheduler`'s `@Scheduled` tick actually run.

```
package com.qdischarge.clinicqueue;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.context.properties.ConfigurationPropertiesScan;
import org.springframework.scheduling.annotation.EnableScheduling;

/**
 * Entry point for the Spring Boot port of the Node/Express "qDischarge"
 * clinic queue backend. Boots an embedded Tomcat server, initializes the
 * "tokens" table on startup, and exposes the same /api/queue and /webhook
 * routes as the original backend/index.js.
 *
 * @EnableScheduling backs service.TreatmentTimingScheduler's periodic
 * re-evaluation of the active queue (real-ETA "go now" triggers +
 * anomaly-control deadline expiry) -- see that class for details.
 */
@SpringBootApplication
@ConfigurationPropertiesScan
@EnableScheduling
public class ClinicQueueApplication {

    public static void main(String[] args) {
        SpringApplication.run(ClinicQueueApplication.class, args);
    }
}
```

3. Package: bot

Everything specific to the WhatsApp conversational flow: the trilingual message catalog, the small enums that match a tap-button id or free-typed text against a menu choice, and the per-phone language/session tracker. None of this package touches the queue's business rules directly — that's QueueManagerService; this package is purely "how do we talk to the patient."

BotMessages.java

src/main/java/com/qdischarge/clinicqueue/bot/BotMessages.java

Every string the bot ever sends, in English, Hindi, and Marathi, as one lookup-table component rather than a properties/i18n bundle — deliberately, so the whole conversation (greeting, language picker, menu, registration prompts, token dashboard, status card, cancel, doctor-access notices) is auditable language-by-language in one file. Uses Java text blocks and `switch` expressions over `Lang` throughout.

```
package com.qdischarge.clinicqueue.bot;

import com.qdischarge.clinicqueue.catalog.MedicalCategory;
import com.qdischarge.clinicqueue.dto.WaButton;
import org.springframework.stereotype.Component;

import java.util.List;

/**
 * Every string the WhatsApp bot sends, in English, Hindi, and Marathi. Kept
 * as one lookup table (rather than a properties/i18n bundle) so the whole
 * conversation -- greeting, language picker, menu, registration, token
 * dashboard, status, cancel -- is easy to audit language-by-language in one
 * file.
 */
@Component
public class BotMessages {

    // -----
    // Greeting + language picker (shown before any language is chosen,
    // so always trilingual)
    // -----

    public String greetingAndLanguagePrompt(String clinicName) {
        return """
            🌸 Welcome to %1$s / %1$s में आपका स्वागत है / %1$s मध्ये आपले स्वागत आहे!

            Please choose your language / कृपया अपनी भाषा चुनें / कृपया तुमची भाषा निवडा:

            1 Type *English* for English
            2 Type *Hindi* for हिंदी
            3 Type *Marathi* for मराठी"""
            .formatted(clinicName);
    }

    public String languageNotUnderstood() {
        return """
            ! Sorry, I didn't understand that. / क्षमा करें, मुझे समझ नहीं आया। / माफ करा, मला समजले नाही.

            Please type *English*, *Hindi*, or *Marathi* to continue. / कृपया आगे बढ़ने के लिए *English*, *Hindi* या *Marathi* टाइप करें। / पुढे
            जाण्यासाठी कृपया *English*, *Hindi* किंवा *Marathi* टाइप करा.""";
    }

    public String languageSwitched(Lang lang) {
        return switch (lang) {
            case EN -> "✅ Language set to English.";
            case HI -> "✅ भाषा हिंदी में सेट कर दी गई है।";
            case MR -> "✅ भाषा मराठी मध्ये सेट केली आहे.";
        };
    }

    // -----
    // Treatment-timing "go now" trigger (see
    // service.QueueManagerService#runTreatmentTimingTick): sent -- together
    // with the Twilio voice call, see service.TwilioStudioCallService --
    // the moment a patient's real routing ETA is no longer comfortably
    // shorter than the queue's remaining treatment time.
    // -----

    public String headingToHospitalNotification(Lang lang, String patientName, int tokenNumber, String departmentName, String hospitalName) {
        return switch (lang) {
            case EN -> "Hello %s, your token number %d for %s at %s is about to be called. We request you to please start heading to the
```

```

hospital now."
        .formatted(patientName, tokenNumber, departmentName, hospitalName);
    case HI -> "नमस्ते %s, %s में %s के लिए आपका टोकन नंबर %d जल्द ही बुलाया जाने वाला है। कृपया अब अस्पताल की ओर रवाना हो जाएं।"
        .formatted(patientName, hospitalName, departmentName, tokenNumber);
    case MR -> "नमस्कार %s, %s येथे %s साठी आपला टोकन क्रमांक %d लवकरच बोलावला जाणार आहे. कृपया आता रुग्णालयाकडे रवाना व्हा."
        .formatted(patientName, hospitalName, departmentName, tokenNumber);
    };
}

// -----
// Welcome menu (STEP: no active token)
// -----

public String welcomeTitle(Lang lang, String clinicName) {
    return switch (lang) {
        case EN -> " 🏥 WELCOME TO " + clinicName.toUpperCase();
        case HI -> " 🏥 " + clinicName + " में आपका स्वागत है";
        case MR -> " 🏥 " + clinicName + " मध्ये आपले स्वागत आहे";
    };
}

public String welcomeDescription(Lang lang) {
    return switch (lang) {
        case EN -> "Welcome to our Smart Queue Management System.\n\nTap an option below, or type it:\n1 Generate Token\n2 Check Status";
        case HI -> "हमारे स्मार्ट कतार प्रबंधन सिस्टम में आपका स्वागत है।\n\nनीचे कोई विकल्प चुनें या टाइप करें:\n1 टोकन बनाएं\n2 स्थिति देखें";
        case MR -> "आमच्या स्मार्ट रांग व्यवस्थापन प्रणालीमध्ये आपले स्वागत आहे.\n\nखालील पर्याय निवडा किंवा टाइप करा:\n1 टोकन तयार करा\n2 स्थिती तपासा";
    };
}

public List<WaButton> welcomeButtons(Lang lang) {
    return switch (lang) {
        case EN -> List.of(new WaButton("btn_generate_token", " 🏥 Generate Token"), new WaButton("btn_check_status", " 🔍 Check Status"));
        case HI -> List.of(new WaButton("btn_generate_token", " 🏥 टोकन बनाएं"), new WaButton("btn_check_status", " 🔍 स्थिति देखें"));
        case MR -> List.of(new WaButton("btn_generate_token", " 🏥 टोकन तयार करा"), new WaButton("btn_check_status", " 🔍 स्थिती तपासा"));
    };
}

// -----
// Registration
// -----

public String nameRegistrationPrompt(Lang lang) {
    return switch (lang) {
        case EN -> " 🧑 *PATIENT REGISTRATION*\n\nPlease reply with your *Full Name* to generate your queue token.\n\n_Example: Yash Dubey_";
        case HI -> " 🧑 *रोगी पंजीकरण*\n\nअपना क्यू टोकन बनाने के लिए कृपया अपना *पूरा नाम* भेजें।\n\n_उदाहरण: यश दुबे_";
        case MR -> " 🧑 *रुग्ण नोंदणी*\n\nतुमचा रांग टोकन तयार करण्यासाठी कृपया तुमचे *पूर्ण नाव* पाठवा.\n\n_उदाहरण: यश दुबे_";
    };
}

public String invalidNameReminder(Lang lang) {
    return switch (lang) {
        case EN -> " 🧑 *NAME REQUIRED*\n\nPlease reply with your actual *Full Name* (e.g. John Doe) to generate your token.";
        case HI -> " 🧑 *नाम आवश्यक है*\n\nअपना टोकन बनाने के लिए कृपया अपना वास्तविक *पूरा नाम* (जैसे: जॉन डो) भेजें।";
        case MR -> " 🧑 *नाव आवश्यक आहे*\n\nतुमचा टोकन तयार करण्यासाठी कृपया तुमचे खरे *पूर्ण नाव* (उदा. जॉन डो) पाठवा.";
    };
}

public String genderPrompt(Lang lang) {
    return switch (lang) {
        case EN -> " 🧑 *GENDER*\n\nThanks! Please tell us your *Gender*.";
        case HI -> " 🧑 *लिंग*\n\nधन्यवाद! कृपया अपना *लिंग* बताएं।";
        case MR -> " 🧑 *लिंग*\n\nधन्यवाद! कृपया तुमचे *लिंग* सांगा.";
    };
}

public List<WaButton> genderButtons(Lang lang) {
    return switch (lang) {
        case EN -> List.of(new WaButton(Gender.MALE.buttonId(), "Male"), new WaButton(Gender.FEMALE.buttonId(), "Female"), new WaButton(Gender.OTHER.buttonId(), "Other"));
        case HI -> List.of(new WaButton(Gender.MALE.buttonId(), "पुरुष"), new WaButton(Gender.FEMALE.buttonId(), "महिला"), new WaButton(Gender.OTHER.buttonId(), "अन्य"));
        case MR -> List.of(new WaButton(Gender.MALE.buttonId(), "पुरुष"), new WaButton(Gender.FEMALE.buttonId(), "स्त्री"), new WaButton(Gender.OTHER.buttonId(), "इतर"));
    };
}

```

```

    };
}

public String invalidGenderReminder(Lang lang) {
    return switch (lang) {
        case EN -> "🚺 *GENDER REQUIRED*\n\nPlease tap one of the options above, or type Male, Female, or Other.";
        case HI -> "🚺 *लिंग आवश्यक है*\n\nकृपया ऊपर दिए गए विकल्पों में से एक चुनें, या पुरुष, महिला, या अन्य टाइप करें।";
        case MR -> "🚺 *लिंग आवश्यक आहे*\n\nकृपया वरील पर्यायांपैकी एक निवडा, किंवा पुरुष, स्त्री किंवा इतर टाइप करा.";
    };
}

public String agePrompt(Lang lang) {
    return switch (lang) {
        case EN -> "👴 *AGE*\n\nThanks! Now please reply with your *Age* (in years).\n\n_Example: 34_";
        case HI -> "👴 *आयु*\n\nधन्यवाद! अब कृपया अपनी *आयु* (वर्षों में) बताएं।\n\n_उदाहरण: 34_";
        case MR -> "👴 *वय*\n\nधन्यवाद! आता कृपया तुमचे *वय* (वर्षांमध्ये) सांगा।\n\n_उदाहरण: 34_";
    };
}

public String invalidAgeReminder(Lang lang) {
    return switch (lang) {
        case EN -> "👴 *VALID AGE REQUIRED*\n\nPlease reply with just a number between 0 and 120 (e.g. 34).";
        case HI -> "👴 *मान्य आयु आवश्यक है*\n\nकृपया केवल 0 से 120 के बीच की संख्या भेजें (जैसे: 34)।";
        case MR -> "👴 *वैध वय आवश्यक आहे*\n\nकृपया फक्त 0 ते 120 मधील संख्या पाठवा (उदा. 34).";
    };
}

// -----
// Category (department) selection
// -----

public String categoryPrompt(Lang lang) {
    String menu = MedicalCategory.numberedMenuText();
    return switch (lang) {
        case EN -> "🏥 *DEPARTMENT*\n\nWhich department do you need? Reply with the number.\n\n" + menu;
        case HI -> "🏥 *विभाग*\n\nआपको किस विभाग की आवश्यकता है? संख्या के साथ उत्तर दें।\n\n" + menu;
        case MR -> "🏥 *विभाग*\n\nतुम्हाला कोणत्या विभागाची गरज आहे? क्रमांकासह उत्तर द्या.\n\n" + menu;
    };
}

public String invalidCategoryReminder(Lang lang) {
    return switch (lang) {
        case EN -> "🏥 *NOT RECOGNIZED*\n\nPlease reply with just the *number* next to the department you need (e.g. 4).";
        case HI -> "🏥 *पहचाना नहीं गया*\n\nकृपया आपको चाहिए विभाग के आगे की *संख्या* के साथ उत्तर दें (जैसे: 4)।";
        case MR -> "🏥 *ओळखले नाही*\n\nकृपया तुम्हाला हव्या असलेल्या विभागाच्या पुढील *क्रमांका*सह उत्तर द्या (उदा. 4).";
    };
}

// -----
// Location (to search nearby hospitals offering the chosen department)
// -----

public String locationPrompt(Lang lang) {
    return switch (lang) {
        case EN -> "📍 *YOUR LOCATION*\n\nTap below to share your current location, so we can find the nearest hospitals for you.\n\n_Can't share it? Reply with a DIGIPIN instead._";
        case HI -> "📍 *आपका स्थान*\n\nनिकटतम अस्पताल खोजने के लिए नीचे टैप करके अपना वर्तमान स्थान साझा करें।\n\n_साझा नहीं कर सकते? इसके बजाय DIGIPIN भेजें।_";
        case MR -> "📍 *तुमचे स्थान*\n\nजवळचे रुग्णालय शोधण्यासाठी खाली टॅप करून तुमचे सध्याचे स्थान शेअर करा.\n\n_शेअर करू शकत नाही? त्याऐवजी DIGIPIN पाठवा._";
    };
}

/** Sent right after {@link #locationPrompt} as a fallback link -- see WebhookController#sendLocationPrompt. */
public String findHospitalLinkPrompt(Lang lang) {
    return switch (lang) {
        case EN -> "Having trouble sharing your location here? Tap this link instead -- it'll ask your browser for permission and show you the same nearest-hospital list.";
        case HI -> "यहाँ स्थान साझा करने में दिक्कत हो रही है? इसके बजाय यह लिंक टैप करें -- यह आपके ब्राउज़र से अनुमति मांगेगा और वही निकटतम अस्पतालों की सूची दिखाएगा।";
        case MR -> "इथे स्थान शेअर करण्यात अडचण येत आहे? त्याऐवजी ही लिंक टॅप करा -- ती तुमच्या ब्राउझरकडून परवानगी मागेल आणि तीच जवळच्या रुग्णालयांची यादी दाखवेल.";
    };
}

```

```

public String findHospitalLinkButtonText(Lang lang) {
    return switch (lang) {
        case EN -> "🌐 Find Hospitals Near Me";
        case HI -> "🌐 पास के अस्पताल खोजें";
        case MR -> "🌐 जवळची रुग्णालये शोधा";
    };
}

public String invalidLocationReminder(Lang lang) {
    return switch (lang) {
        case EN -> "📍 *LOCATION NEEDED*\n\nPlease share your current location (the 📍 /location icon), or type a valid 10-character DIGIPIN.";
        case HI -> "📍 *स्थान आवश्यक है*\n\nकृपया अपना वर्तमान स्थान साझा करें (📍 /स्थान आइकन), या एक मान्य 10-अक्षर का DIGIPIN टाइप करें।";
        case MR -> "📍 *स्थान आवश्यक आहे*\n\nकृपया तुमचे सध्याचे स्थान शेअर करा (📍 /स्थान आयकॉन), किंवा वैध 10-अक्षरी DIGIPIN टाइप करा.";
    };
}

// -----
// Hospital search results (5 at a time, "Show more" for the next 5) and confirmation
// -----

public String hospitalResultsHeader(Lang lang, String category) {
    return switch (lang) {
        case EN -> "🏥 Hospitals offering *" + category + "* near you, nearest first. Tap one to select it.";
        case HI -> "🏥 आपके पास *" + category + "* प्रदान करने वाले अस्पताल, निकटतम पहले। चुनने के लिए एक पर टैप करें।";
        case MR -> "🏥 तुमच्या जवळ *" + category + "* देणारी रुग्णालये, सर्वात जवळचे आधी. निवडण्यासाठी एकावर टॅप करा.";
    };
}

public String showMoreRowTitle(Lang lang) {
    return switch (lang) {
        case EN -> "➕ Show more";
        case HI -> "➕ और दिखाएं";
        case MR -> "➕ आणखी दाखवा";
    };
}

/** Evolution has no tappable list row, so the hint after each results page tells the patient what to type instead (see
WebhookController#isShowMoreCommand). */
public String showMoreHint(Lang lang) {
    return switch (lang) {
        case EN -> "➕ *Type \"more\" to see the next hospitals.";
        case HI -> "➕ अगले अस्पताल देखने के लिए *\"और दिखाएं\" लिखें।";
        case MR -> "➕ पुढील रुग्णालये पाहण्यासाठी *\"आणखी दाखवा\" लिहा.";
    };
}

public String noHospitalsFound(Lang lang, String category) {
    return switch (lang) {
        case EN -> "😞 No hospitals currently offer *" + category + "* near you.\n\nPlease pick a different department.\n\n" +
MedicalCategory.numberedMenuText();
        case HI -> "😞 फिलहाल आपके पास *" + category + "* प्रदान करने वाला कोई अस्पताल नहीं है।\n\nकृपया एक अलग विभाग चुनें।\n\n" +
MedicalCategory.numberedMenuText();
        case MR -> "😞 सध्या तुमच्या जवळ *" + category + "* देणारे कोणतेही रुग्णालय नाही.\n\nकृपया वेगळा विभाग निवडा.\n\n" +
MedicalCategory.numberedMenuText();
    };
}

public String invalidHospitalSelectionReminder(Lang lang) {
    return switch (lang) {
        case EN -> "🏥 Please tap a hospital from the list above, or *Show more* for more options.";
        case HI -> "🏥 कृपया ऊपर दी गई सूची से एक अस्पताल चुनें, या अधिक विकल्पों के लिए *और दिखाएं* पर टैप करें।";
        case MR -> "🏥 कृपया वरील यादीतून एक रुग्णालय निवडा, किंवा अधिक पर्यायांसाठी *आणखी दाखवा* वर टॅप करा.";
    };
}

/** Evolution has no tappable button, so this ends with a "type your choice" instruction instead of "tap" -- see
WebhookController#sendConfirmationCard. */
public String hospitalConfirmationPrompt(Lang lang, String hospitalName, String address, double distanceKm) {
    String addressLine = (address != null && !address.isBlank()) ? "\n📍 " + address : "";
    return switch (lang) {
        case EN -> "🏥 *%s%\n\n~.1f km away\n\nBook your token here?\n\n1️⃣ ✅ *Confirm*\n2️⃣ 🗨️ *Choose Again*\n\n👉 *Type 1* or *Confirm* to book, or *2* / *Choose Again* to pick a different hospital.*\n\n.formatted(hospitalName, addressLine, distanceKm);
        case HI -> "🏥 *%s%\n\n~.1f किमी दूर\n\nक्या यहां अपना टोकन बुक करें?\n\n1️⃣ ✅ *पुष्टि करें*\n2️⃣ 🗨️ *फिर से चुनें*\n\n👉 बुक करने के लिए";
    };
}

```

```

*1* या *पुष्टि करें* लिखें, या दूसरा अस्पताल चुनने के लिए *2* या *फिर से चुनें* लिखें।"
    .formatted(hospitalName, addressLine, distanceKm);
    case MR -> " 🏥 *%s*%s\n 📍 ~%.1f किमी दूर\n\ndे तुमचा टोकन बुक करायचा का?\n\n 📄 ✅ *पुष्टि करा*\n 🔄 🔄 *पुन्हा निवडा*\n\n 🙌 बुक करण्यासाठी
*1* किंवा *पुष्टि करा* लिहा, किंवा वेगळे रुग्णालय निवडण्यासाठी *2* किंवा *पुन्हा निवडा* लिहा."
    .formatted(hospitalName, addressLine, distanceKm);
};
}

public String invalidConfirmationReminder(Lang lang) {
    return switch (lang) {
        case EN -> "Please type *1* or *Confirm* to book this hospital, or *2* / *Choose Again* to pick a different one.";
        case HI -> "इस अस्पताल को बुक करने के लिए *1* या *पुष्टि करें* लिखें, या दूसरा चुनने के लिए *2* या *फिर से चुनें* लिखें।";
        case MR -> "हे रुग्णालय बुक करण्यासाठी *1* किंवा *पुष्टि करा* लिहा, किंवा वेगळे निवडण्यासाठी *2* किंवा *पुन्हा निवडा* लिहा.";
    };
}

/**
 * Sent when {@code confirmBooking} rejects the "Confirm" reply (OPD
 * closing soon, hospital gone, etc.) -- surfaces the actual reason
 * instead of repeating {@link #invalidConfirmationReminder}, which just
 * told the patient to type "1" or "Confirm" again and looped forever since
 * retrying "1" can never fix a closed OPD. Points at *Choose Again*
 * instead, since that's the only reply that can actually help here.
 */
public String bookingFailed(Lang lang, String reason) {
    return switch (lang) {
        case EN -> " ❌ %s\n\nType *2* or *Choose Again* to pick a different hospital.".formatted(reason);
        case HI -> " ❌ %s\n\ndूसरा अस्पताल चुनने के लिए *2* या *फिर से चुनें* लिखें।".formatted(reason);
        case MR -> " ❌ %s\n\nवेगळे रुग्णालय निवडण्यासाठी *2* किंवा *पुन्हा निवडा* लिहा.".formatted(reason);
    };
}

public String alreadyActiveToken(Lang lang, int id, String status) {
    return switch (lang) {
        case EN -> " ⚠️ You already have active Token #" + id + " (" + status.toUpperCase() + ").";
        case HI -> " ⚠️ आपके पास पहले से ही सक्रिय टोकन #" + id + " (" + status.toUpperCase() + ") है।";
        case MR -> " ⚠️ तुमच्याकडे आधीच सक्रिय टोकन #" + id + " (" + status.toUpperCase() + ") आहे.";
    };
}

// -----
// Token dashboard (STEP: token just generated / refreshed)
// -----

public String statusBadge(Lang lang, String status) {
    boolean serving = "serving".equals(status);
    return switch (lang) {
        case EN -> serving ? " 🟢 NOW SERVING!" : " ⌚ WAITING IN QUEUE";
        case HI -> serving ? " 🟢 अभी सेवा में!" : " ⌚ कतार में प्रतीक्षारत";
        case MR -> serving ? " 🟢 आता सेवा सुरू आहे!" : " ⌚ रांगेत प्रतीक्षेत";
    };
}

public String dashboardTitle(Lang lang, int id) {
    return switch (lang) {
        case EN -> " 📄 TOKEN #" + id + " GENERATED!";
        case HI -> " 📄 टोकन #" + id + " जनरेट हुआ!";
        case MR -> " 📄 टोकन #" + id + " तयार झाला!";
    };
}

public String dashboardDescription(Lang lang, String name, Integer age, String statusBadge, String position, int ahead, int waitMinutes) {
    String ageline = ageline(lang, age);
    return switch (lang) {
        case EN -> ""
            Hello %s,%s

            Your queue token has been generated successfully!

            🌟 *Status:* %s
            📍 *Queue Position:* %s
            🧑 *Patients Ahead:* %d
            ⌚ *Estimated Wait:* ~%d mins

            Tap below to open your interactive live queue dashboard!""
    };
}

```

```

        .formatted(name, ageLine, statusBadge, position, ahead, waitMinutes);
    case HI -> ""
        नमस्ते %s,%s

        आपका क्यू टोकन सफलतापूर्वक बन गया है!

        🌟 *स्थिति:* %s
        📍 *कतार में स्थान:* %s
        👤 *आगे मरीज़:* %d
        ⌚ *अनुमानित प्रतीक्षा:* ~%d मिनट

        अपना लाइव कतार डैशबोर्ड खोलने के लिए नीचे टैप करें!""
        .formatted(name, ageLine, statusBadge, position, ahead, waitMinutes);
    case MR -> ""
        नमस्कार %s,%s

        तुमचा रांग टोकन यशस्वीरित्या तयार झाला आहे!

        🌟 *स्थिती:* %s
        📍 *रांगेतील स्थान:* %s
        👤 *पुढे रुग्ण:* %d
        ⌚ *अंदाजे प्रतीक्षा:* ~%d मिनिटे

        तुमचे लाइव रांग डॅशबोर्ड उघडण्यासाठी खाली टॅप करा!""
        .formatted(name, ageLine, statusBadge, position, ahead, waitMinutes);
    };
}

/** "\n 🗳 Age: 34" (or its Hindi/Marathi equivalent), or "" when age isn't known yet -- keeps the greeting line clean either way. */
private String ageLine(Lang lang, Integer age) {
    if (age == null) {
        return "";
    }
    return switch (lang) {
        case EN -> "\n 🗳 Age: " + age;
        case HI -> "\n 🗳 आयु: " + age;
        case MR -> "\n 🗳 वय: " + age;
    };
}

public String liveTrackerButtonText(Lang lang) {
    return switch (lang) {
        case EN -> "🌐 Live Tracker Link";
        case HI -> "🌐 लाइव ट्रैकर लिंक";
        case MR -> "🌐 लाइव ट्रॅकर लिंक";
    };
}

public List<WaButton> quickActionButtons(Lang lang) {
    return switch (lang) {
        case EN -> List.of(new WaButton("btn_check_status", "🔄 Refresh Status"), new WaButton("btn_cancel_token", "❌ Cancel Token"));
        case HI -> List.of(new WaButton("btn_check_status", "🔄 स्थिति ताज़ा करें"), new WaButton("btn_cancel_token", "❌ टोकन रद्द करें"));
        case MR -> List.of(new WaButton("btn_check_status", "🔄 स्थिती रिफ्रेश करा"), new WaButton("btn_cancel_token", "❌ टोकन रद्द करा"));
    };
}

public String quickActionsFooter(Lang lang) {
    return switch (lang) {
        case EN -> "Need to manage your queue ticket?";
        case HI -> "अपना कतार टिकट प्रबंधित करना है?";
        case MR -> "तुमचे रांग तिकीट व्यवस्थापित करायचे आहे का?";
    };
}

// -----
// Status card
// -----

public String statusTitle(Lang lang) {
    return switch (lang) {
        case EN -> "🗳 CURRENT QUEUE STATUS";
        case HI -> "🗳 वर्तमान कतार स्थिति";
        case MR -> "🗳 सध्याची रांग स्थिती";
    };
}
}

```

```

public String statusDescription(Lang lang, int id, String name, Integer age, String position, int ahead, int waitMinutes, String status) {
    String ageSuffix = age != null ? " (" + age + ")" : "";
    return switch (lang) {
        case EN -> ""
            🗑 Token: #d
            👤 Patient: %s
            📍 Position: %s
            👥 People Ahead: %d
            ⌚ Estimated Wait: ~%d mins
            ⚡ Status: %s""
            .formatted(id, name, ageSuffix, position, ahead, waitMinutes, status.toUpperCase());
        case HI -> ""
            🗑 टोकन: #d
            👤 मरीज़: %s
            📍 स्थान: %s
            👥 आगे लोग: %d
            ⌚ अनुमानित प्रतीक्षा: ~%d मिनट
            ⚡ स्थिति: %s""
            .formatted(id, name, ageSuffix, position, ahead, waitMinutes, status.toUpperCase());
        case MR -> ""
            🗑 टोकन: #d
            👤 रुग्ण: %s
            📍 स्थान: %s
            👥 पुढे लोक: %d
            ⌚ अंदाजे प्रतीक्षा: ~%d मिनिटे
            ⚡ स्थिती: %s""
            .formatted(id, name, ageSuffix, position, ahead, waitMinutes, status.toUpperCase());
    };
}

public String noActiveTokenFound(Lang lang) {
    return switch (lang) {
        case EN -> "❌ *No Active Token Found*\n\nSend *Hi* to generate a token.";
        case HI -> "❌ *कोई सक्रिय टोकन नहीं मिला*\n\nटोकन बनाने के लिए *Hi* भेजें!";
        case MR -> "❌ *कोणताही सक्रिय टोकन आढळला नाही*\n\nटोकन तयार करण्यासाठी *Hi* पाठवा.";
    };
}

// -----
// Cancel
// -----

public String tokenCancelled(Lang lang, int id) {
    return switch (lang) {
        case EN -> "❌ *TOKEN CANCELLED*\n\nYour Token #" + id + " has been cancelled.\n\nSend *Hi* anytime to generate a new token.";
        case HI -> "❌ *टोकन रद्द किया गया*\n\nआपका टोकन #" + id + " रद्द कर दिया गया है।\n\nनया टोकन बनाने के लिए कभी भी *Hi* भेजें!";
        case MR -> "❌ *टोकन रद्द केला*\n\nतुमचा टोकन #" + id + " रद्द करण्यात आला आहे.\n\nनवीन टोकन तयार करण्यासाठी केव्हाही *Hi* पाठवा.";
    };
}

public String noActiveTokenToCancel(Lang lang) {
    return switch (lang) {
        case EN -> "❌ No active token to cancel.";
        case HI -> "❌ रद्द करने के लिए कोई सक्रिय टोकन नहीं है!";
        case MR -> "❌ रद्द करण्यासाठी कोणताही सक्रिय टोकन नाही.";
    };
}

// -----
// Doctor record-access grants (see AccessService)
// -----

/** Sent the moment a patient's device claims a doctor's access code (see AccessService#claim). "revoke <id>" is a literal, English-only
command -- kept untranslated so it's always recognized regardless of the reply's language. */
public String accessGranted(Lang lang, String doctorName, String hospitalName, int grantId) {
    String hospitalPart = hospitalName != null ? " (" + hospitalName + ")" : "";
    return switch (lang) {
        case EN -> "👉 *ACCESS GRANTED*\n\nDr. " + doctorName + hospitalPart + " can now view your records.\n\nIf this wasn't you, reply
*revoke " + grantId + " to remove access immediately.";
        case HI -> "👉 *पहुंच प्रदान की गई*\n\nडॉ. " + doctorName + hospitalPart + " अब आपके रिकॉर्ड देख सकते हैं।\n\nअगर यह आपने नहीं किया, तो तुरंत
पहुंच हटाने के लिए *revoke " + grantId + " लिखकर भेजें!";
        case MR -> "👉 *प्रवेश मंजूर*\n\nडॉ. " + doctorName + hospitalPart + " आता तुमचे रेकॉर्ड पाहू शकतात.\n\nहे तुम्ही केले नसेल, तर लगेच प्रवेश
काढण्यासाठी *revoke " + grantId + " असे लिहून पाठवा.";
    };
}

```

```

}

public String accessRevoked(Lang lang, int grantId) {
    return switch (lang) {
        case EN -> "✅ Access #" + grantId + " has been revoked.";
        case HI -> "✅ पहुँच #" + grantId + " हटा दी गई है।";
        case MR -> "✅ प्रवेश #" + grantId + " काढून टाकला आहे.";
    };
}

public String accessRevokeNotFound(Lang lang) {
    return switch (lang) {
        case EN -> "❌ Couldn't find an active access grant with that number for your phone.";
        case HI -> "❌ आपके फ़ोन के लिए उस नंबर से कोई सक्रिय पहुँच नहीं मिली।";
        case MR -> "❌ तुमच्या फोनसाठी त्या क्रमांकाचा कोणताही सक्रिय प्रवेश आढळला नाही.";
    };
}
}

```

ConfirmationChoice.java

src/main/java/com/qdischarge/clinicqueue/bot/ConfirmationChoice.java

The two choices offered after a hospital is picked ("Confirm" / "Choose Again"), matched against either a Meta button id or free-typed text in any of the three languages — Evolution API has no tappable-button primitive, so its "buttons" degrade to a numbered list the patient types back.

```

package com.qdischarge.clinicqueue.bot;

import java.util.Set;

/**
 * The two choices offered after a hospital is picked -- book it, or go back
 * and choose a different one. Evolution API has no tappable button here (see
 * WhatsAppService#formatButtonsAsText, which renders "buttons" as a plain
 * numbered list), so the prompt asks the patient to type their choice and
 * this matches free-typed text in English, Hindi, or Marathi, regardless of
 * which language the session is currently in -- mirroring {@link Intent}.
 */
public enum ConfirmationChoice {
    CONFIRM("btn_confirm_booking", Set.of(
        "1", "confirm", "yes", "y", "book", "confirm booking",
        "पुष्टि करें", "पुष्टि", "हाँ", "हाँ",
        "पुष्टी करा", "पुष्टी", "हो")),
    CHOOSE_AGAIN("btn_choose_again", Set.of(
        "2", "choose again", "change", "no", "n", "back",
        "फिर से चुनें", "फिर से", "नहीं",
        "पुन्हा निवडा", "पुन्हा", "नाही"));

    private final String buttonId;
    private final Set<String> keywords;

    ConfirmationChoice(String buttonId, Set<String> keywords) {
        this.buttonId = buttonId;
        this.keywords = keywords;
    }

    public static ConfirmationChoice match(String buttonId, String cleanMessage) {
        for (ConfirmationChoice choice : values()) {
            if (choice.buttonId.equals(buttonId) || choice.keywords.contains(cleanMessage)) {
                return choice;
            }
        }
        return null;
    }
}

```

Gender.java

src/main/java/com/qdischarge/clinicqueue/bot/Gender.java

The three gender options captured at registration and used to filter hospital search by a hospital's `gender_specific` flag — same match-by-buttonId-or-keyword pattern as `Lang / Intent`.

```
package com.qdischarge.clinicqueue.bot;

import java.util.Set;

/**
 * The three gender options captured during registration (tokens.gender) and
 * used to filter a hospital's gender_specific flag during hospital search --
 * same match-by-buttonId-or-keyword pattern as {@link Lang} and {@link Intent}.
 */
public enum Gender {
    MALE("btn_gender_male", Set.of("male", "m", "पुरुष")),
    FEMALE("btn_gender_female", Set.of("female", "f", "महिला", "स्त्री")),
    OTHER("btn_gender_other", Set.of("other", "o", "अन्य", "इतर"));

    private final String buttonId;
    private final Set<String> keywords;

    Gender(String buttonId, Set<String> keywords) {
        this.buttonId = buttonId;
        this.keywords = keywords;
    }

    public String buttonId() {
        return buttonId;
    }

    /** Lowercase value stored in tokens.gender / matched against hospitals.gender_specific. */
    public String code() {
        return name().toLowerCase();
    }

    public static Gender match(String buttonId, String cleanMessage) {
        for (Gender g : values()) {
            if (g.buttonId.equals(buttonId) || g.keywords.contains(cleanMessage)) {
                return g;
            }
        }
        return null;
    }
}
```

Intent.java

src/main/java/com/qdischarge/clinicqueue/bot/Intent.java

The three top-level menu actions (generate token / check status / cancel token), matched in any supported language, plus `isReservedWord()` which stops a menu command like "status" from ever being accidentally captured as a patient's name.

```
package com.qdischarge.clinicqueue.bot;

import java.util.Set;

/**
 * The three menu actions the WhatsApp bot understands, matched against
 * either a Meta interactive-button id or free-typed text -- in English,
 * Hindi, or Marathi, regardless of which language the session is currently
 * in, so a patient can always fall back to typing the English word too.
 */
public enum Intent {
    GENERATE_TOKEN("btn_generate_token", Set.of(
        "1", "generate token", "token", "new token", "book token",
        "टोकन", "टोकन बनाएं", "टोकन बनाओ",
        "टोकन तयार करा", "नवीन टोकन")),
    CHECK_STATUS("btn_check_status", Set.of(
        "2", "check status", "status",
        "स्थिति", "स्थिति देखें", "स्टेटस",
        "स्थिती", "स्थिती पहा")),
    CANCEL_TOKEN("btn_cancel_token", Set.of(
        "3", "cancel token", "cancel",
        "रद्द करें", "टोकन रद्द करें", "रद्द",
        "रद्द करा", "टोकन रद्द करा"));

    /** Words that mean "start over" in any supported language -- never valid as a patient's name. */
    public static final Set<String> GREETING_WORDS = Set.of("hi", "hello", "hey", "नमस्ते", "नमस्कार", "हाय");

    private final String buttonId;
    private final Set<String> keywords;

    Intent(String buttonId, Set<String> keywords) {
        this.buttonId = buttonId;
        this.keywords = keywords;
    }

    public static Intent match(String buttonId, String cleanMessage) {
        for (Intent intent : values()) {
            if (intent.buttonId.equals(buttonId) || intent.keywords.contains(cleanMessage)) {
                return intent;
            }
        }
        return null;
    }

    /** True for any word this bot treats as a command rather than a real patient name (any supported language). */
    public static boolean isReservedWord(String cleanMessage) {
        if (GREETING_WORDS.contains(cleanMessage)) {
            return true;
        }
        for (Intent intent : values()) {
            if (intent.keywords.contains(cleanMessage)) {
                return true;
            }
        }
        return false;
    }
}
```

Lang.java

src/main/java/com/qdischarge/clinicqueue/bot/Lang.java

The three supported languages. `matchInitial()` additionally accepts bare "1"/"2"/"3" — but only for the very first language prompt, before those digits become the generate/status/cancel menu shortcuts.

```
package com.qdischarge.clinicqueue.bot;

import java.util.Set;

/**
 * The three languages the WhatsApp bot converses in. Each phone number picks
 * one on first contact (see {@link WaSessionService}) and every subsequent
 * reply -- greeting, menu, token dashboard, status, cancel -- is rendered in
 * it via {@link BotMessages}.
 */
public enum Lang {
    EN(Set.of("english", "en"), "btn_lang_en"),
    HI(Set.of("hindi", "hi", "हिंदी"), "btn_lang_hi"),
    MR(Set.of("marathi", "mr", "मराठी"), "btn_lang_mr");

    private final Set<String> keywords;
    private final String buttonId;

    Lang(Set<String> keywords, String buttonId) {
        this.keywords = keywords;
        this.buttonId = buttonId;
    }

    /** DB code stored in wa_sessions.language, e.g. "EN". */
    public static Lang fromCode(String code) {
        if (code == null) {
            return null;
        }
        try {
            return Lang.valueOf(code.trim().toUpperCase());
        } catch (IllegalArgumentException e) {
            return null;
        }
    }

    /**
     * Matches a language selection tap (Meta button id) or typed word
     * ("english"/"hindi"/"marathi", or the native script). Used both for the
     * initial picker and for switching language later. Deliberately excludes
     * bare "1"/"2"/"3" -- those are already the main-menu shortcuts once a
     * language is chosen, see {@link #matchInitial}.
     */
    public static Lang match(String buttonId, String cleanMessage) {
        for (Lang lang : values()) {
            if (lang.buttonId.equals(buttonId) || lang.keywords.contains(cleanMessage)) {
                return lang;
            }
        }
        return null;
    }

    /**
     * Same as {@link #match}, plus "1"/"2"/"3" -- safe only for the very
     * first language prompt, before those digits become the generate/status/
     * cancel menu shortcuts.
     */
    public static Lang matchInitial(String buttonId, String cleanMessage) {
        Lang direct = match(buttonId, cleanMessage);
        if (direct != null) {
            return direct;
        }
        return switch (cleanMessage) {
            case "1" -> EN;
            case "2" -> HI;
            case "3" -> MR;
        };
    }
}
```

```
        default -> null;  
    };  
}  
}
```

WaSessionService.java

src/main/java/com/qdischarge/clinicqueue/bot/WaSessionService.java

Tracks, per phone number, which language a WhatsApp session has picked (`wa_sessions` table) — deliberately independent of the `tokens` table, since a phone needs a language before it has any token row, and the choice persists across however many tokens that phone books over time.

```
package com.qdischarge.clinicqueue.bot;

import lombok.RequiredArgsConstructor;
import org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Map;

/**
 * Tracks, per phone number, which language a WhatsApp bot session has picked
 * (see the wa_sessions table in schema.sql). Deliberately independent of the tokens table:
 * a phone needs a language *before* it has any token row (during the
 * greeting/picker exchange), and keeps it across tokens (new token, cancelled
 * token, completed visit, ...).
 */
@Service
@RequiredArgsConstructor
public class WaSessionService {

    private final NamedParameterJdbcTemplate jdbc;

    public record WaSession(String phone, Lang language, String stage) {
        public boolean awaitingLanguage() {
            return language == null;
        }
    }

    /** Null if this phone has never messaged the bot before. */
    public WaSession get(String phone) {
        List<Map<String, Object>> rows = jdbc.queryForList(
            "SELECT phone, language, stage FROM wa_sessions WHERE phone = :phone", Map.of("phone", phone));
        if (rows.isEmpty()) {
            return null;
        }
        Map<String, Object> row = rows.get(0);
        return new WaSession(phone, Lang.fromCode((String) row.get("language")), (String) row.get("stage"));
    }

    /** First-ever contact from this phone: open a session with no language chosen yet. */
    public void createAwaitingLanguage(String phone) {
        jdbc.update(
            "INSERT INTO wa_sessions (phone, language, stage) VALUES (:phone, NULL, 'awaiting_language') " +
            "ON CONFLICT (phone) DO NOTHING",
            Map.of("phone", phone));
    }

    public void setLanguage(String phone, Lang language) {
        jdbc.update(
            """
            INSERT INTO wa_sessions (phone, language, stage, updated_at)
            VALUES (:phone, :language, 'ready', NOW())
            ON CONFLICT (phone) DO UPDATE SET language = :language, stage = 'ready', updated_at = NOW()
            """,
            Map.of("phone", phone, "language", language.name()));
    }
}
```

4. Package: catalog

A single class defining the fixed, closed set of medical departments the whole system is built around.

MedicalCategory.java

src/main/java/com/qdischarge/clinicqueue/catalog/MedicalCategory.java

The fixed picklist of 33 medical departments/specialties every hospital's offered categories and every token's booked category is validated against — a real queue-partition key, not free text. `match()` resolves a patient's typed reply against the numbered menu: a bare number, an exact case-insensitive match, or an unambiguous substring match (rejecting anything that could mean more than one category, e.g. "cardi").

```
package com.qdischarge.clinicqueue.catalog;

import java.util.List;

/**
 * The fixed set of medical departments/specialties a hospital can offer and
 * a patient can search/book by. Deliberately a flat picklist (not free text)
 * so it's a real queue-partition key (see hospital_departments,
 * tokens.category) rather than an unbounded string -- every hospital's
 * offered categories and every token's booked category are validated
 * against this list.
 */
public final class MedicalCategory {

    public static final List<String> ALL = List.of(
        "General Medicine / Internal Medicine",
        "General Surgery",
        "Family Medicine",
        "Cardiology",
        "Cardiac Surgery",
        "Orthopaedics",
        "Oncology (Medical/Surgical)",
        "Neurology",
        "Neurosurgery",
        "Nephrology",
        "Urology",
        "Gastroenterology",
        "Gastrointestinal/Laparoscopic Surgery",
        "Endocrinology",
        "Pulmonology",
        "Dermatology",
        "Psychiatry",
        "ENT (Otorhinolaryngology)",
        "Obstetrics & Gynaecology",
        "Paediatrics",
        "Paediatric Cardiology",
        "Paediatric Nephrology",
        "Fertility/IVF",
        "Ophthalmology",
        "Dentistry",
        "Physiotherapy",
        "Nutrition & Dietetics",
        "Psychology/Counseling",
        "Bariatric Surgery",
        "Hepatobiliary & Pancreatic Surgery",
        "Colorectal Surgery",
        "Rheumatology",
        "Hematology"
    );

    private MedicalCategory() {
    }

    public static boolean isValid(String name) {
        return canonicalize(name) != null;
    }

    /** Exact (case-insensitive) match only -- returns the canonical, correctly-cased name, or null if unrecognized. */
    public static String canonicalize(String name) {
        if (name == null) {
            return null;
        }
    }
}
```

```

String trimmed = name.trim();
for (String c : ALL) {
    if (c.equalsIgnoreCase(trimmed)) {
        return c;
    }
}
return null;
}

/**
 * Matches a patient's typed reply against the numbered menu (see
 * {@link #numberedMenuText()}: either a bare number ("4") or text that
 * exactly matches (case-insensitive) or uniquely appears as a substring
 * of exactly one category name ("cardiology" -> Cardiology; "cardi" is
 * rejected as ambiguous between Cardiology/Cardiac Surgery/Paediatic
 * Cardiology -- the patient should use the number instead).
 */
public static String match(String input) {
    if (input == null || input.isBlank()) {
        return null;
    }
    String trimmed = input.trim();

    if (trimmed.matches("\\d{1,2}")) {
        int n = Integer.parseInt(trimmed);
        if (n >= 1 && n <= ALL.size()) {
            return ALL.get(n - 1);
        }
        return null;
    }

    String exact = canonicalize(trimmed);
    if (exact != null) {
        return exact;
    }

    String lower = trimmed.toLowerCase();
    List<String> substringHits = ALL.stream().filter(c -> c.toLowerCase().contains(lower)).toList();
    return substringHits.size() == 1 ? substringHits.get(0) : null;
}

/** "1. General Medicine / Internal Medicine\n2. General Surgery\n..." -- the plain numbered picker sent over WhatsApp. */
public static String numberedMenuText() {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < ALL.size(); i++) {
        if (i > 0) {
            sb.append('\n');
        }
        sb.append(i + 1).append(" ").append(ALL.get(i));
    }
    return sb.toString();
}
}

```

5. Package: config

Spring `@Configuration` beans and `ApplicationRunners` that wire up the datasource, schema, thread pools, and startup data seeding. This is also where the app's one `@ConfigurationProperties` class (`AppProperties`) lives, giving every environment variable a typed home.

AppProperties.java

src/main/java/com/qdischarge/clinicqueue/config/AppProperties.java

Typed view of every `app.*` key in `application.yml`, which in turn resolves from environment variables. One field per feature area: admin auth, WhatsApp provider selection, rate limits, the async executor, geo/TomTom/Twilio settings, default hospital seeding, and OTP provider config. `getFrontendUrl()` has a small safety net: it refuses to hand back a `localhost` URL (falls back to the production frontend), since that value ends up in outbound WhatsApp links a patient's phone would otherwise fail to resolve.

```
package com.qdischarge.clinicqueue.config;

import lombok.Data;
import org.springframework.boot.context.properties.ConfigurationProperties;

/**
 * Typed view of the "app.*" configuration keys defined in application.yml,
 * which in turn resolve from environment variables (see README for the
 * full list, including the security-hardening additions on top of the
 * original Node backend's .env).
 */
@Data
@ConfigurationProperties(prefix = "app")
public class AppProperties {

    private String frontendUrl;
    private String clinicName;
    private int tokenExpiryHours;
    private int avgServiceMinutes;

    public String getFrontendUrl() {
        if (frontendUrl == null || frontendUrl.isBlank() || frontendUrl.contains("localhost")) {
            return "https://princete.com";
        }
        return frontendUrl;
    }

    // --- Admin auth ---
    private String adminUsername;
    /** Plaintext dev fallback; hashed in-memory at startup. Prefer adminPasswordHash in production. */
    private String adminPassword;
    /** BCrypt hash of the admin password. Takes precedence over adminPassword when set. */
    private String adminPasswordHash;
    /** HMAC signing key for admin JWTs. Must be set explicitly in production. */
    private String jwtSecret;
    private int jwtExpiryMinutes;

    /** Static secret Meta calls back with during webhook subscription verification (unrelated to admin JWT auth). */
    private String webhookVerifyToken;

    /** Comma-separated list of origins allowed to call this API (CORS). */
    private String allowedOrigins;

    // --- WhatsApp providers ---
    private String waProvider;
    private String metaAccessToken;
    private String metaPhoneNumberId;
    private String metaApiVersion;

    private String evolutionApiUrl;
    private String evolutionApiKey;
    private String instanceName;

    // --- Rate limiting (per client IP, sliding window) ---
    private int rateLimitLoginPerMinute;
    private int rateLimitWebhookPerMinute;
    private int rateLimitCreateTokenPerMinute;

    // --- Async WhatsApp dispatch executor ---
    private int asyncCorePoolSize;
    private int asyncMaxPoolSize;
    private int asyncQueueCapacity;
```

```
// --- Geo / distance-based notification ---
/** Assumed average travel speed used to turn distance into an ETA (straight-line, no live traffic/routing API) -- fallback only, see
tomtomApiKey. */
private double geoAvgSpeedKmh;

// --- Treatment-timing: real routing ETA vs. queue wait, anomaly-control, voice call ---
/** TomTom Routing API key; blank = always use the haversine fallback (see geo.TomTomRoutingService). */
private String tomtomApiKey;
/** Caller-id ("From") number for the outbound "head to the hospital now" call (see service.TwilioStudioCallService). */
private String twilioCallerNumber;
/** Master on/off switch for placing the actual call -- off by default so a fresh checkout never dials anyone. */
private boolean twilioCallEnabled;
/** Extra arrival buffer (minutes) added on top of the travel ETA when deciding it's time to notify+call. */
private int notifyBufferMinutes;
/** How often (ms) the treatment-timing scheduler re-evaluates the active queue (see service.TreatmentTimingScheduler). */
private long timingPollIntervalMs;

// --- Hospital / counters defaults ---
private String hospitalUriSlug;
private String hospitalDigipin;
private Double hospitalLatitude;
private Double hospitalLongitude;
private String hospitalOpenTime;
private String hospitalCloseTime;
private int hospitalActiveCounters;

// --- OTP (Twilio Verify, or a log-only dev fallback) ---
private String otpProvider;
private boolean otpRequiredForRegistration;
private String twilioAccountSid;
private String twilioAuthToken;
private String twilioVerifyServiceSid;
/** Messaging Service SID used by the 'twilio-sms' OTP provider (raw Messages API, not Verify). */
private String twilioMessagingServiceSid;
private int otpTtlMinutes;
private int rateLimitOtpPerMinute;
}
```

AsyncConfig.java

src/main/java/com/qdischarge/clinicqueue/config/AsyncConfig.java

Defines the `whatsappExecutor` bounded thread pool that every outbound WhatsApp/Twilio call runs on (`@Async("whatsappExecutor")`) — so a slow or unreachable third-party API can never exhaust Tomcat's own request-handling threads.

```
package com.qdischarge.clinicqueue.config;

import lombok.RequiredArgsConstructor;
import org.springframework.aop.interceptor.SimpleAsyncUncaughtExceptionHandler;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.scheduling.annotation.AsyncConfigurer;
import org.springframework.scheduling.annotation.EnableAsync;
import org.springframework.scheduling.concurrent.ThreadPoolTaskExecutor;

import java.util.concurrent.Executor;

/**
 * Dedicated, bounded thread pool for outbound WhatsApp API calls, so a slow
 * or unreachable WhatsApp provider can't exhaust Tomcat's request-handling
 * threads. HTTP request threads hand off notification sends here and return
 * to the client immediately (see WhatsAppService).
 */
@Configuration
@EnableAsync
@RequiredArgsConstructor
public class AsyncConfig implements AsyncConfigurer {

    private final AppProperties appProperties;

    @Override
    @Bean(name = "whatsappExecutor")
    public Executor getAsyncExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(appProperties.getAsyncCorePoolSize());
        executor.setMaxPoolSize(appProperties.getAsyncMaxPoolSize());
        executor.setQueueCapacity(appProperties.getAsyncQueueCapacity());
        executor.setThreadNamePrefix("whatsapp-notify-");
        executor.initialize();
        return executor;
    }

    @Override
    public org.springframework.aop.interceptor.AsyncUncaughtExceptionHandler getAsyncUncaughtExceptionHandler() {
        return new SimpleAsyncUncaughtExceptionHandler();
    }
}
```

EvolutionWebhookInitializer.java

src/main/java/com/qdischarge/clinicqueue/config/EvolutionWebhookInitializer.java

On startup, if `WA_PROVIDER=evolution`, registers this app's webhook URL with the Evolution API instance so incoming WhatsApp messages get routed here automatically. Runs on a virtual thread so a slow/unreachable Evolution API never delays Spring Boot's own startup.

```
package com.qdischarge.clinicqueue.config;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.http.HttpEntity;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.http.client.SimpleClientHttpRequestFactory;
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestTemplate;

import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/**
 * Automatically configures Evolution API webhook target to point to
 * * http://host.docker.internal:8088/webhook/whatsapp on application startup,
 * * ensuring incoming WhatsApp messages trigger queue flow seamlessly.
 */
@Component
@RequiredArgsConstructor
@Slf4j
public class EvolutionWebhookInitializer implements ApplicationRunner {

    private final AppProperties appProperties;

    @Value("${server.port:8088}")
    private int serverPort;

    @Override
    public void run(ApplicationArguments args) {
        if (!"evolution".equalsIgnoreCase(appProperties.getWaProvider())) {
            log.info("WA_PROVIDER is set to '{}', skipping Evolution API webhook registration.", appProperties.getWaProvider());
            return;
        }

        String evolutionUrl = appProperties.getEvolutionApiUrl();
        String instance = appProperties.getInstanceName();
        String apiKey = appProperties.getEvolutionApiKey();

        if (evolutionUrl == null || evolutionUrl.isBlank() || instance == null || instance.isBlank()) {
            log.warn("⚠️ EVOLUTION_API_URL or INSTANCE_NAME not set. Skipping webhook setup.");
            return;
        }

        String targetUrl = "http://host.docker.internal:" + serverPort + "/webhook/whatsapp";
        String endpoint = evolutionUrl + "/webhook/set/" + instance;

        log.info("📌 Registering Evolution API Webhook for instance '{}' -> {}", instance, targetUrl);

        // Run asynchronously so network latency/timeout to Evolution API never delays Spring Boot startup
        Thread.ofVirtual().start(() -> {
            try {
                SimpleClientHttpRequestFactory requestFactory = new SimpleClientHttpRequestFactory();
                requestFactory.setConnectTimeout(3000);
                requestFactory.setReadTimeout(5000);
                RestTemplate shortTimeoutTemplate = new RestTemplate(requestFactory);

                HttpHeaders headers = new HttpHeaders();
                headers.setContentType(MediaType.APPLICATION_JSON);
                if (apiKey != null && !apiKey.isBlank()) {

```

```
        headers.set("apikey", apiKey);
    }

    Map<String, Object> webhookConfig = new LinkedHashMap<>();
    webhookConfig.put("enabled", true);
    webhookConfig.put("url", targetUrl);
    webhookConfig.put("byEvents", false);
    webhookConfig.put("base64", false);
    webhookConfig.put("events", List.of("MESSAGES_UPSERT", "SEND_MESSAGE"));

    Map<String, Object> payload = new LinkedHashMap<>();
    payload.put("webhook", webhookConfig);

    shortTimeoutTemplate.postForEntity(endpoint, new HttpEntity<>(payload, headers), Map.class);
    log.info("✅ Evolution API Webhook configured successfully at {}", targetUrl);
} catch (Exception e) {
    log.info("❌ Evolution API Webhook auto-registration note: {} (Evolution API URL: {})", e.getMessage(), evolutionUrl);
}
}
});
}
}
```

HospitalSeedRunner.java

src/main/java/com/qdischarge/clinicqueue/config/HospitalSeedRunner.java

Upserts the single operating hospital's row (keyed by `app.hospital-uri-slug`) from `HOSPITAL_*` env vars on every boot — config is the source of truth, the DB just mirrors it. Leaves the DB's existing location alone if no DIGIPIN or lat/lon is configured, rather than guessing.

```
package com.qdischarge.clinicqueue.config;

import com.qdischarge.clinicqueue.dto.CreateHospitalRequest;
import com.qdischarge.clinicqueue.dto.SetLocationRequest;
import com.qdischarge.clinicqueue.service.HospitalService;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.stereotype.Component;

/**
 * Upserts the single operating hospital's row (uri_slug = app.hospital-uri-slug)
 * from HOSPITAL_* env vars on every boot -- the same "config is the source of
 * truth, DB just mirrors it" pattern as AdminAuthService's credential handling.
 * The "hospitals" table already exists by the time any ApplicationRunner runs,
 * since Hibernate creates the schema (spring.jpa.hibernate.ddl-auto) during
 * context refresh, before runners execute.
 * If neither a DIGIPIN nor lat/lon is configured, it leaves whatever's already
 * in the DB alone -- location is meant to be set explicitly, via env vars or
 * the admin API, not guessed at.
 */
@Component
@RequiredArgsConstructor
@Slf4j
public class HospitalSeedRunner implements ApplicationRunner {

    private final HospitalService hospitalService;
    private final AppProperties appProperties;

    @Override
    public void run(ApplicationArguments args) {
        boolean hasDigipin = appProperties.getHospitalDigipin() != null && !appProperties.getHospitalDigipin().isBlank();
        boolean hasLatLon = appProperties.getHospitalLatitude() != null && appProperties.getHospitalLongitude() != null;

        if (!hasDigipin && !hasLatLon) {
            log.info("No HOSPITAL_DIGIPIN or HOSPITAL_LATITUDE/LONGITUDE configured; leaving hospital '{}' location as-is.",
                    appProperties.getHospitalUriSlug());
            return;
        }

        try {
            SetLocationRequest location = hasDigipin
                ? new SetLocationRequest(appProperties.getHospitalDigipin(), null, null)
                : new SetLocationRequest(null, appProperties.getHospitalLatitude(), appProperties.getHospitalLongitude());

            hospitalService.create(new CreateHospitalRequest(
                appProperties.getHospitalUriSlug(),
                appProperties.getClinicName(),
                null,
                location,
                appProperties.getHospitalOpenTime(),
                appProperties.getHospitalCloseTime(),
                // avgPatientsPerDay: not sourced from env vars -- avgServiceMinutes below is given directly instead.
                null,
                appProperties.getAvgServiceMinutes(),
                // minServiceMinutes: admin-API-only, like the profile fields below -- preserved
                // across re-seeding by HospitalService#create's COALESCE, never wiped by this null.
                null,
                appProperties.getHospitalActiveCounters(),
                // Profile fields (ownership onward) aren't sourced from env vars -- leave them
                // to the admin API. HospitalService#create preserves whatever's already stored
                // on every re-seed instead of wiping them out with these nulls.
                null, null, null, null, null
            ));
        }
    }
}
```

```
        log.info("Hospital '{}' location seeded from configuration.", appProperties.getHospitalUriSlug());
    } catch (Exception e) {
        log.error("⚠️ Failed to seed hospital location at startup: {}", e.getMessage());
    }
}
```

JdbcConfig.java

src/main/java/com/qdischarge/clinicqueue/config/JdbcConfig.java

The one bean definition (`NamedParameterJdbcTemplate`) that every service in this codebase is built on, since there are no JPA repositories anywhere.

```
package com.qdischarge.clinicqueue.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate;

import javax.sql.DataSource;

@Configuration
public class JdbcConfig {

    @Bean
    public NamedParameterJdbcTemplate namedParameterJdbcTemplate(DataSource dataSource) {
        return new NamedParameterJdbcTemplate(dataSource);
    }
}
```

MumbaiHospitalDataSeeder.java

src/main/java/com/qdischarge/clinicqueue/config/MumbaiHospitalDataSeeder.java

Demo/seed data: populates 30+ realistic hospital entries across Thane and Mumbai's western suburbs on every startup, with real-ish coordinates, categories, and OPD hours, for local development and demos without needing to hand-enter hospitals.

```
package com.qdischarge.clinicqueue.config;

import com.qdischarge.clinicqueue.dto.CreateHospitalRequest;
import com.qdischarge.clinicqueue.dto.SetLocationRequest;
import com.qdischarge.clinicqueue.service.HospitalService;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate;
import org.springframework.stereotype.Component;

import java.util.ArrayList;
import java.util.List;

/**
 * Automatically populates 125+ realistic hospital entries across Thane (West/East,
 * Ghodbunder Road, Majiwada, Panchpakhadi, Naupada, Kalwa) and Mumbai Western Suburbs
 * (Kandivali, Borivali, Malad, Goregaon, Andheri, Dahisar, Mira Road) on startup.
 */
@Component
@RequiredArgsConstructor
@Slf4j
public class MumbaiHospitalDataSeeder implements ApplicationRunner {

    private final HospitalService hospitalService;
    private final NamedParameterJdbcTemplate jdbc;

    @Override
    public void run(ApplicationArguments args) {
        try {
            log.info("🌱 Seeding 125+ hospitals across Thane & Mumbai Western Suburbs...");
            List<HospitalSeedData> seedList = getHospitalsList();

            int seeded = 0;
            for (HospitalSeedData h : seedList) {
                try {
                    // digipin left null: the seed data only carries lat/lon, and
                    // HospitalService derives a real DIGIPIN from those via
                    // DigipinService#encode. (This used to pass a fake placeholder
                    // like "4006060001" -- not a real DIGIPIN, just a PIN code plus
                    // a counter -- which failed decode() for every single hospital
                    // since '0'/'1' aren't valid DIGIPIN characters.)
                    SetLocationRequest loc = new SetLocationRequest(null, h.latitude, h.longitude);
                    hospitalService.create(new CreateHospitalRequest(
                        h.slug,
                        h.name,
                        h.address,
                        loc,
                        h.openTime,
                        h.closeTime,
                        null,
                        h.avgServiceMins,
                        5,
                        h.activeCounters,
                        h.ownership,
                        h.yearEst,
                        List.of("NABH", "NABL"),
                        // genderSpecific: null means general/co-ed (open to everyone) --
                        // HospitalService#create only accepts null/"male"/"female", so the
                        // "ALL" literal that used to be here made every single create() call
                        // throw IllegalArgumentException, silently skipped (logged at DEBUG)
                        // by this loop's per-hospital catch below. That's why none of these
                        // 125 hospitals were actually landing in the DB.
                        null,

```

```

        h.categories
    ));
    seeded++;
} catch (Exception e) {
    // create() upserts (ON CONFLICT uri_slug DO UPDATE), so an existing row is
    // never what lands here -- this is always a genuine failure (bad data,
    // DB hiccup, ...), so it needs to be loud, not swallowed at DEBUG where a
    // systemic failure across every single entry (as genderSpecific="ALL" once
    // was) can silently zero out the whole seed run unnoticed.
    log.warn("⚠ Failed to seed hospital {}: {}", h.slug, e.getMessage());
}
}

log.info("✅ Successfully seeded {} hospitals across Thane & Mumbai!", seeded);
} catch (Exception e) {
    log.error("⚠ Failed to seed hospital data: {}", e.getMessage());
}
}

private record HospitalSeedData(
    String slug,
    String name,
    String address,
    double latitude,
    double longitude,
    String openTime,
    String closeTime,
    int avgServiceMins,
    int activeCounters,
    String ownership,
    int yearEst,
    List<String> categories
) {}

private List<HospitalSeedData> getHospitalsList() {
    List<String> allCats = List.of(
        "General Medicine / Internal Medicine",
        "General Surgery",
        "Family Medicine",
        "Cardiology",
        "Orthopaedics",
        "Neurology",
        "Paediatrics",
        "Dermatology",
        "Obstetrics & Gynaecology",
        "ENT (Otorhinolaryngology)",
        "Dentistry",
        "Gastroenterology"
    );

    List<String> stdCat = List.of("General Medicine / Internal Medicine", "Cardiology", "Orthopaedics", "Paediatrics", "Dermatology");
    List<String> surgCat = List.of("General Medicine / Internal Medicine", "General Surgery", "Obstetrics & Gynaecology", "ENT
(Otorhinolaryngology)", "Ophthalmology");
    List<String> multiCat = List.of("General Medicine / Internal Medicine", "Cardiology", "Neurology", "Orthopaedics", "Dermatology",
"Gastroenterology", "Paediatrics");

    List<HospitalSeedData> list = new ArrayList<>();

    // --- THANE HOSPITALS (25 Entries) ---
    list.add(new HospitalSeedData("jupiter-hospital-thane", "Jupiter Hospital Thane", "Eastern Express Highway, Next to Viviana Mall, Thane
West 400606", 19.2016, 72.9785, "08:00", "22:00", 10, 6, "Private", 2007, allCats));
    list.add(new HospitalSeedData("bethany-hospital-thane", "Bethany Hospital Thane", "Pokhran Road No. 2, Opp. Meadows, Thane West
400610", 19.2223, 72.9701, "08:30", "21:30", 10, 5, "Private Trust", 2011, multiCat));
    list.add(new HospitalSeedData("hiranandani-hospital-thane", "Hiranandani Hospital Thane", "Hiranandani Estate, Ghodbunder Road,
Patlipada, Thane West 400607", 19.2458, 72.9742, "09:00", "21:00", 12, 4, "Private", 2009, multiCat));
    list.add(new HospitalSeedData("horizon-prime-hospital-thane", "Horizon Prime Hospital Thane", "Ghodbunder Road, Near Anand Nagar,
Kavesar, Thane West 400615", 19.2612, 72.9733, "08:00", "22:00", 10, 4, "Private", 2018, allCats));
    list.add(new HospitalSeedData("currae-specialty-hospital-thane", "Currae Specialty Hospital", "High Street Mall, Ghodbunder Road,
Kavesar, Thane West 400607", 19.2550, 72.9720, "08:30", "20:30", 10, 3, "Private", 2015, surgCat));
    list.add(new HospitalSeedData("titan-hospital-thane", "Titan Hospital Thane", "Manpada, Ghodbunder Road, Near D-Mart, Thane West
400607", 19.2367, 72.9750, "09:00", "21:00", 10, 3, "Private", 2016, stdCat));
    list.add(new HospitalSeedData("vedant-hospital-thane", "Vedant Hospital Thane", "Vartak Nagar, Pokhran Road No. 1, Thane West 400606",
19.2155, 72.9654, "08:00", "21:00", 10, 3, "Private", 2013, stdCat));
    list.add(new HospitalSeedData("kaushalya-hospital-thane", "Kaushalya Medical Foundation Trust Hospital", "Ganeshwadi, Panchpakhadi,
Near Nitin Company, Thane West 400602", 19.1945, 72.9660, "00:00", "23:59", 8, 5, "Trust", 2002, allCats));

```

```

    list.add(new HospitalSeedData("apex-hospital-thane", "Apex Hospital Thane", "Panchpakhadi, Opp. Service Road, Thane West 400602",
19.1950, 72.9680, "09:00", "21:00", 10, 3, "Private", 2014, multiCat));
    list.add(new HospitalSeedData("thane-health-care-hospital", "Thane Health Care Hospital", "Gokhale Road, Naupada, Near Thane Railway
Station, Thane West 400602", 19.1890, 72.9710, "08:30", "20:30", 10, 3, "Private", 2005, stdCat));
    list.add(new HospitalSeedData("sapphire-hospital-thane", "Sapphire Hospital Thane", "Charai, Near Court Naka, Thane West 400601",
19.1860, 72.9755, "09:00", "21:00", 10, 2, "Private", 2010, surgCat));
    list.add(new HospitalSeedData("platinum-hospital-thane", "Platinum Hospital Thane East", "Station Road, Near Railway Station, Thane
East 400603", 19.1830, 72.9810, "08:00", "22:00", 10, 4, "Private", 2017, allCats));
    list.add(new HospitalSeedData("lifecare-hospital-thane", "Life Care Hospital Naupada", "Vishnu Nagar, Naupada, Thane West 400602",
19.1875, 72.9690, "09:00", "21:00", 10, 3, "Private", 2012, stdCat));
    list.add(new HospitalSeedData("horizon-hospital-naupada", "Horizon Hospital Naupada", "Ram Maruti Road, Naupada, Thane West 400602",
19.1910, 72.9680, "08:30", "21:30", 10, 3, "Private", 2011, multiCat));
    list.add(new HospitalSeedData("aastha-hospital-thane", "Aastha Hospital Teen Hath Naka", "Near Teen Hath Naka Flyover, Eastern Express
Highway, Thane West 400602", 19.1895, 72.9640, "09:00", "21:00", 10, 3, "Private", 2015, surgCat));
    list.add(new HospitalSeedData("orbit-hospital-majiwada", "Orbit Multispecialty Hospital", "Majiwada Flyover Junction, Thane West
400601", 19.2120, 72.9790, "08:00", "22:00", 10, 4, "Private", 2019, allCats));
    list.add(new HospitalSeedData("kevalya-hospital-thane", "Kevalya Hospital Kasarvadavali", "Near Kasarvadavali Naka, Ghodbunder Road,
Thane West 400615", 19.2750, 72.9710, "09:00", "21:00", 10, 3, "Private", 2016, stdCat));
    list.add(new HospitalSeedData("vedam-hospital-thane", "Vedam Hospital Owale", "Owale, Ghodbunder Road, Thane West 400615", 19.2830,
72.9690, "08:30", "20:30", 10, 2, "Private", 2017, stdCat));
    list.add(new HospitalSeedData("csmh-hospital-kalwa", "Chhatrapati Shivaji Maharaj Hospital (CSMH)", "Belapur Road, Kalwa, Thane
400605", 19.2010, 72.9930, "00:00", "23:59", 8, 8, "Government", 1989, allCats));
    list.add(new HospitalSeedData("pinnacle-ortho-thane", "Pinnacle Ortho Centre", "Chandanwadi, Panchpakhadi, Thane West 400602", 19.1970,
72.9730, "09:00", "20:00", 12, 3, "Private", 2008, stdCat));
    list.add(new HospitalSeedData("kalyan-hospital-thane", "Kalyan Hospital & ICU", "Agra Road, Near Castle Mill Naka, Thane West 400601",
19.2040, 72.9720, "08:00", "21:00", 10, 3, "Private", 2013, multiCat));
    list.add(new HospitalSeedData("singhania-hospital-thane", "Singhania Hospital & Medical Centre", "Jekegram, Pokhran Road No. 1, Thane
West 400606", 19.2090, 72.9680, "08:30", "21:30", 10, 4, "Trust", 2000, allCats));
    list.add(new HospitalSeedData("prashanti-hospital-thane", "Prashanti Hospital Thane", "Bhaskar Colony, Naupada, Thane West 400602",
19.1865, 72.9670, "09:00", "21:00", 10, 2, "Private", 2014, stdCat));
    list.add(new HospitalSeedData("siddhi-hospital-manpada", "Siddhi Hospital Manpada", "Gladys Alvares Road, Manpada, Thane West 400607",
19.2310, 72.9730, "08:30", "20:30", 10, 3, "Private", 2016, stdCat));
    list.add(new HospitalSeedData("shree-sai-hospital-waghbil", "Shree Sai Hospital Waghbil", "Waghbil Naka, Ghodbunder Road, Thane West
400615", 19.2660, 72.9735, "09:00", "21:00", 10, 3, "Private", 2018, multiCat));

    // --- MUMBAI WESTERN SUBURBS (Selected Top Entries) ---
    list.add(new HospitalSeedData("apex-kandivali-west", "Apex Super Speciality Hospital", "Link Road, Opp. Shimpoli Signal, Kandivali
West, Mumbai 400067", 19.2105, 72.8512, "08:00", "22:00", 10, 4, "Private", 2008, multiCat));
    list.add(new HospitalSeedData("shatabdi-kandivali-west", "Shatabdi Municipal Hospital", "S.V. Road, Near Station, Kandivali West,
Mumbai 400067", 19.2062, 72.8530, "00:00", "23:59", 8, 8, "Government", 1985, stdCat));
    list.add(new HospitalSeedData("lotus-kandivali-west", "Lotus Multispecialty Hospital", "M.G. Road, Opp. Dena Bank, Kandivali West,
Mumbai 400067", 19.2081, 72.8495, "09:00", "21:00", 12, 3, "Private", 2012, surgCat));
    list.add(new HospitalSeedData("karuna-borivali-west", "Karuna Hospital Borivali", "Junction of S.V. Road & Link Road, Borivali West,
Mumbai 400092", 19.2315, 72.8540, "08:00", "22:00", 10, 5, "Trust", 1990, multiCat));
    list.add(new HospitalSeedData("surana-hospital-malad", "Surana Hospital & Research Centre", "Tank Road, Off S.V. Road, Orlem, Malad
West, Mumbai 400064", 19.1865, 72.8480, "08:30", "21:30", 10, 4, "Private", 2004, multiCat));
    list.add(new HospitalSeedData("lifeline-goregaon-west", "Lifeline Hospital Goregaon", "S.V. Road, Opp. Railway Station, Goregaon West,
Mumbai 400104", 19.1620, 72.8445, "09:00", "21:00", 10, 4, "Private", 2006, stdCat));
    list.add(new HospitalSeedData("kokilaben-andheri-west", "Kokilaben Dhirubhai Ambani Hospital", "Rao Saheb Achutrao Patwardhan Marg,
Four Bungalows, Andheri West, Mumbai 400053", 19.1310, 72.8252, "00:00", "23:59", 10, 10, "Private", 2009, allCats));
    list.add(new HospitalSeedData("criti-care-andheri-west", "Criticare Asia Multispecialty Hospital", "Gulmohar Road, JVPD Scheme, Andheri
West, Mumbai 400049", 19.1165, 72.8310, "08:00", "22:00", 10, 5, "Private", 2001, multiCat));

    return list;
}
}

```

RestTemplateConfig.java

src/main/java/com/qdischarge/clinicqueue/config/RestTemplateConfig.java

The single `RestTemplate` bean (10s connect / 15s read timeout) shared by every outbound HTTP integration — WhatsApp providers, Twilio, TomTom.

```
package com.qdischarge.clinicqueue.config;

import org.springframework.boot.web.client.RestTemplateBuilder;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.client.RestTemplate;

import java.time.Duration;

@Configuration
public class RestTemplateConfig {

    @Bean
    public RestTemplate restTemplate(RestTemplateBuilder builder) {
        return builder
            .setConnectTimeout(Duration.ofSeconds(10))
            .setReadTimeout(Duration.ofSeconds(15))
            .build();
    }
}
```

SchemaInitRunner.java

src/main/java/com/qdischarge/clinicqueue/config/SchemaInitRunner.java

Applies `schema.sql` on every boot, statement-by-statement (splitting on `;`, stripping `--` comments), so one bad/already-applied statement never aborts the rest of the schema setup. This replaced an earlier Flyway-based migration history that had fallen out of sync with the code.

```
package com.qdischarge.clinicqueue.config;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.core.annotation.Order;
import org.springframework.core.io.ClassPathResource;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Component;
import org.springframework.util.StreamUtils;

import java.nio.charset.StandardCharsets;

/**
 * Applies src/main/resources/schema.sql on every boot. Executes each SQL statement
 * individually so that one minor error (e.g. index syntax or missing column) does not
 * abort the remaining table creations.
 */
@Component
@RequiredArgsConstructor
@Slf4j
@Order(1)
public class SchemaInitRunner implements ApplicationRunner {

    private final JdbcTemplate jdbcTemplate;

    @Override
    public void run(ApplicationArguments args) {
        try {
            log.info("🚀 Running database schema initialization...");
            ClassPathResource resource = new ClassPathResource("schema.sql");
            String script = StreamUtils.copyToString(resource.getInputStream(), StandardCharsets.UTF_8);

            // Strip comment-only lines and split by semicolon
            String[] statements = script.split(";");
            int executed = 0;
            int failed = 0;

            for (String statement : statements) {
                // Strip inline -- comments line by line, then re-join and trim
                String cleaned = java.util.Arrays.stream(statement.split("\n"))
                    .map(line -> {
                        int idx = line.indexOf("--");
                        return idx >= 0 ? line.substring(0, idx) : line;
                    })
                    .collect(java.util.stream.Collectors.joining("\n"))
                    .trim();

                // Ignore empty statements
                if (cleaned.isEmpty()) {
                    continue;
                }

                try {
                    jdbcTemplate.execute(cleaned);
                    executed++;
                } catch (Exception e) {
                    failed++;
                    log.debug("Schema statement skipped/warning: {} -> {}", cleaned.length() > 60 ? cleaned.substring(0, 60) + "..." : cleaned,
                        e.getMessage());
                }
            }
        }
    }
}
```

```

        log.info("✅ Database schema initialization completed: {} statements executed ({} skipped/existing).", executed, failed);
    } catch (Exception e) {
        log.error("⚠️ Database schema initialization error: {}", e.getMessage(), e);
    }
}
}

```

WebConfig.java

src/main/java/com/qdischarge/clinicqueue/config/WebConfig.java

Serves a built single-page frontend (if bundled into `src/main/resources/static`) with a client-side-routing fallback to `index.html` — mirrors what the original Node backend did with `express.static` plus a catch-all. `/api/**` and `/webhook/**` are handled by `@RestController` beans first, so they're never swallowed by this fallback.

```

package com.qdischarge.clinicqueue.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.core.io.ClassPathResource;
import org.springframework.core.io.Resource;
import org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
import org.springframework.web.servlet.resource.PathResourceResolver;

import java.io.IOException;

/**
 * Mirrors what the original index.js did with
 * express.static(frontend/dist) + a catch-all fallback: serve a built SPA
 * (if any) from src/main/resources/static, falling back to index.html for
 * client-side routes. CORS is configured centrally in SecurityConfig.
 *
 * /api/** and /webhook/** are handled by @RestController beans, which Spring's
 * RequestMappingHandlerMapping resolves before this resource handler, so they
 * are never swallowed by the SPA fallback below.
 */
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        registry.addResourceHandler("/**")
            .addResourceLocations("classpath:/static/")
            .resourceChain(true)
            .addResolver(new PathResourceResolver() {
                @Override
                protected Resource getResource(String resourcePath, Resource location) throws IOException {
                    Resource requested = location.createRelative(resourcePath);
                    if (requested.exists() && requested.isReadable()) {
                        return requested;
                    }
                    Resource index = new ClassPathResource("/static/index.html");
                    return index.exists() ? index : null;
                }
            });
    }
}

```

6. Package: controller

Every HTTP entry point into the system. Controllers are intentionally thin — validation via Bean Validation annotations on request DTOs, a try/catch translating service-layer exceptions to the right HTTP status, and a consistent `{"success": ..., "data"/"message": ...}` response envelope built by small private helpers in each class.

CounterController.java

src/main/java/com/qdischarge/clinicqueue/controller/CounterController.java

The multi-counter package's live board (public read) plus the two admin actions that free a counter — complete or miss — each of which pulls the next eligible waiting token into that counter, look-ahead style. Every endpoint is scoped to one (hospital, category) department -- see HospitalDepartmentService -- since each department runs its own counters; only relevant once that department's `active_counters > 1`, with a single counter this is equivalent to (and layers on top of) the plain PUT `/api/queue/{id}` flow.

```
package com.qdischarge.clinicqueue.controller;

import com.qdischarge.clinicqueue.catalog.MedicalCategory;
import com.qdischarge.clinicqueue.service.CounterAssignmentService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.LinkedHashMap;
import java.util.Map;

/**
 * Multi-counter package (Advanced): the live counter board (GET, for the
 * display board / admin dashboard, public read) and the two admin actions
 * that free a counter -- complete or miss -- each of which pulls the next
 * eligible waiting token into that counter, look-ahead style. Every
 * endpoint is scoped to one (hospital, category) department -- see
 * HospitalDepartmentService -- since each department runs its own counters;
 * only relevant once that department's active_counters > 1, with a single
 * counter this is equivalent to (and layers on top of) the plain
 * PUT /api/queue/{id} flow.
 */
@RestController
@RequestMapping("/api/counters")
@RequiredArgsConstructor
public class CounterController {

    private final CounterAssignmentService counterAssignmentService;

    @GetMapping
    public ResponseEntity<Map<String, Object>> board(@RequestParam int hospitalId, @RequestParam String category) {
        String canonical = canonicalOrNull(category);
        if (canonical == null) {
            return ResponseEntity.badRequest().body(msg("Unknown category: \"" + category + "\""));
        }
        return ResponseEntity.ok(ok(counterAssignmentService.getBoard(hospitalId, canonical)));
    }

    @PostMapping("/{counterId}/complete")
    public ResponseEntity<Map<String, Object>> complete(@PathVariable int counterId,
                                                       @RequestParam int hospitalId, @RequestParam String category) {
        String canonical = canonicalOrNull(category);
        if (canonical == null) {
            return ResponseEntity.badRequest().body(msg("Unknown category: \"" + category + "\""));
        }
        return ResponseEntity.ok(ok(counterAssignmentService.completeAtCounter(hospitalId, canonical, counterId)));
    }

    @PostMapping("/{counterId}/miss")
    public ResponseEntity<Map<String, Object>> miss(@PathVariable int counterId,
                                                    @RequestParam int hospitalId, @RequestParam String category) {
        String canonical = canonicalOrNull(category);
        if (canonical == null) {
            return ResponseEntity.badRequest().body(msg("Unknown category: \"" + category + "\""));
        }
        return ResponseEntity.ok(ok(counterAssignmentService.missAtCounter(hospitalId, canonical, counterId)));
    }

    private String canonicalOrNull(String category) {
        return MedicalCategory.canonicalize(category);
    }

    private Map<String, Object> ok(Object data) {
        Map<String, Object> m = new LinkedHashMap<>();
        m.put("success", true);
    }
}
```

```
        m.put("data", data);
        return m;
    }

    private Map<String, Object> msg(String message) {
        Map<String, Object> m = new LinkedHashMap<>();
        m.put("success", false);
        m.put("message", message);
        return m;
    }
}
```

DoctorController.java

src/main/java/com/qdischarge/clinicqueue/controller/DoctorController.java

Doctor accounts (phone+OTP, same identity model as patients), joining a hospital via its join code, and the QR/code-based patient-access-request flow: a doctor generates a request here, a patient's device claims it, and from then on the doctor can see that patient's uploaded documents grouped by (name, age).

```
package com.qdischarge.clinicqueue.controller;

import com.qdischarge.clinicqueue.dto.AccessRequestDto;
import com.qdischarge.clinicqueue.dto.DoctorDto;
import com.qdischarge.clinicqueue.dto.DoctorLoginRequest;
import com.qdischarge.clinicqueue.dto.JoinHospitalRequest;
import com.qdischarge.clinicqueue.dto.PatientDocumentDto;
import com.qdischarge.clinicqueue.dto.RegisterDoctorRequest;
import com.qdischarge.clinicqueue.security.CurrentUser;
import com.qdischarge.clinicqueue.service.AccessService;
import com.qdischarge.clinicqueue.service.DoctorService;
import com.qdischarge.clinicqueue.service.PatientDocumentService;
import com.qdischarge.clinicqueue.service.QrCodeService;
import com.qdischarge.clinicqueue.service.TimeSlotService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/**
 * Doctor accounts (phone+OTP, same identity model as patients -- see
 * DoctorService), linking to a hospital via its join code, and the
 * QR/code-based access-request flow: a doctor generates a request here, a
 * patient's device claims it (POST /api/patients/access/{code}/accept), and
 * from then on this doctor can see that patient's uploaded documents,
 * grouped by (name, age) -- see PatientDocumentService#groupByIdentity.
 */
@RestController
@RequestMapping("/api/doctors")
@RequiredArgsConstructor
public class DoctorController {

    private final DoctorService doctorService;
    private final AccessService accessService;
    private final PatientDocumentService patientDocumentService;
    private final QrCodeService qrCodeService;
    private final TimeSlotService timeSlotService;
    private final CurrentUser currentUser;

    @PostMapping("/register")
    public ResponseEntity<Map<String, Object>> register(@Valid @RequestBody RegisterDoctorRequest request) {
        return authResponse(doctorService.register(request.name(), request.phone()));
    }

    @PostMapping("/login")
    public ResponseEntity<Map<String, Object>> login(@Valid @RequestBody DoctorLoginRequest request) {
        return authResponse(doctorService.login(request.phone()));
    }

    private ResponseEntity<Map<String, Object>> authResponse(DoctorService.DoctorAuthResult result) {
        Map<String, Object> body = new LinkedHashMap<>();
        body.put("success", result.success());
        body.put("message", result.message());
        if (result.success()) {
            body.put("token", result.token());
            body.put("data", result.doctor());
        }
        return ResponseEntity.status(result.success() ? 200 : 400).body(body);
    }
}
```

```

}

@PostMapping("/join-hospital")
public ResponseEntity<Map<String, Object>> joinHospital(@Valid @RequestBody JoinHospitalRequest request) {
    try {
        DoctorDto doctor = doctorService.joinHospital(currentUser.requireDoctorId(), request.hospitalCode());
        return ResponseEntity.ok(ok(doctor));
    } catch (IllegalArgumentException e) {
        return ResponseEntity.badRequest().body(msg(e.getMessage()));
    }
}

@GetMapping("/me")
public ResponseEntity<Map<String, Object>> me() {
    return ResponseEntity.ok(ok(doctorService.getById(currentUser.requireDoctorId())));
}

/** Generates a fresh 30-minute access-request code; GET .../qr renders it as a scannable PNG. */
@PostMapping("/access-requests")
public ResponseEntity<Map<String, Object>> createAccessRequest() {
    int doctorId = currentUser.requireDoctorId();
    DoctorDto doctor = doctorService.getById(doctorId);
    if (doctor == null || doctor.getHospitalId() == null) {
        return ResponseEntity.badRequest().body(msg("Join a hospital (POST /api/doctors/join-hospital) before requesting patient access."));
    }
    return ResponseEntity.ok(ok(accessService.createRequest(doctorId)));
}

@GetMapping("/access-requests/{code}/qr")
public ResponseEntity<?> accessRequestQr(@PathVariable String code) {
    AccessRequestDto request = accessService.getByCode(code.trim().toUpperCase());
    if (request == null) {
        return ResponseEntity.status(404).body(msg("Access request not found."));
    }
    try {
        byte[] png = qrCodeService.generatePng(request.getCode(), 350, 2, "#0f172a", "#ffffff");
        return ResponseEntity.ok()
            .contentType(MediaType.IMAGE_PNG)
            .header(HttpHeaders.CACHE_CONTROL, "no-store")
            .body(png);
    } catch (Exception e) {
        return ResponseEntity.status(500).body(err(e));
    }
}

/** The calling doctor's own rostered OPD time slots, soonest first. */
@GetMapping("/me/time-slots")
public ResponseEntity<Map<String, Object>> myTimeSlots() {
    return ResponseEntity.ok(ok(timeSlotService.listForDoctor(currentUser.requireDoctorId())));
}

/** Every patient currently granting this doctor access, grouped by (name, age) with their documents. */
@GetMapping("/patients")
public ResponseEntity<Map<String, Object>> patients() {
    List<String> phones = accessService.listActivePatientPhonesForDoctor(currentUser.requireDoctorId());
    return ResponseEntity.ok(ok(patientDocumentService.groupByIdentity(phones)));
}

/** Download one document's image -- only if this doctor currently has active access to that patient's phone. */
@GetMapping("/patients/documents/{docId}/file")
public ResponseEntity<?> patientDocumentFile(@PathVariable int docId) {
    int doctorId = currentUser.requireDoctorId();
    PatientDocumentDto meta = patientDocumentService.getMetadata(docId);
    if (meta == null) {
        return ResponseEntity.status(404).body(msg("Document not found."));
    }
    List<String> allowedPhones = accessService.listActivePatientPhonesForDoctor(doctorId);
    if (!allowedPhones.contains(meta.getPatientPhone())) {
        return ResponseEntity.status(403).body(msg("You don't currently have access to this patient's records."));
    }
    PatientDocumentService.FileContent file = patientDocumentService.getFile(docId);
    return ResponseEntity.ok()
        .contentType(MediaType.parseMediaType(file.contentType()))
        .header(HttpHeaders.CONTENT_DISPOSITION, "inline; filename=\"" + file.fileName() + "\"");
}

```

```

        .body(file.data());
    }

    /** Upload a prescription or medical report for a patient (if doctor currently has active access). */
    @PostMapping(value = "/patients/documents/upload", consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
    public ResponseEntity<Map<String, Object>> uploadPatientDocument(
        @RequestParam("patientPhone") String patientPhone,
        @RequestParam("patientName") String patientName,
        @RequestParam(name = "patientAge", required = false) Integer patientAge,
        @RequestParam("docType") String docType,
        @RequestParam("file") org.springframework.web.multipart.MultipartFile file) {
        int doctorId = currentUser.requireDoctorId();
        List<String> allowedPhones = accessService.listActivePatientPhonesForDoctor(doctorId);
        if (!allowedPhones.contains(patientPhone)) {
            return ResponseEntity.status(403).body(msg("You don't currently have active access to this patient's records."));
        }
        try {
            DoctorDto doctor = doctorService.getById(doctorId);
            PatientDocumentDto doc = patientDocumentService.upload(
                patientPhone, patientName, patientAge, docType,
                doctor != null ? doctor.getHospitalId() : null, doctorId, file);
            return ResponseEntity.ok(ok(doc));
        } catch (Exception e) {
            return ResponseEntity.badRequest().body(msg(e.getMessage()));
        }
    }

    private Map<String, Object> ok(Object data) {
        Map<String, Object> m = new LinkedHashMap<>();
        m.put("success", true);
        m.put("data", data);
        return m;
    }

    private Map<String, Object> msg(String message) {
        Map<String, Object> m = new LinkedHashMap<>();
        m.put("success", false);
        m.put("message", message);
        return m;
    }

    private Map<String, Object> err(Exception e) {
        Map<String, Object> m = new LinkedHashMap<>();
        m.put("success", false);
        m.put("error", e.getMessage());
        return m;
    }
}

```

HospitalController.java

src/main/java/com/qdischarge/clinicqueue/controller/HospitalController.java

The hospital directory: location, OPD hours, active counter count, and profile. Reads are public (patients need this to pick a hospital); creating/relocating a hospital, reconfiguring a department's counter count, or reading/rotating a doctor join code, are admin-only.

```
package com.qdischarge.clinicqueue.controller;

import com.qdischarge.clinicqueue.catalog.MedicalCategory;
import com.qdischarge.clinicqueue.dto.AssignDoctorLocationRequest;
import com.qdischarge.clinicqueue.dto.CreateHospitalRequest;
import com.qdischarge.clinicqueue.dto.CreateTimeSlotRequest;
import com.qdischarge.clinicqueue.dto.DoctorDto;
import com.qdischarge.clinicqueue.dto.HospitalDepartmentDto;
import com.qdischarge.clinicqueue.dto.HospitalDto;
import com.qdischarge.clinicqueue.dto.SetLocationRequest;
import com.qdischarge.clinicqueue.dto.TimeSlotDto;
import com.qdischarge.clinicqueue.dto.UpdateDepartmentCountersRequest;
import com.qdischarge.clinicqueue.service.DoctorService;
import com.qdischarge.clinicqueue.service.HospitalDepartmentService;
import com.qdischarge.clinicqueue.service.HospitalService;
import com.qdischarge.clinicqueue.service.QueueManagerService;
import com.qdischarge.clinicqueue.service.TimeSlotService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.time.LocalDate;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/**
 * Hospital directory: location (DIGIPIN-based), OPD hours, active counter
 * count, and profile (ownership/year established/accreditation/gender-
 * specific/offered categories). Reads are public (patients need this to
 * pick/see a hospital); creating/relocating a hospital, or reconfiguring a
 * department's counter count, is admin-only (see SecurityConfig).
 */
@RestController
@RequestMapping("/api/hospitals")
@RequiredArgsConstructor
public class HospitalController {

    private final HospitalService hospitalService;
    private final HospitalDepartmentService hospitalDepartmentService;
    private final DoctorService doctorService;
    private final TimeSlotService timeSlotService;

    @GetMapping
    public ResponseEntity<Map<String, Object>> list() {
        List<HospitalDto> hospitals = hospitalService.list();
        return ResponseEntity.ok(ok(hospitals));
    }

    /** The fixed picklist every hospital's "categories" and every token's "category" is validated against. */
    @GetMapping("/categories")
    public ResponseEntity<Map<String, Object>> categories() {
        return ResponseEntity.ok(ok(MedicalCategory.ALL));
    }

    /**
     * Public: hospitals nearest a lat/lon, nearest-first -- the web twin of the
     * WhatsApp bot's "share your location" search (see WebhookController /
     * QueueManagerService#searchAndOfferHospitals), for the "just share your
     * location" web link sent when a patient's WhatsApp app can't reliably
     * deliver a live-location share. category/gender are optional (omit to
     * browse every department); offset/limit page through results 20 at a
     * time, same as the bot -- call again with offset = nextOffset for "load more".
     */
}
```

```

@GetMapping("/nearby")
public ResponseEntity<Map<String, Object>> nearby(
    @RequestParam double lat,
    @RequestParam double lon,
    @RequestParam(required = false) String category,
    @RequestParam(required = false) String gender,
    @RequestParam(defaultValue = "0") int offset,
    @RequestParam(defaultValue = "20") int limit) {
    String canonicalCategory = null;
    if (category != null && !category.isBlank()) {
        canonicalCategory = MedicalCategory.canonicalize(category);
        if (canonicalCategory == null) {
            return ResponseEntity.badRequest().body(msg("Unknown category: \"" + category + "\""));
        }
    }
    int safeOffset = Math.max(0, offset);
    int safeLimit = Math.max(1, Math.min(limit, QueueManagerService.HOSPITAL_PAGE_SIZE));

    HospitalService.HospitalSearchPage page = hospitalService.searchHospitals(
        canonicalCategory, (gender == null || gender.isBlank()) ? null : gender, lat, lon, safeOffset, safeLimit);

    Map<String, Object> data = new LinkedHashMap<>();
    data.put("results", page.results());
    data.put("hasMore", page.hasMore());
    data.put("nextOffset", safeOffset + page.results().size());
    return ResponseEntity.ok(ok(data));
}

@GetMapping("/{uriSlug}")
public ResponseEntity<Map<String, Object>> getBySlug(@PathVariable String uriSlug) {
    HospitalDto hospital = hospitalService.getBySlug(uriSlug);
    if (hospital == null) {
        return ResponseEntity.status(404).body(msg("Hospital not found."));
    }
    return ResponseEntity.ok(ok(hospital));
}

@PostMapping
public ResponseEntity<Map<String, Object>> create(@Valid @RequestBody CreateHospitalRequest request) {
    try {
        return ResponseEntity.ok(ok(hospitalService.create(request)));
    } catch (IllegalArgumentException e) {
        return ResponseEntity.badRequest().body(msg(e.getMessage()));
    }
}

/** Admin-only: the code to hand a doctor so they can self-link their account to this hospital (POST /api/doctors/join-hospital). */
@GetMapping("/{uriSlug}/doctor-join-code")
public ResponseEntity<Map<String, Object>> getDoctorJoinCode(@PathVariable String uriSlug) {
    String code = hospitalService.getDoctorJoinCode(uriSlug);
    if (code == null) {
        return ResponseEntity.status(404).body(msg("Hospital not found."));
    }
    return ResponseEntity.ok(ok(Map.of("doctorJoinCode", code)));
}

/** Admin-only: rotate the join code (e.g. it leaked). Existing doctor links are unaffected. */
@PostMapping("/{uriSlug}/doctor-join-code/regenerate")
public ResponseEntity<Map<String, Object>> regenerateDoctorJoinCode(@PathVariable String uriSlug) {
    String code = hospitalService.regenerateDoctorJoinCode(uriSlug);
    if (code == null) {
        return ResponseEntity.status(404).body(msg("Hospital not found."));
    }
    return ResponseEntity.ok(ok(Map.of("doctorJoinCode", code)));
}

/** Public: which departments this hospital has configured and how many counters each has (defaults to 1 if never set). */
@GetMapping("/{uriSlug}/departments")
public ResponseEntity<Map<String, Object>> departments(@PathVariable String uriSlug) {
    HospitalDto hospital = hospitalService.getBySlug(uriSlug);
    if (hospital == null) {
        return ResponseEntity.status(404).body(msg("Hospital not found."));
    }
    return ResponseEntity.ok(ok(hospitalDepartmentService.list(hospital.getId())));
}

```

```

/** Admin-only: how many counters staff this hospital's (category) department. */
@PutMapping("/{uriSlug}/departments/{category}/counters")
public ResponseEntity<Map<String, Object>> updateDepartmentCounters(@PathVariable String uriSlug, @PathVariable String category,
                                                                    @Valid @RequestBody UpdateDepartmentCountersRequest request) {

    HospitalDto hospital = hospitalService.getBySlug(uriSlug);
    if (hospital == null) {
        return ResponseEntity.status(404).body(msg("Hospital not found."));
    }
    String canonical = MedicalCategory.canonicalize(category);
    if (canonical == null) {
        return ResponseEntity.badRequest().body(msg("Unknown category: \"" + category + "\"."));
    }
    HospitalDepartmentDto updated = hospitalDepartmentService.updateCounters(hospital.getId(), canonical, request.activeCounters());
    return ResponseEntity.ok(ok(updated));
}

/** Admin-only: assigns one of this hospital's doctors to a counter/department location -- see AssignDoctorLocationRequest. */
@PutMapping("/{uriSlug}/doctors/{doctorId}/location")
public ResponseEntity<Map<String, Object>> assignDoctorLocation(@PathVariable String uriSlug, @PathVariable int doctorId,
                                                                @Valid @RequestBody AssignDoctorLocationRequest request) {

    HospitalDto hospital = hospitalService.getBySlug(uriSlug);
    if (hospital == null) {
        return ResponseEntity.status(404).body(msg("Hospital not found."));
    }
    String canonical = MedicalCategory.canonicalize(request.category());
    if (canonical == null) {
        return ResponseEntity.badRequest().body(msg("Unknown category: \"" + request.category() + "\"."));
    }
    try {
        DoctorDto doctor = doctorService.assignLocation(hospital.getId(), doctorId, request.counterId(), canonical);
        return ResponseEntity.ok(ok(doctor));
    } catch (IllegalArgumentException e) {
        return ResponseEntity.badRequest().body(msg(e.getMessage()));
    }
}

/** Public: this hospital's OPD time slots, optionally filtered to one day (?date=yyyy-MM-dd), soonest first. */
@GetMapping("/{uriSlug}/time-slots")
public ResponseEntity<Map<String, Object>> timeSlots(@PathVariable String uriSlug,
                                                    @RequestParam(required = false) String date) {

    HospitalDto hospital = hospitalService.getBySlug(uriSlug);
    if (hospital == null) {
        return ResponseEntity.status(404).body(msg("Hospital not found."));
    }
    LocalDate parsed;
    try {
        parsed = date == null ? null : LocalDate.parse(date);
    } catch (Exception e) {
        return ResponseEntity.badRequest().body(msg("date must be yyyy-MM-dd."));
    }
    return ResponseEntity.ok(ok(timeSlotService.listForHospital(hospital.getId(), parsed)));
}

/** Admin-only: opens one OPD time window on one day and rosters a list of doctors onto it in the same call. */
@PostMapping("/{uriSlug}/time-slots")
public ResponseEntity<Map<String, Object>> createTimeSlot(@PathVariable String uriSlug,
                                                         @Valid @RequestBody CreateTimeSlotRequest request) {

    HospitalDto hospital = hospitalService.getBySlug(uriSlug);
    if (hospital == null) {
        return ResponseEntity.status(404).body(msg("Hospital not found."));
    }
    String canonicalCategory = null;
    if (request.category() != null && !request.category().isBlank()) {
        canonicalCategory = MedicalCategory.canonicalize(request.category());
        if (canonicalCategory == null) {
            return ResponseEntity.badRequest().body(msg("Unknown category: \"" + request.category() + "\"."));
        }
    }
    try {
        CreateTimeSlotRequest canonicalRequest = new CreateTimeSlotRequest(
            request.date(), request.startTime(), request.endTime(), canonicalCategory, request.doctorIds());
        TimeSlotDto slot = timeSlotService.create(hospital.getId(), canonicalRequest);
        return ResponseEntity.ok(ok(slot));
    } catch (IllegalArgumentException e) {

```

```
        return ResponseEntity.badRequest().body(msg(e.getMessage()));
    }
}

@PutMapping("/{uriSlug}/location")
public ResponseEntity<Map<String, Object>> updateLocation(@PathVariable String uriSlug,
                                                         @Valid @RequestBody SetLocationRequest request) {
    try {
        HospitalDto updated = hospitalService.updateLocation(uriSlug, request);
        if (updated == null) {
            return ResponseEntity.status(404).body(msg("Hospital not found."));
        }
        return ResponseEntity.ok(ok(updated));
    } catch (IllegalArgumentException e) {
        return ResponseEntity.badRequest().body(msg(e.getMessage()));
    }
}

private Map<String, Object> ok(Object data) {
    Map<String, Object> m = new LinkedHashMap<>();
    m.put("success", true);
    m.put("data", data);
    return m;
}

private Map<String, Object> msg(String message) {
    Map<String, Object> m = new LinkedHashMap<>();
    m.put("success", false);
    m.put("message", message);
    return m;
}
}
```

LocationController.java

src/main/java/com/qdischarge/clinicqueue/controller/LocationController.java

Stateless DIGIPIN encode/decode utility endpoints — no token/hospital side effects, just wraps `geo.DigipinService`.

```
package com.qdischarge.clinicqueue.controller;

import com.qdischarge.clinicqueue.geo.DigipinService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.LinkedHashMap;
import java.util.Map;

/** Stateless DIGIPIN encode/decode utility -- no token/hospital side effects. */
@RestController
@RequestMapping("/api/location/digipin")
@RequiredArgsConstructor
public class LocationController {

    private final DigipinService digipinService;

    public record EncodeRequest(double latitude, double longitude) {
    }

    public record DecodeRequest(String digipin) {
    }

    @PostMapping("/encode")
    public ResponseEntity<Map<String, Object>> encode(@RequestBody EncodeRequest request) {
        try {
            String digipin = digipinService.encode(request.latitude(), request.longitude());
            Map<String, Object> data = new LinkedHashMap<>();
            data.put("digipin", digipin);
            data.put("formattedDigipin", digipinService.format(digipin));
            return ResponseEntity.ok(ok(data));
        } catch (IllegalArgumentException e) {
            return ResponseEntity.badRequest().body(msg(e.getMessage()));
        }
    }

    @PostMapping("/decode")
    public ResponseEntity<Map<String, Object>> decode(@RequestBody DecodeRequest request) {
        try {
            DigipinService.LatLon latLon = digipinService.decode(request.digipin());
            Map<String, Object> data = new LinkedHashMap<>();
            data.put("latitude", latLon.lat());
            data.put("longitude", latLon.lon());
            return ResponseEntity.ok(ok(data));
        } catch (IllegalArgumentException e) {
            return ResponseEntity.badRequest().body(msg(e.getMessage()));
        }
    }

    private Map<String, Object> ok(Object data) {
        Map<String, Object> m = new LinkedHashMap<>();
        m.put("success", true);
        m.put("data", data);
        return m;
    }

    private Map<String, Object> msg(String message) {
        Map<String, Object> m = new LinkedHashMap<>();
        m.put("success", false);
        m.put("message", message);
        return m;
    }
}
```

OtpController.java

src/main/java/com/qdischarge/clinicqueue/controller/OtpController.java

Public, rate-limited phone-number OTP send/verify endpoints. A successful `/verify` is patient login/registration — it issues a `ROLE_PATIENT` JWT keyed on the phone number, with no separate password ever involved.

```
package com.qdischarge.clinicqueue.controller;

import com.qdischarge.clinicqueue.dto.OtpSendRequest;
import com.qdischarge.clinicqueue.dto.OtpVerifyRequest;
import com.qdischarge.clinicqueue.security.JwtService;
import com.qdischarge.clinicqueue.service.OtpService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.LinkedHashMap;
import java.util.Map;

/**
 * Public, rate-limited (see RateLimitFilter) phone-number OTP endpoints.
 * A verified phone stays "recently verified" for app.otp-ttl-minutes, which
 * QueueManagerService checks when app.otp-required-for-registration=true.
 *
 * A successful /verify also issues a ROLE_PATIENT JWT (subject = the phone
 * number) -- this *is* patient login/registration for the patient-facing
 * self-service endpoints (GET/POST /api/patients/**): there's no separate
 * password, just "prove you own this phone number right now."
 */
@RestController
@RequestMapping("/api/auth/otp")
@RequiredArgsConstructor
public class OtpController {

    private final OtpService otpService;
    private final JwtService jwtService;

    @PostMapping("/send")
    public ResponseEntity<Map<String, Object>> send(@Valid @RequestBody OtpSendRequest request) {
        OtpService.OtpResult result = otpService.sendOtp(request.phone());
        return ResponseEntity.status(result.success() ? 200 : 502).body(body(result.success(), result.message()));
    }

    @PostMapping("/verify")
    public ResponseEntity<Map<String, Object>> verify(@Valid @RequestBody OtpVerifyRequest request) {
        OtpService.OtpResult result = otpService.verifyOtp(request.phone(), request.code());
        Map<String, Object> body = body(result.success(), result.message());
        if (result.success()) {
            body.put("token", jwtService.generatePatientToken(request.phone()));
        }
        return ResponseEntity.status(result.success() ? 200 : 400).body(body);
    }

    private Map<String, Object> body(boolean success, String message) {
        Map<String, Object> m = new LinkedHashMap<>();
        m.put("success", success);
        m.put("message", message);
        return m;
    }
}
```

PatientController.java

src/main/java/com/qdischarge/clinicqueue/controller/PatientController.java

Patient self-service: booking history, document upload/list/download, and the access-grant accept/list/history/revoke actions. Every action here operates on the caller's own phone number taken from their JWT (`CurrentUser`) — there's no "which patient" parameter anywhere.

```
package com.qdischarge.clinicqueue.controller;

import com.qdischarge.clinicqueue.dto.AccessGrantDto;
import com.qdischarge.clinicqueue.dto.PatientDocumentDto;
import com.qdischarge.clinicqueue.security.CurrentUser;
import com.qdischarge.clinicqueue.service.AccessService;
import com.qdischarge.clinicqueue.service.PatientDocumentService;
import com.qdischarge.clinicqueue.service.QueueManagerService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;

import java.io.IOException;
import java.util.LinkedHashMap;
import java.util.Map;

/**
 * Patient self-service: everything here acts on the caller's own phone
 * number, taken from their JWT (see CurrentUser) -- there's no separate
 * "which patient" parameter anywhere, since login *is* proving you own that
 * phone via OTP (OtpController#verify).
 */
@RestController
@RequestMapping("/api/patients")
@RequiredArgsConstructor
public class PatientController {

    private final QueueManagerService queueManagerService;
    private final PatientDocumentService patientDocumentService;
    private final AccessService accessService;
    private final CurrentUser currentUser;

    /** This phone's token-booking history (see QueueManagerService#archiveToHistory). */
    @GetMapping("/history")
    public ResponseEntity<Map<String, Object>> history() {
        return ResponseEntity.ok(ok(queueManagerService.getPatientHistory(currentUser.requirePatientPhone())));
    }

    @PostMapping(value = "/documents", consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
    public ResponseEntity<Map<String, Object>> uploadDocument(
        @RequestParam MultipartFile file,
        @RequestParam String docType,
        @RequestParam String patientName,
        @RequestParam(required = false) Integer patientAge,
        @RequestParam(required = false) Integer hospitalId) {
        try {
            PatientDocumentDto doc = patientDocumentService.upload(
                currentUser.requirePatientPhone(), patientName, patientAge, docType, hospitalId, null, file);
            return ResponseEntity.ok(ok(doc));
        } catch (IllegalArgumentException e) {
            return ResponseEntity.badRequest().body(msg(e.getMessage()));
        } catch (IOException e) {
            return ResponseEntity.status(500).body(err(e));
        }
    }

    /** Every prescription/report this phone has uploaded (for itself or any family member -- see tokens.age in schema.sql). */
    @GetMapping("/documents")
    public ResponseEntity<Map<String, Object>> documents() {
        return ResponseEntity.ok(ok(patientDocumentService.listForPatient(currentUser.requirePatientPhone())));
    }
}
```

```

@GetMapping("/documents/{id}/file")
public ResponseEntity<?> documentFile(@PathVariable int id) {
    String phone = currentUser.requirePatientPhone();
    PatientDocumentDto meta = patientDocumentService.getMetadata(id);
    if (meta == null || !phone.equals(meta.getPatientPhone())) {
        return ResponseEntity.status(404).body(msg("Document not found."));
    }
    PatientDocumentService.FileContent file = patientDocumentService.getFile(id);
    return ResponseEntity.ok()
        .contentType(MediaType.parseMediaType(file.contentType()))
        .header(HttpHeaders.CONTENT_DISPOSITION, "inline; filename=\"" + file.fileName() + "\"")
        .body(file.data());
}

/** Claims a doctor's access-request code (from their QR) -- grants that doctor access and fires the WhatsApp notice. */
@PostMapping("/access/{code}/accept")
public ResponseEntity<Map<String, Object>> acceptAccess(@PathVariable String code) {
    try {
        AccessGrantDto grant = accessService.claim(code, currentUser.requirePatientPhone());
        return ResponseEntity.ok(ok(grant));
    } catch (IllegalArgumentException | IllegalStateException e) {
        return ResponseEntity.badRequest().body(msg(e.getMessage()));
    }
}

/** Doctors who currently have access -- their name + hospital (see AccessGrantDto). */
@GetMapping("/access")
public ResponseEntity<Map<String, Object>> activeAccess() {
    return ResponseEntity.ok(ok(accessService.listActiveForPatient(currentUser.requirePatientPhone())));
}

/** Every grant ever issued from this phone, active or revoked -- "history of accessed doctors". */
@GetMapping("/access/history")
public ResponseEntity<Map<String, Object>> accessHistory() {
    return ResponseEntity.ok(ok(accessService.listHistoryForPatient(currentUser.requirePatientPhone())));
}

@PostMapping("/access/{grantId}/revoke")
public ResponseEntity<Map<String, Object>> revokeAccess(@PathVariable int grantId) {
    boolean revoked = accessService.revoke(grantId, currentUser.requirePatientPhone());
    if (!revoked) {
        return ResponseEntity.status(404).body(msg("No active access grant found with that id for your phone."));
    }
    return ResponseEntity.ok(ok(Map.of("revoked", true)));
}

private Map<String, Object> ok(Object data) {
    Map<String, Object> m = new LinkedHashMap<>();
    m.put("success", true);
    m.put("data", data);
    return m;
}

private Map<String, Object> msg(String message) {
    Map<String, Object> m = new LinkedHashMap<>();
    m.put("success", false);
    m.put("message", message);
    return m;
}

private Map<String, Object> err(Exception e) {
    Map<String, Object> m = new LinkedHashMap<>();
    m.put("success", false);
    m.put("error", e.getMessage());
    return m;
}
}

```

QueueController.java

src/main/java/com/qdischarge/clinicqueue/controller/QueueController.java

The core queue API — admin login, queue reads (whole-queue, current-serving, position, history, token details), token creation, location updates, the pre-registration closing-time check, no-show push-back, the anomaly-control view, the missed-queue admin panel (search/requeue/reject), QR generation, admin verify-by-scan, and status updates. The largest and most route-dense controller in the app.

```
package com.qdischarge.clinicqueue.controller;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.qdischarge.clinicqueue.dto.CreateTokenRequest;
import com.qdischarge.clinicqueue.dto.CreateTokenResult;
import com.qdischarge.clinicqueue.dto.LoginRequest;
import com.qdischarge.clinicqueue.dto.QueueData;
import com.qdischarge.clinicqueue.dto.SetLocationRequest;
import com.qdischarge.clinicqueue.dto.TokenDto;
import com.qdischarge.clinicqueue.dto.UpdateStatusRequest;
import com.qdischarge.clinicqueue.dto.UpdateStatusResult;
import com.qdischarge.clinicqueue.dto.VerifyRequest;
import com.qdischarge.clinicqueue.geo.GeoDistanceService;
import com.qdischarge.clinicqueue.security.AdminAuthService;
import com.qdischarge.clinicqueue.security.JwtService;
import com.qdischarge.clinicqueue.service.QrCodeService;
import com.qdischarge.clinicqueue.service.QueueManagerService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

/**
 * Java port of backend/routes/queue.js. Same paths and response shapes as
 * the original; admin auth on /verify and PUT /:id is now enforced
 * declaratively by SecurityConfig (JWT + ROLE_ADMIN) instead of a manual
 * per-route header check.
 */
@RestController
@RequestMapping("/api/queue")
@RequiredArgsConstructor
@Slf4j
public class QueueController {

    private final QueueManagerService queueManagerService;
    private final QrCodeService qrCodeService;
    private final AdminAuthService adminAuthService;
    private final JwtService jwtService;
    private final ObjectMapper objectMapper;

    @PostMapping("/login")
    public ResponseEntity<Map<String, Object>> login(@Valid @RequestBody LoginRequest request) {
        if (adminAuthService.authenticate(request.username(), request.password())) {
            Map<String, Object> ok = new LinkedHashMap<>();
            ok.put("success", true);
            ok.put("token", jwtService.generateAdminToken(request.username()));
            return ResponseEntity.ok(ok);
        }
        return ResponseEntity.status(401).body(msg("Invalid username or password."));
    }

    /** hospitalId/category are optional -- omit both for the hospital-wide view, or pass both to see one department's queue. */
    @GetMapping
    public ResponseEntity<Map<String, Object>> getQueue(@RequestParam(required = false) Integer hospitalId,
```

```

        @RequestParam(required = false) String category) {

    try {
        QueueData data = queueManagerService.getQueue(hospitalId, category);
        return ResponseEntity.ok(ok(data));
    } catch (Exception e) {
        return ResponseEntity.status(500).body(err(e));
    }
}

@GetMapping("/current")
public ResponseEntity<Map<String, Object>> current(@RequestParam(required = false) Integer hospitalId,
        @RequestParam(required = false) String category) {

    try {
        return ResponseEntity.ok(ok(queueManagerService.getCurrentServingToken(hospitalId, category)));
    } catch (Exception e) {
        return ResponseEntity.status(500).body(err(e));
    }
}

@GetMapping("/position/{phone}")
public ResponseEntity<Map<String, Object>> position(@PathVariable String phone) {
    try {
        TokenDto data = queueManagerService.getPatientPosition(phone);
        if (data == null) {
            return ResponseEntity.status(404).body(msg("No active token found for this phone number."));
        }
        return ResponseEntity.ok(ok(data));
    } catch (Exception e) {
        return ResponseEntity.status(500).body(err(e));
    }
}

/**
 * Every past completed visit under this phone number (potentially for
 * different patients -- see QueueManagerService#archiveToHistory).
 * Admin-only: unlike /position/{phone}, this can surface more than one
 * patient's name/age history under a shared number.
 */
@GetMapping("/history/{phone}")
public ResponseEntity<Map<String, Object>> history(@PathVariable String phone) {
    try {
        return ResponseEntity.ok(ok(queueManagerService.getPatientHistory(phone)));
    } catch (Exception e) {
        return ResponseEntity.status(500).body(err(e));
    }
}

@GetMapping("/token/{id}")
public ResponseEntity<Map<String, Object>> tokenDetails(@PathVariable String id) {
    try {
        TokenDto data = queueManagerService.getTokenDetails(id);
        if (data == null) {
            return ResponseEntity.status(404).body(msg("Token not found."));
        }
        return ResponseEntity.ok(ok(data));
    } catch (Exception e) {
        return ResponseEntity.status(500).body(err(e));
    }
}

@PostMapping
public ResponseEntity<Map<String, Object>> create(@Valid @RequestBody CreateTokenRequest request) {
    try {
        CreateTokenResult result = queueManagerService.createToken(
            request.name(), request.age(), request.gender(), request.category(), request.phone(),
            request.toLocationOrNull(), request.hospitalId());
        Map<String, Object> resp = new LinkedHashMap<>();
        resp.put("success", true);
        resp.put("alreadyExists", result.alreadyExists());
        resp.put("data", result.data());
        return ResponseEntity.ok(resp);
    } catch (IllegalStateException | IllegalArgumentException e) {
        return ResponseEntity.badRequest().body(msg(e.getMessage()));
    } catch (Exception e) {
        return ResponseEntity.status(500).body(err(e));
    }
}

```

```

    }
}

/** Patient shares their current GPS location or manually enters one (DIGIPIN or lat/lon). */
@PostMapping("/{id}/location")
public ResponseEntity<Map<String, Object>> setLocation(@PathVariable String id,
                                                       @Valid @RequestBody SetLocationRequest request) {
    try {
        TokenDto updated = queueManagerService.updateTokenLocation(Integer.parseInt(id), request);
        if (updated == null) {
            return ResponseEntity.status(404).body(msg("Token not found."));
        }
        return ResponseEntity.ok(ok(updated));
    } catch (IllegalStateException | IllegalArgumentException e) {
        return ResponseEntity.badRequest().body(msg(e.getMessage()));
    } catch (Exception e) {
        return ResponseEntity.status(500).body(err(e));
    }
}

/** Pre-registration check: "Hospital is 15 km away, only 30 minutes remain before OPD closes. Continue?" */
@GetMapping("/closing-time-check")
public ResponseEntity<Map<String, Object>> closingTimeCheck(@RequestParam double lat, @RequestParam double lon) {
    try {
        GeoDistanceService.ArrivalFeasibility feasibility = queueManagerService.checkClosingTimeFeasibility(lat, lon);
        return ResponseEntity.ok(ok(feasibility));
    } catch (IllegalStateException e) {
        return ResponseEntity.badRequest().body(msg(e.getMessage()));
    } catch (Exception e) {
        return ResponseEntity.status(500).body(err(e));
    }
}

/**
 * Admin clicks "not come yet" on a called-but-absent waiting patient:
 * instead of marking them missed outright, push them back within their
 * own queue by an exponentially growing number of positions (1, 2, 4,
 * 8, 16, ... then straight to the back) -- see
 * QueueManagerService#pushBackNoShow.
 */
@PostMapping("/{id}/no-show")
public ResponseEntity<Map<String, Object>> noShow(@PathVariable int id) {
    try {
        TokenDto token = queueManagerService.pushBackNoShow(id);
        return ResponseEntity.ok(ok(token));
    } catch (IllegalStateException e) {
        return ResponseEntity.badRequest().body(msg(e.getMessage()));
    } catch (Exception e) {
        return ResponseEntity.status(500).body(err(e));
    }
}

/**
 * Patients already notified+called and still inside their travel-time
 * grace window (see QueueManagerService#runTreatmentTimingTick) --
 * resolves itself automatically (push-back or missed) once the window
 * elapses, this is just a read-only view for the admin dashboard.
 */
@GetMapping("/anomaly-control")
public ResponseEntity<Map<String, Object>> anomalyControlQueue(@RequestParam(required = false) Integer hospitalId,
                                                             @RequestParam(required = false) String category) {
    try {
        return ResponseEntity.ok(ok(queueManagerService.getAnomalyControlQueue(hospitalId, category)));
    } catch (Exception e) {
        return ResponseEntity.status(500).body(err(e));
    }
}

// ---- Missed queue: admin search / requeue-to-front / reject ----

@GetMapping("/missed")
public ResponseEntity<Map<String, Object>> missedQueue(@RequestParam(required = false) Integer hospitalId,
                                                       @RequestParam(required = false) String category) {
    try {
        return ResponseEntity.ok(ok(queueManagerService.getMissedQueue(hospitalId, category)));
    }
}

```

```

    } catch (Exception e) {
        return ResponseEntity.status(500).body(err(e));
    }
}

@GetMapping("/missed/search")
public ResponseEntity<Map<String, Object>> searchMissedQueue(@RequestParam(required = false) String query,
                                                            @RequestParam(required = false) Integer hospitalId,
                                                            @RequestParam(required = false) String category) {
    try {
        return ResponseEntity.ok(ok(queueManagerService.searchMissedQueue(query, hospitalId, category)));
    } catch (Exception e) {
        return ResponseEntity.status(500).body(err(e));
    }
}

@PostMapping("/missed/{id}/requeue")
public ResponseEntity<Map<String, Object>> requeueMissed(@PathVariable int id) {
    try {
        TokenDto token = queueManagerService.requeueMissedToFront(id);
        if (token == null) {
            return ResponseEntity.status(404).body(msg("Token not found in the missed queue."));
        }
        return ResponseEntity.ok(ok(token));
    } catch (Exception e) {
        return ResponseEntity.status(500).body(err(e));
    }
}

@PostMapping("/missed/{id}/reject")
public ResponseEntity<Map<String, Object>> rejectMissed(@PathVariable int id) {
    try {
        TokenDto token = queueManagerService.rejectMissedToken(id);
        if (token == null) {
            return ResponseEntity.status(404).body(msg("Token not found in the missed queue."));
        }
        return ResponseEntity.ok(ok(token));
    } catch (Exception e) {
        return ResponseEntity.status(500).body(err(e));
    }
}

@GetMapping("/qr/{id}")
public ResponseEntity<?> qr(@PathVariable String id) {
    try {
        int tokenId = Integer.parseInt(id);

        Map<String, Object> payload = new LinkedHashMap<>();
        payload.put("tokenId", tokenId);
        payload.put("type", "CLINIC_QUEUE_TOKEN");
        payload.put("timestamp", System.currentTimeMillis());
        String qrPayload = objectMapper.writeValueAsString(payload);

        byte[] png = qrCodeService.generatePng(qrPayload, 350, 2, "#0f172a", "#ffffff");

        return ResponseEntity.ok()
            .contentType(MediaType.IMAGE_PNG)
            .header(HttpHeaders.CACHE_CONTROL, "public, max-age=86400")
            .body(png);
    } catch (Exception e) {
        return ResponseEntity.status(500).body(err(e));
    }
}

@PostMapping("/verify")
public ResponseEntity<Map<String, Object>> verify(@RequestBody(required = false) VerifyRequest request) {
    Object qrData = request != null ? request.qrData() : null;
    Integer targetId = request != null ? request.tokenId() : null;

    if (targetId == null && qrData != null) {
        try {
            if (qrData instanceof Map<?, ?> mapData) {
                targetId = toInt(mapData.get("tokenId"));
            } else if (qrData instanceof String s) {
                if (s.startsWith("{") {

```

```

        JsonNode parsed = objectMapper.readTree(s);
        JsonNode tid = parsed.has("tokenId") ? parsed.get("tokenId") : parsed.get("id");
        targetId = tid != null ? tid.asInt() : null;
    } else {
        Matcher m = Pattern.compile("\\d+").matcher(s);
        if (m.find()) {
            targetId = Integer.parseInt(m.group());
        }
    }
}
} catch (Exception e) {
    log.error("Error parsing QR data: {}", e.getMessage());
}
}

if (targetId == null) {
    return ResponseEntity.badRequest().body(msg("Invalid or missing QR payload / Token ID."));
}

try {
    TokenDto result = queueManagerService.verifyTokenByAdmin(targetId);
    if (result == null) {
        return ResponseEntity.status(404).body(msg("Token #" + targetId + " not found in database."));
    }

    Map<String, Object> resp = new LinkedHashMap<>();
    resp.put("success", true);
    resp.put("message", "🔥 Token #" + targetId + " (" + result.getName()
        + ") verified successfully! Welcome WhatsApp notification sent to patient.");
    resp.put("data", result);
    return ResponseEntity.ok(resp);
} catch (Exception e) {
    log.error("Error verifying token:", e);
    return ResponseEntity.status(500).body(err(e));
}
}

@PutMapping("/{id}")
public ResponseEntity<Map<String, Object>> updateStatus(@PathVariable String id,
    @Valid @RequestBody UpdateStatusRequest request) {

    List<String> validStatuses = List.of("waiting", "serving", "completed", "missed");
    if (!validStatuses.contains(request.status())) {
        return ResponseEntity.badRequest().body(msg("Invalid status."));
    }

    try {
        UpdateStatusResult result = queueManagerService.updateTokenStatus(id, request.status());
        if (result == null) {
            return ResponseEntity.status(404).body(msg("Token not found."));
        }
        return ResponseEntity.ok(ok(result));
    } catch (Exception e) {
        return ResponseEntity.status(500).body(err(e));
    }
}

// ---- response helpers ----

private Map<String, Object> ok(Object data) {
    Map<String, Object> m = new LinkedHashMap<>();
    m.put("success", true);
    m.put("data", data);
    return m;
}

private Map<String, Object> msg(String message) {
    Map<String, Object> m = new LinkedHashMap<>();
    m.put("success", false);
    m.put("message", message);
    return m;
}

private Map<String, Object> err(Exception e) {
    Map<String, Object> m = new LinkedHashMap<>();
    m.put("success", false);

```

```
        m.put("error", e.getMessage());
        return m;
    }

    private Integer toInt(Object o) {
        if (o == null) return null;
        if (o instanceof Number n) return n.intValue();
        try {
            return Integer.parseInt(o.toString());
        } catch (NumberFormatException e) {
            return null;
        }
    }
}
```

WebhookController.java

src/main/java/com/qdischarge/clinicqueue/controller/WebhookController.java

The WhatsApp webhook: Meta's subscription-verification handshake (GET) and the actual inbound-message handler (POST), supporting both the Meta Cloud API and Evolution API payload shapes. Implements the entire server-side conversation state machine described in the architecture overview — by far the largest controller in the codebase, with private helper methods for every interactive message it can send back.

```
package com.qdischarge.clinicqueue.controller;

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.node.MissingNode;
import com.qdischarge.clinicqueue.bot.BotMessages;
import com.qdischarge.clinicqueue.bot.ConfirmationChoice;
import com.qdischarge.clinicqueue.bot.Gender;
import com.qdischarge.clinicqueue.bot.Intent;
import com.qdischarge.clinicqueue.bot.Lang;
import com.qdischarge.clinicqueue.bot.WaSessionService;
import com.qdischarge.clinicqueue.catalog.MedicalCategory;
import com.qdischarge.clinicqueue.config.AppProperties;
import com.qdischarge.clinicqueue.dto.HospitalDto;
import com.qdischarge.clinicqueue.dto.SetLocationRequest;
import com.qdischarge.clinicqueue.dto.TokenDto;
import com.qdischarge.clinicqueue.dto.WaButton;
import com.qdischarge.clinicqueue.dto.WaListRow;
import com.qdischarge.clinicqueue.dto.WaListSection;
import com.qdischarge.clinicqueue.geo.GeoDistanceService;
import com.qdischarge.clinicqueue.service.AccessService;
import com.qdischarge.clinicqueue.service.HospitalService;
import com.qdischarge.clinicqueue.service.QueueManagerService;
import com.qdischarge.clinicqueue.service.WhatsAppService;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

/**
 * Java port of backend/routes/webhook.js. Handles the Meta Cloud API
 * subscription handshake (GET) and incoming WhatsApp messages (POST), for
 * both the Meta Cloud API and Evolution API payload shapes.
 *
 * Every phone's first message goes through a one-time greeting + language
 * picker (English / Hindi / Marathi, tracked in {@link WaSessionService})
 * before it ever sees the main menu -- see {@link #receive}. On Meta, the
 * picker is real tap buttons (btn_lang_en/hi/mr); on Evolution, which has no
 * button primitive, it's a plain-text prompt asking the patient to type
 * "English" / "Hindi" / "Marathi", matched by {@link Lang#matchInitial}.
 * Once a language is set it applies to every reply, and the menu commands
 * (generate/status/cancel, see {@link Intent}) are recognized typed in any
 * of the three languages regardless of which one is active.
 *
 * "Generate Token" walks a returning-or-new patient through registration
 * (name -> gender -> age, skipped straight to department if this phone's
 * identity is already known -- see QueueManagerService#createRegisteringToken)
 * then a department -> location -> hospital search+select+confirm booking
 * flow: hospitals offering the chosen department, open to the patient's
 * gender, are ranked by distance and shown 5 at a time (a "Show more" row
 * for the next 5) as an interactive list; tapping one shows a confirmation
 * card before the token actually joins that hospital's department queue.
 */
@RestController
@RequestMapping("/webhook")
@RequiredArgsConstructor
```

```

@Slf4j
public class WebhookController {

    private final QueueManagerService queueManagerService;
    private final HospitalService hospitalService;
    private final GeoDistanceService geoDistanceService;
    private final WhatsAppService whatsappService;
    private final AppProperties appProperties;
    private final WaSessionService waSessionService;
    private final BotMessages botMessages;
    private final AccessService accessService;

    /** "revoke <id>" -- deliberately a literal, untranslated English command (see BotMessages#accessGranted) so the WhatsApp notice's
instructions always work regardless of the reply's language. */
    private static final Pattern REVOKE_COMMAND = Pattern.compile("^revoke\\s+(\\d+)$", Pattern.CASE_INSENSITIVE);
    private static final Pattern DIGIPIN_PATTERN = Pattern.compile("[A-Z0-9]{10}$");
    private static final String SHOW_MORE_ROW_ID = "show_more";
    private static final String HOSPITAL_ROW_PREFIX = "hosp_";

    // -----
    // WEBHOOK VERIFICATION (GET) for Meta WhatsApp Cloud API
    // -----
    @GetMapping(value = "/whatsapp", produces = MediaType.TEXT_PLAIN_VALUE)
    public ResponseEntity<String> verify(
        @RequestParam(name = "hub.mode", required = false) String mode,
        @RequestParam(name = "hub.verify_token", required = false) String token,
        @RequestParam(name = "hub.challenge", required = false) String challenge) {

        if (mode != null && token != null) {
            if ("subscribe".equals(mode) && token.equals(appProperties.getWebhookVerifyToken())) {
                log.info("✅ META WEBHOOK VERIFIED!");
                return ResponseEntity.ok(challenge);
            }
        }
        return ResponseEntity.status(403).build();
    }
    return ResponseEntity.badRequest().build();
}

// -----
// WEBHOOK RECEIVER (POST) - handles both Meta API and Evolution API
// -----
@PostMapping(value = "/whatsapp", produces = MediaType.TEXT_PLAIN_VALUE)
public ResponseEntity<String> receive(@RequestBody(required = false) JsonNode requestBody) {
    JsonNode body = requestBody != null ? requestBody : MissingNode.getInstance();
    log.info("📩 INCOMING WEBHOOK PAYLOAD: {}", body);

    String fromPhone;
    String pushName;
    String incomingMessage;
    String buttonId;
    Double incomingLat = null;
    Double incomingLon = null;

    if ("whatsapp_business_account".equals(body.path("object").asText())) {
        JsonNode entry = body.path("entry").path(0);
        JsonNode changes = entry.path("changes").path(0);
        JsonNode value = changes.path("value");
        JsonNode message = value.path("messages").path(0);

        if (message.isMissingNode()) {
            return ResponseEntity.ok("EVENT_RECEIVED");
        }

        String rawFrom = message.path("from").asText("");
        fromPhone = rawFrom.startsWith("+") ? rawFrom : "+" + rawFrom;
        pushName = value.path("contacts").path(0).path("profile").path("name").asText("Patient");

        String tmpMessage = "";
        String tmpButtonId = "";
        String msgType = message.path("type").asText("");
        if ("text".equals(msgType)) {
            tmpMessage = message.path("text").path("body").asText("");
        } else if ("interactive".equals(msgType)) {
            String interactiveType = message.path("interactive").path("type").asText("");
            if ("button_reply".equals(interactiveType)) {

```

```

        tmpButtonId = message.path("interactive").path("button_reply").path("id").asText("");
        tmpMessage = message.path("interactive").path("button_reply").path("title").asText("");
    } else if ("list_reply".equals(interactiveType)) {
        tmpButtonId = message.path("interactive").path("list_reply").path("id").asText("");
        tmpMessage = message.path("interactive").path("list_reply").path("title").asText("");
    }
} else if ("location".equals(msgType)) {
    JsonNode loc = message.path("location");
    if (!loc.isMissingNode()) {
        incomingLat = loc.path("latitude").isMissingNode() ? null : loc.path("latitude").asDouble();
        incomingLon = loc.path("longitude").isMissingNode() ? null : loc.path("longitude").asDouble();
    }
}
incomingMessage = tmpMessage;
buttonId = tmpButtonId;
} else {
    JsonNode data = body.has("data") ? body.path("data") : body;

    JsonNode messageData = data.path("message");
    if (messageData.isMissingNode() || messageData.isNull()) {
        messageData = data.path("messages").path(0).path("message");
    }
    if (messageData.isMissingNode() || messageData.isNull()) {
        // Evolution/Baileys fires live-location pings (every update after the initial
        // share) as a "messages.update" event, which nests the patched content one
        // level deeper under "update" instead of directly under "message" -- without
        // this fallback those pings are silently dropped and a live-shared location
        // never resolves.
        messageData = data.path("update").path("message");
    }
    if (messageData.isMissingNode() || messageData.isNull()) {
        messageData = data.path("messages").path(0).path("update").path("message");
    }
    JsonNode keyData = data.path("key");
    if (keyData.isMissingNode() || keyData.isNull()) {
        keyData = data.path("messages").path(0).path("key");
    }

    boolean isFromMe = keyData.path("fromMe").asBoolean(false)
        || data.path("fromMe").asBoolean(false)
        || body.path("fromMe").asBoolean(false)
        || body.path("data").path("key").path("fromMe").asBoolean(false);

    if (isFromMe) {
        return ResponseEntity.ok("EVENT_RECEIVED");
    }

    String remoteJid = textOrNull(keyData.path("remoteJid"));
    if (isBlank(remoteJid)) {
        remoteJid = textOrNull(body.path("remoteJid"));
    }
    if (remoteJid != null && (remoteJid.endsWith("@g.us") || remoteJid.endsWith("@broadcast") || remoteJid.endsWith("@newsletter"))) {
        return ResponseEntity.ok("EVENT_RECEIVED");
    }
    fromPhone = isBlank(remoteJid) ? "" : remoteJid.split("@")[0];
    if (!fromPhone.isEmpty() && !fromPhone.startsWith("+")) {
        fromPhone = "+" + fromPhone;
    }

    String pn = textOrNull(data.path("pushName"));
    if (isBlank(pn)) {
        pn = textOrNull(body.path("pushName"));
    }
    pushName = isBlank(pn) ? "Patient" : pn;

    String listRowId = textOrNull(messageData.path("listResponseMessage").path("singleSelectReply").path("selectedRowId"));

    incomingMessage = firstNonBlank(
        textOrNull(messageData.path("conversation")),
        textOrNull(messageData.path("extendedTextMessage").path("text")),
        textOrNull(messageData.path("buttonsResponseMessage").path("selectedDisplayText")),
        textOrNull(messageData.path("listResponseMessage").path("title"))
    ).trim();

    buttonId = !isBlank(listRowId) ? listRowId : messageData.path("buttonsResponseMessage").path("selectedButtonId").asText("");
}

```

```

        Double[] coords = extractLocationFromPayload(data, messageData, body);
        incomingLat = coords[0];
        incomingLon = coords[1];
    }

    String cleanMessage = incomingMessage.trim().toLowerCase();

    if (fromPhone.isEmpty()) {
        return ResponseEntity.ok("EVENT_RECEIVED");
    }

    log.info("\n 📡 WEBHOOK MSG from [{}] ({}): \"{}\" (ButtonId: \"{}\")",
            fromPhone, pushName, incomingMessage, buttonId);

    try {
        // STEP 0: Greeting + one-time language picker, before anything else.
        WaSessionService.WaSession session = waSessionService.get(fromPhone);
        if (session == null) {
            waSessionService.createAwaitingLanguage(fromPhone);
            sendLanguagePrompt(fromPhone, true);
            return ResponseEntity.ok("EVENT_RECEIVED");
        }
        if (session.awaitingLanguage()) {
            Lang chosen = Lang.matchInitial(buttonId, cleanMessage);
            if (chosen == null) {
                sendLanguagePrompt(fromPhone, false);
                return ResponseEntity.ok("EVENT_RECEIVED");
            }
            waSessionService.setLanguage(fromPhone, chosen);
            sendWelcomeCard(fromPhone, chosen);
            return ResponseEntity.ok("EVENT_RECEIVED");
        }

        Lang lang = session.language();

        // "revoke <id>" -- a patient's fast, always-available response to the access-granted
        // WhatsApp notice (see AccessService#notifyPatientOfNewAccess) if it wasn't them.
        // Checked before anything else so it can never be swallowed by a registration step.
        Matcher revokeMatch = REVOKE_COMMAND.matcher(cleanMessage);
        if (revokeMatch.matches()) {
            int grantId = Integer.parseInt(revokeMatch.group(1));
            boolean revoked = accessService.revoke(grantId, fromPhone);
            whatsappService.sendWhatsAppMessage(fromPhone,
                    revoked ? botMessages.accessRevoked(lang, grantId) : botMessages.accessRevokeNotFound(lang));
            return ResponseEntity.ok("EVENT_RECEIVED");
        }

        TokenDto activeToken = queueManagerService.getActiveToken(fromPhone);

        // STEP 1: Mid-registration/booking, every reply is form input -- checked before any
        // command/language matching below, so e.g. a numeric age or department number like
        // "1"/"2"/"3" can never be swallowed by the generate/status/cancel shortcuts.
        if (activeToken != null && "awaiting_name".equals(activeToken.getSessionStep())) {
            if (Intent.isReservedWord(cleanMessage)) {
                whatsappService.sendWhatsAppMessage(fromPhone, botMessages.invalidNameReminder(lang));
                return ResponseEntity.ok("EVENT_RECEIVED");
            }

            queueManagerService.captureName(activeToken.getId(), incomingMessage);
            sendGenderPrompt(fromPhone, lang);
            return ResponseEntity.ok("EVENT_RECEIVED");
        }

        if (activeToken != null && "awaiting_gender".equals(activeToken.getSessionStep())) {
            Gender gender = Gender.match(buttonId, cleanMessage);
            if (gender == null) {
                whatsappService.sendWhatsAppMessage(fromPhone, botMessages.invalidGenderReminder(lang));
                return ResponseEntity.ok("EVENT_RECEIVED");
            }

            queueManagerService.captureGender(activeToken.getId(), gender.code());
            whatsappService.sendWhatsAppMessage(fromPhone, botMessages.agePrompt(lang));
            return ResponseEntity.ok("EVENT_RECEIVED");
        }
    }

```

```

if (activeToken != null && "awaiting_age".equals(activeToken.getSessionStep())) {
    Integer age = parseAge(cleanMessage);
    if (age == null) {
        whatsappService.sendWhatsAppMessage(fromPhone, botMessages.invalidAgeReminder(lang));
        return ResponseEntity.ok("EVENT_RECEIVED");
    }

    queueManagerService.captureAge(activeToken.getId(), age);
    whatsappService.sendWhatsAppMessage(fromPhone, botMessages.categoryPrompt(lang));
    return ResponseEntity.ok("EVENT_RECEIVED");
}

if (activeToken != null && "awaiting_category".equals(activeToken.getSessionStep())) {
    String category = MedicalCategory.match(incomingMessage);
    if (category == null) {
        whatsappService.sendWhatsAppMessage(fromPhone, botMessages.invalidCategoryReminder(lang));
        return ResponseEntity.ok("EVENT_RECEIVED");
    }

    queueManagerService.captureCategory(activeToken.getId(), category);
    sendLocationPrompt(fromPhone, lang, category);
    return ResponseEntity.ok("EVENT_RECEIVED");
}

if (activeToken != null && "awaiting_location".equals(activeToken.getSessionStep())) {
    SetLocationRequest location = resolveIncomingLocation(incomingLat, incomingLon, cleanMessage);
    if (location == null) {
        whatsappService.sendWhatsAppMessage(fromPhone, botMessages.invalidLocationReminder(lang));
        return ResponseEntity.ok("EVENT_RECEIVED");
    }

    QueueManagerService.HospitalSearchOutcome outcome;
    try {
        outcome = queueManagerService.searchAndOfferHospitals(activeToken.getId(), location);
    } catch (IllegalArgumentException e) {
        whatsappService.sendWhatsAppMessage(fromPhone, botMessages.invalidLocationReminder(lang));
        return ResponseEntity.ok("EVENT_RECEIVED");
    }
    if (outcome == null) {
        whatsappService.sendWhatsAppMessage(fromPhone, botMessages.invalidLocationReminder(lang));
        return ResponseEntity.ok("EVENT_RECEIVED");
    }
    if (outcome.page().results().isEmpty()) {
        whatsappService.sendWhatsAppMessage(fromPhone, botMessages.noHospitalsFound(lang, activeToken.getCategory()));
        return ResponseEntity.ok("EVENT_RECEIVED");
    }
    sendHospitalResultsList(fromPhone, outcome.draft(), outcome.page(), lang);
    return ResponseEntity.ok("EVENT_RECEIVED");
}

if (activeToken != null && "awaiting_hospital_selection".equals(activeToken.getSessionStep())) {
    int offset = Math.max(0, (activeToken.getSearchOffset() != null ? activeToken.getSearchOffset() :
QueueManagerService.HOSPITAL_PAGE_SIZE) - QueueManagerService.HOSPITAL_PAGE_SIZE);
    HospitalService.HospitalSearchPage page = hospitalService.searchHospitals(
        activeToken.getCategory(), activeToken.getGender(), activeToken.getPatientLat(), activeToken.getPatientLon(), offset,
QueueManagerService.HOSPITAL_PAGE_SIZE);

    if (SHOW_MORE_ROW_ID.equals(buttonId) || isShowMoreCommand(cleanMessage)) {
        QueueManagerService.HospitalSearchOutcome outcome = queueManagerService.showMoreHospitals(activeToken.getId());
        if (outcome == null || outcome.page().results().isEmpty()) {
            whatsappService.sendWhatsAppMessage(fromPhone, botMessages.invalidHospitalSelectionReminder(lang));
            return ResponseEntity.ok("EVENT_RECEIVED");
        }
        sendHospitalResultsList(fromPhone, outcome.draft(), outcome.page(), lang);
        return ResponseEntity.ok("EVENT_RECEIVED");
    }

    if (buttonId != null && buttonId.startsWith(HOSPITAL_ROW_PREFIX)) {
        Integer hospitalId = parseInt(buttonId.substring(HOSPITAL_ROW_PREFIX.length()));
        TokenDto selected = hospitalId == null ? null : queueManagerService.selectHospital(activeToken.getId(), hospitalId);
        if (selected != null) {
            sendConfirmationCard(fromPhone, selected, lang);
            return ResponseEntity.ok("EVENT_RECEIVED");
        }
    }
}

```

```

    }

    String cleanMsg = incomingMessage.trim().toLowerCase();

    // Check for "profile N" or "info N" or "details N"
    if (cleanMsg.startsWith("profile ") || cleanMsg.startsWith("info ") || cleanMsg.startsWith("details ") || cleanMsg.endsWith("
profile")) {
        String numStr = cleanMsg.replaceAll("[^0-9]", "");
        Integer idx = parseInt(numStr);
        if (idx != null && page != null && idx >= 1 && idx <= page.results().size()) {
            HospitalService.HospitalMatch match = page.results().get(idx - 1);
            sendHospitalProfile(fromPhone, match.hospital(), match.distanceKm(), lang);
            return ResponseEntity.ok("EVENT_RECEIVED");
        }
    }

    // Check for bare number choice "1", "2", ... "20"
    Integer selectedIdx = parseInt(cleanMsg);
    if (selectedIdx != null && page != null && selectedIdx >= 1 && selectedIdx <= page.results().size()) {
        HospitalDto chosen = page.results().get(selectedIdx - 1).hospital();
        TokenDto selected = queueManagerService.selectHospital(activeToken.getId(), chosen.getId());
        if (selected != null) {
            sendConfirmationCard(fromPhone, selected, lang);
            return ResponseEntity.ok("EVENT_RECEIVED");
        }
    }

    int totalCount = (page != null && page.results() != null) ? page.results().size() : 20;
    whatsappService.sendWhatsAppMessage(fromPhone, "⚠️ Invalid selection. Please reply with a hospital number (*1* to *" +
totalCount + "*) or type *profile 1* to view hospital details.");
    return ResponseEntity.ok("EVENT_RECEIVED");
}

if (activeToken != null && "awaiting_confirmation".equals(activeToken.getSessionStep())) {
    // Evolution renders "buttons" as a plain numbered list (no tappable primitive), so the
    // patient's choice is matched from typed text -- see ConfirmationChoice.
    ConfirmationChoice choice = ConfirmationChoice.match(buttonId, cleanMessage);

    if (choice == ConfirmationChoice.CONFIRM) {
        try {
            TokenDto booked = queueManagerService.confirmBooking(activeToken.getId());
            sendTokenDashboardCard(fromPhone, booked, lang);
        } catch (IllegalStateException e) {
            // Tell the patient why it actually failed (OPD closing soon, hospital
            // gone, ...) instead of the generic "type 1 or Confirm" reminder --
            // retrying Confirm can never fix these, so looping that text was a
            // dead end. See BotMessages#bookingFailed.
            whatsappService.sendWhatsAppMessage(fromPhone, botMessages.bookingFailed(lang, e.getMessage()));
        }
        return ResponseEntity.ok("EVENT_RECEIVED");
    }

    if (choice == ConfirmationChoice.CHOOSE_AGAIN) {
        QueueManagerService.HospitalSearchOutcome outcome = queueManagerService.restartHospitalSelection(activeToken.getId());
        if (outcome == null || outcome.page().results().isEmpty()) {
            whatsappService.sendWhatsAppMessage(fromPhone, botMessages.invalidHospitalSelectionReminder(lang));
            return ResponseEntity.ok("EVENT_RECEIVED");
        }
        sendHospitalResultsList(fromPhone, outcome.draft(), outcome.page(), lang);
        return ResponseEntity.ok("EVENT_RECEIVED");
    }

    whatsappService.sendWhatsAppMessage(fromPhone, botMessages.invalidConfirmationReminder(lang));
    return ResponseEntity.ok("EVENT_RECEIVED");
}

// Let a patient switch language at any later point by typing/tapping it again.
Lang switchTo = Lang.match(buttonId, cleanMessage);
if (switchTo != null && switchTo != lang) {
    waSessionService.setLanguage(fromPhone, switchTo);
    whatsappService.sendWhatsAppMessage(fromPhone, botMessages.languageSwitched(switchTo));
    sendWelcomeCard(fromPhone, switchTo);
    return ResponseEntity.ok("EVENT_RECEIVED");
}

```

```

Intent intent = Intent.match(buttonId, cleanMessage);

// STEP 2: Menu commands -- generate/status/cancel, typed or tapped, in any supported language
if (intent == Intent.GENERATE_TOKEN) {
    if (activeToken == null || "completed".equals(activeToken.getStatus()) || "missed".equals(activeToken.getStatus())) {
        TokenDto draft = queueManagerService.createRegisteringToken(fromPhone);
        if (draft != null && "awaiting_category".equals(draft.getSessionStep())) {
            // A returning phone whose identity (name/gender/age) is already known --
            // skip straight to picking a department instead of re-asking from scratch.
            whatsappService.sendWhatsAppMessage(fromPhone, botMessages.categoryPrompt(lang));
        } else {
            whatsappService.sendWhatsAppMessage(fromPhone, botMessages.nameRegistrationPrompt(lang));
        }
    } else {
        whatsappService.sendWhatsAppMessage(fromPhone,
            botMessages.alreadyActiveToken(lang, activeToken.displayNumber(), activeToken.getStatus()));
        sendTokenDashboardCard(fromPhone, activeToken, lang);
    }
    return ResponseEntity.ok("EVENT_RECEIVED");
}

if (intent == Intent.CHECK_STATUS) {
    sendStatusCard(fromPhone, activeToken, lang);
    return ResponseEntity.ok("EVENT_RECEIVED");
}

if (intent == Intent.CANCEL_TOKEN) {
    if (activeToken != null) {
        queueManagerService.updateTokenStatus(String.valueOf(activeToken.getId()), "missed");
        whatsappService.sendWhatsAppMessage(fromPhone, botMessages.tokenCancelled(lang, activeToken.displayNumber()));
    } else {
        whatsappService.sendWhatsAppMessage(fromPhone, botMessages.noActiveTokenToCancel(lang));
    }
    return ResponseEntity.ok("EVENT_RECEIVED");
}

// STEP 3: Fallback routing
if (activeToken != null && ("waiting".equals(activeToken.getStatus()) || "serving".equals(activeToken.getStatus()))) {
    sendTokenDashboardCard(fromPhone, activeToken, lang);
} else {
    sendWelcomeCard(fromPhone, lang);
}

return ResponseEntity.ok("EVENT_RECEIVED");
} catch (Exception e) {
    log.error("Webhook error:", e);
    return ResponseEntity.status(500).body("{\"success\":false,\"error\":\"\" + e.getMessage() + "\"}");
}

}

/**
 * Meta has a real interactive-button primitive, so the language picker is
 * three tap buttons there. Evolution API has none for this bot (buttons
 * degrade to a poll, which isn't what was wanted here), so it gets a
 * plain-text prompt asking the patient to type the language name instead
 * -- matched back by {@link Lang#matchInitial}.
 */
private void sendLanguagePrompt(String phone, boolean firstContact) {
    String text = firstContact
        ? botMessages.greetingAndLanguagePrompt(appProperties.getClinicName())
        : botMessages.languageNotUnderstood();

    whatsappService.sendWhatsAppMessage(phone, text);
}

// -----
// Interactive message builders
// -----

private void sendWelcomeCard(String phone, Lang lang) {
    String title = botMessages.welcomeTitle(lang, appProperties.getClinicName());
    String description = botMessages.welcomeDescription(lang);
    whatsappService.sendButtonsMessage(phone, title, description, botMessages.welcomeButtons(lang), appProperties.getClinicName());
}

```

```

private void sendGenderPrompt(String phone, Lang lang) {
    whatsappService.sendButtonsMessage(phone, "", botMessages.genderPrompt(lang), botMessages.genderButtons(lang),
    appProperties.getClinicName());
}

/**
 * Native WhatsApp location sharing (and especially live location on
 * Evolution API, which pings via separate "messages.update" events -- see
 * {@code extractLocationFromPayload}) doesn't always land reliably, so
 * alongside the native prompt this also sends a "just tap this link"
 * fallback: a web page that asks the browser for location permission and
 * shows the same nearest-first hospital list the bot would.
 */
private void sendLocationPrompt(String phone, Lang lang, String category) {
    whatsappService.sendLocationRequestMessage(phone, botMessages.locationPrompt(lang));

    String cleanPhone = phone.replaceAll("[^0-9]", "");
    String encodedCategory = java.net.URLEncoder.encode(category, java.nio.charset.StandardCharsets.UTF_8);
    String findHospitalUrl = appProperties.getFrontendUrl() + "/find-hospital?phone=" + cleanPhone + "&category=" + encodedCategory;

    whatsappService.sendUrlButtonMessage(phone, "", botMessages.findHospitalLinkPrompt(lang),
        botMessages.findHospitalLinkButtonText(lang), findHospitalUrl, appProperties.getClinicName());
}

/** "Which hospital?" -- up to 20 results as numbered text list + interactive list rows. */
private void sendHospitalResultsList(String phone, TokenDto draft, HospitalService.HospitalSearchPage page, Lang lang) {
    StringBuilder sb = new StringBuilder();
    sb.append("🏥 *Top Registered Hospitals for ").append(draft.getCategory()).append("*: \n");
    sb.append("-----\n\n");

    int index = 1;
    for (HospitalService.HospitalMatch match : page.results()) {
        HospitalDto h = match.hospital();
        boolean openNow = isOpdOpen(h);
        String statusIcon = openNow ? "🟢 Open" : "🔴 Closed";

        sb.append(index).append(" ").append(h.getName()).append("*\n");
        if (h.getAddress() != null && !h.getAddress().isBlank()) {
            sb.append("📍 ").append(h.getAddress()).append("\n");
        }
        sb.append("🕒 OPD: ").append(formatTime(h.getOpenTime())).append(" - ").append(formatTime(h.getCloseTime()))
            .append(" ").append(statusIcon).append("\n");
        sb.append("📏 Distance: ~%.1f km\n\n".formatted(match.distanceKm()));
        index++;
    }

    sb.append("-----\n");
    sb.append("👉 *Reply with a number (1 to ").append(page.results().size()).append(") to select a hospital for booking.*\n");
    sb.append("💡 *Type \"profile 1\" to view full hospital info & doctor roster.*");
    if (page.hasMore()) {
        sb.append("\n").append(botMessages.showMoreHint(lang));
    }

    whatsappService.sendWhatsAppMessage(phone, sb.toString());
}

/** Evolution has no tappable "Show more" button, so the next page is also requested by typing it, in any supported language. */
private boolean isShowMoreCommand(String cleanMessage) {
    if (cleanMessage == null) {
        return false;
    }
    return switch (cleanMessage.trim()) {
        case "more", "show more", "next", "और दिखाएं", "आणखी दाखवा" -> true;
        default -> false;
    };
}

private void sendHospitalProfile(String phone, HospitalDto h, double distanceKm, Lang lang) {
    StringBuilder sb = new StringBuilder();
    sb.append("🏥 *HOSPITAL PROFILE: ").append(h.getName().toUpperCase()).append("*\n");
    sb.append("-----\n");
    if (h.getOwnership() != null) {
        sb.append("🏠 *Ownership: ").append(h.getOwnership()).append("\n");
    }
    if (h.getYearEstablished() != null) {

```

```

        sb.append("🏥 *Established:* ").append(h.getYearEstablished()).append("\n");
    }
    if (h.getAddress() != null && !h.getAddress().isBlank()) {
        sb.append("📍 *Address:* ").append(h.getAddress()).append("\n");
    }
    if (h.getDigipin() != null) {
        sb.append("🔑 *DIGIPIN:* ").append(h.getDigipin()).append("\n");
    }
    sb.append("🕒 *OPD Hours:* ").append(formatTime(h.getOpenTime())).append(" - ").append(formatTime(h.getCloseTime())).append("\n");
    sb.append("📏 *Distance:* ~%.1f km\n".formatted(distanceKm));
    sb.append("⌚ *Avg Service Time:* ").append(h.getAvgServiceMinutes()).append(" mins per patient\n");
    sb.append("👤 *Active Counters:* ").append(h.getActiveCounters()).append("\n");

    if (h.getCategories() != null && !h.getCategories().isEmpty()) {
        sb.append("\n🏥 *Offered Departments:* \n");
        for (String cat : h.getCategories()) {
            sb.append("    • ").append(cat).append("\n");
        }
    }

    if (h.getAccreditation() != null && !h.getAccreditation().isEmpty()) {
        sb.append("\n🏆 *Accreditations:* ").append(String.join(", ", h.getAccreditation())).append("\n");
    }

    sb.append("\n-----\n");
    sb.append("👉 Reply with hospital number to book a token, or type *menu* to return.");

    whatsappService.sendWhatsAppMessage(phone, sb.toString());
}

private boolean isOpdOpen(HospitalDto h) {
    if (h == null || h.getOpenTime() == null || h.getCloseTime() == null) return true;
    LocalDateTime now = LocalDateTime.now();
    return !now.isBefore(h.getOpenTime()) && now.isBefore(h.getCloseTime());
}

private String formatTime(LocalTime time) {
    if (time == null) return "N/A";
    return time.format(java.time.format.DateTimeFormatter.ofPattern("hh:mm a"));
}

/** "Book here?" -- shown right after a hospital row is tapped, before the token actually joins that hospital's queue. */
private void sendConfirmationCard(String phone, TokenDto selected, Lang lang) {
    HospitalDto hospital = hospitalService.getById(selected.getHospitalId());
    String name = hospital != null ? hospital.getName() : "Hospital";
    String address = hospital != null ? hospital.getAddress() : null;
    double distanceKm = (hospital != null && selected.getPatientLat() != null && selected.getPatientLon() != null)
        ? geoDistanceService.distanceKm(hospital.getLatitude(), hospital.getLongitude(), selected.getPatientLat(),
selected.getPatientLon())
        : 0;
    whatsappService.sendWhatsAppMessage(phone, botMessages.hospitalConfirmationPrompt(lang, name, address, distanceKm));
}

private void sendTokenDashboardCard(String phone, TokenDto token, Lang lang) {
    String cleanPhone = phone.replaceAll("[^0-9]", "");
    String positionText = getOrdinal(token.getPosition());
    int avgServiceTime = appProperties.getAvgServiceMinutes();
    int peopleAhead = token.getPeopleAhead() != null ? token.getPeopleAhead() : 0;
    int estWait = peopleAhead * avgServiceTime;
    String statusBadge = botMessages.statusBadge(lang, token.getStatus());
    // /patient requires a logged-in patient JWT session and ignores any ?phone= query
    // string, so a link built that way just bounces an unauthenticated WhatsApp user to
    // the login screen. /track/{phone} is the public, no-auth lookup route -- it reads
    // the phone straight from the URL and redirects to /token/{id} (see Track.jsx).
    String liveUrl = appProperties.getFrontendUrl() + "/track/" + cleanPhone;

    String title = botMessages.dashboardTitle(lang, token.displayNumber());
    String description = botMessages.dashboardDescription(lang, token.getName(), token.getAge(), statusBadge, positionText, peopleAhead,
estWait);

    whatsappService.sendUrlButtonMessage(phone, title, description, botMessages.liveTrackerButtonText(lang), liveUrl,
appProperties.getClinicName());

    whatsappService.sendButtonsMessage(phone, "", botMessages.quickActionsFooter(lang), botMessages.quickActionButtons(lang), "Quick
Actions");
}

```

```

    }

    private void sendStatusCard(String phone, TokenDto token, Lang lang) {
        if (token == null || "registering_name".equals(token.getStatus())) {
            whatsappService.sendWhatsAppMessage(phone, botMessages.noActiveTokenFound(lang));
            return;
        }
        String cleanPhone = phone.replaceAll("[^0-9]", "");
        String positionText = getOrdinal(token.getPosition());
        int avgServiceTime = appProperties.getAvgServiceMinutes();
        int peopleAhead = token.getPeopleAhead() != null ? token.getPeopleAhead() : 0;
        int estWait = peopleAhead * avgServiceTime;
        // See sendTokenDashboardCard above -- /track/{phone} is the public route, /patient is not.
        String liveUrl = appProperties.getFrontendUrl() + "/track/" + cleanPhone;

        String title = botMessages.statusTitle(lang);
        String description = botMessages.statusDescription(lang, token.displayNumber(), token.getName(), token.getAge(), positionText,
            peopleAhead, estWait, token.getStatus());

        whatsappService.sendUrlButtonMessage(phone, title, description, botMessages.liveTrackerButtonText(lang), liveUrl,
            appProperties.getClinicName());
    }

    /** Accepts a bare number (digits only, after trimming any stray text/punctuation), 0-120. Anything else is rejected. */
    private Integer parseAge(String cleanMessage) {
        String digitsOnly = cleanMessage.replaceAll("[^0-9]", "");
        if (digitsOnly.isEmpty()) {
            return null;
        }
        try {
            int age = Integer.parseInt(digitsOnly);
            return (age >= 0 && age <= 120) ? age : null;
        } catch (NumberFormatException e) {
            return null;
        }
    }

    private Integer parseInt(String s) {
        try {
            return Integer.parseInt(s);
        } catch (NumberFormatException e) {
            return null;
        }
    }

    /** A native location share (lat/lon) wins; otherwise raw lat/lon or Google Maps link; otherwise 10-char DIGIPIN; otherwise unresolved. */
    private SetLocationRequest resolveIncomingLocation(Double lat, Double lon, String cleanMessage) {
        if (lat != null && lon != null) {
            return new SetLocationRequest(null, lat, lon);
        }
        if (cleanMessage != null && !cleanMessage.isBlank()) {
            java.util.regex.Pattern latLonPattern = java.util.regex.Pattern.compile("(~?\\d{1,2}\\.\\.\\d+)?\\s*,\\s*(~?\\d{1,3}\\.\\.\\d+)?");
            java.util.regex.Matcher matcher = latLonPattern.matcher(cleanMessage);
            if (matcher.find()) {
                try {
                    double parsedLat = Double.parseDouble(matcher.group(1));
                    double parsedLon = Double.parseDouble(matcher.group(2));
                    return new SetLocationRequest(null, parsedLat, parsedLon);
                } catch (NumberFormatException ignored) {}
            }
        }
        String candidate = cleanMessage == null ? "" : cleanMessage.trim().toUpperCase();
        if (DIGIPIN_PATTERN.matcher(candidate).matches()) {
            return new SetLocationRequest(candidate, null, null);
        }
        return null;
    }

    private Double[] extractLocationFromPayload(JsonNode data, JsonNode messageData, JsonNode body) {
        List<JsonNode> candidates = List.of(
            messageData.path("locationMessage"),
            messageData.path("liveLocationMessage"),
            messageData.path("viewOnceMessage").path("message").path("locationMessage"),
            messageData.path("viewOnceMessage").path("message").path("liveLocationMessage"),
            messageData.path("ephemeralMessage").path("message").path("locationMessage"),

```

```

        messageData.path("ephemeralMessage").path("message").path("liveLocationMessage"),
        data.path("locationMessage"),
        data.path("liveLocationMessage"),
        data.path("location"),
        body.path("locationMessage"),
        body.path("liveLocationMessage"),
        body.path("location")
    );

    for (JsonNode node : candidates) {
        if (node == null || node.isMissingNode() || node.isNull()) {
            continue;
        }

        Double lat = parseCoord(node, "degreesLatitude", "latitude", "lat", "degLatitude");
        Double lon = parseCoord(node, "degreesLongitude", "longitude", "lon", "lng", "degLongitude");

        if (lat != null && lon != null) {
            log.info(" 📍 Location successfully extracted from WhatsApp payload: lat={}, lon={}", lat, lon);
            return new Double[]{lat, lon};
        }
    }
    return new Double[]{null, null};
}

private Double parseCoord(JsonNode node, String... fieldNames) {
    for (String field : fieldNames) {
        JsonNode f = node.path(field);
        if (!f.isMissingNode() && !f.isNull()) {
            if (f.isNumber()) {
                return f.asDouble();
            } else if (f.isTextual()) {
                try {
                    return Double.parseDouble(f.asText().trim());
                } catch (NumberFormatException ignored) {}
            }
        }
    }
    return null;
}

private String getOrdinal(Integer n) {
    if (n == null || n <= 0) {
        return "0th";
    }
    String[] s = {"th", "st", "nd", "rd"};
    int v = n % 100;
    String suffix = arrAt(s, (v - 20) % 10);
    if (suffix == null) {
        suffix = arrAt(s, v);
    }
    if (suffix == null) {
        suffix = s[0];
    }
    return n + suffix;
}

private String arrAt(String[] arr, int idx) {
    return (idx >= 0 && idx < arr.length) ? arr[idx] : null;
}

// ---- JSON helpers ----

private String textOrNull(JsonNode node) {
    return (node == null || node.isMissingNode() || node.isNull()) ? null : node.asText();
}

private boolean isBlank(String s) {
    return s == null || s.isEmpty();
}

private String firstNonBlank(String... values) {
    for (String v : values) {
        if (!isBlank(v)) {
            return v;
        }
    }
}

```

```
    }  
  }  
  return "";  
}  
}
```

7. Package: dto

Every request/response shape in the app, almost all Java records (immutable, validation-annotated request bodies) or Lombok `@Data` `@Builder` classes (mutable response views built up field-by-field from a JDBC `ResultSet`). None of these are JPA entities — they're populated by hand in each service's row-mapping code.

AccessGrantDto.java

src/main/java/com/qdischarge/clinicqueue/dto/AccessGrantDto.java

A live (or revoked) doctor <-> patient-phone consent grant, carrying the doctor's name/hospital so a patient's access list is self-describing without a second lookup.

```
package com.qdischarge.clinicqueue.dto;

import com.fasterxml.jackson.annotation.JsonInclude;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

/** A live (or revoked) doctor <-> patient-phone consent grant. Carries the doctor's name/hospital so a patient's access list is self-describing without a second lookup. */
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
@JsonInclude(JsonInclude.Include.NON_NULL)
public class AccessGrantDto {
    private Integer id;
    private Integer doctorId;
    private String doctorName;
    private String hospitalName;
    private String patientPhone;
    private LocalDateTime grantedAt;
    private LocalDateTime revokedAt;
    /** 'patient' -- the only actor allowed to revoke today. */
    private String revokedBy;
}
```

AccessRequestDto.java

src/main/java/com/qdischarge/clinicqueue/dto/AccessRequestDto.java

A doctor's outstanding access request — a short code + QR, not yet tied to any patient until claimed.

```
package com.qdischarge.clinicqueue.dto;

import com.fasterxml.jackson.annotation.JsonInclude;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

/** A doctor's outstanding "let me view your records" request -- a short code + QR (GET .../qr), not yet tied to any patient until claimed. */
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
@JsonInclude(JsonInclude.Include.NON_NULL)
public class AccessRequestDto {
    private Integer id;
    private String code;
    private Integer doctorId;
    /** pending | claimed | expired */
    private String status;
    private LocalDateTime createdAt;
    private LocalDateTime expiresAt;
}
```

AssignDoctorLocationRequest.java

src/main/java/com/qdischarge/clinicqueue/dto/AssignDoctorLocationRequest.java

Admin-only request body: assigns a doctor to a counter number within a department.

```
package com.qdischarge.clinicqueue.dto;

import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;

/**
 * Admin-only: assigns one of a hospital's doctors to a physical location in
 * its queue system -- a counter number (like every other counter, see
 * CounterAssignmentService) within a given department. Both fields are
 * required together; a hospital either gives a doctor a seat or it doesn't.
 */
public record AssignDoctorLocationRequest(
    @Min(value = 1, message = "counterId must be at least 1.")
    int counterId,
    /** Must be a valid catalog.MedicalCategory name (case-insensitive). */
    @NotBlank(message = "category is required.")
    String category) {
}
```

CreateHospitalRequest.java

src/main/java/com/qdischarge/clinicqueue/dto/CreateHospitalRequest.java

The full hospital-registration payload: location, OPD hours, either a direct `avgServiceMinutes` or a derived-from- `avgPatientsPerDay` figure, and the optional profile fields (ownership, year established, accreditation, gender restriction, offered categories) that are preserved across re-seeding rather than wiped on every boot.

```
package com.qdischarge.clinicqueue.dto;

import jakarta.validation.Valid;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;

import java.util.List;

/**
 * location, openTime/closeTime are asked of every hospital at registration.
 * avgServiceMinutes (how long one patient takes) is normally *not* filled in
 * directly -- instead give avgPatientsPerDay (how many patients the hospital
 * treats on an average day) and HospitalService derives it from that plus
 * the open/close hours: (open hours in minutes) / avgPatientsPerDay.
 * avgServiceMinutes, if given, is an explicit override that wins over that
 * calculation. minServiceMinutes is separate and asked for directly -- the
 * floor a hospital commits to per patient, not derived from anything.
 *
 * The five profile fields (ownership onward) are all optional and preserved
 * across re-seeding on every boot (see HospitalService#create) -- a null here
 * leaves whatever's already stored alone rather than wiping it out, same as
 * doctorJoinCode. categories, when given, must each be a valid
 * catalog.MedicalCategory name.
 */
public record CreateHospitalRequest(
    @NotBlank(message = "uriSlug is required.")
    @jakarta.validation.constraints.Pattern(regexp = "^[a-z0-9-]{2,100}$",
        message = "uriSlug must be lowercase letters, digits, and hyphens only.")
    String uriSlug,
    @NotBlank(message = "name is required.")
    String name,
    String address,
    @NotNull(message = "location is required.")
    @Valid
    SetLocationRequest location,
    String openTime,
    String closeTime,
    @Min(value = 1, message = "avgPatientsPerDay must be at least 1.")
    Integer avgPatientsPerDay,
    @Min(value = 1, message = "avgServiceMinutes must be at least 1.")
    Integer avgServiceMinutes,
    /** The floor a hospital commits to per patient -- purely informational (not derived, unlike avgServiceMinutes); must not exceed
    avgServiceMinutes when both are given. */
    @Min(value = 1, message = "minServiceMinutes must be at least 1.")
    Integer minServiceMinutes,
    @Min(value = 1, message = "activeCounters must be at least 1.")
    Integer activeCounters,
    /** Free text; "Private" / "Trust" / "Government" / "Chain-affiliated" are the expected values. */
    String ownership,
    @Min(value = 1800, message = "yearEstablished looks invalid.")
    Integer yearEstablished,
    List<String> accreditation,
    /** Null/omitted = general/co-ed. Otherwise must be "male" or "female". */
    String genderSpecific,
    /** Each entry must be a valid catalog.MedicalCategory name (case-insensitive). */
    List<String> categories) {
}
```

CreateTimeSlotRequest.java

src/main/java/com/qdischarge/clinicqueue/dto/CreateTimeSlotRequest.java

Admin-only: one OPD time window on one day, assigned to a list of doctors in the same call.

```
package com.qdischarge.clinicqueue.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotEmpty;
import jakarta.validation.constraints.NotNull;

import java.time.LocalDate;
import java.util.List;

/**
 * Admin-only: one OPD time window on one day, assigned to a list of doctors
 * in the same call -- a hospital can have several of these on the same
 * date (call this endpoint once per slot) each with its own doctor roster.
 */
public record CreateTimeSlotRequest(
    @NotNull(message = "date is required.")
    LocalDate date,
    @NotBlank(message = "startTime is required (HH:mm).")
    String startTime,
    @NotBlank(message = "endTime is required (HH:mm).")
    String endTime,
    /** Optional; must be a valid catalog.MedicalCategory name (case-insensitive) if given. */
    String category,
    @NotEmpty(message = "doctorIds must list at least one doctor.")
    List<Integer> doctorIds) {
}
```

CreateTokenRequest.java

src/main/java/com/qdischarge/clinicqueue/dto/CreateTokenRequest.java

The direct/API token-booking payload (as opposed to the WhatsApp flow) — name/age/gender/category/phone plus an optional location (DIGIPIN or lat/lon) and hospital selection.

```
package com.qdischarge.clinicqueue.dto;

import jakarta.validation.constraints.Max;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;

public record CreateTokenRequest(
    String name,
    @Min(value = 0, message = "age must be at least 0.")
    @Max(value = 120, message = "age must be at most 120.")
    Integer age,
    @Pattern(regexp = "(?i)^(male|female|other)$", message = "gender must be male, female, or other.")
    String gender,
    // Which department (catalog.MedicalCategory) this token is queued in -- every
    // queue-shaping read/write is now scoped by (hospital, category), see QueueManagerService.
    @NotBlank(message = "category is required.")
    String category,
    @NotBlank(message = "Phone number is required.")
    @Pattern(regexp = "[+0-9][0-9 ()-]{4,20}$", message = "Phone number is invalid.")
    String phone,
    // Optional: patient's current (or manually entered) location, as a DIGIPIN or lat/lon.
    // When given, the distance-based "get ready" notification window is computed right away.
    String digipin,
    Double latitude,
    Double longitude,
    // Optional: which hospital (see GET /api/hospitals/nearby) this token books into --
    // omit to fall back to this deployment's single operating hospital (app.hospital-uri-slug).
    Integer hospitalId) {

    public SetLocationRequest toLocationOrNull() {
        if ((digipin == null || digipin.isBlank()) && (latitude == null || longitude == null)) {
            return null;
        }
        return new SetLocationRequest(digipin, latitude, longitude);
    }
}
```

CreateTokenResult.java

src/main/java/com/qdischarge/clinicqueue/dto/CreateTokenResult.java

Mirrors `{ alreadyExists, data }` from the original Node `createToken()` — `alreadyExists` is true when the phone already had an active token, so the caller can distinguish "booked" from "here's your existing one."

```
package com.qdischarge.clinicqueue.dto;

/** Mirrors { alreadyExists, data } returned by queueManager.js#createToken(). */
public record CreateTokenResult(boolean alreadyExists, TokenDto data) {
}
```

DoctorDto.java

src/main/java/com/qdischarge/clinicqueue/dto/DoctorDto.java

A doctor account: phone+OTP identity, optionally linked to one hospital, plus their assigned counter/department location within that hospital's queue system.

```
package com.qdischarge.clinicqueue.dto;

import com.fasterxml.jackson.annotation.JsonInclude;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

/** A doctor account: phone+OTP identity (see DoctorService), linked to at most one hospital via that hospital's join code. */
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
@JsonInclude(JsonInclude.Include.NON_NULL)
public class DoctorDto {
    private Integer id;
    private String phone;
    private String name;
    private Integer hospitalId;
    /** Populated on joined reads (e.g. GET /api/doctors/me) -- null when the doctor hasn't linked a hospital yet. */
    private String hospitalName;
    /**
     * This doctor's assigned physical location within their hospital's queue
     * system -- a counter number, exactly like every other counter (see
     * CounterAssignmentService), plus which department (catalog.MedicalCategory)
     * that counter belongs to. Set by the hospital (admin), not the doctor --
     * see HospitalController's PUT ../doctors/{doctorId}/location.
     */
    private Integer counterId;
    private String category;
    private LocalDateTime createdAt;
}
```

DoctorLoginRequest.java

src/main/java/com/qdischarge/clinicqueue/dto/DoctorLoginRequest.java

Returning-doctor login payload — just the phone number; identity is proven by a prior OTP verification.

```
package com.qdischarge.clinicqueue.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;

/** Returning doctor: re-verify OTP for this phone, then call this to get a fresh doctor JWT. */
public record DoctorLoginRequest(
    @NotBlank(message = "Phone number is required.")
    @Pattern(regexp = "^[+0-9][0-9 ()-]{4,20}$", message = "Phone number is invalid.")
    String phone) {
}
```

DoctorPatientGroupDto.java

src/main/java/com/qdischarge/clinicqueue/dto/DoctorPatientGroupDto.java

One patient's complete accessible record set as a doctor sees it: every document under a phone the doctor has active access to, grouped by (name, age) rather than phone alone, since one WhatsApp number can be used by different family members.

```
package com.qdischarge.clinicqueue.dto;

import com.fasterxml.jackson.annotation.JsonInclude;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.util.List;

/**
 * One person's complete accessible record, as a doctor sees it: every
 * document under a phone number the doctor has active access to, grouped by
 * (name, age) rather than by phone alone -- since one WhatsApp number can be
 * used to book/upload for different family members (see tokens.age in schema.sql).
 * A phone the doctor was just granted access to but that has no documents
 * yet still shows up here (name/age null, empty documents) so the doctor
 * can see *that* they have access even before any record exists.
 */
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
@JsonInclude(JsonInclude.Include.NON_NULL)
public class DoctorPatientGroupDto {
    private String patientPhone;
    private String name;
    private Integer age;
    private List<PatientDocumentDto> documents;
}
```

HospitalDepartmentDto.java

src/main/java/com/qdischarge/clinicqueue/dto/HospitalDepartmentDto.java

Row/view of `hospital_departments`: one (hospital, category) queue's counter count and optional department-specific average service time.

```
package com.qdischarge.clinicqueue.dto;

import com.fasterxml.jackson.annotation.JsonInclude;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

/**
 * Row/view of "hospital_departments": one (hospital, category) queue's
 * counter count. See catalog.MedicalCategory for valid category values and
 * QueueManagerService for how (hospitalId, category) partitions the queue.
 */
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
@JsonInclude(JsonInclude.Include.NON_NULL)
public class HospitalDepartmentDto {
    private Integer id;
    private Integer hospitalId;
    private String category;
    private Integer activeCounters;
    /** Null = falls back to the hospital's avgServiceMinutes. */
    private Integer avgServiceMinutes;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
}
```

HospitalDto.java

src/main/java/com/qdischarge/clinicqueue/dto/HospitalDto.java

The full hospital record: location (DIGIPIN + lat/lon), OPD hours, service-time settings, and the profile fields (ownership, year established, accreditation, gender restriction, offered categories).

```
package com.qdischarge.clinicqueue.dto;

import com.fasterxml.jackson.annotation.JsonInclude;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;
import java.time.LocalTime;
import java.util.List;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
@JsonInclude(JsonInclude.Include.NON_NULL)
public class HospitalDto {
    private Integer id;
    private String uriSlug;
    private String name;
    private String address;
    private String digipin;
    private String formattedDigipin;
    private Double latitude;
    private Double longitude;
    private LocalTime openTime;
    private LocalTime closeTime;
    private Integer avgServiceMinutes;
    /** The floor this hospital commits to per patient (asked directly at registration, not derived). */
    private Integer minServiceMinutes;
    private Integer activeCounters;

    // --- Hospital profile (see catalog.MedicalCategory for valid categories) ---
    /** Free text; "Private" / "Trust" / "Government" / "Chain-affiliated" are the expected values. */
    private String ownership;
    private Integer yearEstablished;
    private List<String> accreditation;
    /** Null = general/co-ed; "male" or "female" = this hospital only serves that gender. */
    private String genderSpecific;
    private List<String> categories;

    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
}
```

JoinHospitalRequest.java

src/main/java/com/qdischarge/clinicqueue/dto/JoinHospitalRequest.java

A doctor's request body when linking their account to a hospital via its join code.

```
package com.qdischarge.clinicqueue.dto;

import jakarta.validation.constraints.NotBlank;

public record JoinHospitalRequest(
    @NotBlank(message = "hospitalCode is required.")
    String hospitalCode) {
}
```

LoginRequest.java

src/main/java/com/qdischarge/clinicqueue/dto/LoginRequest.java

Admin login payload (username/password).

```
package com.qdischarge.clinicqueue.dto;

import jakarta.validation.constraints.NotBlank;

public record LoginRequest(@NotBlank String username, @NotBlank String password) {
}
```

OtpSendRequest.java

src/main/java/com/qdischarge/clinicqueue/dto/OtpSendRequest.java

Phone number to send an OTP to.

```
package com.qdischarge.clinicqueue.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;

public record OtpSendRequest(
    @NotBlank(message = "Phone number is required.")
    @Pattern(regexp = "^+[0-9][0-9 ()-]{4,20}$", message = "Phone number is invalid.")
    String phone) {
}
```

OtpVerifyRequest.java

src/main/java/com/qdischarge/clinicqueue/dto/OtpVerifyRequest.java

Phone number plus the 4-8 digit OTP code to verify.

```
package com.qdischarge.clinicqueue.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;

public record OtpVerifyRequest(
    @NotBlank(message = "Phone number is required.")
    @Pattern(regexp = "^+[0-9][0-9 ()-]{4,20}$", message = "Phone number is invalid.")
    String phone,
    @NotBlank(message = "OTP code is required.")
    @Pattern(regexp = "[0-9]{4,8}$", message = "OTP code must be 4-8 digits.")
    String code) {
}
```

PatientDocumentDto.java

src/main/java/com/qdischarge/clinicqueue/dto/PatientDocumentDto.java

Metadata for one uploaded prescription/report photo — deliberately never carries the file bytes themselves, so listing a patient's documents never drags BYTEA payloads across the wire.

```
package com.qdischarge.clinicqueue.dto;

import com.fasterxml.jackson.annotation.JsonInclude;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

/**
 * Metadata for one uploaded prescription/report photo -- deliberately never
 * carries the file bytes themselves (see PatientDocumentService.FileContent
 * for that), so listing a patient's documents never pulls BYTEA payloads
 * across the wire. Fetch the actual image via the dedicated .../file route.
 */
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
@JsonInclude(JsonInclude.Include.NON_NULL)
public class PatientDocumentDto {
    private Integer id;
    private String patientPhone;
    private String patientName;
    private Integer patientAge;
    /** 'prescription' or 'report'. */
    private String docType;
    private Integer hospitalId;
    private Integer uploadedByDoctorId;
    private String fileName;
    private String contentType;
    private Integer fileSize;
    private LocalDateTime createdAt;
}
```

QueueData.java

src/main/java/com/qdischarge/clinicqueue/dto/QueueData.java

Mirrors the object returned by the original `getQueue()`: the token list, the currently-serving token id, and aggregate stats.

```
package com.qdischarge.clinicqueue.dto;

import java.util.List;

/** Mirrors the object returned by queueManager.js#getQueue(). */
public record QueueData(List<TokenDto> tokens, Integer currentServing, Stats stats) {
}
```

RegisterDoctorRequest.java

src/main/java/com/qdischarge/clinicqueue/dto/RegisterDoctorRequest.java

New-doctor registration payload: name + phone, called right after the phone's OTP is verified.

```
package com.qdischarge.clinicqueue.dto;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Pattern;

/** Called right after the phone's OTP is verified (see OtpService#isPhoneVerifiedRecently) -- that's the doctor's proof of identity, same as
patient registration. */
public record RegisterDoctorRequest(
    @NotBlank(message = "Name is required.")
    String name,
    @NotBlank(message = "Phone number is required.")
    @Pattern(regexp = "[+0-9][0-9 (-)]{4,20}$", message = "Phone number is invalid.")
    String phone) {
}
```

SetLocationRequest.java

src/main/java/com/qdischarge/clinicqueue/dto/SetLocationRequest.java

A location given as either a DIGIPIN or raw lat/lon — used both for setting a hospital's location and a patient sharing theirs. Exactly one form must be present; the service layer derives the other.

```
package com.qdischarge.clinicqueue.dto;

import jakarta.validation.constraints.DecimalMax;
import jakarta.validation.constraints.DecimalMin;

/**
 * A location given either as a DIGIPIN or as raw lat/lon -- used both for
 * "manually add/set a hospital's location" and for a patient sharing their
 * current or manually-entered location. Exactly one of the two forms must
 * be present; the service layer derives the other.
 */
public record SetLocationRequest(
    String digipin,
    @DecimalMin(value = "-90", message = "Latitude must be between -90 and 90.")
    @DecimalMax(value = "90", message = "Latitude must be between -90 and 90.")
    Double latitude,
    @DecimalMin(value = "-180", message = "Longitude must be between -180 and 180.")
    @DecimalMax(value = "180", message = "Longitude must be between -180 and 180.")
    Double longitude) {

    public boolean hasDigipin() {
        return digipin != null && !digipin.isBlank();
    }

    public boolean hasLatLon() {
        return latitude != null && longitude != null;
    }
}
```

Stats.java

src/main/java/com/qdischarge/clinicqueue/dto/Stats.java

Mirrors the `stats` object literal from the original `getQueue()`: total/waiting/serving/completed/missed counts.

```
package com.qdischarge.clinicqueue.dto;

/** Mirrors the `stats` object literal returned by queueManager.js#getQueue(). */
public record Stats(int total, int waiting, int serving, int completed, int missed) {
}
```

TimeSlotDto.java

src/main/java/com/qdischarge/clinicqueue/dto/TimeSlotDto.java

Row/view of `time_slots`: one OPD time window on one day for one hospital (optionally scoped to one department), plus the list of doctors rostered onto it.

```
package com.qdischarge.clinicqueue.dto;

import com.fasterxml.jackson.annotation.JsonInclude;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.util.List;

/**
 * Row/view of "time_slots": one OPD time window on one day, for one hospital
 * (and optionally scoped to one department), assigned to whichever doctors
 * are rostered onto it -- see TimeSlotService. A hospital can have any
 * number of these on the same slot_date; each is independent.
 */
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
@JsonInclude(JsonInclude.Include.NON_NULL)
public class TimeSlotDto {
    private Integer id;
    private Integer hospitalId;
    /** Null = hospital-wide slot, not scoped to one department. */
    private String category;
    private LocalDate slotDate;
    private LocalTime startTime;
    private LocalTime endTime;
    private List<Integer> doctorIds;
    private LocalDateTime createdAt;
}
```

TokenDto.java

src/main/java/com/qdischarge/clinicqueue/dto/TokenDto.java

Row/view of the `tokens` table — the single most complex DTO in the app, since it fields position/geo/priority information only some callers populate. `@JsonInclude(NON_NULL)` means unused fields simply don't appear in a given response, replicating how the original returned different subsets of fields from different functions. `displayNumber()` is the patient-facing "Token #N", falling back to 0 rather than the global `id` for the rare row where `dailyNumber` isn't set.

```
package com.qdischarge.clinicqueue.dto;

import com.fasterxml.jackson.annotation.JsonInclude;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

/**
 * Row/view of the "tokens" table. Different queueManager.js functions
 * returned different subsets of these fields (e.g. getActiveToken omitted
 * completedAt/isVerified, getCurrentServingToken omitted isVerified/verifiedAt).
 * We replicate that by leaving unused fields null here -- @JsonInclude.NON_NULL
 * means they simply don't appear in the JSON response, matching the shape of
 * the original plain object literals.
 */
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
@JsonInclude(JsonInclude.Include.NON_NULL)
public class TokenDto {

    private Integer id;
    /** Patient-facing "Token #N" -- resets per (hospital, category, day), unlike `id` which is one global sequence across every
    hospital/department. See QueueManagerService#nextDailyNumber. */
    private Integer dailyNumber;
    private String phone;
    private String name;
    private Integer age;
    private String gender;
    /** The department (catalog.MedicalCategory) this token is queued in -- see QueueManagerService. */
    private String category;
    private String status;
    private String sessionStep;
    private LocalDateTime createdAt;
    private LocalDateTime servedAt;
    private LocalDateTime completedAt;
    private LocalDateTime missedAt;
    private Boolean isVerified;
    private LocalDateTime verifiedAt;

    // Extra fields populated only by position-aware lookups
    // (getPatientPosition / getTokenDetails / verifyTokenByAdmin).
    private Integer position;
    private Integer peopleAhead;
    private Integer currentServing;

    // --- Geo / real-ETA treatment timing (see geo/TomTomRoutingService,
    // service/TreatmentTimingScheduler) ---
    private Integer hospitalId;
    private String hospitalName;
    private String patientDigipin;
    private Double patientLat;
    private Double patientLon;
    private Double distanceKm;
    /** TomTom-routed one-way ETA to the hospital, minutes -- fetched once at booking/location-set time. */
    private Double travelMinutes;
    /** Last-computed "minutes of queue work still ahead of this patient" -- display only, always recomputed live before any decision (see
    QueueManagerService#runTreatmentTimingTick). */
```

```
private Double treatmentRemainingMinutes;
private LocalDateTime notifiedReadyAt;
/** Non-null while the patient is inside their post-notify grace window (see QueueManagerService#handleNoShowOrMiss). */
private LocalDateTime anomalyControlUntil;

// --- Missed-queue / requeue / floating-point positioning (see
// QueueManagerService#movePatientToPosition) ---
private Double priorityRank;
private LocalDateTime rejectedAt;
/** How many times reception has clicked "not come yet" on this token -- drives the exponential push-back (see
QueueManagerService#pushBackNoShow). */
private Integer noShowCount;

// --- Multi-counter package only ---
private Integer counterId;

// --- Booking-in-progress (hospital search/selection, pre-confirmation) ---
private Integer searchOffset;

/**
 * The number to actually show a patient in any "Token #N" message --
 * {@link #dailyNumber} when it's known, or 0 for the rare legacy/edge-case
 * row where it isn't, rather than falling back to {@link #id} (a global
 * sequence across every hospital/department, which is what caused
 * patients to see e.g. "#17" for the 1st booking of the day in their
 * department).
 */
public int displayNumber() {
    return dailyNumber != null ? dailyNumber : 0;
}
}
```

TokenHistoryDto.java

src/main/java/com/qdischarge/clinicqueue/dto/TokenHistoryDto.java

Row/view of the `token_history` table: a permanent record of one completed visit, archived independently of the live `tokens` row the moment it's marked completed.

```
package com.qdischarge.clinicqueue.dto;

import com.fasterxml.jackson.annotation.JsonInclude;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

/**
 * Row/view of the "token_history" table: a permanent record of a completed
 * visit, archived by QueueManagerService the moment a token is marked
 * 'completed' (service actually finished). Lets one phone number's past
 * visits -- for potentially different patients (different name/age each
 * time) -- be looked up (GET /api/queue/history/{phone}) independently of
 * the live "tokens" table, which only tracks the current/active booking.
 */
@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
@JsonInclude(JsonInclude.Include.NON_NULL)
public class TokenHistoryDto {
    private Integer id;
    private Integer tokenId;
    private String phone;
    private String name;
    private Integer age;
    private String gender;
    private String category;
    private Integer hospitalId;
    private Integer counterId;
    private LocalDateTime createdAt;
    private LocalDateTime servedAt;
    private LocalDateTime completedAt;
    private LocalDateTime archivedAt;
}
```

UpdateDepartmentCountersRequest.java

src/main/java/com/qdischarge/clinicqueue/dto/UpdateDepartmentCountersRequest.java

Admin-only: how many counters should staff a given (hospital, category) department.

```
package com.qdischarge.clinicqueue.dto;

import jakarta.validation.constraints.Min;

/** Admin-only: how many counters staff a given (hospital, category) department. */
public record UpdateDepartmentCountersRequest(
    @Min(value = 1, message = "activeCounters must be at least 1.")
    int activeCounters) {
}
```

UpdateStatusRequest.java

src/main/java/com/qdischarge/clinicqueue/dto/UpdateStatusRequest.java

The new status (waiting/serving/completed/missed) for a PUT to a token.

```
package com.qdischarge.clinicqueue.dto;

import jakarta.validation.constraints.NotBlank;

public record UpdateStatusRequest(@NotBlank(message = "Invalid status.") String status) {
}
```

UpdateStatusResult.java

src/main/java/com/qdischarge/clinicqueue/dto/UpdateStatusResult.java

Mirrors { token, nextServing } from the original updateTokenStatus().

```
package com.qdischarge.clinicqueue.dto;

/** Mirrors { token, nextServing } returned by queueManager.js#updateTokenStatus(). */
public record UpdateStatusResult(TokenDto token, Integer nextServing) {
}
```

VerifyRequest.java

src/main/java/com/qdischarge/clinicqueue/dto/VerifyRequest.java

Either a raw QR payload (object or string) or a direct token id, for the admin scan-to-verify endpoint.

```
package com.qdischarge.clinicqueue.dto;

/** Either qrData (raw QR payload -- object or string) or a direct tokenId may be supplied. */
public record VerifyRequest(Object qrData, Integer tokenId) {
}
```

WaButton.java

src/main/java/com/qdischarge/clinicqueue/dto/WaButton.java

A quick-reply button: { id, displayText }, mirrors the shape used throughout the original whatsappSender.js.

```
package com.qdischarge.clinicqueue.dto;

/** A quick-reply button, mirrors { id, displayText } used throughout whatsappSender.js. */
public record WaButton(String id, String displayText) {
}
```

WaListRow.java

src/main/java/com/qdischarge/clinicqueue/dto/WaListRow.java

One tappable row in an interactive WhatsApp list message.

```
package com.qdischarge.clinicqueue.dto;

/** One tappable row in an interactive list message (WhatsAppService#sendListMessage). */
public record WaListRow(String id, String title, String description) {
}
```

WaListSection.java

src/main/java/com/qdischarge/clinicqueue/dto/WaListSection.java

One titled group of rows in an interactive WhatsApp list message.

```
package com.qdischarge.clinicqueue.dto;

import java.util.List;

/** One titled group of rows in an interactive list message (WhatsAppService#sendListMessage). */
public record WaListSection(String title, List<WaListRow> rows) {
}
```

8. Package: exception

The application-wide safety net for anything not already caught inside a controller method.

GlobalExceptionHandler.java

src/main/java/com/qdischarge/clinicqueue/exception/GlobalExceptionHandler.java

A `@RestControllerAdvice` covering malformed JSON bodies, Bean Validation failures (surfacing the first field error's message), missing static resources (404, matching Express's default when there's no bundled frontend), client-disconnect exceptions (logged quietly at DEBUG rather than as errors), and a final catch-all that returns a consistent `{"success": false, "error": ...}` body for anything else.

```
package com.qdischarge.clinicqueue.exception;

import lombok.extern.slf4j.Slf4j;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.http.converter.HttpMessageNotReadableException;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import org.springframework.web.servlet.resource.NoResourceFoundException;

import java.util.LinkedHashMap;
import java.util.Map;

/**
 * Safety net for anything not already caught inside a controller method
 * (each controller mirrors the original Express routes' own try/catch, so
 * this mostly covers things like malformed JSON request bodies).
 */
@RestControllerAdvice
@Slf4j
public class GlobalExceptionHandler {

    @ExceptionHandler(HttpMessageNotReadableException.class)
    public ResponseEntity<Map<String, Object>> handleBadJson(HttpMessageNotReadableException e) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(body(false, "Invalid or malformed request body."));
    }

    /** Bean validation failure on a @Valid request body (e.g. blank phone/username/status). */
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>> handleValidation(MethodArgumentNotValidException e) {
        String message = e.getBindingResult().getFieldErrors().stream()
            .findFirst()
            .map(fe -> fe.getDefaultMessage())
            .orElse("Invalid request.");
        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(body(false, message));
    }

    /**
     * No matching static resource and no bundled frontend/index.html to fall
     * back to (see WebConfig's SPA resolver) -> 404, same as Express's
     * default handling when res.sendFile() can't find frontend/dist/index.html.
     */
    @ExceptionHandler(NoResourceFoundException.class)
    public ResponseEntity<Void> handleNoResource(NoResourceFoundException e) {
        return ResponseEntity.notFound().build();
    }

    /**
     * Handles client disconnects silently (e.g. browser closed tab, ngrok tunnel timeout,
     * or connection aborted by host machine while writing response).
     */
    @ExceptionHandler({
        org.springframework.web.context.request.async.AsyncRequestNotUsableException.class,
        org.apache.catalina.connector.ClientAbortException.class
    })
    public void handleClientDisconnect(Exception e) {
        log.debug("Client closed connection before response completed: {}", e.getMessage());
    }

    @ExceptionHandler(java.io.IOException.class)
    public void handleIOException(java.io.IOException e) {
        String msg = e.getMessage() != null ? e.getMessage().toLowerCase() : "";
        if (msg.contains("aborted") || msg.contains("broken pipe") || msg.contains("connection reset")) {
```

```
        log.debug("Client connection aborted: {}", e.getMessage());
    } else {
        log.error("I/O error during request processing: {}", e.getMessage());
    }
}

@ExceptionHandler(Exception.class)
public ResponseEntity<Map<String, Object>> handleGeneric(Exception e) {
    // Ignore client abort exceptions wrapped inside other exceptions
    Throwable cause = e;
    while (cause != null) {
        String msg = cause.getMessage() != null ? cause.getMessage().toLowerCase() : "";
        if (msg.contains("aborted by the software in your host machine") || msg.contains("broken pipe") || msg.contains("connection
reset")) {
            log.debug("Client aborted request: {}", cause.getMessage());
            return null;
        }
        cause = cause.getCause();
    }
    log.error("Unhandled error", e);
    Map<String, Object> body = new LinkedHashMap<>();
    body.put("success", false);
    body.put("error", e.getMessage());
    return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(body);
}

private Map<String, Object> body(boolean success, String message) {
    Map<String, Object> body = new LinkedHashMap<>();
    body.put("success", success);
    body.put("message", message);
    return body;
}
}
```

9. Package: geo

Everything location-related: encoding/decoding India Post's DIGIPIN addressing scheme, straight-line distance math, and the real (traffic-aware) routing integration with TomTom.

DigipinService.java

src/main/java/com/qdischarge/clinicqueue/geo/DigipinService.java

A faithful Java port of India Post's official DIGIPIN encoder/decoder — the same 4×4 grid, bounding box, and per-level narrowing logic as the reference implementation. DIGIPIN recursively divides India's bounding box into a 4×4 grid for 10 levels, giving ~4m×4m resolution, so every hospital and every patient location in this system is addressable as a single 10-character code independent of street addresses.

```
package com.qdischarge.clinicqueue.geo;

import org.springframework.stereotype.Service;

/**
 * Java port of India Post's official DIGIPIN encoder/decoder
 * (github.com/INDIAPOST-gov/digipin, src/digipin.js) -- same 4x4 grid,
 * bounding box, and per-level narrowing logic, character-for-character.
 *
 * DIGIPIN divides India's bounding box (lat 2.5-38.5N, lon 63.5-99.5E) into
 * a 4x4 grid recursively for 10 levels, giving ~4m x 4m resolution. Every
 * hospital and every patient location in this system is addressable as a
 * single 10-character DIGIPIN, independent of street addresses or postal
 * codes -- which is the point of using it here: a hospital's "location" is
 * just its DIGIPIN (settable manually or derived from lat/lon), and a
 * patient's current location is encoded the same way for distance math.
 */
@Service
public class DigipinService {

    private static final char[][] GRID = {
        {'F', 'C', '9', '8'},
        {'J', '3', '2', '7'},
        {'K', '4', '5', '6'},
        {'L', 'M', 'P', 'T'},
    };

    private static final double MIN_LAT = 2.5;
    private static final double MAX_LAT = 38.5;
    private static final double MIN_LON = 63.5;
    private static final double MAX_LON = 99.5;

    private static final java.util.regex.Pattern VALID_CHARS =
        java.util.regex.Pattern.compile("^[23456789CJKLMPT]{10}$");

    /** Encodes a lat/lon pair (must fall within India's bounding box) into a 10-char DIGIPIN. */
    public String encode(double lat, double lon) {
        if (lat < MIN_LAT || lat > MAX_LAT) {
            throw new IllegalArgumentException("Latitude out of DIGIPIN range (" + MIN_LAT + " to " + MAX_LAT + ")");
        }
        if (lon < MIN_LON || lon > MAX_LON) {
            throw new IllegalArgumentException("Longitude out of DIGIPIN range (" + MIN_LON + " to " + MAX_LON + ")");
        }

        double minLat = MIN_LAT, maxLat = MAX_LAT, minLon = MIN_LON, maxLon = MAX_LON;
        StringBuilder pin = new StringBuilder(10);

        for (int level = 1; level <= 10; level++) {
            double latDiv = (maxLat - minLat) / 4;
            double lonDiv = (maxLon - minLon) / 4;

            // Rows run north-to-south (row 0 = northernmost quarter), matching the reference impl.
            int row = clamp(3 - (int) Math.floor((lat - minLat) / latDiv));
            int col = clamp((int) Math.floor((lon - minLon) / lonDiv));

            pin.append(GRID[row][col]);

            double newMaxLat = minLat + latDiv * (4 - row);
            double newMinLat = minLat + latDiv * (3 - row);
            maxLat = newMaxLat;
            minLat = newMinLat;

            double newMinLon = minLon + lonDiv * col;
            double newMaxLon = newMinLon + lonDiv;
        }
    }
}
```

```

        minLon = newMinLon;
        maxLon = newMaxLon;
    }

    return pin.toString();
}

private int clamp(int v) {
    return Math.max(0, Math.min(v, 3));
}

/** Decodes a 10-char DIGIPIN back to its cell's center lat/lon. */
public LatLon decode(String digipin) {
    if (digipin == null) {
        throw new IllegalArgumentException("DIGIPIN must not be null.");
    }
    String pin = digipin.trim().toUpperCase();
    if (pin.length() != 10) {
        throw new IllegalArgumentException("DIGIPIN must be exactly 10 characters.");
    }
    if (!INVALID_CHARS.matcher(pin).matches()) {
        throw new IllegalArgumentException(
            "Invalid DIGIPIN. Only 2,3,4,5,6,7,8,9,C,J,K,L,M,P,F,T are permitted, no separators.");
    }

    double minLat = MIN_LAT, maxLat = MAX_LAT, minLon = MIN_LON, maxLon = MAX_LON;

    for (int i = 0; i < 10; i++) {
        char c = pin.charAt(i);
        int ri = -1, ci = -1;
        outer:
        for (int r = 0; r < 4; r++) {
            for (int cc = 0; cc < 4; cc++) {
                if (GRID[r][cc] == c) {
                    ri = r;
                    ci = cc;
                    break outer;
                }
            }
        }
        if (ri < 0) {
            throw new IllegalArgumentException("Invalid character in DIGIPIN: " + c);
        }

        double latDiv = (maxLat - minLat) / 4;
        double lonDiv = (maxLon - minLon) / 4;

        double lat1 = maxLat - latDiv * (ri + 1);
        double lat2 = maxLat - latDiv * ri;
        double lon1 = minLon + lonDiv * ci;
        double lon2 = minLon + lonDiv * (ci + 1);

        minLat = lat1;
        maxLat = lat2;
        minLon = lon1;
        maxLon = lon2;
    }

    double centerLat = (minLat + maxLat) / 2;
    double centerLon = (minLon + maxLon) / 2;
    return new LatLon(round6(centerLat), round6(centerLon));
}

private double round6(double v) {
    return Math.round(v * 1_000_000.0) / 1_000_000.0;
}

/** Cosmetic "XXX-XXX-XXXX" grouping for display only; encode()/decode() always use the plain 10-char form. */
public String format(String rawDigipin) {
    if (rawDigipin == null || rawDigipin.length() != 10) {
        return rawDigipin;
    }
    return rawDigipin.substring(0, 3) + "-" + rawDigipin.substring(3, 6) + "-" + rawDigipin.substring(6, 10);
}

```

```

    public record LatLon(double lat, double lon) {
    }
}

```

GeoDistanceService.java

src/main/java/com/qdischarge/clinicqueue/geo/GeoDistanceService.java

Straight-line-only (haversine) distance and travel-time math — deliberately approximate, used only for the pre-registration "can you make it before closing?" check and as `TomTomRoutingService`'s fallback. The precise, minutes-based "go now" trigger patients are actually notified against lives in `TomTomRoutingService` + `QueueManagerService`.

```

package com.qdischarge.clinicqueue.geo;

import com.qdischarge.clinicqueue.config.AppProperties;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

/**
 * Distance-aware queue math: how far a patient is from the hospital and how
 * long that takes to travel. Straight-line-only -- no roads/traffic -- so
 * it's used for two narrow, tolerant-of-approximation jobs: the
 * pre-registration "can you make it before closing?" check
 * (checkClosingTime), and as geo.TomTomRoutingService's fallback when a
 * real routing ETA isn't available. The precise, minutes-based "go now"
 * trigger patients are actually notified/called against lives in
 * geo.TomTomRoutingService + service.QueueManagerService#runTreatmentTimingTick.
 */
@Service
@RequiredArgsConstructor
public class GeoDistanceService {

    private final AppProperties appProperties;

    private static final double EARTH_RADIUS_KM = 6371.0;

    /** Great-circle distance between two points, in kilometers. */
    public double distanceKm(double lat1, double lon1, double lat2, double lon2) {
        double dLat = Math.toRadians(lat2 - lat1);
        double dLon = Math.toRadians(lon2 - lon1);
        double a = Math.sin(dLat / 2) * Math.sin(dLat / 2)
            + Math.cos(Math.toRadians(lat1)) * Math.cos(Math.toRadians(lat2))
            * Math.sin(dLon / 2) * Math.sin(dLon / 2);
        double c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
        return EARTH_RADIUS_KM * c;
    }

    /** Straight-line-distance-based travel time estimate, in minutes (no live traffic/routing API involved). */
    public double estimatedTravelMinutes(double distanceKm) {
        double speedKmh = Math.max(1, appProperties.getGeoAvgSpeedKmh());
        return (distanceKm / speedKmh) * 60.0;
    }

    public record ArrivalFeasibility(boolean canMakeIt, double distanceKm, double travelMinutes, double minutesUntilClose) {
    }

    /** Can the patient realistically reach the hospital before OPD closes? */
    public ArrivalFeasibility checkClosingTime(double distanceKm, double travelMinutes, double minutesUntilClose) {
        // A little slack for parking/walking in, not just door-to-door driving time.
        boolean canMakeIt = travelMinutes + 5 <= minutesUntilClose;
        return new ArrivalFeasibility(canMakeIt, distanceKm, travelMinutes, minutesUntilClose);
    }
}

```

TomTomRoutingService.java

src/main/java/com/qdischarge/clinicqueue/geo/TomTomRoutingService.java

Real, roads/traffic-aware travel-time estimate from a patient's location to the hospital via TomTom's Routing API (`calculateRoute` , with `traffic=true`), called once at booking/location-set time. Falls back to `GeoDistanceService` 's straight-line estimate whenever no `TOMTOM_API_KEY` is configured, or the call itself fails (network/outage/bad key) — same third-party-failure philosophy as `OtpService` 's Twilio fallback. The response is parsed from TomTom's `routes[0].summary.{lengthInMeters, travelTimeInSeconds}` shape.

```
package com.qdischarge.clinicqueue.geo;

import com.qdischarge.clinicqueue.config.AppProperties;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestTemplate;

import java.util.List;
import java.util.Map;

/**
 * Real (roads/traffic-aware) travel-time estimate from a patient's location
 * to the hospital, via TomTom's Routing API -- the "reaching time" the
 * treatment-timing trigger (see service.QueueManagerService#runTreatmentTimingTick)
 * compares against how much queue time is left. Called once, at
 * booking/location-set time (QueueManagerService#applyPatientLocation), not
 * re-pollled per patient -- traffic conditions aren't re-estimated after
 * that, only the queue side of the comparison is.
 *
 * Falls back to {@link GeoDistanceService}'s straight-line haversine
 * estimate -- rather than failing the booking -- whenever no API key is
 * configured, or the TomTom call itself fails (network/outage/bad key):
 * this mirrors how the rest of this codebase treats third-party
 * integrations (see OtpService's Twilio dummy-credential fallback).
 */
@Service
@RequiredArgsConstructor
@Slf4j
public class TomTomRoutingService {

    private final AppProperties appProperties;
    private final RestTemplate restTemplate;
    private final GeoDistanceService geoDistanceService;

    public record RouteEstimate(double distanceKm, double travelMinutes, boolean fromTomTom) {
    }

    /** hospitallat/Lon = route destination, patientlat/Lon = route origin. */
    public RouteEstimate estimate(double hospitallat, double hospitallon, double patientlat, double patientlon) {
        String apiKey = appProperties.getTomtomApiKey();
        if (apiKey == null || apiKey.isBlank()) {
            return fallback(hospitallat, hospitallon, patientlat, patientlon);
        }
        try {
            return callTomTom(apiKey, patientlat, patientlon, hospitallat, hospitallon);
        } catch (RuntimeException e) {
            log.error("✖ TomTom routing error ({})) -- falling back to straight-line estimate.", e.getMessage());
            return fallback(hospitallat, hospitallon, patientlat, patientlon);
        }
    }

    @SuppressWarnings("unchecked")
    private RouteEstimate callTomTom(String apiKey, double originlat, double originlon, double destlat, double destlon) {
        String url = "https://api.tomtom.com/routing/1/calculateRoute/%s,%s:%s,%s/json?key=%s&traffic=true"
            .formatted(originlat, originlon, destlat, destlon, apiKey);

        Map<String, Object> body = restTemplate.getForObject(url, Map.class);
        if (body == null) {
            throw new IllegalStateException("Empty response from TomTom");
        }
        List<Map<String, Object>> routes = (List<Map<String, Object>>) body.get("routes");
```

```
    if (routes == null || routes.isEmpty()) {
        throw new IllegalStateException("No routes in TomTom response");
    }
    Map<String, Object> summary = (Map<String, Object>) routes.get(0).get("summary");
    double lengthMeters = ((Number) summary.get("lengthInMeters")).doubleValue();
    double travelTimeSeconds = ((Number) summary.get("travelTimeInSeconds")).doubleValue();

    return new RouteEstimate(lengthMeters / 1000.0, travelTimeSeconds / 60.0, true);
}

private RouteEstimate fallback(double hospitalLat, double hospitalLon, double patientLat, double patientLon) {
    double distanceKm = geoDistanceService.distanceKm(hospitalLat, hospitalLon, patientLat, patientLon);
    double travelMinutes = geoDistanceService.estimatedTravelMinutes(distanceKm);
    return new RouteEstimate(distanceKm, travelMinutes, false);
}
}
```

10. Package: ratelimit

A minimal, dependency-free, in-memory sliding-window rate limiter guarding the endpoints most worth protecting from abuse.

RateLimitFilter.java

src/main/java/com/qdischarge/clinicqueue/ratelimit/RateLimitFilter.java

A servlet filter that recognizes four specific routes — admin login, patient token creation, the WhatsApp webhook, and OTP send/verify — and applies a per-client-IP request cap to each (limits configurable via `AppProperties`). Everything else is left unthrottled. Wired explicitly into `SecurityConfig`'s filter chain rather than auto-registered, so it runs exactly once per request.

```
package com.qdischarge.clinicqueue.ratelimit;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.qdischarge.clinicqueue.config.AppProperties;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.MediaType;
import org.springframework.lang.NonNull;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.time.Duration;
import java.util.LinkedHashMap;
import java.util.Map;

/**
 * Guards the endpoints most worth protecting from abuse: admin login
 * (brute force), patient token creation, and the public WhatsApp webhook.
 * Everything else is left unthrottled, matching the original's behavior.
 *
 * Deliberately not a @Component: it's wired into SecurityConfig's filter
 * chain explicitly (see SecurityConfig#ratelimitFilter) so it runs exactly
 * once per request instead of also being auto-registered as a
 * container-level servlet filter.
 */
@RequiredArgsConstructor
public class RateLimitFilter extends OncePerRequestFilter {

    private final RateLimiterService rateLimiterService;
    private final AppProperties appProperties;
    private final ObjectMapper objectMapper;

    private static final Duration WINDOW = Duration.ofMinutes(1);

    @Override
    protected void doFilterInternal(@NonNull HttpServletRequest request,
                                    @NonNull HttpServletResponse response,
                                    @NonNull FilterChain filterChain) throws ServletException, IOException {
        String path = request.getRequestURI();
        String method = request.getMethod();

        Integer limit = null;
        String bucket = null;

        if ("POST".equals(method) && path.equals("/api/queue/login")) {
            limit = appProperties.getRateLimitLoginPerMinute();
            bucket = "login";
        } else if ("POST".equals(method) && path.equals("/api/queue")) {
            limit = appProperties.getRateLimitCreateTokenPerMinute();
            bucket = "create-token";
        } else if ("POST".equals(method) && path.equals("/webhook/whatsapp")) {
            limit = appProperties.getRateLimitWebhookPerMinute();
            bucket = "webhook";
        } else if ("POST".equals(method) && (path.equals("/api/auth/otp/send") || path.equals("/api/auth/otp/verify"))) {
            limit = appProperties.getRateLimitOtpPerMinute();
            bucket = "otp";
        }

        if (limit != null) {
            String key = bucket + ":" + clientIp(request);
```

```
        if (!rateLimiterService.tryAcquire(key, limit, WINDOW)) {
            writeTooManyRequests(response);
            return;
        }

        filterChain.doFilter(request, response);
    }

    private String clientIp(HttpServletRequest request) {
        String xForwardedFor = request.getHeader("X-Forwarded-For");
        if (xForwardedFor != null && !xForwardedFor.isBlank()) {
            return xForwardedFor.split(",")[0].trim();
        }
        return request.getRemoteAddr();
    }

    private void writeTooManyRequests(HttpServletResponse response) throws IOException {
        response.setStatus(HttpStatus.TOO_MANY_REQUESTS.value());
        response.setContentType(MediaType.APPLICATION_JSON_VALUE);
        Map<String, Object> body = new LinkedHashMap<>();
        body.put("success", false);
        body.put("message", "Too many requests. Please slow down and try again shortly.");
        objectMapper.writeValue(response.getWriter(), body);
    }
}
```

RateLimiterService.java

src/main/java/com/qdischarge/clinicqueue/ratelimit/RateLimiterService.java

The actual sliding-window algorithm: a `ConcurrentHashMap` of per-key timestamp dequeues, pruned of anything older than the window on each call. Good enough for a single instance; a multi-replica deployment would need a Redis-backed limiter to share the window across nodes.

```
package com.qdischarge.clinicqueue.ratelimit;

import org.springframework.stereotype.Service;

import java.time.Duration;
import java.time.Instant;
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.concurrent.ConcurrentHashMap;

/**
 * Simple in-memory sliding-window rate limiter, keyed per (client IP, route).
 * Good enough to blunt brute-force/abuse against a single instance; for a
 * multi-replica deployment, swap this for a Redis-backed limiter so the
 * window is shared across nodes (see README's scaling notes).
 */
@Service
public class RateLimiterService {

    private final ConcurrentHashMap<String, Deque<Instant>> windows = new ConcurrentHashMap<>();

    /** Returns true if the call is allowed under the given limit, recording it if so. */
    public boolean tryAcquire(String key, int maxRequests, Duration window) {
        Instant now = Instant.now();
        Instant cutoff = now.minus(window);

        Deque<Instant> timestamps = windows.computeIfAbsent(key, k -> new ArrayDeque<>());
        synchronized (timestamps) {
            while (!timestamps.isEmpty() && timestamps.peekFirst().isBefore(cutoff)) {
                timestamps.pollFirst();
            }
            if (timestamps.size() >= maxRequests) {
                return false;
            }
            timestamps.addLast(now);
            return true;
        }
    }
}
```

11. Package: security

Stateless JWT authentication/authorization for the whole API, plus the admin credential check. Everything the original Express app exposed without auth stays public here too; only genuinely admin/patient/doctor-scoped routes are locked down, declaratively, in `SecurityConfig`.

AdminAuthService.java

`src/main/java/com/qdischarge/clinicqueue/security/AdminAuthService.java`

Resolves the effective admin credential check: prefers a pre-computed BCrypt hash (`ADMIN_PASSWORD_HASH` , the production choice), falling back to hashing a plaintext `ADMIN_PASSWORD` once at startup for local/dev use. Either way, login is always a constant-time BCrypt comparison, never a plaintext `==` .

```
package com.qdischarge.clinicqueue.security;

import com.qdischarge.clinicqueue.config.AppProperties;
import jakarta.annotation.PostConstruct;
import lombok.RequiredArgsConstructor;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

import java.util.Objects;

/**
 * Resolves the effective admin credential check. Prefers ADMIN_PASSWORD_HASH
 * (a pre-computed BCrypt hash, the right choice for production); falls back
 * to hashing the plaintext ADMIN_PASSWORD once at startup for local/dev use
 * so the original .env-style config keeps working. Either way, the actual
 * login comparison is a constant-time BCrypt match, never a plaintext ==.
 */
@Service
@RequiredArgsConstructor
public class AdminAuthService {

    private final AppProperties appProperties;
    private final PasswordEncoder passwordEncoder;

    private String effectiveHash;

    @PostConstruct
    void init() {
        String configuredHash = appProperties.getAdminPasswordHash();
        if (configuredHash != null && !configuredHash.isBlank()) {
            effectiveHash = configuredHash;
        } else {
            String plaintext = appProperties.getAdminPassword() != null ? appProperties.getAdminPassword() : "";
            effectiveHash = passwordEncoder.encode(plaintext);
        }
    }

    public boolean authenticate(String username, String password) {
        if (username == null || password == null) {
            return false;
        }
        boolean usernameMatches = Objects.equals(username, appProperties.getAdminUsername());
        boolean passwordMatches = passwordEncoder.matches(password, effectiveHash);
        return usernameMatches && passwordMatches;
    }
}
```

CurrentUser.java

src/main/java/com/qdischarge/clinicqueue/security/CurrentUser.java

Pulls the JWT-decoded identity for the current request out of Spring Security's context, so controllers can identify "who's calling" — `requirePatientPhone()` or `requireDoctorId()` — without re-parsing the header or an extra DB lookup.

```
package com.qdischarge.clinicqueue.security;

import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;

/**
 * Pulls the JwtAuthFilter-decoded token (subject/role/doctorId) for the
 * current request out of the SecurityContext, so patient- and
 * doctor-scoped controllers can identify "who's calling" (their own phone
 * number, or their doctorId) without re-parsing the Authorization header or
 * doing an extra DB lookup.
 */
@Component
public class CurrentUser {

    /** Null if there's no authenticated caller on this request (public endpoint hit anonymously). */
    public JwtService.DecodedToken get() {
        Authentication auth = SecurityContextHolder.getContext().getAuthentication();
        if (auth == null || !(auth.getDetails() instanceof JwtService.DecodedToken decoded)) {
            return null;
        }
        return decoded;
    }

    /** The calling patient's phone number (JWT subject). Throws if there's no valid ROLE_PATIENT token -- SecurityConfig should already have
    rejected that request. */
    public String requirePatientPhone() {
        JwtService.DecodedToken decoded = get();
        if (decoded == null || !JwtService.PATIENT_ROLE.equals(decoded.role())) {
            throw new IllegalStateException("No authenticated patient on this request.");
        }
        return decoded.subject();
    }

    /** The calling doctor's id (JWT claim). Throws if there's no valid ROLE_DOCTOR token -- SecurityConfig should already have rejected that
    request. */
    public int requireDoctorId() {
        JwtService.DecodedToken decoded = get();
        if (decoded == null || !JwtService.DOCTOR_ROLE.equals(decoded.role()) || decoded.doctorId() == null) {
            throw new IllegalStateException("No authenticated doctor on this request.");
        }
        return decoded.doctorId();
    }
}
```

JwtAuthFilter.java

src/main/java/com/qdischarge/clinicqueue/security/JwtAuthFilter.java

Parses `Authorization: Bearer <jwt>` on every request and, if valid, populates the security context with a `ROLE_ADMIN` / `ROLE_PATIENT` / `ROLE_DOCTOR` authentication, stashing the decoded token (subject, role, doctorId) in the authentication's "details" slot for `CurrentUser` to read back.

```
package com.qdischarge.clinicqueue.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpHeaders;
import org.springframework.lang.NonNull;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.util.List;

/**
 * Parses "Authorization: Bearer <jwt>" and, if valid, populates the
 * SecurityContext with a ROLE_ADMIN / ROLE_PATIENT / ROLE_DOCTOR
 * authentication (see JwtService). The decoded token itself -- subject,
 * role, and doctorId for doctor tokens -- is stashed in the authentication's
 * "details" slot so controllers can pull it via CurrentUser without a second
 * DB lookup. Routes that don't require auth simply proceed with no
 * authentication set.
 */
@Component
@RequiredArgsConstructor
public class JwtAuthFilter extends OncePerRequestFilter {

    private final JwtService jwtService;

    @Override
    protected void doFilterInternal(@NonNull HttpServletRequest request,
                                    @NonNull HttpServletResponse response,
                                    @NonNull FilterChain filterChain) throws ServletException, IOException {
        String authHeader = request.getHeader(HttpHeaders.AUTHORIZATION);

        if (authHeader != null && authHeader.startsWith("Bearer ")) {
            String token = authHeader.substring(7);
            JwtService.DecodedToken decoded = jwtService.decode(token);
            if (decoded != null) {
                var authentication = new UsernamePasswordAuthenticationToken(
                    decoded.subject(), null, List.of(new SimpleGrantedAuthority("ROLE_" + decoded.role())));
                authentication.setDetails(decoded);
                SecurityContextHolder.getContext().setAuthentication(authentication);
            }
        }

        filterChain.doFilter(request, response);
    }
}
```

JwtService.java

src/main/java/com/qdischarge/clinicqueue/security/JwtService.java

Issues and validates signed, short-lived JWTs for all three caller kinds. Generates a random per-process signing key (with a loud warning) if `JWT_SECRET` isn't set — safe for a single dev instance, but explicitly called out as unsafe for a multi-instance deployment (every restart invalidates every session, and different instances wouldn't agree on a key).

```
package com.qdischarge.clinicqueue.security;

import com.qdischarge.clinicqueue.config.AppProperties;
import io.jsonwebtoken.Claims;
import io.jsonwebtoken.JwtException;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;
import jakarta.annotation.PostConstruct;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;

import javax.crypto.SecretKey;
import java.security.SecureRandom;
import java.time.Duration;
import java.time.Instant;
import java.util.Base64;
import java.util.Date;
import java.util.Map;

/**
 * Issues and validates signed, short-lived session tokens for all three
 * kinds of caller this API has: ROLE_ADMIN (reception/staff, unchanged from
 * the original single static ADMIN_TOKEN), ROLE_PATIENT (issued to a phone
 * number the moment its OTP is verified -- see OtpController), and
 * ROLE_DOCTOR (issued on doctor registration/login, carries a doctorId claim
 * so doctor-scoped endpoints know which doctor is calling without a second
 * DB lookup). The token itself is opaque to callers, which just store
 * whatever string comes back and replay it as a Bearer header.
 */
@Service
@RequiredArgsConstructor
@Slf4j
public class JwtService {

    private static final String ROLE_CLAIM = "role";
    private static final String DOCTOR_ID_CLAIM = "doctorId";

    public static final String ADMIN_ROLE = "ADMIN";
    public static final String PATIENT_ROLE = "PATIENT";
    public static final String DOCTOR_ROLE = "DOCTOR";

    private final AppProperties appProperties;
    private SecretKey signingKey;

    @PostConstruct
    void init() {
        String secret = appProperties.getJwtSecret();
        if (secret == null || secret.isBlank()) {
            log.warn("⚠️ JWT_SECRET is not set. Generating a random signing key for this process only -- "
                + "all sessions (admin/patient/doctor) will be invalidated on restart, and this is NOT safe "
                + "for a multi-instance deployment. Set JWT_SECRET explicitly in production.");
            byte[] random = new byte[64];
            new SecureRandom().nextBytes(random);
            signingKey = Keys.hmacShaKeyFor(random);
        } else {
            byte[] keyBytes = decodeSecret(secret);
            signingKey = Keys.hmacShaKeyFor(keyBytes);
        }
    }

    private byte[] decodeSecret(String secret) {
        try {
            byte[] decoded = Base64.getDecoder().decode(secret);
        }
    }
}
```

```

        if (decoded.length >= 32) {
            return decoded;
        }
    } catch (IllegalArgumentException ignored) {
        // not base64 -- fall through and use raw UTF-8 bytes
    }
    return secret.getBytes(java.nio.charset.StandardCharsets.UTF_8);
}

public String generateAdminToken(String username) {
    return generateToken(username, ADMIN_ROLE, Map.of());
}

/** Issued the moment a phone number's OTP is verified (see OtpController#verify). */
public String generatePatientToken(String phone) {
    return generateToken(phone, PATIENT_ROLE, Map.of());
}

/** Carries doctorId so doctor-scoped endpoints (e.g. GET /api/doctors/patients) don't need a phone->id lookup on every call. */
public String generateDoctorToken(String phone, int doctorId) {
    return generateToken(phone, DOCTOR_ROLE, Map.of(DOCTOR_ID_CLAIM, doctorId));
}

private String generateToken(String subject, String role, Map<String, Object> extraClaims) {
    Instant now = Instant.now();
    Instant expiry = now.plus(Duration.ofMinutes(appProperties.getJwtExpiryMinutes()));
    var builder = Jwts.builder()
        .subject(subject)
        .claim(ROLE_CLAIM, role)
        .issuedAt(Date.from(now))
        .expiration(Date.from(expiry));
    extraClaims.forEach(builder::claim);
    return builder.signWith(signingKey).compact();
}

/** Decoded token identity: subject (username for admin, phone for patient/doctor), role, and doctorId (doctor tokens only). */
public record DecodedToken(String subject, String role, Integer doctorId) {
}

/** Null if the token is missing, malformed, expired, or has an unrecognized role. */
public DecodedToken decode(String token) {
    try {
        Claims claims = Jwts.parser()
            .verifyWith(signingKey)
            .build()
            .parseSignedClaims(token)
            .getPayload();
        String role = String.valueOf(claims.get(ROLE_CLAIM));
        if (!ADMIN_ROLE.equals(role) && !PATIENT_ROLE.equals(role) && !DOCTOR_ROLE.equals(role)) {
            return null;
        }
        Integer doctorId = claims.get(DOCTOR_ID_CLAIM, Integer.class);
        return new DecodedToken(claims.getSubject(), role, doctorId);
    } catch (JwtException | IllegalArgumentException e) {
        return null;
    }
}

/** Back-compat convenience for existing admin-only call sites: subject if valid and ROLE_ADMIN, else null. */
public String validateAndGetSubject(String token) {
    DecodedToken decoded = decode(token);
    return (decoded != null && ADMIN_ROLE.equals(decoded.role())) ? decoded.subject() : null;
}
}

```

RestAccessDeniedHandler.java

src/main/java/com/qdischarge/clinicqueue/security/RestAccessDeniedHandler.java

A validly-authenticated caller without the required role gets a JSON 403, in the same response shape as every other error in the app.

```
package com.qdischarge.clinicqueue.security;

import com.fasterxml.jackson.databind.ObjectMapper;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.RequiredArgsConstructor;
import org.springframework.http.MediaType;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.web.access.AccessDeniedHandler;
import org.springframework.stereotype.Component;

import java.io.IOException;
import java.util.LinkedHashMap;
import java.util.Map;

/** A validly-authenticated caller without the required role -- same JSON error shape as the 401 case. */
@Component
@RequiredArgsConstructor
public class RestAccessDeniedHandler implements AccessDeniedHandler {

    private final ObjectMapper objectMapper;

    @Override
    public void handle(HttpServletRequest request, HttpServletResponse response, AccessDeniedException accessDeniedException)
        throws IOException {
        response.setStatus(HttpServletResponse.SC_FORBIDDEN);
        response.setContentType(MediaType.APPLICATION_JSON_VALUE);
        Map<String, Object> body = new LinkedHashMap<>();
        body.put("success", false);
        body.put("message", "Forbidden. Admin privileges are required for this action.");
        objectMapper.writeValue(response.getWriter(), body);
    }
}
```

RestAuthenticationEntryPoint.java

src/main/java/com/qdischarge/clinicqueue/security/RestAuthenticationEntryPoint.java

A missing/invalid token gets a JSON 401 — keeps the exact `{"success":false,"message":...}` shape the original Express middleware returned.

```
package com.qdischarge.clinicqueue.security;

import com.fasterxml.jackson.databind.ObjectMapper;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import lombok.RequiredArgsConstructor;
import org.springframework.http.MediaType;
import org.springframework.security.core.AuthenticationException;
import org.springframework.security.web.AuthenticationEntryPoint;
import org.springframework.stereotype.Component;

import java.io.IOException;
import java.util.LinkedHashMap;
import java.util.Map;

/** Keeps the same {"success":false,"message":"..."} JSON shape the original Express middleware returned on 401. */
@Component
@RequiredArgsConstructor
public class RestAuthenticationEntryPoint implements AuthenticationEntryPoint {

    private final ObjectMapper objectMapper;

    @Override
    public void commence(HttpServletRequest request, HttpServletResponse response, AuthenticationException authException)
        throws IOException {
        response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
        response.setContentType(MediaType.APPLICATION_JSON_VALUE);
        Map<String, Object> body = new LinkedHashMap<>();
        body.put("success", false);
        body.put("message", "Unauthorized. Admin token is missing or invalid.");
        objectMapper.writeValue(response.getWriter(), body);
    }
}
```

SecurityConfig.java

src/main/java/com/qdischarge/clinicqueue/security/SecurityConfig.java

The central security wiring: stateless sessions, CORS (allow-list from `ALLOWED_ORIGINS`), CSRF disabled (no cookies involved), security headers (frame-deny, HSTS), and the full explicit per-route role matrix — admin-only actions, doctor-only `/api/doctors/**` (except register/login themselves, which are OTP-gated instead), patient-only `/api/patients/**`, and everything else left public. Registers `RateLimitFilter` and `JwtAuthFilter` ahead of Spring's own authentication filter.

```
package com.qdischarge.clinicqueue.security;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.qdischarge.clinicqueue.config.AppProperties;
import com.qdischarge.clinicqueue.ratelimit.RateLimitFilter;
import com.qdischarge.clinicqueue.ratelimit.RateLimiterService;
import lombok.RequiredArgsConstructor;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.Arrays;
import java.util.List;

/**
 * Stateless, JWT-based security for the admin-only endpoints. Everything the
 * original queue.js/webhook.js exposed without auth (patient check-in, queue
 * reads, QR generation, the WhatsApp webhook) stays public; only
 * POST /api/queue/verify and PUT /api/queue/{id} require ROLE_ADMIN, exactly
 * like the original's authenticateAdmin middleware.
 */
@Configuration
@EnableWebSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final AppProperties appProperties;
    private final JwtAuthFilter jwtAuthFilter;
    private final RestAuthenticationEntryPoint authenticationEntryPoint;
    private final RestAccessDeniedHandler accessDeniedHandler;
    private final RateLimiterService rateLimiterService;
    private final ObjectMapper objectMapper;

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public RateLimitFilter rateLimitFilter() {
        return new RateLimitFilter(rateLimiterService, appProperties, objectMapper);
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .cors(cors -> cors.configurationSource(corsConfigurationSource()))
            .csrf(csrf -> csrf.disable()) // stateless REST API, no cookies/session
            .sessionManagement(sm -> sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .exceptionHandling(ex -> ex
                .authenticationEntryPoint(authenticationEntryPoint)
                .accessDeniedHandler(accessDeniedHandler))
    }
}
```

```

.headers(headers -> headers
    .contentTypeOptions(opts -> {})
    .frameOptions(frame -> frame.deny())
    .httpStrictTransportSecurity(hsts -> hsts.includeSubDomains(true)))
.authorizeHttpRequests(auth -> auth
    // NOTE: JwtAuthFilter now authenticates three token kinds (ROLE_ADMIN/
    // ROLE_PATIENT/ROLE_DOCTOR, see JwtService), so every admin-only route below
    // must say hasRole("ADMIN") explicitly -- a bare authenticated() would also
    // accept a valid patient or doctor token, which is not what any of these mean.
    .requestMatchers(HttpMethod.POST, "/api/queue/verify").hasRole("ADMIN")
    .requestMatchers(HttpMethod.PUT, "/api/queue/*").hasRole("ADMIN")
    // Missed-queue admin panel: staff-only search/requeue/reject over PII.
    .requestMatchers("/api/queue/missed", "/api/queue/missed/**").hasRole("ADMIN")
    // Per-phone visit history can span more than one patient's name/age -- staff-only.
    .requestMatchers("/api/queue/history/**").hasRole("ADMIN")
    // Hospital directory reads (location/URI/OPD hours) are public; creating,
    // relocating a hospital, or reading/rotating its doctor join code are admin actions.
    .requestMatchers(HttpMethod.POST, "/api/hospitals").hasRole("ADMIN")
    .requestMatchers(HttpMethod.PUT, "/api/hospitals/*/location").hasRole("ADMIN")
    .requestMatchers(HttpMethod.PUT, "/api/hospitals/*/departments/*/counters").hasRole("ADMIN")
    .requestMatchers("/api/hospitals/*/doctor-join-code", "/api/hospitals/*/doctor-join-code/**").hasRole("ADMIN")
    // Hospital assigns one of its doctors to a counter/department location.
    .requestMatchers(HttpMethod.PUT, "/api/hospitals/*/doctors/*/location").hasRole("ADMIN")
    // Time-slot reads (a hospital's OPD schedule) are public; opening one is admin-only.
    .requestMatchers(HttpMethod.POST, "/api/hospitals/*/time-slots").hasRole("ADMIN")
    // Multi-counter admin actions (complete/miss/reassign a counter).
    .requestMatchers(HttpMethod.POST, "/api/counters/**").hasRole("ADMIN")
    // Admin pushes a called-but-absent patient back in their own queue (exponential backoff).
    .requestMatchers(HttpMethod.POST, "/api/queue/*/no-show").hasRole("ADMIN")
    // Doctor registration/login prove identity via OTP (see OtpController), not a
    // doctor JWT yet -- everything else a doctor does needs the token that returns.
    .requestMatchers(HttpMethod.POST, "/api/doctors/register", "/api/doctors/login").permitAll()
    .requestMatchers("/api/doctors/**").hasRole("DOCTOR")
    // Every /api/patients/** action is self-service on the caller's own phone
    // number (identified from the token, see CurrentUser) -- there's no "public"
    // patient endpoint here, login *is* OtpController's /verify.
    .requestMatchers("/api/patients/**").hasRole("PATIENT")
    .requestMatchers("/actuator/health/**", "/actuator/info").permitAll()
    .requestMatchers("/actuator/**").hasRole("ADMIN")
    .anyRequest().permitAll())
.addFilterBefore(rateLimitFilter(), UsernamePasswordAuthenticationFilter.class)
.addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);

return http.build();
}

@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration configuration = new CorsConfiguration();
    List<String> origins = Arrays.stream(appProperties.getAllowedOrigins().split(","))
        .map(String::trim)
        .filter(s -> !s.isEmpty())
        .toList();
    configuration.setAllowedOrigins(origins);
    configuration.setAllowedOriginPatterns(List.of(""));
    configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS", "PATCH"));
    configuration.setAllowedHeaders(List.of(""));
    configuration.setAllowCredentials(false);

    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", configuration);
    return source;
}
}

```

12. Package: service

The business logic layer — where almost every SQL statement in the app actually lives. This is the largest package, dominated by `QueueManagerService`, the queue engine everything else calls into.

AccessService.java

src/main/java/com/qdischarge/clinicqueue/service/AccessService.java

The doctor <-> patient consent model: generates a short-lived access request (code + QR) for a doctor, lets a patient's device claim it (creating a revocable grant and firing a WhatsApp notice), and handles revocation — enforced by a `WHERE patient_phone = :phone` clause, not just a caller check, so only the granting patient can ever revoke their own grant.

```
package com.qdischarge.clinicqueue.service;

import com.qdischarge.clinicqueue.bot.BotMessages;
import com.qdischarge.clinicqueue.bot.Lang;
import com.qdischarge.clinicqueue.bot.WaSessionService;
import com.qdischarge.clinicqueue.dto.AccessGrantDto;
import com.qdischarge.clinicqueue.dto.AccessRequestDto;
import lombok.RequiredArgsConstructor;
import org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate;
import org.springframework.stereotype.Service;

import java.security.SecureRandom;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Timestamp;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Map;

/**
 * The doctor <-> patient consent model: a doctor generates a short-lived
 * access *request* (code + QR), a patient's device claims it, which creates
 * a revocable access *grant* and fires a WhatsApp notice the patient can
 * reply to (see BotMessages#accessGranted / WebhookController's "revoke"
 * command) to undo it immediately if it wasn't them.
 */
@Service
@RequiredArgsConstructor
public class AccessService {

    private final NamedParameterJdbcTemplate jdbc;
    private final WhatsAppService whatsappService;
    private final WaSessionService waSessionService;
    private final BotMessages botMessages;

    private static final SecureRandom RANDOM = new SecureRandom();
    private static final String CODE_ALPHABET = "ABCDEFGHJKLMNPQRSTUVWXYZ23456789";
    private static final int CODE_LENGTH = 8;
    private static final int REQUEST_EXPIRY_MINUTES = 30;

    public AccessRequestDto createRequest(int doctorId) {
        String code = generateCode();
        LocalDateTime expiresAt = LocalDateTime.now().plusMinutes(REQUEST_EXPIRY_MINUTES);
        Integer id = jdbc.queryForObject(
            "INSERT INTO access_requests (code, doctor_id, expires_at) VALUES (:code, :doctorId, :expiresAt) RETURNING id",
            Map.of("code", code, "doctorId", doctorId, "expiresAt", expiresAt), Integer.class);
        return AccessRequestDto.builder()
            .id(id).code(code).doctorId(doctorId).status("pending")
            .createdAt(LocalDateTime.now()).expiresAt(expiresAt).build();
    }

    /** Looks up a pending request by its code -- used to render the QR PNG without needing the numeric id. */
    public AccessRequestDto getByCode(String code) {
        List<AccessRequestDto> rows = jdbc.query(
            "SELECT id, code, doctor_id, status, created_at, expires_at FROM access_requests WHERE code = :code",
            Map.of("code", code), AccessService::mapRequest);
        return rows.isEmpty() ? null : rows.get(0);
    }

    /** Patient's device claims a doctor's access code -- creates the grant and notifies the patient over WhatsApp. */
    public AccessGrantDto claim(String code, String patientPhone) {
        List<Map<String, Object>> rows = jdbc.queryForList(
            "SELECT id, doctor_id, status, expires_at FROM access_requests WHERE code = :code",
            Map.of("code", code.trim().toUpperCase()));
    }
}
```

```

    if (rows.isEmpty()) {
        throw new IllegalArgumentException("Invalid access code.");
    }
    Map<String, Object> req = rows.get(0);
    int requestId = (Integer) req.get("id");
    String status = (String) req.get("status");
    LocalDateTime expiresAt = ((Timestamp) req.get("expires_at")).toLocalDateTime();
    int doctorId = (Integer) req.get("doctor_id");

    if (!"pending".equals(status) || expiresAt.isBefore(LocalDateTime.now())) {
        throw new IllegalStateException("This access code has expired or was already used.");
    }

    jdbc.update("UPDATE access_requests SET status = 'claimed', claimed_by_phone = :phone, claimed_at = NOW() WHERE id = :id",
        Map.of("phone", patientPhone, "id", requestId));

    Integer grantId = jdbc.queryForObject(
        "INSERT INTO access_grants (doctor_id, patient_phone, source_request_id) VALUES (:doctorId, :phone, :requestId) RETURNING id",
        Map.of("doctorId", doctorId, "phone", patientPhone, "requestId", requestId), Integer.class);

    AccessGrantDto grant = getGrant(grantId);
    notifyPatientOfNewAccess(grant);
    return grant;
}

/** Only the granting patient can revoke -- ownership is enforced by the WHERE clause, not just the caller. */
public boolean revoke(int grantId, String patientPhone) {
    int updated = jdbc.update(
        "UPDATE access_grants SET revoked_at = NOW(), revoked_by = 'patient' WHERE id = :id AND patient_phone = :phone AND revoked_at IS NULL",
        Map.of("id", grantId, "phone", patientPhone));
    return updated > 0;
}

public List<AccessGrantDto> listActiveForPatient(String phone) {
    return jdbc.query(JOINED_GRANT_SELECT + " WHERE g.patient_phone = :phone AND g.revoked_at IS NULL ORDER BY g.granted_at DESC",
        Map.of("phone", phone), AccessService::mapGrant);
}

/** Every grant this phone has ever issued, active or revoked -- "history of accessed doctors". */
public List<AccessGrantDto> listHistoryForPatient(String phone) {
    return jdbc.query(JOINED_GRANT_SELECT + " WHERE g.patient_phone = :phone ORDER BY g.granted_at DESC",
        Map.of("phone", phone), AccessService::mapGrant);
}

/** Phone numbers this doctor currently has active (non-revoked) access to -- feeds PatientDocumentService#groupByIdentity. */
public List<String> listActivePatientPhonesForDoctor(int doctorId) {
    return jdbc.query(
        "SELECT DISTINCT patient_phone FROM access_grants WHERE doctor_id = :doctorId AND revoked_at IS NULL",
        Map.of("doctorId", doctorId), (rs, i) -> rs.getString("patient_phone"));
}

private AccessGrantDto getGrant(int id) {
    List<AccessGrantDto> rows = jdbc.query(JOINED_GRANT_SELECT + " WHERE g.id = :id", Map.of("id", id), AccessService::mapGrant);
    return rows.isEmpty() ? null : rows.get(0);
}

private void notifyPatientOfNewAccess(AccessGrantDto grant) {
    WaSessionService.WaSession session = waSessionService.get(grant.getPatientPhone());
    Lang lang = (session != null && session.language() != null) ? session.language() : Lang.EN;
    whatsappService.sendWhatsAppMessage(grant.getPatientPhone(),
        botMessages.accessGranted(lang, grant.getDoctorName(), grant.getHospitalName(), grant.getId()));
}

private String generateCode() {
    StringBuilder sb = new StringBuilder(CODE_LENGTH);
    for (int i = 0; i < CODE_LENGTH; i++) {
        sb.append(CODE_ALPHABET.charAt(RANDOM.nextInt(CODE_ALPHABET.length())));
    }
    return sb.toString();
}

private static final String JOINED_GRANT_SELECT = ""
    SELECT g.id, g.doctor_id, d.name AS doctor_name, h.name AS hospital_name,
        g.patient_phone, g.granted_at, g.revoked_at, g.revoked_by

```

```
FROM access_grants g
JOIN doctors d ON d.id = g.doctor_id
LEFT JOIN hospitals h ON h.id = d.hospital_id
""";

private static AccessGrantDto mapGrant(ResultSet rs, int rowNum) throws SQLException {
    Timestamp grantedAt = rs.getTimestamp("granted_at");
    Timestamp revokedAt = rs.getTimestamp("revoked_at");
    return AccessGrantDto.builder()
        .id(rs.getInt("id"))
        .doctorId(rs.getInt("doctor_id"))
        .doctorName(rs.getString("doctor_name"))
        .hospitalName(rs.getString("hospital_name"))
        .patientPhone(rs.getString("patient_phone"))
        .grantedAt(grantedAt != null ? grantedAt.toLocalDateTime() : null)
        .revokedAt(revokedAt != null ? revokedAt.toLocalDateTime() : null)
        .revokedBy(rs.getString("revoked_by"))
        .build();
}

private static AccessRequestDto mapRequest(ResultSet rs, int rowNum) throws SQLException {
    Timestamp createdAt = rs.getTimestamp("created_at");
    Timestamp expiresAt = rs.getTimestamp("expires_at");
    return AccessRequestDto.builder()
        .id(rs.getInt("id"))
        .code(rs.getString("code"))
        .doctorId(rs.getInt("doctor_id"))
        .status(rs.getString("status"))
        .createdAt(createdAt != null ? createdAt.toLocalDateTime() : null)
        .expiresAt(expiresAt != null ? expiresAt.toLocalDateTime() : null)
        .build();
}
}
```

CounterAssignmentService.java

src/main/java/com/qdischarge/clinicqueue/service/CounterAssignmentService.java

The multi-counter package's dynamic look-ahead assignment logic, active once a department's `active_counters > 1`. Implements the product spec's worked example precisely: when one counter frees up, only *that* counter gets refilled, others keep serving what they already had. Also runs `refreshReservations()`, an advisory-only mechanism that earmarks upcoming eligible patients to each busy counter and pings them "you're up next for Counter X" ahead of time — never the actual authority on who's served next, that's still `QueueManagerService`'s claim logic.

```
package com.qdischarge.clinicqueue.service;

import com.qdischarge.clinicqueue.dto.TokenDto;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.jdbc.core.BeanPropertyRowMapper;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Advanced/multi-counter package: dynamic look-ahead counter assignment.
 * Same "tokens" table and the same base queue (single source of truth,
 * intentionally *not* a forked copy of the schema/service -- see README) --
 * this is what turns on once a (hospital, category) department's
 * active_counters > 1 (see HospitalDepartmentService). Counter numbers are
 * scoped to that department: "Counter 1" of Cardiology and "Counter 1" of
 * Orthopaedics at the same hospital are two different physical counters,
 * every query below is scoped by (hospitalId, category) alongside counterId.
 *
 * Behavior, matching the product spec's worked example for 4 counters:
 * tokens 1-4 are assigned to counters 1-4 to start. When counter 3 (serving
 * token 7, i.e. the batch 5-8) finishes, only counter 3 is refilled with the
 * next queued token (9) -- counters 1, 2, 4 keep serving whatever they
 * already had (5, 6, 8), giving 5, 6, 9, 8. A token that doesn't show up is
 * left alone (admin "waits") until the admin explicitly calls missAtCounter,
 * which frees that counter for the next eligible token exactly like a normal
 * completion would.
 *
 * Look-Ahead Window = 2 x active counters (informational: how many upcoming
 * waiting tokens the admin display board should show as "on deck").
 *
 * Reservations (see refreshReservations): while counters A, B, C are all mid-
 * treatment, a distinct upcoming eligible token is earmarked to each one
 * ("reserved_counter_id") and pinged "you're up next for Counter X" -- purely
 * advisory display/notification, layered on top of the actual claim logic in
 * QueueManagerService, which alone still decides who is genuinely served next
 * once a counter frees up.
 */
@Service
@RequiredArgsConstructor
@Slf4j
public class CounterAssignmentService {

    private final NamedParameterJdbcTemplate jdbc;
    private final QueueManagerService queueManagerService;
    private final HospitalDepartmentService hospitalDepartmentService;
    private final WhatsAppService whatsappService;

    private static final RowMapper<TokenDto> TOKEN_ROW_MAPPER = new BeanPropertyRowMapper<>(TokenDto.class);

    public record CounterState(int counterId, TokenDto servingToken) {
    }

    /** One busy counter's earmarked upcoming patient -- see refreshReservations. */
}
```

```

public record CounterReservation(int counterId, TokenDto token) {
}

public record CounterBoard(int activeCounters, List<CounterState> counters, List<TokenDto> lookAhead,
    List<CounterReservation> upcoming) {
}

public CounterBoard getBoard(int hospitalId, String category) {
    int activeCounters = hospitalDepartmentService.activeCounters(hospitalId, category);
    ensureInitialAssignment(hospitalId, category, activeCounters);
    refreshReservations(hospitalId, category, activeCounters);

    List<CounterState> counters = new ArrayList<>();
    for (int c = 1; c <= activeCounters; c++) {
        List<TokenDto> rows = jdbc.query(
            """
            SELECT * FROM tokens WHERE status = 'serving' AND counter_id = :c
            AND hospital_id = :hospitalId AND category = :category LIMIT 1
            """,
            Map.of("c", c, "hospitalId", hospitalId, "category", category), TOKEN_ROW_MAPPER);
        counters.add(new CounterState(c, rows.isEmpty() ? null : rows.get(0)));
    }

    int lookAheadSize = 2 * activeCounters;
    List<TokenDto> lookAhead = jdbc.query(
        """
        SELECT * FROM tokens WHERE status = 'waiting' AND hospital_id = :hospitalId AND category = :category
        ORDER BY COALESCE(priority_rank, id) ASC LIMIT :n
        """,
        Map.of("hospitalId", hospitalId, "category", category, "n", lookAheadSize), TOKEN_ROW_MAPPER);

    List<CounterReservation> upcoming = jdbc.query(
        """
        SELECT * FROM tokens WHERE status = 'waiting' AND reserved_counter_id IS NOT NULL
        AND hospital_id = :hospitalId AND category = :category
        ORDER BY reserved_counter_id ASC
        """,
        Map.of("hospitalId", hospitalId, "category", category),
        (rs, rowNum) -> new CounterReservation(rs.getInt("reserved_counter_id"), TOKEN_ROW_MAPPER.mapRow(rs, rowNum)));

    return new CounterBoard(activeCounters, counters, lookAhead, upcoming);
}

/** Counter finished normally: token -> completed, next eligible waiting token takes the freed counter. */
public CounterBoard completeAtCounter(int hospitalId, String category, int counterId) {
    finishCounter(hospitalId, category, counterId, "completed");
    claimIntoCounter(hospitalId, category, counterId);
    return getBoard(hospitalId, category);
}

/** Patient at this counter didn't show (admin gives up waiting): token -> missed, freed counter pulls the next one. */
public CounterBoard missAtCounter(int hospitalId, String category, int counterId) {
    finishCounter(hospitalId, category, counterId, "missed");
    claimIntoCounter(hospitalId, category, counterId);
    return getBoard(hospitalId, category);
}

/** First-time setup: nobody assigned to any counter yet in this department -> seed counters 1..N off the front of its queue. */
private void ensureInitialAssignment(int hospitalId, String category, int activeCounters) {
    Integer alreadyAssigned = jdbc.queryForObject(
        """
        SELECT COUNT(*)::int FROM tokens
        WHERE status = 'serving' AND counter_id IS NOT NULL AND hospital_id = :hospitalId AND category = :category
        """,
        Map.of("hospitalId", hospitalId, "category", category), Integer.class);
    if (alreadyAssigned != null && alreadyAssigned > 0) {
        return;
    }
    for (int c = 1; c <= activeCounters; c++) {
        claimIntoCounter(hospitalId, category, c);
    }
}

private void finishCounter(int hospitalId, String category, int counterId, String status) {
    String timeColumn = "completed".equals(status) ? "completed_at" : "missed_at";

```

```

Map<String, Object> params = new HashMap<>();
params.put("status", status);
params.put("c", counterId);
params.put("hospitalId", hospitalId);
params.put("category", category);
List<TokenDto> rows = jdbc.query(
    "UPDATE tokens SET status = :status, " + timeColumn + " = NOW(), counter_id = NULL "
    + "WHERE counter_id = :c AND status = 'serving' AND hospital_id = :hospitalId AND category = :category RETURNING *",
    params, TOKEN_ROW_MAPPER);
if (rows.isEmpty()) {
    return;
}
TokenDto token = rows.get(0);
if ("completed".equals(status)) {
    whatsappService.sendWhatsAppMessage(token.getPhone(),
        "✅ Token #" + token.displayNumber() + " has been served. Thank you for visiting! 🙏");
} else {
    whatsappService.sendWhatsAppMessage(token.getPhone(),
        "⚠️ You missed your turn at Counter " + counterId + " for Token #" + token.displayNumber()
        + ".\n\n🔴 Please send \"Hi\" again to generate a new token.");
}
}

private void claimIntoCounter(int hospitalId, String category, int counterId) {
    // The multi-counter variant (unlike single-counter's) reaches past a
    // still-graced front-of-line token instead of blocking every counter
    // on it -- see QueueManagerService#claimNextEligibleWaitingTokenForCounter.
    TokenDto claimed = queueManagerService.claimNextEligibleWaitingTokenForCounter(hospitalId, category);
    if (claimed == null) {
        return;
    }
    // reserved_counter_id = NULL is defensive: a status='waiting' row can
    // carry a reservation, but the instant it's claimed it's no longer
    // 'waiting' anyway, so this just avoids a stale tag lingering on the row.
    jdbc.update("UPDATE tokens SET counter_id = :c, reserved_counter_id = NULL WHERE id = :id",
        Map.of("c", counterId, "id", claimed.getId()));
    whatsappService.sendWhatsAppMessage(claimed.getPhone(),
        "🎉 It's your turn! Please proceed to Counter " + counterId + " for Token #" + claimed.displayNumber() + ".");

    int activeCounters = hospitalDepartmentService.activeCounters(hospitalId, category);
    refreshReservations(hospitalId, category, activeCounters);
}

/**
 * Advisory only -- never changes who's actually served next (that's still
 * QueueManagerService#claimNextEligibleWaitingToken's call, at the moment
 * a counter genuinely frees up). While multiple counters are
 * simultaneously mid-treatment, this earmarks a *distinct* upcoming
 * eligible waiting token to each busy counter that doesn't already have
 * one, and pings that patient "you're up next for Counter X" so they can
 * be ready before their counter is even free -- the "next 3 patients set
 * at A, B, C" behavior. A counter that already has a valid reservation is
 * left untouched (no notification churn); an existing reservation is
 * dropped -- and its counter re-filled -- only once its held token is no
 * longer waiting, or fails the same unverified/no-longer-graced
 * eligibility check a real claim would apply. Called after every claim
 * (so a newly-busy counter gets a reservation right away) and on every
 * board read (so a reservation invalidated by e.g. a manual reposition --
 * see QueueManagerService#movePatientToPosition -- gets refilled even
 * without another claim event).
 */
private void refreshReservations(int hospitalId, String category, int activeCounters) {
    List<Integer> busyCounters = jdbc.query(
        """
        SELECT DISTINCT counter_id FROM tokens WHERE status = 'serving' AND counter_id IS NOT NULL
        AND counter_id <= :activeCounters AND hospital_id = :hospitalId AND category = :category
        """,
        Map.of("activeCounters", activeCounters, "hospitalId", hospitalId, "category", category),
        (rs, rowNum) -> rs.getInt("counter_id"));
    if (busyCounters.isEmpty()) {
        return; // nobody's actually mid-treatment -- nothing to earmark ahead for
    }

    // Existing reservations: an unverified token whose grace has lapsed
    // (or was never notified/graced at all) would be skipped-and-resolved

```

```

// by a real claim just like at claim time -- drop the reservation so
// its counter gets re-filled below.
List<Map<String, Object>> existing = jdbc.queryForList(
    """
    SELECT id, reserved_counter_id, is_verified, notified_ready_at FROM tokens
    WHERE status = 'waiting' AND reserved_counter_id IS NOT NULL
    AND hospital_id = :hospitalId AND category = :category
    """,
    Map.of("hospitalId", hospitalId, "category", category));

List<Integer> countersStillValid = new ArrayList<>();
for (Map<String, Object> row : existing) {
    boolean verified = Boolean.TRUE.equals(row.get("is_verified"));
    boolean wasNotified = row.get("notified_ready_at") != null;
    if (wasNotified && !verified) {
        jdbc.update("UPDATE tokens SET reserved_counter_id = NULL WHERE id = :id", Map.of("id", row.get("id")));
    } else {
        countersStillValid.add((Integer) row.get("reserved_counter_id"));
    }
}

List<Integer> needsReservation = new ArrayList<>();
for (Integer c : busyCounters) {
    if (!countersStillValid.contains(c) && !needsReservation.contains(c)) {
        needsReservation.add(c);
    }
}
if (needsReservation.isEmpty()) {
    return;
}
Collections.sort(needsReservation);

List<Map<String, Object>> candidates = jdbc.queryForList(
    """
    SELECT id, phone, is_verified, notified_ready_at FROM tokens
    WHERE status = 'waiting' AND reserved_counter_id IS NULL
    AND hospital_id = :hospitalId AND category = :category
    ORDER BY COALESCE(priority_rank, id) ASC
    """,
    Map.of("hospitalId", hospitalId, "category", category));

int nextCounterIdx = 0;
for (Map<String, Object> candidate : candidates) {
    if (nextCounterIdx >= needsReservation.size()) {
        break;
    }
    boolean verified = Boolean.TRUE.equals(candidate.get("is_verified"));
    boolean wasNotified = candidate.get("notified_ready_at") != null;
    if (wasNotified && !verified) {
        continue; // same eligibility skip as a real claim -- not ready to be earmarked yet
    }
    int counterId = needsReservation.get(nextCounterIdx++);
    int tokenId = (Integer) candidate.get("id");
    // Guarded like every other claim/finish update in this class --
    // status='waiting' AND reserved_counter_id IS NULL still true --
    // so a concurrent refreshReservations run (e.g. two admin actions
    // landing close together) can't double-book the same token onto
    // two counters, and we only notify once we know we actually won.
    int updated = jdbc.update(
        "UPDATE tokens SET reserved_counter_id = :c WHERE id = :id AND status = 'waiting' AND reserved_counter_id IS NULL",
        Map.of("c", counterId, "id", tokenId));
    if (updated == 0) {
        nextCounterIdx--; // lost the race -- this counter's slot is still open, retry it on the next candidate
        continue;
    }
    whatsappService.sendWhatsAppMessage((String) candidate.get("phone"),
        "🔔 You're up next for Counter " + counterId + "! Please be ready -- we'll call you there shortly.");
}
}
}

```

HospitalDepartmentService.java

src/main/java/com/qdischarge/clinicqueue/service/HospitalDepartmentService.java

Manages `hospital_departments` — one row per (hospital, category) queue, defaulting to 1 counter when not explicitly configured, created on demand rather than requiring an explicit admin setup step first.

```
package com.qdischarge.clinicqueue.service;

import com.qdischarge.clinicqueue.dto.HospitalDepartmentDto;
import lombok.RequiredArgsConstructor;
import org.springframework.jdbc.core.BeanPropertyRowMapper;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Map;

/**
 * hospital_departments: one row per (hospital, category) queue the
 * "multi-counter package" and QueueManagerService's department-scoped
 * queries key off of -- how many counters staff that department. A missing
 * row just means "not configured yet", defaulting to 1 counter; rows are
 * created on demand (see {@link #ensure}) rather than requiring an explicit
 * admin setup step before a department can be booked into.
 */
@Service
@RequiredArgsConstructor
public class HospitalDepartmentService {

    private final NamedParameterJdbcTemplate jdbc;

    private static final RowMapper<HospitalDepartmentDto> ROW_MAPPER = new BeanPropertyRowMapper<>(HospitalDepartmentDto.class);

    /** Idempotent: creates the (hospital, category) row with default active_counters=1 if it doesn't exist yet. */
    public void ensure(int hospitalId, String category) {
        jdbc.update(
            "INSERT INTO hospital_departments (hospital_id, category) VALUES (:hospitalId, :category) " +
            "ON CONFLICT (hospital_id, category) DO NOTHING",
            Map.of("hospitalId", hospitalId, "category", category));
    }

    /** Called whenever a hospital's offered categories are set/updated (HospitalService#create). */
    public void syncFromHospital(int hospitalId, List<String> categories) {
        if (categories == null) {
            return;
        }
        for (String category : categories) {
            ensure(hospitalId, category);
        }
    }

    public HospitalDepartmentDto get(int hospitalId, String category) {
        List<HospitalDepartmentDto> rows = jdbc.query(
            "SELECT * FROM hospital_departments WHERE hospital_id = :hospitalId AND category = :category",
            Map.of("hospitalId", hospitalId, "category", category), ROW_MAPPER);
        return rows.isEmpty() ? null : rows.get(0);
    }

    /** Active counters for this department, defaulting to 1 if it hasn't been configured yet. */
    public int activeCounters(int hospitalId, String category) {
        HospitalDepartmentDto dept = get(hospitalId, category);
        return dept != null ? Math.max(1, dept.getActiveCounters()) : 1;
    }

    public List<HospitalDepartmentDto> list(int hospitalId) {
        return jdbc.query("SELECT * FROM hospital_departments WHERE hospital_id = :hospitalId ORDER BY category ASC",
            Map.of("hospitalId", hospitalId), ROW_MAPPER);
    }

    public HospitalDepartmentDto updateCounters(int hospitalId, String category, int activeCounters) {
```

```
ensure(hospitalId, category);
List<HospitalDepartmentDto> rows = jdbc.query(
    """
        UPDATE hospital_departments SET active_counters = :activeCounters, updated_at = NOW()
        WHERE hospital_id = :hospitalId AND category = :category
        RETURNING *
    """,
    Map.of("activeCounters", Math.max(1, activeCounters), "hospitalId", hospitalId, "category", category),
    ROW_MAPPER);
return rows.isEmpty() ? null : rows.get(0);
}
```

HospitalService.java

src/main/java/com/qdischarge/clinicqueue/service/HospitalService.java

The hospital directory: DIGIPIN-based location resolution, nearest-first search (filtered by category and gender-restriction, paginated 20 at a time), and the upsert-with-COALESCE `create()` that lets `HospitalSeedRunner` / `MumbaiHospitalDataSeeder` re-run on every boot without ever clobbering an admin-edited profile field or a doctor's saved join code.

```
package com.qdischarge.clinicqueue.service;

import com.qdischarge.clinicqueue.catalog.MedicalCategory;
import com.qdischarge.clinicqueue.config.AppProperties;
import com.qdischarge.clinicqueue.dto.CreateHospitalRequest;
import com.qdischarge.clinicqueue.dto.HospitalDto;
import com.qdischarge.clinicqueue.dto.SetLocationRequest;
import com.qdischarge.clinicqueue.geo.DigipinService;
import com.qdischarge.clinicqueue.geo.GeoDistanceService;
import lombok.RequiredArgsConstructor;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate;
import org.springframework.stereotype.Service;

import java.security.SecureRandom;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Comparator;
import java.util.List;
import java.util.Map;

/**
 * Hospital directory: location (DIGIPIN-based), OPD hours, counter count,
 * and profile (ownership/year established/accreditation/gender-specific/
 * offered categories -- see catalog.MedicalCategory). A hospital's
 * "location" is deliberately just a DIGIPIN + the lat/lon it decodes to --
 * settable either directly as a DIGIPIN or as raw lat/lon (a DIGIPIN is then
 * derived), which is what "setting the location to a hospital URI" means in
 * practice: every hospital has a stable uriSlug and a DIGIPIN address.
 */
@Service
@RequiredArgsConstructor
public class HospitalService {

    private final NamedParameterJdbcTemplate jdbc;
    private final DigipinService digipinService;
    private final GeoDistanceService geoDistanceService;
    private final HospitalDepartmentService hospitalDepartmentService;
    private final AppProperties appProperties;

    /**
     * categories/accreditation are stored as comma-separated TEXT (not a
     * Postgres array -- see schema.sql), so HospitalDto can't be mapped
     * with a plain BeanPropertyRowMapper.
     */
    private static final RowMapper<HospitalDto> ROW_MAPPER = (rs, rowNum) -> HospitalDto.builder()
        .id(rs.getInt("id"))
        .uriSlug(rs.getString("uri_slug"))
        .name(rs.getString("name"))
        .address(rs.getString("address"))
        .digipin(rs.getString("digipin"))
        .latitude(rs.getDouble("latitude"))
        .longitude(rs.getDouble("longitude"))
        .openTime(rs.getObject("open_time", LocalTime.class))
        .closeTime(rs.getObject("close_time", LocalTime.class))
        .avgServiceMinutes(rs.getInt("avg_service_minutes"))
        .minServiceMinutes((Integer) rs.getObject("min_service_minutes"))
        .activeCounters(rs.getInt("active_counters"))
        .ownership(rs.getString("ownership"))
        .yearEstablished((Integer) rs.getObject("year_established"))
        .accreditation(splitCsv(rs.getString("accreditation")))
        .genderSpecific(rs.getString("gender_specific"))
```

```

        .categories(splitCsv(rs.getString("categories")))
        .createdAt(rs.getObject("created_at", LocalDateTime.class))
        .updatedAt(rs.getObject("updated_at", LocalDateTime.class))
        .build();

/** Excludes 0/0/1/I/L -- easy to read aloud/type when handing a doctor their hospital's join code. */
private static final String JOIN_CODE_ALPHABET = "ABCDEFGHJKLMNPQRSTUVWXYZ23456789";
private static final SecureRandom RANDOM = new SecureRandom();

public HospitalDto getBySlug(String uriSlug) {
    List<HospitalDto> rows = jdbc.query(
        "SELECT * FROM hospitals WHERE uri_slug = :slug", Map.of("slug", uriSlug), ROW_MAPPER);
    return finish(rows.isEmpty() ? null : rows.get(0));
}

public HospitalDto getById(int id) {
    List<HospitalDto> rows = jdbc.query(
        "SELECT * FROM hospitals WHERE id = :id", Map.of("id", id), ROW_MAPPER);
    return finish(rows.isEmpty() ? null : rows.get(0));
}

/** The hospital this single-tenant deployment operates as (app.hospital-uri-slug). */
public HospitalDto getOperatingHospital() {
    HospitalDto hospital = getBySlug(appProperties.getHospitalUriSlug());
    if (hospital == null) {
        // schema.sql always seeds a 'main' row; if a custom slug was configured
        // without creating that hospital yet, fall back to it so the single-
        // counter flows (which need *a* hospital) never hard-fail.
        hospital = getBySlug("main");
    }
    return hospital;
}

public List<HospitalDto> list() {
    return jdbc.query("SELECT * FROM hospitals ORDER BY name ASC", ROW_MAPPER).stream()
        .map(this::finish).toList();
}

public record HospitalMatch(HospitalDto hospital, double distanceKm) {
}

public record HospitalSearchPage(List<HospitalMatch> results, boolean hasMore) {
}

/**
 * Hospitals offering the given category (case-insensitive, or every
 * category if null), open to the given patient gender (a hospital with
 * no gender_specific flag accepts everyone; one flagged "male"/"female"
 * only accepts a matching patient gender -- a patient who is "other"
 * only matches unflagged hospitals; gender null skips this filter
 * entirely), nearest first. offset/limit page through the ranked list --
 * the WhatsApp booking flow and the "share your location" web page both
 * show 20 at a time with a "Show more"/"Load more" for the next page
 * (see QueueManagerService#searchAndOfferHospitals, HospitalController#nearby).
 */
public HospitalSearchPage searchHospitals(String category, String gender, double lat, double lon, int offset, int limit) {
    List<HospitalDto> candidates = list().stream()
        .filter(h -> category == null || (h.getCategories() != null
            && h.getCategories().stream().anyMatch(c -> c.equalsIgnoreCase(category))))
        .filter(h -> gender == null || h.getGenderSpecific() == null || h.getGenderSpecific().equalsIgnoreCase(gender))
        .toList();

    List<HospitalMatch> ranked = candidates.stream()
        .map(h -> new HospitalMatch(h, geoDistanceService.distanceKm(lat, lon, h.getLatitude(), h.getLongitude())))
        .sorted(Comparator.comparingDouble(HospitalMatch::distanceKm))
        .toList();

    int from = Math.min(offset, ranked.size());
    int to = Math.min(offset + limit, ranked.size());
    return new HospitalSearchPage(ranked.subList(from, to), to < ranked.size());
}

/** Same fallback schema.sql's placeholder row and DEFAULT clause use: a 9-to-5, 8-hour OPD day. */
private static final long DEFAULT_OPEN_MINUTES = 480;

```

```

public HospitalDto create(CreateHospitalRequest req) {
    LatLon resolved = resolveLocation(req.location());
    LocalTime open = req.openTime() != null ? LocalTime.parse(req.openTime()) : LocalTime.of(9, 0);
    LocalTime close = req.closeTime() != null ? LocalTime.parse(req.closeTime()) : LocalTime.of(17, 0);
    int avgServiceMinutes = resolveAvgServiceMinutes(req, open, close);
    if (req.minServiceMinutes() != null && req.minServiceMinutes() > avgServiceMinutes) {
        throw new IllegalArgumentException("minServiceMinutes cannot exceed avgServiceMinutes.");
    }

    if (req.genderSpecific() != null && !req.genderSpecific().isBlank()
        && !req.genderSpecific().equalsIgnoreCase("male") && !req.genderSpecific().equalsIgnoreCase("female")) {
        throw new IllegalArgumentException("genderSpecific must be \"male\" or \"female\" (or omitted for general/co-ed).");
    }
    List<String> canonicalCategories = canonicalizeCategories(req.categories());

    // doctor_join_code and the new profile columns (ownership onward): a fresh join-code
    // candidate is always generated, but on an upsert of an already-existing hospital,
    // COALESCE keeps whatever value it already had unless this call explicitly supplies a
    // new one -- a doctor's saved join code (or an admin-edited profile field) must never
    // silently change/reset just because HospitalSeedRunner re-upserts on every boot without
    // knowing about it.
    String sql = ""
        INSERT INTO hospitals (uri_slug, name, address, digipin, latitude, longitude, open_time, close_time,
            avg_service_minutes, min_service_minutes, active_counters, doctor_join_code, ownership, year_established, accreditation,
            gender_specific, categories)
        VALUES (:slug, :name, :address, :digipin, :lat, :lon, :open, :close, :avgService, :minService, :counters, :joinCode,
            :ownership, :yearEstablished, :accreditation, :genderSpecific, :categories)
        ON CONFLICT (uri_slug) DO UPDATE SET
            name = EXCLUDED.name, address = EXCLUDED.address, digipin = EXCLUDED.digipin,
            latitude = EXCLUDED.latitude, longitude = EXCLUDED.longitude,
            open_time = EXCLUDED.open_time, close_time = EXCLUDED.close_time,
            avg_service_minutes = EXCLUDED.avg_service_minutes,
            min_service_minutes = COALESCE(EXCLUDED.min_service_minutes, hospitals.min_service_minutes),
            active_counters = EXCLUDED.active_counters,
            doctor_join_code = COALESCE(hospitals.doctor_join_code, EXCLUDED.doctor_join_code),
            ownership = COALESCE(EXCLUDED.ownership, hospitals.ownership),
            year_established = COALESCE(EXCLUDED.year_established, hospitals.year_established),
            accreditation = COALESCE(EXCLUDED.accreditation, hospitals.accreditation),
            gender_specific = COALESCE(EXCLUDED.gender_specific, hospitals.gender_specific),
            categories = COALESCE(EXCLUDED.categories, hospitals.categories),
            updated_at = CURRENT_TIMESTAMP
        RETURNING *
    """;
    Map<String, Object> params = new java.util.HashMap<>();
    params.put("slug", req.uriSlug());
    params.put("name", req.name());
    params.put("address", req.address() == null ? "" : req.address());
    params.put("digipin", resolved.digipin());
    params.put("lat", resolved.lat());
    params.put("lon", resolved.lon());
    params.put("open", open);
    params.put("close", close);
    params.put("avgService", avgServiceMinutes);
    params.put("minService", req.minServiceMinutes());
    params.put("counters", req.activeCounters() == null ? 1 : req.activeCounters());
    params.put("joinCode", generateJoinCode());
    params.put("ownership", req.ownership());
    params.put("yearEstablished", req.yearEstablished());
    params.put("accreditation", joinCsv(req.accreditation()));
    params.put("genderSpecific", req.genderSpecific() == null ? null : req.genderSpecific().toLowerCase());
    params.put("categories", joinCsv(canonicalCategories));

    List<HospitalDto> rows = jdbc.query(sql, params, ROW_MAPPER);
    HospitalDto hospital = finish(rows.get(0));
    if (canonicalCategories != null) {
        hospitalDepartmentService.syncFromHospital(hospital.getId(), canonicalCategories);
    }
    return hospital;
}

private List<String> canonicalizeCategories(List<String> categories) {
    if (categories == null) {
        return null;
    }
    List<String> canonical = new ArrayList<>();

```

```

    for (String c : categories) {
        String match = MedicalCategory.canonicalize(c);
        if (match == null) {
            throw new IllegalArgumentException("Unknown category: \"" + c + "\". See GET /api/hospitals/categories for the valid list.");
        }
        canonical.add(match);
    }
    return canonical;
}

/** Admin-only: the join code doctors enter to link their account to this hospital (never exposed on public reads). */
public String getDoctorJoinCode(String uriSlug) {
    List<String> rows = jdbc.query(
        "SELECT doctor_join_code FROM hospitals WHERE uri_slug = :slug",
        Map.of("slug", uriSlug), (rs, i) -> rs.getString("doctor_join_code"));
    return rows.isEmpty() ? null : rows.get(0);
}

/** Admin-only: rotates the join code (e.g. if it leaked) -- existing doctor->hospital links are untouched, only future joins need the new code. */
public String regenerateDoctorJoinCode(String uriSlug) {
    String code = generateJoinCode();
    int updated = jdbc.update(
        "UPDATE hospitals SET doctor_join_code = :code, updated_at = CURRENT_TIMESTAMP WHERE uri_slug = :slug",
        Map.of("code", code, "slug", uriSlug));
    return updated > 0 ? code : null;
}

/** Used by DoctorService#joinHospital -- a doctor typing in their hospital's code. */
public HospitalDto findByDoctorJoinCode(String code) {
    List<HospitalDto> rows = jdbc.query(
        "SELECT * FROM hospitals WHERE doctor_join_code = :code", Map.of("code", code), ROW_MAPPER);
    return rows.isEmpty() ? null : finish(rows.get(0));
}

private String generateJoinCode() {
    StringBuilder sb = new StringBuilder(10);
    for (int i = 0; i < 10; i++) {
        sb.append(JOIN_CODE_ALPHABET.charAt(RANDOM.nextInt(JOIN_CODE_ALPHABET.length())));
    }
    return sb.toString();
}

/**
 * avgServiceMinutes (how long one patient's visit takes, which drives
 * every wait-time estimate shown to patients) is normally derived rather
 * than asked for directly: given how many hours the hospital is open and
 * how many patients it treats on an average day, one patient's share of
 * the day is (open minutes) / (patients per day). An explicit
 * avgServiceMinutes always overrides that math; with neither given, falls
 * back to the original flat default of 10 minutes/patient.
 */
private int resolveAvgServiceMinutes(CreateHospitalRequest req, LocalTime open, LocalTime close) {
    if (req.avgServiceMinutes() != null) {
        return req.avgServiceMinutes();
    }
    if (req.avgPatientsPerDay() != null) {
        long openMinutes = java.time.Duration.between(open, close).toMinutes();
        if (openMinutes <= 0) {
            openMinutes = DEFAULT_OPEN_MINUTES;
        }
        return (int) Math.max(1, openMinutes / req.avgPatientsPerDay());
    }
    return 10;
}

public HospitalDto updateLocation(String uriSlug, SetLocationRequest location) {
    LatLon resolved = resolveLocation(location);
    List<HospitalDto> rows = jdbc.query(
        """
        UPDATE hospitals
        SET digipin = :digipin, latitude = :lat, longitude = :lon, updated_at = CURRENT_TIMESTAMP
        WHERE uri_slug = :slug
        RETURNING *
        """,

```

```
        Map.of("digipin", resolved.digipin(), "lat", resolved.lat(), "lon", resolved.lon(), "slug", uriSlug),
        ROW_MAPPER);
    return rows.isEmpty() ? null : finish(rows.get(0));
}

/** Resolves a DIGIPIN-or-lat/lon request into both forms; DIGIPIN is the source of truth when given. */
public LatLon resolveLocation(SetLocationRequest location) {
    if (location.hasDigipin()) {
        DigipinService.LatLon decoded = digipinService.decode(location.digipin());
        return new LatLon(location.digipin().trim().toUpperCase(), decoded.lat(), decoded.lon());
    }
    if (location.hasLatLon()) {
        String digipin = digipinService.encode(location.latitude(), location.longitude());
        return new LatLon(digipin, location.latitude(), location.longitude());
    }
    throw new IllegalArgumentException("Provide either a digipin or both latitude and longitude.");
}

private HospitalDto finish(HospitalDto h) {
    if (h != null && h.getDigipin() != null) {
        h.setFormattedDigipin(digipinService.format(h.getDigipin()));
    }
    return h;
}

private static List<String> splitCsv(String csv) {
    if (csv == null || csv.isBlank()) {
        return null;
    }
    return Arrays.stream(csv.split(",")).map(String::trim).filter(s -> !s.isEmpty()).toList();
}

private static String joinCsv(List<String> values) {
    if (values == null || values.isEmpty()) {
        return null;
    }
    return String.join(", ", values);
}

public record LatLon(String digipin, double lat, double lon) {
}
}
```

OtpService.java

src/main/java/com/qdischarge/clinicqueue/service/OtpService.java

Phone-number OTP verification behind three interchangeable providers — `twilio` (delegates entirely to Twilio Verify), `twilio-sms` (self-owned generation/expiry/matching, delivered via Twilio's plain Messages API), and `log` (dev-friendly: generates and logs its own code, no external account needed). Also implements a Twilio→WhatsApp delivery fallback when an SMS send itself fails, with `verifyOtp()` transparently figuring out which path a given phone's code came from.

```
package com.qdischarge.clinicqueue.service;

import com.qdischarge.clinicqueue.config.AppProperties;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.HttpEntity;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.util.LinkedMultiValueMap;
import org.springframework.util.MultiValueMap;
import org.springframework.web.client.HttpStatusCodeException;
import org.springframework.web.client.RestClientException;
import org.springframework.web.client.RestTemplate;

import java.security.SecureRandom;
import java.util.List;
import java.util.Map;

/**
 * Phone-number OTP verification, gating (optionally) patient registration --
 * "message OTP auth with Twilio". Three providers, selected by app.otp-provider:
 *
 * - "twilio": delegates code generation/expiry/matching entirely to Twilio's
 *   Verify API (Send + VerificationCheck), the standard way to do SMS OTP
 *   without owning that state yourself. Needs TWILIO_ACCOUNT_SID /
 *   TWILIO_AUTH_TOKEN / TWILIO_VERIFY_SERVICE_SID; ships with dummy
 *   placeholders by default (see .env.example) that fail cleanly (401, not
 *   a crash) until real credentials are set.
 * - "twilio-sms": we own generation/expiry/matching (like "log" below), but
 *   deliver the code as a real SMS via Twilio's plain Messages API through a
 *   Messaging Service (POST ../Messages.json with MessagingServiceSid),
 *   rather than the Verify API. Needs TWILIO_ACCOUNT_SID / TWILIO_AUTH_TOKEN
 *   / TWILIO_MESSAGING_SERVICE_SID.
 * - "log" (default, dev-friendly): generates its own 6-digit code, stores
 *   only its BCrypt hash (otp_verifications.code_hash) with an expiry and
 *   an attempt cap, and logs the plaintext code instead of sending an SMS --
 *   so OTP flows are fully testable with zero external accounts.
 *
 * Twilio -> WhatsApp fallback: if a Twilio Verify or Twilio SMS send call
 * itself fails (account issue, network, Twilio outage -- not "wrong code",
 * which is a normal verify failure), sendOtp() falls back to generating/
 * hashing its own code exactly like the "log" provider does, but delivers it
 * over WhatsApp (via WhatsAppService, which has its own Meta<->Evolution
 * failover) instead of SMS. verifyOtp() then has to know which path a given
 * phone's code came from without any extra state: it checks for a pending
 * unverified local row first (present whenever the code was generated
 * locally -- "log", "twilio-sms", or a WhatsApp fallback) and verifies
 * against that; only if there isn't one does it ask Twilio's
 * VerificationCheck, which is only reachable when "twilio" (Verify) is
 * configured and its own send succeeded.
 *
 * Either way, a successful verifyOtp() records a "phone verified recently"
 * ledger row so QueueManagerService can gate token creation on it
 * uniformly, regardless of which provider/path actually did the checking.
 */
@Service
@RequiredArgsConstructor
@Slf4j
public class OtpService {
```

```

private final AppProperties appProperties;
private final NamedParameterJdbcTemplate jdbc;
private final PasswordEncoder passwordEncoder;
private final RestTemplate restTemplate;
private final WhatsAppService whatsappService;

private static final int MAX_ATTEMPTS = 5;
private static final SecureRandom RANDOM = new SecureRandom();

private boolean isTwilio() {
    return "twilio".equalsIgnoreCase(appProperties.getOtpProvider());
}

private boolean isTwilioSms() {
    return "twilio-sms".equalsIgnoreCase(appProperties.getOtpProvider());
}

public record OtpResult(boolean success, String message) {
}

public OtpResult sendOtp(String phone) {
    if (isTwilio()) {
        return sendViaTwilio(phone);
    }
    if (isTwilioSms()) {
        return sendViaTwilioSms(phone);
    }
    return sendViaLog(phone);
}

public OtpResult verifyOtp(String phone, String code) {
    // A pending local row only exists when sendOtp() fell back to WhatsApp delivery
    // (see sendViaTwilio) -- in that case Twilio never has a code to check, so verify
    // locally instead. Otherwise this phone's code was sent by whichever provider is
    // configured (Twilio's Verify API owns the check, or it's the "log" provider).
    boolean useLocalVerify = !isTwilio() || hasPendingLocalOtp(phone);
    OtpResult result = useLocalVerify ? verifyViaLog(phone, code) : verifyViaTwilio(phone, code);
    if (result.success()) {
        recordVerifiedLedger(phone);
    }
    return result;
}

/** Has this phone completed OTP verification within the configured TTL? Used to gate token creation. */
public boolean isPhoneVerifiedRecently(String phone) {
    Integer count = jdbc.queryForObject(
        """
        SELECT COUNT(*)::int FROM otp_verifications
        WHERE phone = :phone AND verified = TRUE
        AND verified_at > NOW() - (:ttl || ' minutes')::INTERVAL
        """,
        Map.of("phone", phone, "ttl", String.valueOf(appProperties.getOtpTtlMinutes())), Integer.class);
    return count != null && count > 0;
}

// -----
// "log" provider: we own generation, hashing, expiry, attempts
// -----

private OtpResult sendViaLog(String phone) {
    String code = generateAndStoreLocalOtp(phone);
    log.info("🔒 [DEV OTP] Code for {} is {} (valid {} min). No real SMS was sent -- set OTP_PROVIDER=twilio for that.",
        phone, code, appProperties.getOtpTtlMinutes());
    return new OtpResult(true, "OTP sent.");
}

/** Generates a 6-digit code, stores only its BCrypt hash, and returns the plaintext to whoever's delivering it. */
private String generateAndStoreLocalOtp(String phone) {
    String code = String.format("%06d", RANDOM.nextInt(1_000_000));
    String hash = passwordEncoder.encode(code);

    jdbc.update(
        """
        INSERT INTO otp_verifications (phone, code_hash, purpose, expires_at)

```

```

VALUES (:phone, :hash, 'registration', NOW() + (:ttl || ' minutes')::INTERVAL)
""",
Map.of("phone", phone, "hash", hash, "ttl", String.valueOf(appProperties.getOtpTtlMinutes())));
return code;
}

/** Is there a locally-generated, unverified, unexpired code pending for this phone? Only true after a Twilio->WhatsApp fallback send. */
private boolean hasPendingLocalOtp(String phone) {
    Integer count = jdbc.queryForObject(
        """
        SELECT COUNT(*)::int FROM otp_verifications
        WHERE phone = :phone AND verified = FALSE AND expires_at > NOW()
        """,
        Map.of("phone", phone), Integer.class);
    return count != null && count > 0;
}

private OtpResult verifyViaLog(String phone, String code) {
    List<Map<String, Object>> rows = jdbc.queryForList(
        """
        SELECT id, code_hash, attempts FROM otp_verifications
        WHERE phone = :phone AND verified = FALSE AND expires_at > NOW()
        ORDER BY created_at DESC LIMIT 1
        """,
        Map.of("phone", phone));

    if (rows.isEmpty()) {
        return new OtpResult(false, "No active OTP for this number. Request a new one.");
    }

    Map<String, Object> row = rows.get(0);
    int id = (Integer) row.get("id");
    int attempts = (Integer) row.get("attempts");
    String hash = (String) row.get("code_hash");

    if (attempts >= MAX_ATTEMPTS) {
        return new OtpResult(false, "Too many incorrect attempts. Request a new OTP.");
    }

    jdbc.update("UPDATE otp_verifications SET attempts = attempts + 1 WHERE id = :id", Map.of("id", id));

    if (!passwordEncoder.matches(code, hash)) {
        return new OtpResult(false, "Incorrect OTP.");
    }

    jdbc.update("UPDATE otp_verifications SET verified = TRUE, verified_at = NOW() WHERE id = :id", Map.of("id", id));
    return new OtpResult(true, "OTP verified.");
}

// -----
// "twilio" provider: Twilio Verify owns generation/expiry/matching
// -----

private HttpHeaders twilioHeaders() {
    HttpHeaders headers = new HttpHeaders();
    headers.setContentType(MediaType.APPLICATION_FORM_URLENCODED);
    headers.setBasicAuth(appProperties.getTwilioAccountSid(), appProperties.getTwilioAuthToken());
    return headers;
}

private OtpResult sendViaTwilio(String phone) {
    String url = "https://verify.twilio.com/v2/Services/%s/Verifications"
        .formatted(appProperties.getTwilioVerifyServiceSid());
    MultiValueMap<String, String> body = new LinkedMultiValueMap<>();
    body.add("To", phone);
    body.add("Channel", "sms");
    try {
        restTemplate.postForEntity(url, new HttpEntity<>(body, twilioHeaders()), Map.class);
        return new OtpResult(true, "OTP sent via SMS.");
    } catch (RestClientException e) {
        log.error("✗ Twilio Verify send error for {}: {} -- falling back to WhatsApp delivery.", phone, extractError(e));
        return sendViaWhatsAppFallback(phone);
    }
}

```

```
// -----
// "twilio-sms" provider: we own generation/expiry/matching, Twilio's
// plain Messages API (not Verify) just delivers the SMS
// -----

private OtpResult sendViaTwilioSms(String phone) {
    String code = generateAndStoreLocalOtp(phone);
    String url = "https://api.twilio.com/2010-04-01/Accounts/%s/Messages.json"
        .formatted(appProperties.getTwilioAccountSid());
    MultiValueMap<String, String> body = new LinkedMultiValueMap<>();
    body.add("To", phone);
    body.add("MessagingServiceSid", appProperties.getTwilioMessagingServiceSid());
    body.add("Body", "Your verification code is " + code + " (valid " + appProperties.getOtpTtlMinutes() + " minutes).");
    try {
        restTemplate.postForEntity(url, new HttpEntity<>(body, twilioHeaders()), Map.class);
        return new OtpResult(true, "OTP sent via SMS.");
    } catch (RestClientException e) {
        log.error("✗ Twilio Messages API send error for {}: {} -- falling back to WhatsApp delivery.", phone, extractError(e));
        whatsappService.sendWhatsAppMessage(phone,
            "🔔 Your verification code is " + code + " (valid " + appProperties.getOtpTtlMinutes()
                + " minutes). We couldn't reach you by SMS just now, so we sent this over WhatsApp instead.");
        return new OtpResult(true, "Could not send OTP via SMS right now -- sent your verification code via WhatsApp instead.");
    }
}

/**
 * Twilio's SMS send failed (account/network/Twilio outage) -- rather than
 * leave the patient stuck, generate our own code (same as the "log"
 * provider) and deliver it over WhatsApp instead, via WhatsAppService
 * (which itself fails over between Meta and Evolution). verifyOtp() picks
 * this path back up automatically via hasPendingLocalOtp().
 */
private OtpResult sendViaWhatsAppFallback(String phone) {
    String code = generateAndStoreLocalOtp(phone);
    whatsappService.sendWhatsAppMessage(phone,
        "🔔 Your verification code is " + code + " (valid " + appProperties.getOtpTtlMinutes()
            + " minutes). We couldn't reach you by SMS just now, so we sent this over WhatsApp instead.");
    return new OtpResult(true, "Could not send OTP via SMS right now -- sent your verification code via WhatsApp instead.");
}

private OtpResult verifyViaTwilio(String phone, String code) {
    String url = "https://verify.twilio.com/v2/Services/%s/VerificationCheck"
        .formatted(appProperties.getTwilioVerifyServiceSid());
    MultiValueMap<String, String> body = new LinkedMultiValueMap<>();
    body.add("To", phone);
    body.add("Code", code);
    try {
        var response = restTemplate.postForEntity(url, new HttpEntity<>(body, twilioHeaders()), Map.class);
        String status = response.getBody() != null ? String.valueOf(response.getBody().get("status")) : null;
        if ("approved".equals(status)) {
            return new OtpResult(true, "OTP verified.");
        }
        return new OtpResult(false, "Incorrect or expired OTP.");
    } catch (RestClientException e) {
        log.error("✗ Twilio Verify check error for {}: {}", phone, extractError(e));
        return new OtpResult(false, "Could not verify OTP right now. Please try again.");
    }
}

private String extractError(RestClientException e) {
    if (e instanceof HttpStatusCodeException hsce) {
        return hsce.getStatusCode() + " " + hsce.getResponseBodyAsString();
    }
    return e.getMessage();
}

private void recordVerifiedLedger(String phone) {
    jdbc.update(
        """
        INSERT INTO otp_verifications (phone, code_hash, purpose, expires_at, verified, verified_at)
        VALUES (:phone, 'EXTERNALLY_VERIFIED', 'registration', NOW() + (:ttl || ' minutes')::INTERVAL, TRUE, NOW())
        """,
        Map.of("phone", phone, "ttl", String.valueOf(appProperties.getOtpTtlMinutes())));
}
```

```
}  
}
```

PatientDocumentService.java

src/main/java/com/qdischarge/clinicqueue/service/PatientDocumentService.java

Prescription/report photo uploads, stored directly as Postgres `BYTEA` (no cloud storage configured). Listing always projects metadata-only columns — only `getFile()` touches the actual file bytes — and `groupByIdentity()` is what powers a doctor's patient list, grouping documents by (phone, name, age) since one WhatsApp number can be shared across a family.

```
package com.qdischarge.clinicqueue.service;

import com.qdischarge.clinicqueue.dto.DoctorPatientGroupDto;
import com.qdischarge.clinicqueue.dto.PatientDocumentDto;
import lombok.RequiredArgsConstructor;
import org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate;
import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;

import java.io.IOException;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Timestamp;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Set;

/**
 * Prescription/report photo uploads, stored as Postgres BYTEA (this project
 * has no cloud storage configured -- see patient_documents in schema.sql). Listing
 * always projects metadata-only columns; only #getFile touches file_data, so
 * a patient's document list never drags BYTEA payloads across the wire.
 */
@Service
@RequiredArgsConstructor
public class PatientDocumentService {

    private final NamedParameterJdbcTemplate jdbc;

    private static final Set<String> VALID_DOC_TYPES = Set.of("prescription", "report");
    private static final long MAX_FILE_SIZE_BYTES = 8L * 1024 * 1024; // 8MB -- see BYTEA-in-Postgres storage decision

    private static final String METADATA_COLUMNS =
        "id, patient_phone, patient_name, patient_age, doc_type, hospital_id, uploaded_by_doctor_id, file_name, content_type, file_size, created_at";

    public PatientDocumentDto upload(String patientPhone, String patientName, Integer patientAge, String docType,
        Integer hospitalId, Integer uploadedByDoctorId, MultipartFile file) throws IOException {
        if (file == null || file.isEmpty()) {
            throw new IllegalArgumentException("A file is required.");
        }
        if (file.getSize() > MAX_FILE_SIZE_BYTES) {
            throw new IllegalArgumentException("File too large -- max 8MB.");
        }
        String normalizedType = docType == null ? "" : docType.trim().toLowerCase();
        if (!VALID_DOC_TYPES.contains(normalizedType)) {
            throw new IllegalArgumentException("docType must be 'prescription' or 'report'.");
        }
        if (patientName == null || patientName.isBlank()) {
            throw new IllegalArgumentException("patientName is required.");
        }

        Map<String, Object> params = new HashMap<>();
        params.put("phone", patientPhone);
        params.put("name", patientName);
        params.put("age", patientAge);
        params.put("docType", normalizedType);
        params.put("hospitalId", hospitalId);
        params.put("uploadedByDoctorId", uploadedByDoctorId);
        params.put("fileName", file.getOriginalFilename() != null ? file.getOriginalFilename() : "upload");
        params.put("contentType", file.getContentType() != null ? file.getContentType() : "application/octet-stream");
    }
}
```

```

        params.put("fileSize", (int) file.getSize());
        params.put("fileData", file.getBytes());

        Integer id = jdbc.queryForObject(
            """
            INSERT INTO patient_documents (patient_phone, patient_name, patient_age, doc_type, hospital_id, uploaded_by_doctor_id,
file_name, content_type, file_size, file_data)
            VALUES (:phone, :name, :age, :docType, :hospitalId, :uploadedByDoctorId, :fileName, :contentType, :fileSize, :fileData)
            RETURNING id
            """, params, Integer.class);
        return getMetadata(id);
    }

    public List<PatientDocumentDto> listForPatient(String phone) {
        return jdbc.query(
            "SELECT " + METADATA_COLUMNS + " FROM patient_documents WHERE patient_phone = :phone ORDER BY created_at DESC",
            Map.of("phone", phone), PatientDocumentService::mapMetadata);
    }

    public PatientDocumentDto getMetadata(int id) {
        List<PatientDocumentDto> rows = jdbc.query(
            "SELECT " + METADATA_COLUMNS + " FROM patient_documents WHERE id = :id",
            Map.of("id", id), PatientDocumentService::mapMetadata);
        return rows.isEmpty() ? null : rows.get(0);
    }

    public record FileContent(byte[] data, String contentType, String fileName) {
    }

    public FileContent getFile(int id) {
        List<Map<String, Object>> rows = jdbc.queryForList(
            "SELECT file_data, content_type, file_name FROM patient_documents WHERE id = :id", Map.of("id", id));
        if (rows.isEmpty()) {
            return null;
        }
        Map<String, Object> r = rows.get(0);
        return new FileContent((byte[]) r.get("file_data"), (String) r.get("content_type"), (String) r.get("file_name"));
    }

    /**
     * A doctor's full accessible record set: every document under any of the
     * given phone numbers, grouped by (phone, name, age) rather than phone
     * alone -- see DoctorPatientGroupDto. A phone with zero documents still
     * appears (empty group) so "I was just granted access" is visible even
     * before any record exists.
     */
    public List<DoctorPatientGroupDto> groupByIdentity(List<String> accessiblePhones) {
        if (accessiblePhones.isEmpty()) {
            return List.of();
        }
        List<Map<String, Object>> rows = jdbc.queryForList(
            "SELECT " + METADATA_COLUMNS + " FROM patient_documents WHERE patient_phone IN (:phones) " +
            "ORDER BY patient_phone, patient_name, patient_age, created_at DESC",
            Map.of("phones", accessiblePhones));

        Map<String, DoctorPatientGroupDto> groups = new LinkedHashMap<>();
        for (Map<String, Object> row : rows) {
            String phone = (String) row.get("patient_phone");
            String name = (String) row.get("patient_name");
            Integer age = (Integer) row.get("patient_age");
            String key = phone + "|" + name + "|" + age;
            DoctorPatientGroupDto group = groups.computeIfAbsent(key, k -> DoctorPatientGroupDto.builder()
                .patientPhone(phone).name(name).age(age).documents(new ArrayList<>()).build());
            group.getDocuments().add(mapMetadataRow(row));
        }

        for (String phone : accessiblePhones) {
            boolean hasAnyGroup = groups.keySet().stream().anyMatch(k -> k.startsWith(phone + "|"));
            if (!hasAnyGroup) {
                groups.put(phone + "|", DoctorPatientGroupDto.builder()
                    .patientPhone(phone).documents(List.of()).build());
            }
        }
        return new ArrayList<>(groups.values());
    }
}

```

```
private static PatientDocumentDto mapMetadataRow(ResultSet rs, int rowNum) throws SQLException {
    return mapMetadataRow(toRowMap(rs));
}

private static Map<String, Object> toRowMap(ResultSet rs) throws SQLException {
    Map<String, Object> row = new HashMap<>();
    row.put("id", rs.getInt("id"));
    row.put("patient_phone", rs.getString("patient_phone"));
    row.put("patient_name", rs.getString("patient_name"));
    row.put("patient_age", (Integer) rs.getObject("patient_age"));
    row.put("doc_type", rs.getString("doc_type"));
    row.put("hospital_id", (Integer) rs.getObject("hospital_id"));
    row.put("uploaded_by_doctor_id", (Integer) rs.getObject("uploaded_by_doctor_id"));
    row.put("file_name", rs.getString("file_name"));
    row.put("content_type", rs.getString("content_type"));
    row.put("file_size", (Integer) rs.getObject("file_size"));
    row.put("created_at", rs.getTimestamp("created_at"));
    return row;
}

private static PatientDocumentDto mapMetadataRow(Map<String, Object> row) {
    Timestamp createdAt = (Timestamp) row.get("created_at");
    return PatientDocumentDto.builder()
        .id((Integer) row.get("id"))
        .patientPhone((String) row.get("patient_phone"))
        .patientName((String) row.get("patient_name"))
        .patientAge((Integer) row.get("patient_age"))
        .docType((String) row.get("doc_type"))
        .hospitalId((Integer) row.get("hospital_id"))
        .uploadedByDoctorId((Integer) row.get("uploaded_by_doctor_id"))
        .fileName((String) row.get("file_name"))
        .contentType((String) row.get("content_type"))
        .fileSize((Integer) row.get("file_size"))
        .createdAt(createdAt != null ? createdAt.toLocalDateTime() : null)
        .build();
}
}
```

QrCodeService.java

src/main/java/com/qdischarge/clinicqueue/service/QrCodeService.java

PNG QR code generation via ZXing — a direct Java port of the original's `qrcode` npm package usage (same size/margin/color parameters).

```
package com.qdischarge.clinicqueue.service;

import com.google.zxing.BarcodeFormat;
import com.google.zxing.EncodeHintType;
import com.google.zxing.WriterException;
import com.google.zxing.client.j2se.MatrixToImageConfig;
import com.google.zxing.client.j2se.MatrixToImageWriter;
import com.google.zxing.common.BitMatrix;
import com.google.zxing.qrcode.QRCodeWriter;
import com.google.zxing.qrcode.decoder.ErrorCorrectionLevel;
import org.springframework.stereotype.Service;

import javax.imageio.ImageIO;
import java.awt.Color;
import java.awt.image.BufferedImage;
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.util.Map;

/**
 * Java port of the "qrcode" npm package usage in routes/queue.js
 * (QRCode.toBuffer with type 'png', width 350, margin 2, dark/light colors).
 */
@Service
public class QrCodeService {

    public byte[] generatePng(String content, int size, int margin, String darkHex, String lightHex)
        throws WriterException, IOException {
        Map<EncodeHintType, Object> hints = Map.of(
            EncodeHintType.MARGIN, margin,
            EncodeHintType.ERROR_CORRECTION, ErrorCorrectionLevel.M);

        BitMatrix matrix = new QRCodeWriter().encode(content, BarcodeFormat.QR_CODE, size, size, hints);

        MatrixToImageConfig config = new MatrixToImageConfig(
            Color.decode(darkHex).getRGB(),
            Color.decode(lightHex).getRGB());

        BufferedImage image = MatrixToImageWriter.toBufferedImage(matrix, config);

        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        ImageIO.write(image, "png", baos);
        return baos.toByteArray();
    }
}
```

QueueManagerService.java

src/main/java/com/qdischarge/clinicqueue/service/QueueManagerService.java

The core of the entire application — roughly 1,400 lines implementing every queue-shaping rule: the full booking flow (name→gender→age→category→location→hospital search/select→confirm), direct/API booking, the real-ETA treatment-timing tick (notify + call trigger, anomaly-control grace windows), token status transitions and history archiving, the missed-queue admin panel (search/requeue-to-front/reject), and floating-point "lexorank"-style priority repositioning that lets a token jump the queue without renumbering anyone else. Nearly every method is explicitly scoped by `(hospitalId, category)`, since a hospital runs one independent queue per department it offers. See the Architecture Overview above for the treatment-timing and priority-rank concepts in more depth.

```
package com.qdischarge.clinicqueue.service;

import com.qdischarge.clinicqueue.bot.BotMessages;
import com.qdischarge.clinicqueue.bot.Lang;
import com.qdischarge.clinicqueue.bot.WaSessionService;
import com.qdischarge.clinicqueue.catalog.MedicalCategory;
import com.qdischarge.clinicqueue.config.AppProperties;
import com.qdischarge.clinicqueue.dto.CreateTokenResult;
import com.qdischarge.clinicqueue.dto.HospitalDto;
import com.qdischarge.clinicqueue.dto.QueueData;
import com.qdischarge.clinicqueue.dto.SetLocationRequest;
import com.qdischarge.clinicqueue.dto.Stats;
import com.qdischarge.clinicqueue.dto.TokenDto;
import com.qdischarge.clinicqueue.dto.TokenHistoryDto;
import com.qdischarge.clinicqueue.dto.UpdateStatusResult;
import com.qdischarge.clinicqueue.geo.GeoDistanceService;
import com.qdischarge.clinicqueue.geo.TomTomRoutingService;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.jdbc.core.BeanPropertyRowMapper;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate;
import org.springframework.stereotype.Service;

import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Java port of backend/utils/queueManager.js, extended with: hospital-aware,
 * DIGIPIN/distance-based patient locations and dynamic "get ready" notification
 * windows (geo/GeoDistanceService), a dedicated missed-token queue with
 * search + requeue-to-front + reject for reception staff, priority-rank
 * based ordering so a requested token can jump to the front of the line
 * without renumbering everyone else, and a search+select+confirm hospital
 * booking flow (see the "Booking flow" section below).
 *
 * Every queue-shaping method is scoped by (hospitalId, category): a hospital
 * doesn't run one queue, it runs one queue *per department it offers* (see
 * catalog.MedicalCategory / hospital_departments) -- each with its own
 * position counting, missed-queue, and counter assignment. Reads that accept
 * a nullable (hospitalId, category) pair (getQueue/getCurrentServingToken/
 * getMissedQueue/searchMissedQueue) fall back to an unscoped, hospital-wide
 * view when both are omitted, for callers (e.g. today's admin dashboard)
 * that don't yet filter by department.
 */
@Service
@RequiredArgsConstructor
@Slf4j
public class QueueManagerService {

    private final NamedParameterJdbcTemplate jdbc;
    private final WhatsAppService whatsappService;
    private final AppProperties appProperties;
    private final HospitalService hospitalService;
    private final HospitalDepartmentService hospitalDepartmentService;
    private final GeoDistanceService geoDistanceService;
    private final OtpService otpService;
```

```

/** Real routing ETA for the treatment-timing "go now" trigger (see runTreatmentTimingTick). */
private final TomTomRoutingService tomTomRoutingService;
/** Voice-call half of the "go now" trigger (see runTreatmentTimingTick). */
private final TwilioStudioCallService twilioStudioCallService;
private final BotMessages botMessages;
/** Per-phone language preference (see resolveLang) -- same source the WhatsApp bot itself uses. */
private final WaSessionService waSessionService;

private static final RowMapper<TokenDto> TOKEN_ROW_MAPPER = new BeanPropertyRowMapper<>(TokenDto.class);
private static final RowMapper<TokenHistoryDto> TOKEN_HISTORY_ROW_MAPPER = new BeanPropertyRowMapper<>(TokenHistoryDto.class);
/** Ordering expression: a requested-to-front token's negative priority_rank sorts before every plain id. */
private static final String QUEUE_ORDER = "COALESCE(priority_rank, id) ASC";
/** How many hospitals the WhatsApp booking flow shows per page ("5 options, then Show more"). */
public static final int HOSPITAL_PAGE_SIZE = 20;
/** R formula headstart: the first 3 genuinely-ahead tokens don't add wait time (see computeTreatmentRemainingMinutes). */
private static final int ETA_WAITING_HEADSTART_TOKENS = 3;
/** R formula fixed buffer, in minutes (see computeTreatmentRemainingMinutes). */
private static final double ETA_FIXED_BUFFER_MINUTES = 2.0;

// -----
// Expired-token cleanup (runs before every read, same as the original)
// -----
private void cleanExpiredTokens() {
    String sql = ""
        UPDATE tokens
        SET status = 'missed', missed_at = NOW()
        WHERE status IN ('waiting', 'serving', 'registering_name')
        AND created_at < NOW() - (:hours || ' hours')::INTERVAL
        """;

    try {
        jdbc.update(sql, Map.of("hours", String.valueOf(appProperties.getTokenExpiryHours())));
    } catch (Exception e) {
        log.error("Error cleaning expired tokens: {}", e.getMessage());
    }
}

/** Unscoped, hospital-wide view (today's admin dashboard default). */
public QueueData getQueue() {
    return getQueue(null, null);
}

/** hospitalId/category null = no filter on that dimension (see class docs). */
public QueueData getQueue(Integer hospitalId, String category) {
    cleanExpiredTokens();

    StringBuilder sql = new StringBuilder("SELECT * FROM tokens WHERE status != 'registering_name'");
    Map<String, Object> params = new HashMap<>();
    appendScope(sql, params, hospitalId, category);
    sql.append(" ORDER BY ").append(QUEUE_ORDER);

    List<TokenDto> tokens = jdbc.query(sql.toString(), params, TOKEN_ROW_MAPPER);

    int total = tokens.size();
    long waiting = tokens.stream().filter(t -> "waiting".equals(t.getStatus())).count();
    long serving = tokens.stream().filter(t -> "serving".equals(t.getStatus())).count();
    long completed = tokens.stream().filter(t -> "completed".equals(t.getStatus())).count();
    long missed = tokens.stream().filter(t -> "missed".equals(t.getStatus())).count();

    Integer currentServing = tokens.stream()
        .filter(t -> "serving".equals(t.getStatus()))
        .map(TokenDto::getId)
        .findFirst()
        .orElse(null);

    Stats stats = new Stats(total, (int) waiting, (int) serving, (int) completed, (int) missed);
    return new QueueData(tokens, currentServing, stats);
}

private void appendScope(StringBuilder sql, Map<String, Object> params, Integer hospitalId, String category) {
    if (hospitalId != null) {
        sql.append(" AND hospital_id = :hospitalId");
        params.put("hospitalId", hospitalId);
    }
    if (category != null && !category.isBlank()) {
        sql.append(" AND category = :category");
    }
}

```

```

        params.put("category", category);
    }
}

public TokenDto getCurrentServingToken(Integer hospitalId, String category) {
    cleanExpiredTokens();

    StringBuilder sql = new StringBuilder("SELECT * FROM tokens WHERE status = 'serving'");
    Map<String, Object> params = new HashMap<>();
    appendScope(sql, params, hospitalId, category);
    sql.append(" ORDER BY served_at DESC LIMIT 1");

    List<TokenDto> rows = jdbc.query(sql.toString(), params, TOKEN_ROW_MAPPER);
    if (rows.isEmpty()) {
        return null;
    }
    TokenDto t = rows.get(0);
    return TokenDto.builder()
        .id(t.getId()).name(t.getName()).phone(t.getPhone()).status(t.getStatus())
        .sessionStep(t.getSessionStep()).createdAt(t.getCreatedAt()).servedAt(t.getServedAt())
        .counterId(t.getCounterId())
        .build();
}

public TokenDto getPatientPosition(String phone) {
    cleanExpiredTokens();

    String sql = ""
        SELECT * FROM tokens
        WHERE (phone = :phone OR phone = '+' || :phone OR phone = REPLACE(:phone, '+', '') OR phone LIKE '%' || :phone)
        AND status IN ('waiting', 'serving')
        LIMIT 1
        """;

    List<TokenDto> rows = jdbc.query(sql, Map.of("phone", phone), TOKEN_ROW_MAPPER);
    if (rows.isEmpty()) {
        return null;
    }
    TokenDto token = rows.get(0);
    QueueData queueData = getQueue(token.getHospitalId(), token.getCategory());

    if ("serving".equals(token.getStatus())) {
        return withGeoFields(TokenDto.builder()
            .id(token.getId()).name(token.getName()).phone(token.getPhone()).status(token.getStatus())
            .sessionStep(token.getSessionStep()).position(0).peopleAhead(0)
            .currentServing(queueData.currentServing())
            .isVerified(token.getIsVerified()).verifiedAt(token.getVerifiedAt())
            .build(), token);
    }

    int peopleAhead = countWaitingAhead(token);

    return withGeoFields(TokenDto.builder()
        .id(token.getId()).name(token.getName()).phone(token.getPhone()).status(token.getStatus())
        .sessionStep(token.getSessionStep()).position(peopleAhead + 1).peopleAhead(peopleAhead)
        .currentServing(queueData.currentServing())
        .isVerified(token.getIsVerified()).verifiedAt(token.getVerifiedAt())
        .build(), token);
}

public TokenDto getTokenDetails(String idStr) {
    cleanExpiredTokens();

    int id = Integer.parseInt(idStr);
    TokenDto token = getRaw(id);
    if (token == null) {
        return null;
    }
    QueueData queueData = getQueue(token.getHospitalId(), token.getCategory());

    if (!"waiting".equals(token.getStatus())) {
        return withGeoFields(TokenDto.builder()
            .id(token.getId()).name(token.getName()).phone(token.getPhone()).status(token.getStatus())
            .sessionStep(token.getSessionStep()).position(0).peopleAhead(0)
            .currentServing(queueData.currentServing())
            .isVerified(token.getIsVerified()).verifiedAt(token.getVerifiedAt())

```

```

        .missedAt(token.getMissedAt()).rejectedAt(token.getRejectedAt())
        .build(), token);
    }

    int peopleAhead = countWaitingAhead(token);

    return withGeoFields(TokenDto.builder()
        .id(token.getId()).name(token.getName()).phone(token.getPhone()).status(token.getStatus())
        .sessionStep(token.getSessionStep()).position(peopleAhead + 1).peopleAhead(peopleAhead)
        .currentServing(queueData.currentServing())
        .isVerified(token.getIsVerified()).verifiedAt(token.getVerifiedAt())
        .build(), token);
}

/** Copies the geo/priority fields from a freshly-read raw row onto a purpose-built response DTO. */
private TokenDto withGeoFields(TokenDto target, TokenDto raw) {
    target.setDailyNumber(raw.getDailyNumber());
    target.setHospitalId(raw.getHospitalId());
    if (raw.getHospitalId() != null) {
        HospitalDto h = hospitalService.getById(raw.getHospitalId());
        if (h != null) {
            target.setHospitalName(h.getName());
        }
    }
    target.setCategory(raw.getCategory());
    target.setGender(raw.getGender());
    target.setAge(raw.getAge());
    target.setPatientDigipin(raw.getPatientDigipin());
    target.setDistanceKm(raw.getDistanceKm());
    target.setTravelMinutes(raw.getTravelMinutes());
    target.setTreatmentRemainingMinutes(raw.getTreatmentRemainingMinutes());
    target.setNotifiedReadyAt(raw.getNotifiedReadyAt());
    target.setAnomalyControlUntil(raw.getAnomalyControlUntil());
    target.setPriorityRank(raw.getPriorityRank());
    target.setCounterId(raw.getCounterId());
    return target;
}

/** Plain, unenriched row read by id -- no position/geo computation, just what's on the row. */
private TokenDto getRaw(int id) {
    List<TokenDto> rows = jdbc.query("SELECT * FROM tokens WHERE id = :id", Map.of("id", id), TOKEN_ROW_MAPPER);
    return rows.isEmpty() ? null : rows.get(0);
}

private int countWaitingAhead(TokenDto token) {
    Map<String, Object> params = new HashMap<>();
    params.put("sortKey", token.getPriorityRank() != null ? token.getPriorityRank() : token.getId());
    params.put("hospitalId", token.getHospitalId());
    params.put("category", token.getCategory());
    Integer count = jdbc.queryForObject(
        """
        SELECT COUNT(*)::int FROM tokens WHERE status = 'waiting' AND COALESCE(priority_rank, id) < :sortKey
        AND hospital_id IS NOT DISTINCT FROM :hospitalId AND category IS NOT DISTINCT FROM :category
        """,
        params, Integer.class);
    return count == null ? 0 : count;
}

public TokenDto getActiveToken(String phone) {
    cleanExpiredTokens();

    String sql = """
    SELECT * FROM tokens
    WHERE (phone = :phone OR phone = '+' || :phone OR phone = REPLACE(:phone, '+', '') OR phone LIKE '%' || :phone)
    AND status IN ('waiting', 'serving', 'registering_name')
    LIMIT 1
    """;
    List<TokenDto> rows = jdbc.query(sql, Map.of("phone", phone), TOKEN_ROW_MAPPER);
    return rows.isEmpty() ? null : rows.get(0);
}

// -----
// Booking flow: name -> gender -> age -> category -> location ->
// search/select hospital -> confirm. A returning phone that already has
// a known identity (see findKnownIdentity) skips straight from the draft

```

```

// token to "awaiting_category".
// -----

public record KnownIdentity(String name, Integer age, String gender) {
}

/**
 * Best-known (name, age, gender) for this phone from either a past
 * completed visit (token_history) or a past registration that got at
 * least that far (tokens, excluding the placeholder "Patient" default
 * and the just-created draft, which never has age/gender yet) --
 * whichever is most recent. Null if this phone has never given all three
 * before.
 */
private KnownIdentity findKnownIdentity(String phone) {
    String sql = """
        SELECT name, age, gender FROM (
            SELECT name, age, gender, completed_at AS ts FROM token_history WHERE phone = :phone
            UNION ALL
            SELECT name, age, gender, created_at AS ts FROM tokens
            WHERE phone = :phone AND name <> 'Patient' AND age IS NOT NULL AND gender IS NOT NULL
        ) known
        WHERE name IS NOT NULL AND age IS NOT NULL AND gender IS NOT NULL
        ORDER BY ts DESC NULLS LAST
        LIMIT 1
    """;
    List<KnownIdentity> rows = jdbc.query(sql, Map.of("phone", phone),
        (rs, i) -> new KnownIdentity(rs.getString("name"), (Integer) rs.getObject("age"), rs.getString("gender")));
    return rows.isEmpty() ? null : rows.get(0);
}

/**
 * Opens (or returns the existing) draft registration row for this phone.
 * A phone this system already knows (name/age/gender from a past visit
 * or attempt) is pre-filled and jumped straight to "awaiting_category" --
 * only a genuinely new phone is asked for name/gender/age from scratch.
 */
public TokenDto createRegisteringToken(String phone) {
    TokenDto active = getActiveToken(phone);
    if (active != null) {
        return active;
    }
    String sql = """
        INSERT INTO tokens (name, phone, status, session_step)
        VALUES ('Patient', :phone, 'registering_name', 'awaiting_name')
        RETURNING *
    """;
    List<TokenDto> rows = jdbc.query(sql, Map.of("phone", phone), TOKEN_ROW_MAPPER);
    if (rows.isEmpty()) {
        return null;
    }
    TokenDto draft = rows.get(0);

    KnownIdentity known = findKnownIdentity(phone);
    if (known == null) {
        return draft;
    }
    List<TokenDto> updated = jdbc.query(
        "UPDATE tokens SET name = :name, age = :age, gender = :gender, session_step = 'awaiting_category' WHERE id = :id RETURNING *",
        Map.of("name", known.name(), "age", known.age(), "gender", known.gender(), "id", draft.getId()),
        TOKEN_ROW_MAPPER);
    return updated.isEmpty() ? draft : updated.get(0);
}

/** STEP 1: name captured -> next asks gender. */
public TokenDto captureName(int tokenId, String name) {
    List<TokenDto> rows = jdbc.query(
        "UPDATE tokens SET name = :name, session_step = 'awaiting_gender' WHERE id = :id RETURNING *",
        Map.of("name", name, "id", tokenId), TOKEN_ROW_MAPPER);
    return rows.isEmpty() ? null : rows.get(0);
}

/** STEP 2: gender captured -> next asks age. */
public TokenDto captureGender(int tokenId, String gender) {
    List<TokenDto> rows = jdbc.query(

```

```

        "UPDATE tokens SET gender = :gender, session_step = 'awaiting_age' WHERE id = :id RETURNING *",
        Map.of("gender", gender, "id", tokenId), TOKEN_ROW_MAPPER);
    return rows.isEmpty() ? null : rows.get(0);
}

/** STEP 3: age captured -> next asks category (department). Token is not queued yet -- that only happens on confirmBooking. */
public TokenDto captureAge(int tokenId, int age) {
    List<TokenDto> rows = jdbc.query(
        "UPDATE tokens SET age = :age, session_step = 'awaiting_category' WHERE id = :id RETURNING *",
        Map.of("age", age, "id", tokenId), TOKEN_ROW_MAPPER);
    return rows.isEmpty() ? null : rows.get(0);
}

/** STEP 4: category captured -> next asks for the patient's location, to search hospitals by. */
public TokenDto captureCategory(int tokenId, String category) {
    List<TokenDto> rows = jdbc.query(
        "UPDATE tokens SET category = :category, session_step = 'awaiting_location' WHERE id = :id RETURNING *",
        Map.of("category", category, "id", tokenId), TOKEN_ROW_MAPPER);
    return rows.isEmpty() ? null : rows.get(0);
}

public record HospitalSearchOutcome(TokenDto draft, HospitalService.HospitalSearchPage page) {
}

/**
 * STEP 5: location captured -> hospitals offering this token's category,
 * open to its gender, ranked nearest-first, first page. Empty results
 * bounce the draft back to "awaiting_category" instead of stranding the
 * patient on a dead-end step.
 */
public HospitalSearchOutcome searchAndOfferHospitals(int tokenId, SetLocationRequest location) {
    HospitalService.LatLon resolved = hospitalService.resolveLocation(location);
    List<TokenDto> rows = jdbc.query(
        """
        UPDATE tokens
        SET patient_digipin = :digipin, patient_lat = :lat, patient_lon = :lon, search_offset = 0
        WHERE id = :id
        RETURNING *
        """,
        Map.of("digipin", resolved.digipin(), "lat", resolved.lat(), "lon", resolved.lon(), "id", tokenId),
        TOKEN_ROW_MAPPER);
    if (rows.isEmpty()) {
        return null;
    }
    TokenDto draft = rows.get(0);

    HospitalService.HospitalSearchPage page = hospitalService.searchHospitals(
        draft.getCategory(), draft.getGender(), resolved.lat(), resolved.lon(), 0, HOSPITAL_PAGE_SIZE);

    if (page.results().isEmpty()) {
        List<TokenDto> reverted = jdbc.query(
            "UPDATE tokens SET session_step = 'awaiting_category' WHERE id = :id RETURNING *",
            Map.of("id", tokenId), TOKEN_ROW_MAPPER);
        return new HospitalSearchOutcome(reverted.isEmpty() ? draft : reverted.get(0), page);
    }

    jdbc.update("UPDATE tokens SET search_offset = :offset, session_step = 'awaiting_hospital_selection' WHERE id = :id",
        Map.of("offset", HOSPITAL_PAGE_SIZE, "id", tokenId));
    return new HospitalSearchOutcome(getRaw(tokenId), page);
}

/** "Show more" -- next page of the same search, from wherever search_offset last left off. */
public HospitalSearchOutcome showMoreHospitals(int tokenId) {
    TokenDto draft = getRaw(tokenId);
    if (draft == null || draft.getPatientLat() == null || draft.getPatientLon() == null) {
        return null;
    }
    int offset = draft.getSearchOffset() != null ? draft.getSearchOffset() : 0;
    HospitalService.HospitalSearchPage page = hospitalService.searchHospitals(
        draft.getCategory(), draft.getGender(), draft.getPatientLat(), draft.getPatientLon(), offset, HOSPITAL_PAGE_SIZE);
    jdbc.update("UPDATE tokens SET search_offset = :offset WHERE id = :id",
        Map.of("offset", offset + HOSPITAL_PAGE_SIZE, "id", tokenId));
    return new HospitalSearchOutcome(draft, page);
}

```

```

/** "Choose Again" (declined the confirmation card) -- clears the tentative pick and re-shows results from the top. */
public HospitalSearchOutcome restartHospitalSelection(int tokenId) {
    jdbc.update("UPDATE tokens SET search_offset = 0, hospital_id = NULL WHERE id = :id", Map.of("id", tokenId));
    return showMoreHospitals(tokenId);
}

/** STEP 6: patient taps a hospital row -> tentatively selected, next asks for a yes/no confirmation. */
public TokenDto selectHospital(int tokenId, int hospitalId) {
    HospitalDto hospital = hospitalService.getById(hospitalId);
    if (hospital == null) {
        return null;
    }
    List<TokenDto> rows = jdbc.query(
        "UPDATE tokens SET hospital_id = :hospitalId, session_step = 'awaiting_confirmation' WHERE id = :id RETURNING **",
        Map.of("hospitalId", hospitalId, "id", tokenId), TOKEN_ROW_MAPPER);
    return rows.isEmpty() ? null : rows.get(0);
}

/** STEP 7: patient confirms -> the draft actually joins its (hospital, category) queue. */
public TokenDto confirmBooking(int tokenId) {
    TokenDto draft = getRaw(tokenId);
    if (draft == null || draft.getHospitalId() == null) {
        throw new IllegalStateException("No hospital selected yet.");
    }
    HospitalDto hospital = hospitalService.getById(draft.getHospitalId());
    if (hospital == null) {
        throw new IllegalStateException("Selected hospital no longer exists.");
    }
    if (hospital.getCloseTime() != null) {
        java.time.LocalDateTime now = java.time.LocalDateTime.now();
        java.time.LocalDateTime closeTime = hospital.getCloseTime();
        java.time.LocalDateTime cutOffTime = closeTime.minusMinutes(30);
        if (!now.isBefore(cutOffTime) && now.isBefore(closeTime.plusHours(2))) {
            throw new IllegalStateException("Online token booking for today is closed because "
                + hospital.getName() + " OPD closes at " + closeTime + " (within 30 minutes). Please visit the reception counter directly
or book for tomorrow.");
        }
    }
    hospitalDepartmentService.ensure(hospital.getId(), draft.getCategory());

    jdbc.update(
        """
        UPDATE tokens SET status = 'waiting', session_step = 'menu', created_at = NOW(),
            daily_number = :dailyNumber
        WHERE id = :id
        """,
        Map.of("id", tokenId, "dailyNumber", nextDailyNumber(hospital.getId(), draft.getCategory())));

    if (draft.getPatientLat() != null && draft.getPatientLon() != null) {
        try {
            applyPatientLocation(tokenId, new SetLocationRequest(draft.getPatientDigipin(), draft.getPatientLat(), draft.getPatientLon()),
hospital);
        } catch (Exception e) {
            log.error("Could not apply patient location on booking confirm for token {}: {}", tokenId, e.getMessage());
        }
    }
    return getTokenDetails(String.valueOf(tokenId));
}

// -----
// Direct/API booking (not via the WhatsApp search flow) -- books into
// whichever hospital the patient picked (see FindHospital.jsx / the
// "nearby" search), falling back to this deployment's single operating
// hospital only when none was given, for callers that never offer a
// hospital picker (e.g. a single-hospital deployment's own booking form).
// Still requires a category since every token belongs to a
// (hospital, category) queue now.
// -----

/** age/gender/location are all optional -- when location is given (current GPS or manually entered), the distance-based notify window is
computed immediately. hospitalId is optional -- omit it to book into this deployment's single operating hospital. */
public CreateTokenResult createToken(String name, Integer age, String gender, String category, String phone, SetLocationRequest location,
Integer hospitalId) {
    cleanExpiredTokens();

```

```

    if (appProperties.isOtpRequiredForRegistration() && !otpService.isPhoneVerifiedRecently(phone)) {
        throw new IllegalStateException("Please verify your phone number with the OTP sent via SMS before registering.");
    }

    String canonicalCategory = MedicalCategory.canonicalize(category);
    if (canonicalCategory == null) {
        throw new IllegalArgumentException("Unknown category: \"" + category + "\". See GET /api/hospitals/categories for the valid
list.");
    }

    TokenDto active = getActiveToken(phone);
    if (active != null) {
        return new CreateTokenResult(true, getTokenDetails(String.valueOf(active.getId())));
    }

    HospitalDto hospital;
    if (hospitalId != null) {
        hospital = hospitalService.getById(hospitalId);
        if (hospital == null) {
            throw new IllegalArgumentException("Selected hospital not found.");
        }
    } else {
        hospital = hospitalService.getOperatingHospital();
    }
    if (hospital != null && hospital.getCloseTime() != null) {
        java.time.LocalDateTime now = java.time.LocalDateTime.now();
        java.time.LocalDateTime closeTime = hospital.getCloseTime();
        java.time.LocalDateTime cutOffTime = closeTime.minusMinutes(30);
        if (!now.isBefore(cutOffTime) && now.isBefore(closeTime.plusHours(2))) {
            throw new IllegalStateException("Online token booking for today is closed because OPD closes at "
                + closeTime + " (within 30 minutes). Please visit the reception counter directly or book for tomorrow.");
        }
    }

    String defaultName = (name == null || name.isBlank()) ? "Patient" : name;
    if (hospital != null) {
        hospitalDepartmentService.ensure(hospital.getId(), canonicalCategory);
    }

    Integer resolvedHospitalId = hospital != null ? hospital.getId() : null;
    Map<String, Object> insertParams = new HashMap<>();
    insertParams.put("name", defaultName);
    insertParams.put("age", age);
    insertParams.put("gender", gender == null ? null : gender.toLowerCase());
    insertParams.put("category", canonicalCategory);
    insertParams.put("phone", phone);
    insertParams.put("hospitalId", resolvedHospitalId);
    insertParams.put("dailyNumber", nextDailyNumber(resolvedHospitalId, canonicalCategory));

    Integer newId = jdbc.queryForObject(
        """
        INSERT INTO tokens (name, age, gender, category, phone, status, session_step, hospital_id, daily_number)
        VALUES (:name, :age, :gender, :category, :phone, 'waiting', 'menu', :hospitalId, :dailyNumber)
        RETURNING id
        """,
        insertParams, Integer.class);

    if (location != null && hospital != null) {
        try {
            applyPatientLocation(newId, location, hospital);
        } catch (Exception e) {
            log.error("Could not apply patient location for token {}: {}", newId, e.getMessage());
        }
    }

    return new CreateTokenResult(false, getTokenDetails(String.valueOf(newId)));
}

/**
 * The patient-facing "Token #N" for a fresh (hospital, category) booking made
 * today -- 1, 2, 3, ... independently for every hospital's every department,
 * unlike the raw `id` column (a single sequence shared across every hospital
 * and department in the whole system, which is what patients used to be
 * shown -- confusing when e.g. a patient is 1st in line but sees "Token #13"
 * because 12 other tokens already exist elsewhere today). Not a DB sequence:

```

```

* just "one more than today's highest daily_number in this (hospital,
* category)" -- good enough for a display label under normal booking
* volume/concurrency; a true collision just means two patients momentarily
* share a display number, which self-corrects on the next booking.
*/
private Integer nextDailyNumber(Integer hospitalId, String category) {
    Map<String, Object> params = new HashMap<>();
    params.put("hospitalId", hospitalId);
    params.put("category", category);
    Integer max = jdbc.queryForObject(
        """
        SELECT MAX(daily_number) FROM tokens
        WHERE hospital_id IS NOT DISTINCT FROM :hospitalId AND category IS NOT DISTINCT FROM :category
        AND created_at::date = CURRENT_DATE
        """,
        params, Integer.class);
    return (max == null ? 0 : max) + 1;
}

/** Sets/updates a patient's location on an existing token and recomputes distance + notify window from it. */
public TokenDto updateTokenLocation(int tokenId, SetLocationRequest location) {
    TokenDto token = getRaw(tokenId);
    if (token == null) {
        return null;
    }
    HospitalDto hospital = token.getHospitalId() != null
        ? hospitalService.getById(token.getHospitalId())
        : hospitalService.getOperatingHospital();
    if (hospital == null) {
        throw new IllegalStateException("Hospital location is not configured yet.");
    }
    applyPatientLocation(tokenId, location, hospital);
    return getTokenDetails(String.valueOf(tokenId));
}

/**
 * Resolves the patient's location to a real TomTom routing ETA (falling
 * back to the straight-line estimate when TomTom isn't available -- see
 * TomTomRoutingService) and stores it as this token's "reaching time".
 * Clears any in-flight notify/anomaly-control state -- a relocation
 * invalidates whatever plan was computed off the old position -- so
 * runTreatmentTimingTick treats this token as freshly eligible again.
 */
private void applyPatientLocation(int tokenId, SetLocationRequest location, HospitalDto hospital) {
    HospitalService.LatLon resolved = hospitalService.resolveLocation(location);
    TomTomRoutingService.RouteEstimate route = tomTomRoutingService.estimate(
        hospital.getLatitude(), hospital.getLongitude(), resolved.lat(), resolved.lon());

    jdbc.update(
        """
        UPDATE tokens
        SET patient_digipin = :digipin, patient_lat = :lat, patient_lon = :lon,
            distance_km = :distanceKm, travel_minutes = :travelMinutes,
            treatment_remaining_minutes = NULL, notified_ready_at = NULL, anomaly_control_until = NULL
        WHERE id = :id
        """,
        Map.of("digipin", resolved.digipin(), "lat", resolved.lat(), "lon", resolved.lon(),
            "distanceKm", route.distanceKm(), "travelMinutes", route.travelMinutes(), "id", tokenId));
}

/**
 * Can this patient realistically reach the hospital before OPD closes,
 * given their current distance? Pure read -- callers decide what to do
 * with a "no" (e.g. show "Continue anyway / Choose another hospital").
 */
public GeoDistanceService.ArrivalFeasibility checkClosingTimeFeasibility(double lat, double lon) {
    HospitalDto hospital = hospitalService.getOperatingHospital();
    if (hospital == null) {
        throw new IllegalStateException("Hospital location is not configured yet.");
    }
    double distanceKm = geoDistanceService.distanceKm(hospital.getLatitude(), hospital.getLongitude(), lat, lon);
    double travelMinutes = geoDistanceService.estimatedTravelMinutes(distanceKm);

    java.time.LocalDateTime now = java.time.LocalDateTime.now();
    long minutesUntilClose = java.time.Duration.between(now, hospital.getCloseTime()).toMinutes();

```

```

        return geoDistanceService.checkClosingTime(distanceKm, travelMinutes, minutesUntilClose);
    }

    // -----
    // Treatment-timing: real routing ETA ("reaching time") vs. how many
    // minutes of queue work are still ahead of the patient ("treatment
    // remaining time"), driving the "go now" WhatsApp + voice-call trigger
    // and the anomaly-control grace window. Polled by
    // service.TreatmentTimingScheduler on a fixed interval (unlike the old
    // straight-line system, this can't rely on "runs on every queue read" --
    // a notify/call has to fire on a wall-clock deadline even with nobody
    // polling the API right now).
    // -----

    /** One scheduler tick: fire any due "go now" triggers, then resolve any expired anomaly-control grace windows. */
    public void runTreatmentTimingTick() {
        try {
            evaluateNotifyTriggers();
        } catch (Exception e) {
            log.error("Error evaluating treatment-timing notify triggers: {}", e.getMessage());
        }
        try {
            evaluateAnomalyExpiry();
        } catch (Exception e) {
            log.error("Error evaluating anomaly-control expiry: {}", e.getMessage());
        }
    }

    /**
     * Every waiting, not-yet-notified, not-yet-arrived token with a known
     * travel ETA: recompute how many minutes of queue work are still ahead
     * of it (persisted for display either way), and the moment that's no
     * longer comfortably longer than the travel ETA (+ the configured
     * buffer), fire the WhatsApp notification + voice call together and
     * open its anomaly-control grace window.
     */
    private void evaluateNotifyTriggers() {
        List<Map<String, Object>> candidates = jdbc.queryForList(
            """
            SELECT id FROM tokens
            WHERE status = 'waiting' AND travel_minutes IS NOT NULL AND notified_ready_at IS NULL
            AND is_verified IS NOT TRUE
            """, Collections.emptyMap());

        for (Map<String, Object> row : candidates) {
            TokenDto token = getRaw((Integer) row.get("id"));
            if (token == null || token.getHospitalId() == null || token.getCategory() == null || token.getTravelMinutes() == null) {
                continue;
            }
            HospitalDto hospital = hospitalService.getById(token.getHospitalId());
            if (hospital == null) {
                continue;
            }

            double treatmentRemaining = computeTreatmentRemainingMinutes(token, hospital);
            jdbc.update("UPDATE tokens SET treatment_remaining_minutes = :v WHERE id = :id",
                Map.of("v", treatmentRemaining, "id", token.getId()));

            double travelMinutes = token.getTravelMinutes();
            double buffer = appProperties.getNotifyBufferMinutes();
            if (treatmentRemaining <= travelMinutes + buffer) {
                fireHeadToHospitalTrigger(token, hospital, travelMinutes, treatmentRemaining);
            }
        }
    }

    /**
     * R = ((Ta >= 3 ? Ta - 3 : 0) * minimum-time-to-treat + 2) / active-counters,
     * for this token's own (hospital, category) department. Ta is the count
     * of *genuinely* waiting tokens ahead -- see countGenuinelyAheadForEta:
     * a token that's unverified but still inside its anomaly-control grace
     * window doesn't hold up anyone behind it for this estimate, since it's
     * effectively set aside (not competing for a counter right now) until
     * its grace ends and it's either pushed back or marked missed.
     */

```

```

private double computeTreatmentRemainingMinutes(TokenDto token, HospitalDto hospital) {
    int ta = countGenuinelyAheadForEta(token);
    int activeCounters = hospitalDepartmentService.activeCounters(token.getHospitalId(), token.getCategory());
    Integer minServiceMinutes = hospital.getMinServiceMinutes() != null
        ? hospital.getMinServiceMinutes() : hospital.getAvgServiceMinutes();
    double minimumTime = minServiceMinutes != null ? minServiceMinutes : appProperties.getAvgServiceMinutes();

    int aheadBeyondHeadstart = ta >= ETA_WAITING_HEADSTART_TOKENS ? (ta - ETA_WAITING_HEADSTART_TOKENS) : 0;
    return (aheadBeyondHeadstart * minimumTime + ETA_FIXED_BUFFER_MINUTES) / Math.max(1, activeCounters);
}

/**
 * Ta for the R estimate above only -- waiting tokens strictly ahead of
 * this one, excluding any currently parked in their anomaly-control
 * grace window (see class docs / handleNoShowOrMiss). Deliberately a
 * separate query from countWaitingAhead(): that one still needs to count
 * every waiting token, graced or not, for patient-facing "your position"
 * displays and for the missed-vs-push-back boundary in
 * handleNoShowOrMiss -- only the ETA estimate treats a graced token as
 * temporarily out of the way.
 */
private int countGenuinelyAheadForEta(TokenDto token) {
    Map<String, Object> params = new HashMap<>();
    params.put("sortKey", token.getPriorityRank() != null ? token.getPriorityRank() : token.getId());
    params.put("hospitalId", token.getHospitalId());
    params.put("category", token.getCategory());
    Integer count = jdbc.queryForObject(
        """
        SELECT COUNT(*)::int FROM tokens WHERE status = 'waiting' AND COALESCE(priority_rank, id) < :sortKey
        AND hospital_id IS NOT DISTINCT FROM :hospitalId AND category IS NOT DISTINCT FROM :category
        AND NOT (is_verified IS NOT TRUE AND anomaly_control_until IS NOT NULL AND anomaly_control_until > NOW())
        """,
        params, Integer.class);
    return count == null ? 0 : count;
}

/**
 * Fires once per notify cycle: marks the token notified, opens an
 * anomaly-control grace window sized to whatever slack the patient's
 * travel time buys them beyond their queue wait (0 if none -- the
 * "reaching time < remaining time" case falls straight through to the
 * exponential push-back almost immediately, per the product spec), and
 * sends the localized WhatsApp message + Twilio voice call together.
 */
private void fireHeadToHospitalTrigger(TokenDto token, HospitalDto hospital, double travelMinutes, double treatmentRemaining) {
    double slackMinutes = Math.max(0, travelMinutes - treatmentRemaining);
    int updated = jdbc.update(
        "UPDATE tokens SET notified_ready_at = NOW(), anomaly_control_until = NOW() + (:slack || ' minutes')::INTERVAL "
        + "WHERE id = :id AND notified_ready_at IS NULL",
        Map.of("slack", String.valueOf(slackMinutes), "id", token.getId()));
    if (updated == 0) {
        return; // lost a race with another tick/instance -- already notified
    }

    Lang lang = resolveLang(token.getPhone());
    String message = botMessages.headingToHospitalNotification(
        lang, token.getName(), token.displayNumber(), token.getCategory(), hospital.getName());
    whatsappService.sendWhatsAppMessage(token.getPhone(), message);
    twilioStudioCallService.triggerHeadToHospitalCall(token.getPhone(), message, lang);
}

/** This phone's chosen WhatsApp bot language (see WaSessionService), defaulting to English if it's never messaged the bot. */
private Lang resolveLang(String phone) {
    WaSessionService.WaSession session = waSessionService.get(phone);
    return (session != null && session.language() != null) ? session.language() : Lang.EN;
}

/**
 * Every waiting, not-yet-arrived token whose anomaly-control grace
 * window has elapsed: push it back exponentially (same 1, 2, 4, 8, 16,
 * ... rule as an admin's manual "not come yet"), unless it's already
 * sitting at the very back of its department's queue with nowhere left
 * to push back to -- at that point it's genuinely missed.
 */
private void evaluateAnomalyExpiry() {

```

```

List<Map<String, Object>> due = jdbc.queryForList(
    """
        SELECT id FROM tokens
        WHERE status = 'waiting' AND anomaly_control_until IS NOT NULL AND anomaly_control_until <= NOW()
        AND is_verified IS NOT TRUE
    """, Collections.emptyMap());

for (Map<String, Object> row : due) {
    TokenDto token = getRaw((Integer) row.get("id"));
    if (token != null && "waiting".equals(token.getStatus())) {
        handleNoShowOrMiss(token);
    }
}

}

/**
 * Shared by evaluateAnomalyExpiry() above and claimNextEligibleWaitingToken's
 * "still not verified when their turn comes up" branch: push the token
 * back exponentially if there's still room to (reuses pushBackNoShow, so
 * the same no_show_count keeps growing 1, 2, 4, 8, 16, ... regardless of
 * which of the two callers triggered it); otherwise it's already at the
 * back of its own department's queue with nowhere further to go, so it's
 * marked missed instead.
 */
private void handleNoShowOrMiss(TokenDto token) {
    int queueSize = otherWaitingRanksInOrder(token).size() + 1;
    int currentPosition = countWaitingAhead(token) + 1;
    if (currentPosition >= queueSize) {
        updateTokenStatus(String.valueOf(token.getId()), "missed");
    } else {
        pushBackNoShow(token.getId());
    }
}

private void sendTurnNotification(String phone, int displayNumber) {
    whatsappService.sendWhatsAppMessage(phone,
        "👉 It's your turn now! Please come to the counter for Token #" + displayNumber + ".");
}

private void sendNextInLineNotification(String phone, int displayNumber) {
    whatsappService.sendWhatsAppMessage(phone,
        "👉 You are next in line for Token #" + displayNumber + "! There is only 1 person ahead of you. Please be ready.");
}

public UpdateStatusResult updateTokenStatus(String idStr, String status) {
    int tokenId = Integer.parseInt(idStr);

    if ("serving".equals(status)) {
        TokenDto target = getRaw(tokenId);
        if (target == null) {
            return null;
        }
    }

    Map<String, Object> scopeParams = new HashMap<>();
    scopeParams.put("id", tokenId);
    scopeParams.put("hospitalId", target.getHospitalId());
    scopeParams.put("category", target.getCategory());
    jdbc.update(
        """
            UPDATE tokens SET status = 'completed', completed_at = NOW() WHERE status = 'serving' AND id != :id
            AND hospital_id IS NOT DISTINCT FROM :hospitalId AND category IS NOT DISTINCT FROM :category
        """,
        scopeParams);

    List<TokenDto> rows = jdbc.query(
        "UPDATE tokens SET status = 'serving', served_at = NOW(), no_show_count = 0 WHERE id = :id RETURNING *",
        Map.of("id", tokenId), TOKEN_ROW_MAPPER);
    if (rows.isEmpty()) {
        return null;
    }
    TokenDto token = rows.get(0);
    sendTurnNotification(token.getPhone(), token.displayNumber());

    TokenDto view = TokenDto.builder()
        .id(token.getId()).phone(token.getPhone()).name(token.getName()).status(token.getStatus())

```

```

        .sessionStep(token.getSessionStep()).createdAt(token.getCreatedAt()).servedAt(token.getServedAt())
        .build();
    return new UpdateStatusResult(view, null);
}

if ("completed".equals(status) || "missed".equals(status)) {
    String timeColumn = "completed".equals(status) ? "completed_at" : "missed_at";
    String updateSql = "UPDATE tokens SET status = :status, " + timeColumn + " = NOW() WHERE id = :id RETURNING *";
    List<TokenDto> rows = jdbc.query(updateSql, Map.of("status", status, "id", tokenId), TOKEN_ROW_MAPPER);
    if (rows.isEmpty()) {
        return null;
    }
    TokenDto updatedToken = rows.get(0);

    if ("completed".equals(status)) {
        archiveToHistory(updatedToken);
        whatsappService.sendWhatsAppMessage(updatedToken.getPhone(),
            "✅ Token #" + updatedToken.displayNumber() + " has been served. Thank you for visiting "
            + appProperties.getClinicName() + "! 🏥");
    } else {
        whatsappService.sendWhatsAppMessage(updatedToken.getPhone(),
            "⚠️ You missed your turn for Token #" + updatedToken.displayNumber()
            + ".\n\n👉 Please send \"Hi\" or \"Hello\" again to generate a new token.");
    }

    Integer nextServingId = advanceToNextEligibleWaiting(updatedToken.getHospitalId(), updatedToken.getCategory());

    TokenDto view = TokenDto.builder()
        .id(updatedToken.getId()).phone(updatedToken.getPhone()).name(updatedToken.getName())
        .status(updatedToken.getStatus()).sessionStep(updatedToken.getSessionStep())
        .createdAt(updatedToken.getCreatedAt()).servedAt(updatedToken.getServedAt())
        .completedAt(updatedToken.getCompletedAt()).missedAt(updatedToken.getMissedAt())
        .build();
    return new UpdateStatusResult(view, nextServingId);
}

// Fallback branch: only remaining valid status is 'waiting' (route validation
// restricts status to waiting/serving/completed/missed).
List<TokenDto> rows = jdbc.query(
    "UPDATE tokens SET status = :status, served_at = NULL, completed_at = NULL, missed_at = NULL WHERE id = :id RETURNING *",
    Map.of("status", status, "id", tokenId), TOKEN_ROW_MAPPER);
if (rows.isEmpty()) {
    return null;
}
TokenDto token = rows.get(0);
TokenDto view = TokenDto.builder()
    .id(token.getId()).phone(token.getPhone()).name(token.getName()).status(token.getStatus())
    .sessionStep(token.getSessionStep()).createdAt(token.getCreatedAt())
    .build();
return new UpdateStatusResult(view, null);
}

/**
 * Copies a just-completed token into the permanent token_history ledger.
 * Doesn't touch/delete the "tokens" row -- today's live stats/queue view
 * (getQueue()) still count it exactly as before; this just also gives it
 * a durable, per-phone-searchable home (see getPatientHistory) that
 * outlives it. The same phone number was already free to book a new
 * token -- for a different patient, different name/age -- the instant
 * this one left 'waiting'/'serving'/'registering_name'; this is purely
 * the audit trail for that.
 */
private void archiveToHistory(TokenDto token) {
    // Map.of() rejects null values outright, and age/gender/category/hospitalId/counterId/
    // servedAt can legitimately be null -- a plain HashMap tolerates them.
    Map<String, Object> params = new HashMap<>();
    params.put("tokenId", token.getId());
    params.put("phone", token.getPhone());
    params.put("name", token.getName());
    params.put("age", token.getAge());
    params.put("gender", token.getGender());
    params.put("category", token.getCategory());
    params.put("hospitalId", token.getHospitalId());
    params.put("counterId", token.getCounterId());
    params.put("createdAt", token.getCreatedAt());

```

```

        params.put("servedAt", token.getServedAt());
        params.put("completedAt", token.getCompletedAt() != null ? token.getCompletedAt() : java.time.LocalDateTime.now());

        jdbc.update(
            """
            INSERT INTO token_history (token_id, phone, name, age, gender, category, hospital_id, counter_id, created_at, served_at,
completed_at)
            VALUES (:tokenId, :phone, :name, :age, :gender, :category, :hospitalId, :counterId, :createdAt, :servedAt, :completedAt)
            """,
            params);
    }

    /** Every past completed visit under this phone number, most recent first -- across however many different patients have used it. */
    public List<TokenHistoryDto> getPatientHistory(String phone) {
        return jdbc.query(
            "SELECT * FROM token_history WHERE phone = :phone ORDER BY completed_at DESC NULLS LAST, id DESC",
            Map.of("phone", phone), TOKEN_HISTORY_ROW_MAPPER);
    }

    /**
     * Picks the next waiting token in this (hospitalId, category) department
     * to serve, in queue order, auto-skipping (marking 'missed', no manual
     * intervention needed) any token that was already sent its
     * distance-based "get ready" ping but never checked in (barcode-scanned
     * / is_verified) by the time its turn came up -- the "Missed Priority
     * Window" rule. Also pings the following token as "next in line" once a
     * token is actually put into service.
     */
    private Integer advanceToNextEligibleWaiting(Integer hospitalId, String category) {
        TokenDto claimed = claimNextEligibleWaitingToken(hospitalId, category);
        if (claimed == null) {
            return null;
        }
        sendTurnNotification(claimed.getPhone(), claimed.displayNumber());

        Map<String, Object> params = new HashMap<>();
        params.put("hospitalId", hospitalId);
        params.put("category", category);
        List<Map<String, Object>> upNext = jdbc.queryForList(
            """
            SELECT id, phone, daily_number FROM tokens WHERE status = 'waiting'
            AND hospital_id IS NOT DISTINCT FROM :hospitalId AND category IS NOT DISTINCT FROM :category
            ORDER BY %s LIMIT 1
            """,
            params);
        if (!upNext.isEmpty()) {
            Integer dailyNumber = (Integer) upNext.get(0).get("daily_number");
            sendNextInLineNotification((String) upNext.get(0).get("phone"), dailyNumber != null ? dailyNumber : 0);
        }
        return claimed.getId();
    }

    /**
     * Atomically claims the front-most waiting token in this
     * (hospitalId, category) department and flips it to 'serving'. A token
     * that's been notified but still isn't checked in (is_verified) is
     * skipped -- someone not physically present can't be served -- but only
     * *penalized* (pushed back exponentially, or marked missed if there's
     * nowhere left to push back to, via handleNoShowOrMiss) once its
     * anomaly-control grace window has actually elapsed; while they're still
     * within their travel-time grace they're left exactly where they are and
     * simply tried again next time a counter frees up. Shared by the
     * single-counter flow above and by CounterAssignmentService for the
     * multi-counter package -- neither sends the "your turn" notification
     * here, since the multi-counter caller needs to mention which counter to
     * go to.
     */
    public TokenDto claimNextEligibleWaitingToken(Integer hospitalId, String category) {
        int guard = 0;
        while (guard++ < 200) {
            Map<String, Object> scopeParams = new HashMap<>();
            scopeParams.put("hospitalId", hospitalId);
            scopeParams.put("category", category);
            List<Map<String, Object>> candidates = jdbc.queryForList(
                """

```

```

        SELECT id, is_verified, notified_ready_at, anomaly_control_until FROM tokens WHERE status = 'waiting'
        AND hospital_id IS NOT DISTINCT FROM :hospitalId AND category IS NOT DISTINCT FROM :category
        ORDER BY %s LIMIT 1
        """".formatted(QUEUE_ORDER),
        scopeParams);
    if (candidates.isEmpty()) {
        return null;
    }
    Map<String, Object> candidate = candidates.get(0);
    int id = (Integer) candidate.get("id");
    boolean verified = Boolean.TRUE.equals(candidate.get("is_verified"));
    boolean wasNotified = candidate.get("notified_ready_at") != null;
    java.time.LocalDateTime anomalyUntil = (java.time.LocalDateTime) candidate.get("anomaly_control_until");

    if (wasNotified && !verified) {
        boolean stillWithinGrace = anomalyUntil != null && anomalyUntil.isAfter(java.time.LocalDateTime.now());
        if (stillWithinGrace) {
            // The one desk this method serves has nothing else it can do --
            // the front of the line is still legitimately allowed time to
            // arrive, so this claim attempt simply fails; the caller (or the
            // next admin/scheduler trigger) tries again later. Returning here
            // instead of looping is a pure efficiency fix -- looping would
            // just re-select this exact same unchanged row up to `guard`
            // times for the same outcome.
            return null;
        }
        TokenDto fullToken = getRaw(id);
        if (fullToken != null) {
            handleNoShowOrMiss(fullToken);
        }
        continue; // that push-back/miss changed the front of the line -- re-check it
    }

    List<TokenDto> claimedRows = jdbc.query(
        "UPDATE tokens SET status = 'serving', served_at = NOW(), no_show_count = 0 WHERE id = :id AND status = 'waiting' RETURNING
        id, is_verified, notified_ready_at, anomaly_control_until, no_show_count,
        Map.of(\"id\", id), TOKEN_ROW_MAPPER);
    if (claimedRows.isEmpty()) {
        continue; // lost a race (e.g. concurrent admin action) -- retry
    }
    return claimedRows.get(0);
}

log.error("claimNextEligibleWaitingToken() hit its safety guard without resolving -- check for a stuck loop.");
return null;
}

/**
 * Multi-counter variant of claimNextEligibleWaitingToken(). The
 * single-counter version above is correct to block on a still-graced
 * front-of-line token -- with exactly one desk, there's nothing else it
 * could usefully do. With N counters genuinely running in parallel,
 * though, that same "wait for the front" rule would freeze every counter
 * over one patient's commute, even while other counters are free and
 * other, fully eligible patients are waiting right behind them. This
 * version scans the whole department's waiting list in one query instead
 * of re-querying LIMIT 1, and -- unlike the single-counter version --
 * actually skips PAST a still-graced token to try the next candidate,
 * without ever touching it: it's left exactly where it is, still first
 * in line, simply not eligible to be the answer to *this* particular
 * "which counter's about to free up" question. A token whose grace has
 * actually expired is still resolved (pushed back / marked missed)
 * exactly as the single-counter version does.
 */
public TokenDto claimNextEligibleWaitingTokenForCounter(Integer hospitalId, String category) {
    int guard = 0;
    while (guard++ < 200) {
        Map<String, Object> scopeParams = new HashMap<>();
        scopeParams.put("hospitalId", hospitalId);
        scopeParams.put("category", category);
        List<Map<String, Object>> candidates = jdbc.queryForList(
            """"
            SELECT id, is_verified, notified_ready_at, anomaly_control_until FROM tokens WHERE status = 'waiting'
            AND hospital_id IS NOT DISTINCT FROM :hospitalId AND category IS NOT DISTINCT FROM :category
            ORDER BY %s
            """".formatted(QUEUE_ORDER),

```

```

        scopeParams);
    if (candidates.isEmpty()) {
        return null;
    }

    boolean rankingChanged = false;
    for (Map<String, Object> candidate : candidates) {
        int id = (Integer) candidate.get("id");
        boolean verified = Boolean.TRUE.equals(candidate.get("is_verified"));
        boolean wasNotified = candidate.get("notified_ready_at") != null;
        java.time.LocalDateTime anomalyUntil = (java.time.LocalDateTime) candidate.get("anomaly_control_until");

        if (wasNotified && !verified) {
            boolean stillWithinGrace = anomalyUntil != null && anomalyUntil.isAfter(java.time.LocalDateTime.now());
            if (stillWithinGrace) {
                continue; // leave them exactly where they are -- try the next candidate in this same pass instead
            }
            TokenDto fullToken = getRaw(id);
            if (fullToken != null) {
                handleNoShowOrMiss(fullToken);
            }
            rankingChanged = true;
            break; // ranks shifted underneath this pass -- rescan from a clean ordering
        }

        List<TokenDto> claimedRows = jdbc.query(
            "UPDATE tokens SET status = 'serving', served_at = NOW(), no_show_count = 0 WHERE id = :id AND status = 'waiting'
RETURNING **",
            Map.of("id", id), TOKEN_ROW_MAPPER);
        if (!claimedRows.isEmpty()) {
            return claimedRows.get(0);
        }
        // lost a race (e.g. concurrent admin action) -- try the next candidate in this same pass
    }
    if (!rankingChanged) {
        return null; // scanned every waiting token this pass -- nobody's actually eligible right now
    }
}
log.error("claimNextEligibleWaitingTokenForCounter() hit its safety guard without resolving -- check for a stuck loop.");
return null;
}

public TokenDto verifyTokenByAdmin(int id) {
    List<TokenDto> rows = jdbc.query(
        "UPDATE tokens SET is_verified = TRUE, verified_at = NOW() WHERE id = :id RETURNING **",
        Map.of("id", id), TOKEN_ROW_MAPPER);
    if (rows.isEmpty()) {
        return null;
    }
    TokenDto token = rows.get(0);
    TokenDto details = getTokenDetails(String.valueOf(token.getId()));

    Integer pos = details != null ? details.getPosition() : null;
    String positionText = (pos != null && pos != 0) ? (pos == 1 ? "1st" : pos + "th") : "In Queue";

    String welcomeMsg = ""
        🧑 *WELCOME TO THE HOSPITAL!*

        Hello *%s*,

        Your queue Token *##d* has been scanned and *VERIFIED* by our reception desk! ✅

        📍 *Status:* Checked-In & Verified
        👤 *Queue Position:* %s

        Thank you for visiting %s. Please take a seat in our waiting room. We will notify you on WhatsApp as soon as your turn arrives!
        🧑 ""

        .formatted(token.getName(), token.displayNumber(), positionText, appProperties.getClinicName());

    whatsappService.sendWhatsAppMessage(token.getPhone(), welcomeMsg);

    return details;
}

```

```

// -----
// Anomaly-control: patients already notified/called, still inside their
// travel-time grace window -- see runTreatmentTimingTick. Read-only;
// resolution (push-back or missed) happens automatically once the
// window elapses (evaluateAnomalyExpiry).
// -----

/** Waiting tokens currently inside an active anomaly-control grace window, soonest-to-expire first. hospitalId/category null = no filter.
*/
public List<TokenDto> getAnomalyControlQueue(Integer hospitalId, String category) {
    cleanExpiredTokens();
    StringBuilder sql = new StringBuilder(
        "SELECT * FROM tokens WHERE status = 'waiting' AND anomaly_control_until IS NOT NULL AND anomaly_control_until > NOW()");
    Map<String, Object> params = new HashMap<>();
    appendScope(sql, params, hospitalId, category);
    sql.append(" ORDER BY anomaly_control_until ASC");
    return jdbc.query(sql.toString(), params, TOKEN_ROW_MAPPER);
}

// -----
// Missed queue: admin search, requeue-to-front, reject
// -----

/** Unscoped, hospital-wide view. */
public List<TokenDto> getMissedQueue() {
    return getMissedQueue(null, null);
}

/** Ordered oldest-missed-first, newest-missed-last (a patient who missed later sits at the bottom). hospitalId/category null = no filter.
*/
public List<TokenDto> getMissedQueue(Integer hospitalId, String category) {
    cleanExpiredTokens();
    StringBuilder sql = new StringBuilder("SELECT * FROM tokens WHERE status = 'missed'");
    Map<String, Object> params = new HashMap<>();
    appendScope(sql, params, hospitalId, category);
    sql.append(" ORDER BY missed_at ASC NULLS LAST, id ASC");
    return jdbc.query(sql.toString(), params, TOKEN_ROW_MAPPER);
}

public List<TokenDto> searchMissedQueue(String query) {
    return searchMissedQueue(query, null, null);
}

/** Search the missed queue by token id (exact) or phone number (partial match); hospitalId/category null = no filter. */
public List<TokenDto> searchMissedQueue(String query, Integer hospitalId, String category) {
    cleanExpiredTokens();
    String q = query == null ? "" : query.trim();
    if (q.isEmpty()) {
        return getMissedQueue(hospitalId, category);
    }

    Map<String, Object> params = new HashMap<>();
    params.put("phoneLike", "%" + q + "%");

    StringBuilder sql = new StringBuilder("SELECT * FROM tokens WHERE status = 'missed' AND (");
    if (q.matches("\\d+")) {
        params.put("idExact", Integer.parseInt(q));
        sql.append("id = :idExact OR phone LIKE :phoneLike");
    } else {
        sql.append("phone LIKE :phoneLike");
    }
    sql.append(")");
    appendScope(sql, params, hospitalId, category);
    sql.append(" ORDER BY missed_at ASC NULLS LAST, id ASC");

    return jdbc.query(sql.toString(), params, TOKEN_ROW_MAPPER);
}

/** Puts a missed token back into the waiting queue of its own department, at the very front (position #1). */
public TokenDto requeueMissedToFront(int id) {
    TokenDto missed = getRaw(id);
    if (missed == null) {
        return null;
    }
    Map<String, Object> scopeParams = new HashMap<>();

```

```

scopeParams.put("hospitalId", missed.getHospitalId());
scopeParams.put("category", missed.getCategory());
Double minRank = jdbc.queryForObject(
    """
    SELECT COALESCE(MIN(priority_rank), 0)::float8 FROM tokens WHERE status = 'waiting'
    AND hospital_id IS NOT DISTINCT FROM :hospitalId AND category IS NOT DISTINCT FROM :category
    """,
    scopeParams, Double.class);
double newRank = Math.min(minRank == null ? 0.0 : minRank, 0.0) - 1;

List<TokenDto> rows = jdbc.query(
    """
    UPDATE tokens
    SET status = 'waiting', priority_rank = :rank,
        missed_at = NULL, served_at = NULL, completed_at = NULL, rejected_at = NULL,
        notified_ready_at = NULL
    WHERE id = :id AND status = 'missed'
    RETURNING *
    """, Map.of("id", id, "rank", newRank), TOKEN_ROW_MAPPER);

if (rows.isEmpty()) {
    return null;
}
TokenDto token = rows.get(0);
whatsappService.sendWhatsAppMessage(token.getPhone(),
    "✅ Good news! Reception has reinstated your Token #" + token.displayNumber()
    + " and moved it to the FRONT of the queue. Please come to the counter now.");
return getTokenDetails(String.valueOf(id));
}

/** Closes out a missed token permanently -- it will not be auto-cleaned or reappear anywhere active. */
public TokenDto rejectMissedToken(int id) {
    List<TokenDto> rows = jdbc.query(
        "UPDATE tokens SET status = 'rejected', rejected_at = NOW() WHERE id = :id AND status = 'missed' RETURNING *",
        Map.of("id", id), TOKEN_ROW_MAPPER);
    if (rows.isEmpty()) {
        return null;
    }
    TokenDto token = rows.get(0);
    whatsappService.sendWhatsAppMessage(token.getPhone(),
        "❌ Your missed Token #" + token.displayNumber() + " has been closed by reception. "
        + "Please send \"Hi\" to generate a new token if you'd still like to visit.");
    return token;
}

// -----
// Floating-point priority repositioning + exponential no-show push-back.
// Both reorder a *waiting* token within its own (hospital, category)
// queue by giving it a new priority_rank sitting between two existing
// neighbors' ranks -- fractional/"lexorank"-style indexing, so nobody
// else's rank needs to change. E.g. ranks 1, 2, 3, 4 -> moving the
// rank-1 token to sit between the 2nd and 3rd tokens gives it something
// like 2.5, leaving 2, 2.5, 3, 4 -- everyone else untouched.
// -----

/**
 * Moves a waiting token to the given 1-based position within its own
 * department's waiting queue by computing a new priority_rank between
 * its new neighbors, without renumbering anyone else. targetPosition is
 * clamped to [1, queueSize] -- position 1 is the very front, queueSize
 * is the very back. Service-layer only for now (no HTTP endpoint yet --
 * see prompt.txt).
 */
public TokenDto movePatientToPosition(int tokenId, int targetPosition) {
    TokenDto token = getRaw(tokenId);
    if (token == null || !"waiting".equals(token.getStatus())) {
        throw new IllegalStateException("Only a waiting token can be repositioned.");
    }

    List<Double> otherRanks = otherWaitingRanksInOrder(token);
    int clamped = Math.max(1, Math.min(targetPosition, otherRanks.size() + 1));
    Double before = clamped >= 2 ? otherRanks.get(clamped - 2) : null;
    Double after = clamped <= otherRanks.size() ? otherRanks.get(clamped - 1) : null;

    double newRank;

```

```

    if (before == null && after == null) {
        newRank = 1.0; // only token in the queue
    } else if (before == null) {
        newRank = after - 1.0; // new front of the line
    } else if (after == null) {
        newRank = before + 1.0; // new back of the line
    } else {
        newRank = before + (after - before) / 2.0;
        if (newRank <= before || newRank >= after) {
            // Float precision exhausted between these two neighbors (an
            // extreme number of repeated inserts into the same gap) --
            // spread the whole department's ranks back out to integers
            // and retry once with fresh headroom.
            renumberQueue(token.getHospitalId(), token.getCategory());
            return movePatientToPosition(tokenId, targetPosition);
        }
    }
}

// Also drops any multi-counter reservation this token was holding (see
// CounterAssignmentService#refreshReservations) -- its old position no
// longer means it's up next for whichever counter had earmarked it.
jdbc.update("UPDATE tokens SET priority_rank = :rank, reserved_counter_id = NULL WHERE id = :id",
    Map.of("rank", newRank, "id", tokenId));
return getTokenDetails(String.valueOf(tokenId));
}

/**
 * Reception clicks "not come yet" on a called patient who hasn't shown
 * up -- instead of marking them missed outright, push them back within
 * their own waiting queue by an exponentially growing number of
 * positions each time this happens to the *same* token: 1st click skips
 * 1 position, 2nd skips 2, then 4, 8, 16, ... If the queue doesn't have
 * that many waiting patients left (or the skip has grown past what's
 * left), the token goes straight to the back instead. no_show_count
 * resets to 0 the moment the token actually gets served. Also clears
 * notified_ready_at/anomaly_control_until -- pushing a token back changes
 * where it sits relative to the front, so it needs to become eligible for
 * the "go now" trigger (see runTreatmentTimingTick) again at its new
 * position instead of staying marked as already-notified forever.
 */
public TokenDto pushBackNoShow(int tokenId) {
    TokenDto token = getRaw(tokenId);
    if (token == null || !"waiting".equals(token.getStatus())) {
        throw new IllegalStateException("Only a waiting token can be pushed back.");
    }

    int noShowCount = token.getNoShowCount() == null ? 0 : token.getNoShowCount();
    int skip = noShowCount == 0 ? 1 : (1 << noShowCount); // 1, 2, 4, 8, 16, ...
    int queueSize = otherWaitingRanksInOrder(token).size() + 1; // + this token itself
    int targetPosition = Math.min(skip + 1, queueSize); // not enough patients left -> straight to the back

    jdbc.update(
        "UPDATE tokens SET no_show_count = :count, notified_ready_at = NULL, anomaly_control_until = NULL WHERE id = :id",
        Map.of("count", noShowCount + 1, "id", tokenId));

    TokenDto moved = movePatientToPosition(tokenId, targetPosition);
    whatsappService.sendWhatsAppMessage(token.getPhone(),
        "📞 We called Token #" + token.displayNumber() + " but you weren't ready, so you've been moved back "
        + skip + " position" + (skip == 1 ? "" : "s") + " in the queue. Please stay nearby.");
    return moved;
}

/** Every *other* waiting token in this token's own (hospital, category) department, in queue order. */
private List<Double> otherWaitingRanksInOrder(TokenDto token) {
    Map<String, Object> scopeParams = new HashMap<>();
    scopeParams.put("hospitalId", token.getHospitalId());
    scopeParams.put("category", token.getCategory());
    scopeParams.put("id", token.getId());
    return jdbc.queryForList(
        """
        SELECT COALESCE(priority_rank, id)::float8 FROM tokens
        WHERE status = 'waiting' AND id != :id
        AND hospital_id IS NOT DISTINCT FROM :hospitalId AND category IS NOT DISTINCT FROM :category
        ORDER BY COALESCE(priority_rank, id) ASC
        """, scopeParams, Double.class);
}

```

```
}

/** Resets a department's waiting queue back to plain integer ranks (1, 2, 3, ...) in its current order. */
private void renumberQueue(Integer hospitalId, String category) {
    Map<String, Object> scopeParams = new HashMap<>();
    scopeParams.put("hospitalId", hospitalId);
    scopeParams.put("category", category);
    List<Integer> orderedIds = jdbc.queryForList(
        """
        SELECT id FROM tokens WHERE status = 'waiting'
        AND hospital_id IS NOT DISTINCT FROM :hospitalId AND category IS NOT DISTINCT FROM :category
        ORDER BY COALESCE(priority_rank, id) ASC
        """, scopeParams, Integer.class);
    for (int i = 0; i < orderedIds.size(); i++) {
        jdbc.update("UPDATE tokens SET priority_rank = :rank WHERE id = :id",
            Map.of("rank", (double) (i + 1), "id", orderedIds.get(i)));
    }
}
}
```

TimeSlotService.java

src/main/java/com/qdischarge/clinicqueue/service/TimeSlotService.java

OPD time windows (`time_slots` / `time_slot_doctors`): opens one time window on one day and rosters a list of doctors onto it in a single call. A hospital can hold any number of slots on the same day with no uniqueness constraint — avoiding overlaps is the caller's responsibility.

```
package com.qdischarge.clinicqueue.service;

import com.qdischarge.clinicqueue.dto.CreateTimeSlotRequest;
import com.qdischarge.clinicqueue.dto.TimeSlotDto;
import lombok.RequiredArgsConstructor;
import org.springframework.jdbc.core.namedparam.MapSqlParameterSource;
import org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate;
import org.springframework.stereotype.Service;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;

/**
 * time_slots / time_slot_doctors: OPD time windows within a day, each
 * assigned to a list of doctors in one call (see CreateTimeSlotRequest). A
 * hospital can hold any number of slots on the same day -- there's no
 * uniqueness constraint beyond the primary key, so overlapping/duplicate
 * slots are the caller's responsibility to avoid.
 */
@Service
@RequiredArgsConstructor
public class TimeSlotService {

    private final NamedParameterJdbcTemplate jdbc;

    public TimeSlotDto create(int hospitalId, CreateTimeSlotRequest req) {
        LocalDateTime start = LocalDateTime.parse(req.startTime());
        LocalDateTime end = LocalDateTime.parse(req.endTime());
        if (!end.isAfter(start)) {
            throw new IllegalArgumentException("endTime must be after startTime.");
        }

        MapSqlParameterSource params = new MapSqlParameterSource()
            .addValue("hospitalId", hospitalId)
            .addValue("category", req.category())
            .addValue("date", req.date())
            .addValue("start", start)
            .addValue("end", end);

        Integer slotId = jdbc.queryForObject(
            """
            INSERT INTO time_slots (hospital_id, category, slot_date, start_time, end_time)
            VALUES (:hospitalId, :category, :date, :start, :end)
            RETURNING id
            """, params, Integer.class);

        for (Integer doctorId : new java.util.LinkedHashSet<>(req.doctorIds())) {
            jdbc.update(
                "INSERT INTO time_slot_doctors (time_slot_id, doctor_id) VALUES (:slotId, :doctorId)",
                Map.of("slotId", slotId, "doctorId", doctorId));
        }
        return getById(slotId);
    }

    public TimeSlotDto getById(int id) {
        List<TimeSlotDto> rows = query("WHERE ts.id = :id", Map.of("id", id));
        return rows.isEmpty() ? null : rows.get(0);
    }

    /** All of a hospital's slots, optionally filtered to one day; soonest first. */
    public List<TimeSlotDto> listForHospital(int hospitalId, LocalDate date) {
        if (date != null) {

```

```
        return query("WHERE ts.hospital_id = :hospitalId AND ts.slot_date = :date",
            Map.of("hospitalId", hospitalId, "date", date));
    }
    return query("WHERE ts.hospital_id = :hospitalId", Map.of("hospitalId", hospitalId));
}

/** Every slot this doctor has been rostered onto, soonest first. */
public List<TimeSlotDto> listForDoctor(int doctorId) {
    return query(
        "WHERE ts.id IN (SELECT time_slot_id FROM time_slot_doctors WHERE doctor_id = :doctorId)",
        Map.of("doctorId", doctorId));
}

private List<TimeSlotDto> query(String whereClause, Map<String, Object> params) {
    List<TimeSlotDto> slots = jdbc.query(
        """
        SELECT ts.* FROM time_slots ts %s
        ORDER BY ts.slot_date ASC, ts.start_time ASC, ts.id ASC
        """.formatted(whereClause),
        params, (rs, rowNum) -> TimeSlotDto.builder()
            .id(rs.getInt("id"))
            .hospitalId(rs.getInt("hospital_id"))
            .category(rs.getString("category"))
            .slotDate(rs.getObject("slot_date", LocalDate.class))
            .startTime(rs.getObject("start_time", LocalTime.class))
            .endTime(rs.getObject("end_time", LocalTime.class))
            .createdAt(rs.getObject("created_at", java.time.LocalDateTime.class))
            .build());

    for (TimeSlotDto slot : slots) {
        List<Integer> doctorIds = jdbc.queryForList(
            "SELECT doctor_id FROM time_slot_doctors WHERE time_slot_id = :id ORDER BY doctor_id ASC",
            Map.of("id", slot.getId(), Integer.class);
        slot.setDoctorIds(new ArrayList<>(doctorIds));
    }
    return slots;
}
}
```

TreatmentTimingScheduler.java

src/main/java/com/qdischarge/clinicqueue/service/TreatmentTimingScheduler.java

The thin wall-clock timer trigger for the real-ETA notify/call logic — all the actual SQL and business rules live in `QueueManagerService`. Ticks are gated on `ApplicationReadyEvent` so the very first `fixedDelay` tick (which fires as soon as the scheduling infrastructure comes up) can never race ahead of `HospitalSeedRunner` and query tables/columns that don't exist yet.

```
package com.qdischarge.clinicqueue.service;

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.boot.context.event.ApplicationReadyEvent;
import org.springframework.context.event.EventListener;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

import java.util.concurrent.atomic.AtomicBoolean;

/**
 * Thin timer trigger for the real-ETA "go now" notify/call and
 * anomaly-control expiry logic (see
 * QueueManagerService#runTreatmentTimingTick). Deliberately just a tick --
 * all the tokens-table SQL and business rules live in QueueManagerService,
 * same as every other queue rule in this app.
 *
 * A wall-clock timer (rather than the old "runs opportunistically on every
 * queue read" pattern -- see QueueManagerService class docs) is required
 * here: a notify/call has to fire, and an elapsed anomaly-control window has
 * to resolve, on its own deadline even if nobody happens to be polling the
 * API at that moment.
 *
 * The very first fixedDelay tick fires as soon as the scheduling
 * infrastructure comes up during context refresh -- which can race ahead of
 * HospitalSeedRunner (a plain ApplicationRunner that runs afterwards, on the
 * main thread) and query tables/columns that don't exist yet. Gate ticks on
 * ApplicationReadyEvent, which only fires once every runner has finished, so
 * ticks before that are skipped rather than erroring.
 */
@Component
@RequiredArgsConstructor
@Slf4j
public class TreatmentTimingScheduler {

    private final QueueManagerService queueManagerService;

    private final AtomicBoolean ready = new AtomicBoolean(false);

    @EventListener(ApplicationReadyEvent.class)
    public void onReady() {
        ready.set(true);
    }

    @Scheduled(fixedDelayString = "${app.timing-poll-interval-ms:15000}")
    public void tick() {
        if (!ready.get()) {
            return;
        }
        queueManagerService.runTreatmentTimingTick();
    }
}
```

TwilioStudioCallService.java

src/main/java/com/qdischarge/clinicqueue/service/TwilioStudioCallService.java

Places the "head to the hospital now" voice call via Twilio's plain Voice Calls API with inline TwiML (a `<Say>` of the message text) — no Studio Flow or public callback URL needed. No-ops (logs and returns) when calling is disabled or no caller number is configured, so a fresh checkout never dials anyone by accident.

```
package com.qdischarge.clinicqueue.service;

import com.qdischarge.clinicqueue.bot.Lang;
import com.qdischarge.clinicqueue.config.AppProperties;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.HttpEntity;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.scheduling.annotation.Async;
import org.springframework.stereotype.Service;
import org.springframework.util.LinkedMultiValueMap;
import org.springframework.util.MultiValueMap;
import org.springframework.web.client.HttpStatusCodeException;
import org.springframework.web.client.RestClientException;
import org.springframework.web.client.RestTemplate;

import java.util.Map;

/**
 * Places the "your token is about to be called, head to the hospital now"
 * voice call, via Twilio's plain Voice Calls API
 * (https://api.twilio.com/2010-04-01/Accounts/{AccountSid}/Calls.json) with
 * inline TwiML (a "TwiML" param, not a Url/Studio Flow) -- plain REST
 * (RestTemplate + Basic Auth), same pattern as OtpService's Twilio Verify/
 * Messages calls, rather than pulling in the Twilio Java SDK for one
 * endpoint. No Studio Flow or public callback URL needed: the TwiML
 * (a <Say> of the message text) is handed to Twilio directly in the
 * request, so Account SID + Auth Token + caller number is enough.
 *
 * Fired alongside the WhatsApp notification from
 * QueueManagerService#runTreatmentTimingTick -- both channels, same
 * trigger, same moment.
 *
 * No-ops (logs and returns) rather than dialing anyone when calling is
 * disabled or the caller number isn't configured -- mirrors OtpService's
 * "fails cleanly on dummy credentials" default, so a fresh checkout never
 * places a real phone call.
 */
@Service
@RequiredArgsConstructor
@Slf4j
public class TwilioStudioCallService {

    private final AppProperties appProperties;
    private final RestTemplate restTemplate;

    private boolean isConfigured() {
        if (!appProperties.isTwilioCallEnabled()) {
            return false;
        }
        String caller = appProperties.getTwilioCallerNumber();
        return caller != null && !caller.isBlank();
    }

    /** Fire-and-forget: never blocks or fails the caller's request thread on a slow/unreachable Twilio. */
    @Async("whatsappExecutor")
    public void triggerHeadToHospitalCall(String toPhone, String message, Lang lang) {
        if (!isConfigured()) {
            log.info("🚫 Twilio call skipped for {} (calling disabled or caller number not configured).",
                    formatPhone(toPhone));
            return;
        }
        try {
```

```

String url = "https://api.twilio.com/2010-04-01/Accounts/%s/Calls.json"
    .formatted(appProperties.getTwilioAccountSid());

MultiValueMap<String, String> body = new LinkedMultiValueMap<>();
body.add("To", toPhone);
body.add("From", appProperties.getTwilioCallerNumber());
body.add("Twiml", buildTwiml(message, lang));

HttpHeaders headers = new HttpHeaders();
headers.setContentType(MediaType.APPLICATION_FORM_URLENCODED);
headers.setBasicAuth(appProperties.getTwilioAccountSid(), appProperties.getTwilioAuthToken());

restTemplate.postForEntity(url, new HttpEntity<>(body, headers), Map.class);
log.info("📞 Twilio call triggered for {}", formatPhone(toPhone));
} catch (RestClientException e) {
    log.error("❌ Twilio Call error for {}: {}", formatPhone(toPhone), extractError(e));
} catch (Exception e) {
    log.error("❌ Could not trigger Twilio call for {}: {}", formatPhone(toPhone), e.getMessage());
}
}

/** Twilio TTS locale for <Say>; Marathi has no dedicated classic voice, so it falls back to Hindi. */
private String sayLanguage(Lang lang) {
    if (lang == Lang.HI || lang == Lang.MR) {
        return "hi-IN";
    }
    return "en-IN";
}

private String buildTwiml(String message, Lang lang) {
    String escaped = message == null ? "" : message
        .replace("&", "&amp;")
        .replace("<", "&lt;")
        .replace(">", "&gt;")
        .replace("\"", "&quot;")
        .replace("'", "&apos;");
    return "<Response><Say language=\"%s\">%s</Say></Response>".formatted(sayLanguage(lang), escaped);
}

private String formatPhone(String phone) {
    return phone == null ? "" : phone.replaceAll("[^0-9+]", "");
}

private String extractError(RestClientException e) {
    if (e instanceof HttpStatusCodeException hsce) {
        return hsce.getStatusCode() + " " + hsce.getResponseBodyAsString();
    }
    return e.getMessage();
}
}

```

WhatsAppService.java

src/main/java/com/qdischarge/clinicqueue/service/WhatsAppService.java

All outbound WhatsApp sends, exclusively via the Evolution API's plain-text send endpoint — interactive concepts (buttons, lists, URL cards) that Evolution has no native primitive for are all rendered down to clean formatted text (numbered options, a footer line) rather than failing. Every public method is `@Async("whatsappExecutor")`, so nothing here ever blocks the calling request thread.

```
package com.qdischarge.clinicqueue.service;

import com.qdischarge.clinicqueue.config.AppProperties;
import com.qdischarge.clinicqueue.dto.WaButton;
import com.qdischarge.clinicqueue.dto.WaListRow;
import com.qdischarge.clinicqueue.dto.WaListSection;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.HttpEntity;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.scheduling.annotation.Async;
import org.springframework.stereotype.Service;
import org.springframework.web.client.HttpStatusException;
import org.springframework.web.client.RestClientException;
import org.springframework.web.client.RestTemplate;

import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/**
 * Service for sending WhatsApp notifications exclusively via Evolution API.
 * Converts interactive messages, buttons, and links into clean, formatted text messages.
 */
@Service
@RequiredArgsConstructor
@Slf4j
public class WhatsAppService {

    private final RestTemplate restTemplate;
    private final AppProperties appProperties;

    private String formatPhone(String phone) {
        return phone == null ? "" : phone.replaceAll("^0-9", "");
    }

    private String extractError(RestClientException e) {
        if (e instanceof HttpStatusException hsce) {
            return hsce.getResponseBodyAsString();
        }
        return e.getMessage();
    }

    // -----
    // Public, async, fire-and-forget entry points
    // -----

    @Async("whatsappExecutor")
    public void sendWhatsAppMessage(String phone, String text) {
        sendWhatsAppMessageSync(phone, text);
    }

    @Async("whatsappExecutor")
    public void sendButtonsMessage(String phone, String title, String description, List<WaButton> buttons, String footer) {
        String effectiveFooter = footer != null ? footer : "qDischarge Smart Queue";
        String formattedText = formatButtonsAsText(title, description, buttons, effectiveFooter);
        sendWhatsAppMessageSync(phone, formattedText);
    }

    @Async("whatsappExecutor")
    public void sendUrlButtonMessage(String phone, String title, String description, String buttonText, String urlTarget, String footer) {
        String text = (title != null && !title.isEmpty() ? title + "\n\n" : "") + description + "\n\n" + " " + urlTarget;
    }
}
```

```

        sendWhatsAppMessageSync(phone, text);
    }

    @Async("whatsappExecutor")
    public void sendPollMessage(String phone, String question, List<String> options) {
        String text = question + "\n\n" + String.join("\n", options);
        sendWhatsAppMessageSync(phone, text);
    }

    @Async("whatsappExecutor")
    public void sendListMessage(String phone, String title, String description, List<WaListSection> sections, String footer, String buttonText)
{
        String text = plainTextFallback(title, description, sections);
        sendWhatsAppMessageSync(phone, text);
    }

    @Async("whatsappExecutor")
    public void sendLocationRequestMessage(String phone, String bodyText) {
        sendWhatsAppMessageSync(phone, bodyText);
    }

    // -----
    // Synchronous implementation via Evolution API
    // -----

    private Map<String, Object> sendWhatsAppMessageSync(String phone, String text) {
        String cleaned = formatPhone(phone);
        if (cleaned.isEmpty()) {
            log.warn("⚠ Cannot send WhatsApp message: phone number is empty");
            return null;
        }

        String url = appProperties.getEvolutionApiUrl() + "/message/sendText/" + appProperties.getInstanceName();
        log.info("\n📡 [EVOLUTION API] Sending Text to {}:{}\n\n", cleaned, text);

        Map<String, Object> body = new LinkedHashMap<>();
        body.put("number", cleaned);
        body.put("text", text);

        HttpHeaders headers = new HttpHeaders();
        headers.setContentType(MediaType.APPLICATION_JSON);
        headers.set("apikey", appProperties.getEvolutionApiKey());

        try {
            ResponseEntity<Map> response = restTemplate.postForEntity(url, new HttpEntity<>(body, headers), Map.class);
            log.info("✅ Evolution API text sent successfully to {}", cleaned);
            return response.getBody();
        } catch (RestClientException e) {
            String errStr = extractError(e);
            if (errStr != null && errStr.contains("Connection Closed")) {
                log.warn("⚠ Evolution API Notice for {}: {}: Cannot send message to the bot's own connected number (Connection Closed). Please
test from a separate mobile number.", cleaned);
            } else {
                log.error("❌ Evolution API error for {}: {}", cleaned, errStr);
            }
            return null;
        }
    }

    private String formatButtonsAsText(String title, String description, List<WaButton> buttons, String footer) {
        StringBuilder sb = new StringBuilder();
        if (title != null && !title.isEmpty()) {
            sb.append(title).append("\n\n");
        }
        if (description != null && !description.isEmpty()) {
            sb.append(description);
        }
        if (buttons != null && !buttons.isEmpty()) {
            sb.append("\n\n");
            for (int i = 0; i < buttons.size(); i++) {
                sb.append(i + 1).append(" ").append(buttons.get(i).displayText()).append("\n");
            }
        }
        if (footer != null && !footer.isEmpty()) {
            sb.append("\n_").append(footer).append("_");
        }
    }

```

```
    }
    return sb.toString().trim();
}

private String plainTextFallback(String title, String description, List<WaListSection> sections) {
    StringBuilder sb = new StringBuilder();
    if (title != null && !title.isEmpty()) {
        sb.append(title).append("\n\n");
    }
    if (description != null && !description.isEmpty()) {
        sb.append(description).append("\n\n");
    }
    if (sections != null) {
        for (WaListSection section : sections) {
            for (WaListRow row : section.rows()) {
                sb.append("• ").append(row.title());
                if (row.description() != null && !row.description().isEmpty()) {
                    sb.append(" -- ").append(row.description());
                }
                sb.append('\n');
            }
        }
    }
    return sb.toString().trim();
}
}
```

13. Database Schema — schema.sql

The single source of truth for the database structure, executed idempotently on every boot by SchemaInitRunner (every statement is CREATE ... IF NOT EXISTS / ADD COLUMN IF NOT EXISTS, so it's always safe to re-run against a fresh or partially-populated database). All tables live in a dedicated clinicqueue Postgres schema rather than public — set as the connection's default schema via currentSchema=clinicqueue on the JDBC URL — specifically so this app never collides with unrelated tables that might already exist in a shared database's public schema (see the comment above the doctors table for the incident that motivated this).

Table	Purpose
hospitals	Location (DIGIPIN + lat/lon), OPD hours, counter count, and profile (ownership, accreditation, gender restriction, offered categories). Seeded with a placeholder "main" row that HospitalSeedRunner overwrites on boot.
tokens	The live queue — one row per patient ticket, carrying booking-flow state, geo/ETA fields, priority-rank ordering, and multi-counter assignment. The busiest table in the schema, with a dozen indexes supporting the query patterns in QueueManagerService.
otp_verifications	Phone OTP hashes, expiry, and attempt counts (see OtpService).
wa_sessions	Per-phone WhatsApp bot language/stage (see WaSessionService).
token_history	Durable per-visit ledger, archived the moment a token completes.
doctors	Phone+OTP doctor accounts, linked to a hospital and a counter/department.
patient_documents	Uploaded prescription/report photos, stored as BYTEA (no cloud storage).
access_requests / access_grants	The doctor-patient consent model — a short-lived request (code+QR), and the revocable grant created once a patient claims one.
hospital_departments	One row per (hospital, category) queue's counter count.
time_slots / time_slot_doctors	OPD time windows and their rostered doctors.

```
-- =====
-- Idempotent schema definition, executed automatically on every boot by
-- Spring Boot's SQL initializer (spring.sql.init.mode=always, see
-- application.yml) via the spring-boot-starter-jdbc dependency already on
-- the classpath. This app talks to Postgres entirely through plain
-- NamedParameterJdbcTemplate SQL -- there are no @Entity classes anywhere,
-- so spring.jpa.hibernate.ddl-auto has nothing to generate DDL from and
-- Flyway's versioned-migration-history model (which fell out of sync with
-- the code and produced "bad SQL grammar" errors) has been replaced by this
-- single, always-safe-to-rerun file instead. Every statement here is
-- CREATE ... IF NOT EXISTS / ADD COLUMN IF NOT EXISTS, so running it against
-- a fresh DB or a DB that already has some/all of this schema both just
-- work -- mirrors the original Node backend's ad hoc CREATE TABLE IF NOT
-- EXISTS approach (see backend/utlis/db.js).
--
-- Everything below lives in its own "clinicqueue" schema, not "public" --
-- the JDBC URL's currentSchema=clinicqueue param (see application.yml) makes
-- it the connection's default, so every unqualified CREATE TABLE/reference
-- here (and every query in the app) resolves inside it. That keeps this app
-- from colliding with whatever else already lives in "public" on the target
-- database (e.g. a shared Supabase project's own "users"/"doctors" tables --
-- CREATE TABLE IF NOT EXISTS is a no-op against an existing same-named
-- table, and ADD COLUMN IF NOT EXISTS would then silently bolt this app's
-- columns onto it instead of creating its own).
-- =====

CREATE SCHEMA IF NOT EXISTS clinicqueue;
```

```

-----
-- hospitals: location (DIGIPIN-based), OPD hours, counters, and profile.
-- See HospitalService.
-----

```

```

CREATE TABLE IF NOT EXISTS hospitals (
  id SERIAL PRIMARY KEY,
  uri_slug VARCHAR(100) NOT NULL UNIQUE,
  name VARCHAR(200) NOT NULL,
  address TEXT,
  digipin CHAR(10) NOT NULL,
  latitude DOUBLE PRECISION NOT NULL,
  longitude DOUBLE PRECISION NOT NULL,
  open_time TIME NOT NULL DEFAULT '09:00',
  close_time TIME NOT NULL DEFAULT '17:00',
  avg_service_minutes INT NOT NULL DEFAULT 10,
  min_service_minutes INT,
  active_counters INT NOT NULL DEFAULT 1,
  doctor_join_code VARCHAR(20),
  ownership VARCHAR(50),
  year_established INT,
  accreditation TEXT,
  gender_specific VARCHAR(10),
  categories TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```

-- Self-heals a "hospitals" table created by an earlier version of this file.

```

```

ALTER TABLE hospitals
  ADD COLUMN IF NOT EXISTS min_service_minutes INT,
  ADD COLUMN IF NOT EXISTS doctor_join_code VARCHAR(20),
  ADD COLUMN IF NOT EXISTS ownership VARCHAR(50),
  ADD COLUMN IF NOT EXISTS year_established INT,
  ADD COLUMN IF NOT EXISTS accreditation TEXT,
  ADD COLUMN IF NOT EXISTS gender_specific VARCHAR(10),
  ADD COLUMN IF NOT EXISTS categories TEXT;

```

```

CREATE UNIQUE INDEX IF NOT EXISTS idx_hospitals_doctor_join_code
  ON hospitals (doctor_join_code) WHERE doctor_join_code IS NOT NULL;

```

```

-- Placeholder default hospital row (India Gate, Delhi, as a non-null
-- placeholder location) -- HospitalSeedRunner overwrites this from
-- HOSPITAL_* env vars on every boot, so this row is just a safe fallback
-- if the app ever starts against a fresh DB before that runner executes.
INSERT INTO hospitals (uri_slug, name, address, digipin, latitude, longitude, open_time, close_time, avg_service_minutes, active_counters)
SELECT 'main', 'Main Hospital', 'Address not yet configured', '3C3P39L8T4', 28.6139, 77.2090, '09:00', '17:00', 10, 1
WHERE NOT EXISTS (SELECT 1 FROM hospitals WHERE uri_slug = 'main');

```

```

-----
-- tokens: one row per patient queue ticket. See QueueManagerService.
-----

```

```

CREATE TABLE IF NOT EXISTS tokens (
  id SERIAL PRIMARY KEY,
  phone VARCHAR(50) NOT NULL,
  name VARCHAR(100) NOT NULL,
  age INT,
  gender VARCHAR(10),
  category VARCHAR(100),
  status VARCHAR(20) DEFAULT 'waiting',
  session_step VARCHAR(50) DEFAULT 'welcome',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  served_at TIMESTAMP NULL,
  completed_at TIMESTAMP NULL,
  missed_at TIMESTAMP NULL,
  rejected_at TIMESTAMP NULL,
  is_verified BOOLEAN DEFAULT FALSE,
  verified_at TIMESTAMP NULL,
  hospital_id INT,
  patient_digipin CHAR(10),
  patient_lat DOUBLE PRECISION,
  patient_lon DOUBLE PRECISION,
  distance_km DOUBLE PRECISION,
  travel_minutes DOUBLE PRECISION,
  treatment_remaining_minutes DOUBLE PRECISION,
  notified_ready_at TIMESTAMP,

```

```

    anomaly_control_until TIMESTAMP,
    priority_rank BIGINT,
    counter_id INT,
    reserved_counter_id INT,
    no_show_count INT NOT NULL DEFAULT 0,
    search_offset INT NOT NULL DEFAULT 0,
    daily_number INT
);

-- Self-heals a "tokens" table created by an earlier version of this file.
-- Each ALTER is a separate statement so one failure cannot block the others.
ALTER TABLE tokens ADD COLUMN IF NOT EXISTS age INT;
ALTER TABLE tokens ADD COLUMN IF NOT EXISTS gender VARCHAR(10);
ALTER TABLE tokens ADD COLUMN IF NOT EXISTS category VARCHAR(100);
ALTER TABLE tokens ADD COLUMN IF NOT EXISTS rejected_at TIMESTAMP;
ALTER TABLE tokens ADD COLUMN IF NOT EXISTS hospital_id INT;
ALTER TABLE tokens ADD COLUMN IF NOT EXISTS patient_digipin CHAR(10);
ALTER TABLE tokens ADD COLUMN IF NOT EXISTS patient_lat DOUBLE PRECISION;
ALTER TABLE tokens ADD COLUMN IF NOT EXISTS patient_lon DOUBLE PRECISION;
ALTER TABLE tokens ADD COLUMN IF NOT EXISTS distance_km DOUBLE PRECISION;
ALTER TABLE tokens ADD COLUMN IF NOT EXISTS travel_minutes DOUBLE PRECISION;
ALTER TABLE tokens ADD COLUMN IF NOT EXISTS treatment_remaining_minutes DOUBLE PRECISION;
ALTER TABLE tokens ADD COLUMN IF NOT EXISTS notified_ready_at TIMESTAMP;
ALTER TABLE tokens ADD COLUMN IF NOT EXISTS anomaly_control_until TIMESTAMP;
ALTER TABLE tokens ADD COLUMN IF NOT EXISTS priority_rank BIGINT;
ALTER TABLE tokens ADD COLUMN IF NOT EXISTS counter_id INT;
ALTER TABLE tokens ADD COLUMN IF NOT EXISTS reserved_counter_id INT;
ALTER TABLE tokens ADD COLUMN IF NOT EXISTS no_show_count INT NOT NULL DEFAULT 0;
ALTER TABLE tokens ADD COLUMN IF NOT EXISTS search_offset INT NOT NULL DEFAULT 0;
ALTER TABLE tokens ADD COLUMN IF NOT EXISTS daily_number INT;

-- Speeds up the hot paths in QueueManagerService: phone lookups, status-
-- filtered queue reads, the "how many waiting tokens have a smaller
-- id/priority_rank" position count, and the department-scoped FIFO scan.
CREATE INDEX IF NOT EXISTS idx_tokens_phone ON tokens (phone);
CREATE INDEX IF NOT EXISTS idx_tokens_status ON tokens (status);
CREATE INDEX IF NOT EXISTS idx_tokens_status_id ON tokens (status, id);
CREATE INDEX IF NOT EXISTS idx_tokens_created_at ON tokens (created_at);
CREATE INDEX IF NOT EXISTS idx_tokens_hospital_id ON tokens (hospital_id);
CREATE INDEX IF NOT EXISTS idx_tokens_missed_at ON tokens (missed_at);
CREATE INDEX IF NOT EXISTS idx_tokens_priority_rank ON tokens (priority_rank);
CREATE INDEX IF NOT EXISTS idx_tokens_counter_id ON tokens (counter_id);
CREATE INDEX IF NOT EXISTS idx_tokens_reserved_counter_id ON tokens (reserved_counter_id);
CREATE INDEX IF NOT EXISTS idx_tokens_category ON tokens (category);
CREATE INDEX IF NOT EXISTS idx_tokens_hospital_category_status ON tokens (hospital_id, category, status);

-----
-- otp_verifications: phone-number OTP hashes/expiry/attempts. See OtpService.
-----
CREATE TABLE IF NOT EXISTS otp_verifications (
    id SERIAL PRIMARY KEY,
    phone VARCHAR(50) NOT NULL,
    code_hash VARCHAR(255) NOT NULL,
    purpose VARCHAR(50) NOT NULL DEFAULT 'registration',
    expires_at TIMESTAMP NOT NULL,
    attempts INT NOT NULL DEFAULT 0,
    verified BOOLEAN NOT NULL DEFAULT FALSE,
    verified_at TIMESTAMP,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX IF NOT EXISTS idx_otp_phone ON otp_verifications (phone);
CREATE INDEX IF NOT EXISTS idx_otp_phone_verified ON otp_verifications (phone, verified, expires_at);

-----
-- wa_sessions: per-phone WhatsApp bot language/stage. See bot.WaSessionService.
-----
CREATE TABLE IF NOT EXISTS wa_sessions (
    phone VARCHAR(32) PRIMARY KEY,
    language VARCHAR(2),
    stage VARCHAR(20) NOT NULL DEFAULT 'awaiting_language',
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-----

```

```
-- token_history: durable per-phone visit ledger, archived the moment a token
-- is marked 'completed'. See QueueManagerService#archiveToHistory.
-----

CREATE TABLE IF NOT EXISTS token_history (
  id SERIAL PRIMARY KEY,
  token_id INT NOT NULL,
  phone VARCHAR(50) NOT NULL,
  name VARCHAR(100) NOT NULL,
  age INT,
  gender VARCHAR(10),
  category VARCHAR(100),
  hospital_id INT,
  counter_id INT,
  created_at TIMESTAMP,
  served_at TIMESTAMP,
  completed_at TIMESTAMP,
  archived_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

ALTER TABLE token_history
  ADD COLUMN IF NOT EXISTS gender VARCHAR(10),
  ADD COLUMN IF NOT EXISTS category VARCHAR(100),
  ADD COLUMN IF NOT EXISTS hospital_id INT,
  ADD COLUMN IF NOT EXISTS counter_id INT;

CREATE INDEX IF NOT EXISTS idx_token_history_phone ON token_history (phone);
CREATE INDEX IF NOT EXISTS idx_token_history_token_id ON token_history (token_id);

-----
-- doctors: phone+OTP accounts, linked to one hospital via its join code, and
-- assigned a counter/department. See DoctorService.
-----

CREATE TABLE IF NOT EXISTS doctors (
  id SERIAL PRIMARY KEY,
  phone VARCHAR(50) NOT NULL UNIQUE,
  name VARCHAR(100) NOT NULL,
  hospital_id INT,
  counter_id INT,
  category VARCHAR(100),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

ALTER TABLE doctors
  ADD COLUMN IF NOT EXISTS id SERIAL,
  ADD COLUMN IF NOT EXISTS hospital_id INT,
  ADD COLUMN IF NOT EXISTS counter_id INT,
  ADD COLUMN IF NOT EXISTS category VARCHAR(100);

-- Earlier versions of this app connected straight to a Supabase project's default
-- "postgres" DB with no schema of its own, so this CREATE TABLE landed on top of that
-- project's *own* "doctors"/"users" tables sitting in "public" (Supabase Auth's
-- auth.users, judging by their column shape -- encrypted_password, confirmation_token,
-- raw_app_meta_data, etc. -- or a public-schema table referencing it) instead of
-- creating this app's own: "doctors" there was keyed on a NOT NULL "ehrid" column with
-- a foreign key into that "users" table, which this app's INSERTs never populated ("null
-- value in column \"ehrid\", then a FK violation once a random UUID silenced that).
-- Now that every table in this file lives in its own "clinicqueue" schema (see the
-- CREATE SCHEMA above and currentSchema= in application.yml's datasource url), the
-- CREATE TABLE above always creates a fresh doctors table rather than colliding with
-- one from another project living in "public" -- no ehrid/FK workaround needed.

CREATE INDEX IF NOT EXISTS idx_doctors_hospital_id ON doctors (hospital_id);

-----
-- patient_documents: uploaded prescription/report photos, stored as BYTEA
-- (no cloud storage configured). See PatientDocumentService.
-----

CREATE TABLE IF NOT EXISTS patient_documents (
  id SERIAL PRIMARY KEY,
  patient_phone VARCHAR(50) NOT NULL,
  patient_name VARCHAR(100) NOT NULL,
  patient_age INT,
  doc_type VARCHAR(20) NOT NULL, -- 'prescription' or 'report'
  hospital_id INT,
```

```

    uploaded_by_doctor_id INT,
    file_name VARCHAR(255) NOT NULL,
    content_type VARCHAR(100) NOT NULL,
    file_size INT NOT NULL,
    file_data BYTEA NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX IF NOT EXISTS idx_patient_documents_phone ON patient_documents (patient_phone);
CREATE INDEX IF NOT EXISTS idx_patient_documents_identity ON patient_documents (patient_phone, patient_name, patient_age);

-- -----
-- access_requests / access_grants: a doctor's outstanding "let me view your
-- records" request (code + QR), and the revocable consent grant created once
-- a patient claims one. See AccessService.
-- -----
CREATE TABLE IF NOT EXISTS access_requests (
    id SERIAL PRIMARY KEY,
    code VARCHAR(20) NOT NULL UNIQUE,
    doctor_id INT NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'pending', -- pending | claimed | expired
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    expires_at TIMESTAMP NOT NULL,
    claimed_by_phone VARCHAR(50),
    claimed_at TIMESTAMP
);

CREATE INDEX IF NOT EXISTS idx_access_requests_code ON access_requests (code);

CREATE TABLE IF NOT EXISTS access_grants (
    id SERIAL PRIMARY KEY,
    doctor_id INT NOT NULL,
    patient_phone VARCHAR(50) NOT NULL,
    source_request_id INT,
    granted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    revoked_at TIMESTAMP,
    revoked_by VARCHAR(20) -- 'patient' or 'doctor'
);

CREATE INDEX IF NOT EXISTS idx_access_grants_patient_phone ON access_grants (patient_phone);
CREATE INDEX IF NOT EXISTS idx_access_grants_doctor_id ON access_grants (doctor_id);
CREATE INDEX IF NOT EXISTS idx_access_grants_active ON access_grants (patient_phone, doctor_id);

-- -----
-- hospital_departments: one row per (hospital, category) queue -- how many
-- counters staff that department. See HospitalDepartmentService.
-- -----
CREATE TABLE IF NOT EXISTS hospital_departments (
    id SERIAL PRIMARY KEY,
    hospital_id INT NOT NULL,
    category VARCHAR(100) NOT NULL,
    active_counters INT NOT NULL DEFAULT 1,
    avg_service_minutes INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Backs both lookups and the ON CONFLICT (hospital_id, category) upsert in
-- HospitalDepartmentService#ensure.
CREATE UNIQUE INDEX IF NOT EXISTS idx_hospital_departments_unique ON hospital_departments (hospital_id, category);

-- -----
-- time_slots / time_slot_doctors: OPD time windows within a day, each
-- assigned to a list of doctors. See TimeSlotService.
-- -----
CREATE TABLE IF NOT EXISTS time_slots (
    id SERIAL PRIMARY KEY,
    hospital_id INT NOT NULL,
    category VARCHAR(100),
    slot_date DATE NOT NULL,
    start_time TIME NOT NULL,
    end_time TIME NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```
CREATE INDEX IF NOT EXISTS idx_time_slots_hospital_date ON time_slots (hospital_id, slot_date);

CREATE TABLE IF NOT EXISTS time_slot_doctors (
  time_slot_id INT NOT NULL,
  doctor_id INT NOT NULL,
  PRIMARY KEY (time_slot_id, doctor_id)
);

CREATE INDEX IF NOT EXISTS idx_time_slot_doctors_doctor ON time_slot_doctors (doctor_id);
```

14. Configuration — application.yml

Spring Boot configuration. Virtually every value is an environment-variable placeholder with a dev-friendly default (`${VAR:default}`), following the original Node backend's `.env` variable naming so migrating a deployment's environment config over was mechanical. Notable points: a local `.env` file is loaded via `spring-dotenv` for non-Docker runs; the `datasource` URL disables server-side prepared statements (`prepareThreshold=0`) for compatibility with Supabase's transaction-mode connection pooler, and sets a dedicated `clinicqueue` schema via `currentSchema=`; and schema application is handled by the custom `SchemaInitRunner` rather than Spring Boot's own SQL initializer, specifically so a DB outage at boot is logged rather than crashing the whole application context.

```
server:
  port: ${PORT:8088}
  ssl:
    enabled: ${SERVER_SSL_ENABLED:false}
    key-store: ${SERVER_SSL_KEY_STORE:}
    key-store-password: ${SERVER_SSL_KEY_STORE_PASSWORD:}
    key-store-type: ${SERVER_SSL_KEY_STORE_TYPE:PKCS12}
    key-alias: ${SERVER_SSL_KEY_ALIAS:}

spring:
  config:
    # Load a local .env (KEY=VALUE, same format docker-compose reads) into the
    # Spring Environment for non-Docker runs (mvn spring-boot:run / java -jar).
    # "optional:" so startup doesn't fail when it's absent (e.g. in CI/Docker,
    # where real env vars are supplied instead). Never committed -- see .gitignore.
    import: optional:file:.env[.properties]
  application:
    name: clinic-queue-backend
  datasource:
    # prepareThreshold=0 disables pgjdbc's server-side prepared statements. Required
    # for Supabase's transaction-mode pooler (port 6543): PgBouncer hands each
    # transaction a different physical backend connection, so a statement name
    # cached by the driver on one connection collides ("prepared statement \"S_n\"
    # already exists") the next time it's reused on a different one. Harmless
    # against a direct/session-mode connection too, so it's left unconditional
    # rather than gated on which port is configured.
    #
    # currentSchema puts every unqualified table this app touches inside its own
    # "clinicqueue" schema (created by schema.sql) instead of "public" -- needed
    # whenever DB_DATABASE points at a database that already has its own
    # unrelated tables in public (e.g. a shared Supabase project with its own
    # "users"/"doctors"). Without this, schema.sql's CREATE TABLE IF NOT EXISTS /
    # ADD COLUMN IF NOT EXISTS silently bolts this app's columns onto those
    # foreign tables instead of creating its own -- that's what caused the
    # "ehrid"/FK-to-users errors on doctor registration. pgjdbc sends this as a
    # startup-packet option (-c search_path=...), which -- unlike a runtime SET
    # -- is safe with PgBouncer/Supervisor transaction pooling.
    url: jdbc:postgresql://${DB_HOST:localhost}:${DB_PORT:5432}/${DB_DATABASE:postgres}?
    sslmode=${DB_SSL_MODE:prefer}&prepareThreshold=0&currentSchema=${DB_SCHEMA:clinicqueue}
    username: ${DB_USER:postgres}
    password: ${DB_PASSWORD:postgres}
    driver-class-name: org.postgresql.Driver
  hikari:
    initialization-fail-timeout: -1
    # A remote pooled host (e.g. Supabase's pgbouncer) can take several
    # seconds just to hand back a connection -- 10s was cutting it close
    # under any extra latency/load, so a churned-out connection would
    # occasionally miss the deadline and surface as "Failed to obtain JDBC
    # Connection" (see QueueManagerService's scheduled ticks and, silently,
    # any webhook request that needed a connection at the same moment).
    connection-timeout: 30000
    maximum-pool-size: ${DB_POOL_MAX_SIZE:5}
    minimum-idle: ${DB_POOL_MIN_IDLE:2}
    # idle-timeout/max-lifetime this short (30s/60s) forced HikariCP to
    # keep retiring and re-establishing connections against a remote host
    # whose handshake alone can take several seconds -- constant churn that
    # was the actual cause of the connection-timeout spikes above. Give
    # connections a realistic lifespan instead (max-lifetime kept under a
```

```

    # typical pooler's own ~30min connection recycle window).
    idle-timeout: 600000
    max-lifetime: 1500000
  jackson:
    serialization:
      write-dates-as-timestamps: false
  web:
    resources:
      add-mappings: true
  servlet:
    multipart:
      # Spring Boot defaults to 1MB, which would reject prescription/report photo
      # uploads well under PatientDocumentService's own 8MB cap -- give it headroom.
      max-file-size: 10MB
      max-request-size: 10MB
  sql:
    init:
      # This app has no JPA/Hibernate entities anywhere (plain NamedParameterJdbcTemplate
      # throughout -- see JdbcConfig), so there's nothing for ddl-auto to generate DDL
      # from. Schema instead comes from src/main/resources/schema.sql -- but applied by
      # config/SchemaInitRunner, NOT Spring Boot's own auto-configured SQL initializer:
      # that initializer's continue-on-error only covers failures *within* the script,
      # not a failed-to-connect DB at boot, which it treats as fatal and crashes the
      # whole application context. SchemaInitRunner instead swallows any failure (DB
      # unreachable, or a bad statement) exactly like the old FlywayMigrationRunner did,
      # so a DB outage at startup is logged and doesn't take the app down with it.
      mode: never

# Application-specific settings. Most env var names deliberately match the
# original Node backend's .env; see README for the security-hardening additions.
app:
  frontend-url: ${FRONTEND_URL:https://princete.com}
  clinic-name: ${CLINIC_NAME:qDischarge}
  token-expiry-hours: ${TOKEN_EXPIRY_HOURS:24}
  avg-service-minutes: ${AVG_SERVICE_MINUTES:10}

  admin-username: ${ADMIN_USERNAME:admin}
  admin-password: ${ADMIN_PASSWORD:adminpass}
  admin-password-hash: ${ADMIN_PASSWORD_HASH:}
  jwt-secret: ${JWT_SECRET:}
  jwt-expiry-minutes: ${JWT_EXPIRY_MINUTES:720}

  webhook-verify-token: ${WEBHOOK_VERIFY_TOKEN:clinic_queue_token}
  allowed-origins:
    ${ALLOWED_ORIGINS:https://princete.com,https://www.princete.com,http://princete.com,http://www.princete.com,http://localhost:5173,http://localhost:3000,http://localhost:8080}

  # WhatsApp provider: 'meta' (Meta Cloud API) or 'evolution' (Evolution API)
  wa-provider: ${WA_PROVIDER:evolution}
  meta-access-token: ${META_ACCESS_TOKEN:}
  meta-phone-number-id: ${META_PHONE_NUMBER_ID:1337438019442398}
  meta-api-version: ${META_API_VERSION:v25.0}

  evolution-api-url: ${EVOLUTION_API_URL:http://localhost:8080}
  evolution-api-key: ${EVOLUTION_API_KEY:}
  instance-name: ${INSTANCE_NAME:clinic-bot}

  rate-limit-login-per-minute: ${RATE_LIMIT_LOGIN_PER_MINUTE:10}
  rate-limit-webhook-per-minute: ${RATE_LIMIT_WEBHOOK_PER_MINUTE:120}
  rate-limit-create-token-per-minute: ${RATE_LIMIT_CREATE_TOKEN_PER_MINUTE:20}

  async-core-pool-size: ${ASYNC_CORE_POOL_SIZE:4}
  async-max-pool-size: ${ASYNC_MAX_POOL_SIZE:16}
  async-queue-capacity: ${ASYNC_QUEUE_CAPACITY:500}

  # Distance -> ETA fallback (see geo/GeoDistanceService; used only when TomTom is unavailable)
  geo-avg-speed-kmh: ${GEO_AVG_SPEED_KMH:25}

  # Real routing ETA (TomTom) vs. queue treatment-time-remaining, anomaly-control
  # grace window, and the "head to the hospital now" voice call (see
  # geo/TomTomRoutingService, service/TreatmentTimingScheduler, service/TwilioStudioCallService).
  # Blank TOMTOM_API_KEY -> falls back to the haversine estimate above instead of failing booking.
  tomtom-api-key: ${TOMTOM_API_KEY:}
  twilio-caller-number: ${TWILIO_CALLER_NUMBER:}
  twilio-call-enabled: ${TWILIO_CALL_ENABLED:false}

```

```

notify-buffer-minutes: ${NOTIFY_BUFFER_MINUTES:2}
timing-poll-interval-ms: ${TIMING_POLL_INTERVAL_MS:15000}

# Default single hospital, seeded into the "hospitals" table on first migration.
# Location can be given as a DIGIPIN, or as lat/lon (a DIGIPIN is then derived).
hospital-uri-slug: ${HOSPITAL_URI_SLUG:main}
hospital-digipin: ${HOSPITAL_DIGIPIN:}
hospital-latitude: ${HOSPITAL_LATITUDE:}
hospital-longitude: ${HOSPITAL_LONGITUDE:}
hospital-open-time: ${HOSPITAL_OPEN_TIME:09:00}
hospital-close-time: ${HOSPITAL_CLOSE_TIME:17:00}
hospital-active-counters: ${HOSPITAL_ACTIVE_COUNTERS:1}

# OTP verification: 'log' logs the code instead of sending an SMS (dev
# default, no Twilio account needed); 'twilio' delegates generation/expiry/
# matching entirely to Twilio's Verify API; 'twilio-sms' self-generates the
# code (like 'log') and sends it via Twilio's plain Messages API through a
# Messaging Service instead.
otp-provider: ${OTP_PROVIDER:log}
otp-required-for-registration: ${OTP_REQUIRED_FOR_REGISTRATION:false}
twilio-account-sid: ${TWILIO_ACCOUNT_SID:ACdummy000000000000000000000000}
twilio-auth-token: ${TWILIO_AUTH_TOKEN:dummy_auth_token}
twilio-verify-service-sid: ${TWILIO_VERIFY_SERVICE_SID:VAdummy000000000000000000000000}
twilio-messaging-service-sid: ${TWILIO_MESSAGING_SERVICE_SID:}
otp-ttl-minutes: ${OTP_TTL_MINUTES:10}
rate-limit-otp-per-minute: ${RATE_LIMIT_OTP_PER_MINUTE:5}

management:
  endpoints:
    web:
      exposure:
        include: health,info
  endpoint:
    health:
      show-details: when-authorized
      # Exposes /actuator/health/liveness (process is alive -- ignores DB)
      # and /actuator/health/readiness (DB reachable). Point container/
      # orchestrator restart policies at liveness and load-balancer routing
      # at readiness, so a transient DB outage doesn't kill healthy instances.
      probes:
        enabled: true
  health:
    defaults:
      enabled: true
    livenessState:
      enabled: true
    readinessState:
      enabled: true

logging:
  level:
    com.qdischarge.clinicqueue: INFO

```

15. Build — pom.xml

Maven build definition, Spring Boot 3.3.5 parent, Java 21. Notable dependencies beyond the standard Spring Boot starters (web, jdbc, security, validation, actuator): jjwt for JWT signing/parsing, zxing for QR code generation, spring-dotenv for loading a local .env file into Spring's environment on non-Docker runs, and the PostgreSQL JDBC driver. There is no JPA/Hibernate starter — consistent with the rest of the codebase never using @Entity classes.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.5</version>
    <relativePath/>
  </parent>

  <groupId>com.qdischarge</groupId>
  <artifactId>clinic-queue-backend</artifactId>
  <version>1.0.0</version>
  <name>clinic-queue-backend</name>
  <description>Spring Boot port of the qDischarge Smart Clinic Queue Management System (originally Node/Express)</description>
  <packaging>jar</packaging>

  <properties>
    <java.version>21</java.version>
    <zxing.version>3.5.3</zxing.version>
    <lombok.version>1.18.46</lombok.version>
    <jjwt.version>0.12.6</jjwt.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-jdbc</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-security</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>

    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-actuator</artifactId>
    </dependency>

    <!-- JWT for admin session tokens (signed, short-lived, stateless) -->
    <dependency>
      <groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-api</artifactId>
      <version>${jjwt.version}</version>
    </dependency>
    <dependency>
      <groupId>io.jsonwebtoken</groupId>
```

```

    <artifactId>jjwt-impl</artifactId>
    <version>${jjwt.version}</version>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>${jjwt.version}</version>
    <scope>runtime</scope>
</dependency>

<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
</dependency>

<!-- Auto-loads .env into Spring's Environment for local (non-Docker) runs,
      so DB_HOST/DB_PORT/etc. in .env actually reach application.yml's
      ${DB_HOST:localhost}-style placeholders. Docker Compose doesn't need
      this since it injects env vars itself, but `mvn spring-boot:run` and
      `java -jar` do, and previously silently fell back to localhost. -->
<dependency>
    <groupId>me.paulschwarz</groupId>
    <artifactId>spring-dotenv</artifactId>
    <version>4.0.0</version>
</dependency>

<!-- QR code generation (mirrors the "qrcode" npm package) -->
<dependency>
    <groupId>com.google.zxing</groupId>
    <artifactId>core</artifactId>
    <version>${zxing.version}</version>
</dependency>
<dependency>
    <groupId>com.google.zxing</groupId>
    <artifactId>javase</artifactId>
    <version>${zxing.version}</version>
</dependency>

<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
</dependency>

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
</dependencies>

<build>
    <finalName>clinic-queue-backend</finalName>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <configuration>
                <excludes>
                    <exclude>
                        <groupId>org.projectlombok</groupId>
                        <artifactId>lombok</artifactId>
                    </exclude>
                </excludes>
            </configuration>
        </plugin>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <configuration>
                <annotationProcessorPaths>
                    <path>
                        <groupId>org.projectlombok</groupId>

```

```
        <artifactId>lombok</artifactId>
        <version>${lombok.version}</version>
      </path>
    </annotationProcessorPaths>
  </configuration>
</plugin>
</plugins>
</build>

</project>
```